

Multi-Authority Encryption with Malicious Authorities

Rishab Goyal
UW-Madison*

Saikumar Yadugiri
UW-Madison[†]

Abstract

This work addresses fundamental gaps in our current understanding of multi-authority encryption systems. We design multi-authority functional encryption (MAFE) for circuits from *minimal assumptions* in, both, the collusion-resistant and bounded-collusion models. Under the additional assumption of either SXDH or LWE, we show that any adversary that also corrupts any k out of n local authorities (such that $\binom{n}{k} = \text{poly}(\lambda)$) cannot break MAFE security, in both collusion models. Additionally, we show that any MAFE scheme, where an adversary can corrupt more than $O(\log \lambda)$ authorities implies obfuscation, thus our MAFE scheme is optimal for a wide range of parameters.

We also construct an adaptively-secure multi-authority attribute-based encryption (MAABE) scheme for all polynomial-sized predicates under the hardness of witness encryption in the standard model. Prior to this work, the only MAABE construction (beyond log-depth monotone circuits) relied on sub-exponentially-secure obfuscation. We show many applications of our multi-authority encryption schemes in a variety of contexts such as decentralized tracing and threshold cryptography.

Contents

1	Introduction	1
1.1	Applications	3
2	Technical Overview	5
2.1	MAFE with Malicious Authorities	6
2.2	MAFE with Honest Authorities from Minimal Assumptions	8
2.2.1	Bounded Collusion MAFE from PKE	8
2.2.2	Collusion Resistant MAFE from Functional Encryption	10
2.3	Multi-Authority ABE from Witness Encryption	12
3	MAFE from Public-Key Encryption	14
3.1	Definition	14
3.2	Preliminaries	16
3.3	Construction	17

*Email: rishab@cs.wisc.edu.

[†]Email: saikumar@cs.wisc.edu.

4	Bounded Collusion MAFE with Malicious Authorities	18
4.1	Definition	18
4.2	MAFE with Adaptive Corruptions from HSS	19
5	Collusion Resistant MAFE from Functional Encryption	21
5.1	Definition	22
5.2	Preliminaries	22
5.3	Selectively Secure MAFE with Batched Key Generation	23
5.4	Adaptively Secure MAFE with Batched Key Generation	25
5.5	MAFE from FE	27
6	Collusion Resistant MAFE with Malicious Authorities	28
6.1	Definition	28
6.2	Construction	29
7	MA-ABE from Witness Encryption	31
7.1	Definitions	31
7.2	MA-ABE with Batched Key Generation	32
7.3	MA-ABE from WE	34
7.4	MA-ABE with Malicious Authorities	35
A	Additional Preliminaries	45
B	Relevant Material from [GY24]	48
B.1	Distributed Client-Server Framework	48
B.2	Augmented Statistical Lemmas	50
C	MAFE with Trusted Setup	50
C.1	Construction	51
D	Proofs from Section 3	52
E	MAFE with Dynamic Collusions	57
E.1	Definitions	57
E.2	Preliminaries	58
E.3	Weakly Optimal tMAFE from IBE and tMAFE	58
E.4	Dynamic MAFE from Minimal Assumptions	69
F	Homomorphic Secret Sharing with Strong Security	70
G	Proofs of Theorem 4.4	71
H	Generic Compilers for Authority Corruptions	77
I	Proofs from Section 5.3	79

J	One-Ciphertext Private-Key MAFE	84
J.1	Definition	85
J.2	Preliminaries	86
J.3	One-Key One-Challenge SIM-secure MCFE for P/Poly	86
J.4	One-Key Many-Challenge MCFE for P/Poly	94
J.5	One-CT Private-Key MAFE	95
K	Proofs from Section 5.4	97
L	Vector Non-Committing Encryption from IBE	103
L.1	Definition: Vector Non-Committing Encryption	103
L.2	Preliminaries	104
L.3	VNCE from NCE	105
M	Proofs from Section 5.5	110
N	Proofs from Section 6.2	114
O	Lower Bounds on MAFE with Authority Corruptions	119
O.1	Preliminaries	119
O.2	Definitions	120
O.3	XiO from fst1MAFE	121
O.4	Weakly Size-Succinct FE from stMAFE	122
P	Distributed Functional Tag System	124
P.1	Definition	124
P.2	Preliminaries	125
P.3	Construction	127
Q	Proofs from Section 7.2	134
R	Full MA-ABE from VNCE	140
S	Proofs from Section 7.4	144

1 Introduction

Multi-authority encryption model [Cha07, CC09, LW11, BCG⁺17b] is a well-established model to shield advanced encryption systems from the key escrow problem [Rog15]. It decentralizes the job of the *singular trusted* master key authority, in monolithic systems such as attribute-based (ABE) and functional encryption (FE) [SW05, GPSW06, BW07, KSW08, BSW11], to a group of a-synchronized, non-interacting, and mistrusting local key authorities. For example, entities such as the Department of Motor Vehicles (DMV), Department of Homeland Security (DHS), Internal Revenue Service (IRS) can come together to provide fine-grained access to encrypted data, while shielding from adversarial corruption of one or more such entities. Decentralization to multiple key authorities diversifies the trusted computing base and reduces the risk due to corruption of, or malicious behavior by, one or more key authorities.

Over the last two decades, a lot of research has focused on designing expressive multi-authority encryption systems given their vast applications across a huge swathe of cryptography. Despite great interest and significant effort, we have just a handful multi-authority encryption systems. These can be broadly categorized as follows:

- **Sub-exponentially-hard obfuscation.** Using the heavy hammer of sub-exponentially-hard indistinguishability obfuscation [BGI⁺12, GGH⁺16] and one-way functions, we have multi-authority functional encryption (MAFE) for all polynomial-sized circuits [BCG⁺17b].
- **All-or-nothing-encryption for monotone NC¹ and weaker predicates.** The second encompasses a long line of research [Cha07, CC09, LW11, MJ18, Kim19, WFL19, OT20, DKW21, AG21, AGT21, DKW23, DP23] giving various multi-authority attribute-based encryption (MAABE) schemes and inner-product extensions under a variety of standard assumptions. The state-of-the-art is MAABE for monotone boolean formulas based on pairings, and subset policies/bounded-size DNF formulas based on lattices.
- **Honest authorities and bounded collusion.** The third line of work designs MAFE/ABE [GGL24, GY24] for all polynomial-sized circuits, but with significantly weaker guarantees. The state-of-the-art is MAFE for general circuits from sub-exponentially-hard public-key encryption (PKE) in the random oracle model (ROM). However, its security only holds as long as *all* local key authorities are ‘honest’ and only an a-priori bounded number of keys get corrupted. *Requiring every local authority to be honest goes against the original motivation.*

This has left significant gaps in our current understanding of multi-authority encryption systems. In particular, outside of sub-exponentially-hard obfuscation, we do not have any multi-authority encryption system, beyond all-or-nothing encryption for NC¹ predicates, that is secure in presence of malicious authorities. This prompted the following question central to this work.

(Q1) *Can we design MAFE for circuits secure against malicious authorities without obfuscation?*

The above question is completely open in, both, the collusion-resistant and bounded-collusion model. Unfortunately, the state of affairs is not much better even when we additionally restrict all local authorities to be always honest. Consider the following natural question—

(Q2) *If all local authorities are honest, can we design MAFE from minimal assumptions?*

Unfortunately, irrespective of the collusion model (collusion-resistant or bounded-collusion), the above question is still unresolved. This is very much unlike the traditional single-authority model

where analogous questions have long been resolved [SS10, GVW12, AJ15, BV15, AV19]. The fact that above questions are still open highlight the difficulty in designing MAFE, irrespective of whether the local authorities are honest or malicious.

Furthermore, even if we restrict to the attribute-based encryption setting (i.e., all-or-nothing encryption), we do not have any MAABE construction that supports general predicates without sub-exponential obfuscation. This brings us to the final question that we study:

(Q3) *Can we design collusion-resistant MAABE for circuits without obfuscation?*

This work. We answer these three questions, thereby addressing the fundamental gaps in our current understanding of multi-authority encryption.

We answer Q1 and design MAFE for circuits, where an adversary can corrupt any k out of n local authorities as long as $\binom{n}{k} = \text{poly}(\lambda)$. We give two constructions — where our first construction is a bounded-collusion MAFE scheme under the hardness of either SXDH or LWE; and our second construction is a fully-collusion-resistant MAFE under the hardness of poly-hard (single-authority) functional encryption and either SXDH or LWE. Additionally, we show that any MAFE scheme, where an adversary can corrupt more than $O(\log \lambda)$ authority bits, implies obfuscation.¹ Therefore, our solution to Q1 optimally solves the problem of malicious authorities for a wide range of parameters. Later, we discuss some questions along Q1 that we leave as interesting open problems.

Next, we completely resolve Q2 and design MAFE for circuits from minimal assumptions in, both, the collusion-resistant and bounded-collusion models as long as the authorities do not act maliciously.

Finally, we design a fully-collusion-resistant MAABE scheme for all polynomial-sized predicates under the hardness of witness encryption [GGSW13]. Given witness encryption is known to be a fundamentally weaker primitive than obfuscation, with multiple ‘obfuscation-free’ candidates [BIOW20, Tsa22, VWW22], our work addresses Q3 and reduces the strength of the assumptions sufficient for MAABE. Our MAABE scheme does not rely on random oracles and we also prove it to be adaptively secure without complexity leveraging. The only prior MAABE construction that does not rely on ROM is by Waters et al. [WWW22], and supports subset predicates under a non-falsifiable evasive lattice assumption. We summarize our results in Tables 1 and 2.

Bounded Collusion MAFE			Collusion Resistant MAFE		
Reference	Assumption	Authorities	Reference	Assumption	Authorities
[GY24]	Sub-exp PKE + ROM	Honest	[BCG ⁺ 17b]	Sub-exp iO + OWF	Malicious
Thm 3.6	PKE	Honest	Thm 5.10	FE	Honest
Thm 4.4	LWE/SXDH	Malicious	Thm 6.3	FE + LWE/SXDH	Malicious

Table 1: Comparison of our results with prior works on MAFE for P/Poly circuits with adaptive security in bounded collusion (left) and collusion resistant (right) settings.

¹As we explain later, the phrase “authority bits” refers to as the total number of attribute bits controlled by a specific group of authorities.

Collusion Resistant MAABE			
Reference	Assumption	Policy Class	Security
[LW11] ... [DKW23]	Pairings + ROM	NC ¹	Adaptive
[DKW21]	LWE + ROM	DNF	Very Selective
[WWW22]	Evasive LWE	Subset Policies	Very Selective
Thm 7.5	WE	P/Poly	Adaptive

Table 2: Comparison of our results with prior works on MA-ABE. All of these results support malicious authorities.

1.1 Applications

Beyond the traditional applications as discussed in many prior works [Cha07, CC09, LW11, BCG⁺17b], our multi-authority encryption schemes could be very useful in a variety of new contexts such as decentralized tracing and threshold cryptography.

I: Multi-authority traitor tracing. In a traitor tracing (TT) system [CFN94], a central authority samples a master public key mpk , master secret key msk , and recipient secret keys $\{\text{sk}_i\}_i$, such that a ciphertext ct created using mpk can be decrypted by any recipient key sk_i . A special feature of TT is that if any subset S of recipients collude to create a useful pirate decoding device D , then the central authority can identify at least one user in S as a ‘traitor’ without accusing honest recipients. Unfortunately, the assumption of a central trusted party in TT is too strong, an adversary that can compromise the trusted authority can decrypt any ciphertext, while also can never be traced since tracing can only identify corrupt recipient keys. In light of these limitations, recent works [Luo24, BLM⁺24] propose an ad-hoc/registration-based model. While these do distribute the work of a trusted authority, they put too much overhead on each recipient and require strong synchronization between recipients. For instance, any new user that wants to join the system would trigger a full system update.

We propose a new decentralized TT system called *multi-authority traitor tracing* (MATT). Our goal is to replace the single trusted authority with a group of a-synchronized, non-interacting, and mistrusting local authorities. As in standard TT, each authority samples a secret key for each recipient, and a recipient combines these partial secret keys to decrypt a ciphertext. MATT provides sufficient decentralization, wherein the system’s semantic security as well as traceability guarantee hold even if some of the local authorities are malicious. Moreover, it does not suffer from the registration and high setup overhead as do the ad-hoc/registration-based TT [Luo24, BLM⁺24]. Overall, it weakens the trust assumption in TT with asynchronous setup and minimal overhead on each recipient.

Our observation is that the seminal private linear broadcast encryption (PLBE) paradigm introduced by Boneh-Sahai-Waters [BSW06] can be naturally generalized to the multi-authority model as an important building block for MATT. (We refer to [GKW18] for more details.) Now, to design a multi-authority version of PLBE, we can generalize the Goyal-Koppula-Waters [GKW18] approach that combined ABE and a dual-mode FE scheme (called mixed FE) for designing optimally-efficient PLBE. In particular, by combining our bounded-collusion MAFE construction with lockable obfuscation [GKW17, WZ17], as in [CVW⁺18], we can design a multi-authority version of mixed FE, and by further combining it with our fully-collusion-resistant MAABE construction, we can design MATT as proposed above. Since our focus is on designing new MAFE and MAABE systems, we highlight MATT as an interesting application, and leave further exploration of MATT as an

interesting research direction.

II: Beyond threshold policies in distributed encryption with silent setup. Threshold encryption [Des88, Fra90, DF90] is a fundamental cryptographic primitive in threshold cryptography. Typically, there is an interactive setup phase in threshold encryption to distribute the secret key amongst n parties. An encryptor can select a threshold t (between 1 and n) during encryption such that the resulting ciphertext ct can be decrypted by any subset of t (or more) parties that possess a distributed key share. In a recent work, Garg et al. [GKPW24] posed the question – is threshold encryption without an interactive setup possible? This is regarded as the silent setup setting. [GKPW24] answered it positively and designed a compact threshold encryption system. This was recently generalized by Devadas et al. [DJWW25] to distributed monotone-policy encryption.

Our multi-authority encryption systems can be used to design a distributed encryption system for arbitrary monotone policies generically. Essentially, each party can non-interactively and asynchronously sample their individual keys using our MAFE/MAABE system and publish those on a public bulletin board. Therefore, no interaction between parties is required during setup as authorities in multi-authority encryption already work asynchronously. Further, an encryptor can dynamically decide the group of parties as well as a threshold t (or any other more general access structure ϕ), and encrypt the message μ under the corresponding public keys. Later, any set of authorized parties (as per ϕ) can decrypt the resulting ciphertext. Moreover, an interesting feature of designing threshold encryption using MAFE/MAABE is that these provide a natural notion of key delegation too. That is, we could have clients who are not interested in establishing or managing their secret keys, and want to otherwise be a part of the distributed encryption system. Such clients can simply request/purchase a delegated key, and any party (who did setup) can oblige its request and give it a predicate key for an appropriate GID. While a generic approach might not immediately yield strong succinctness guarantees (w.r.t. n , number of parties involved in setup), it does open a pathway to general monotone policies beyond threshold access structures. We expect future works to draw on this connection, and use multi-authority encryption to design more expressive distributed systems.

III: Beyond threshold policies in distributed signatures. Similar to threshold encryption, the concept of decentralization has also been well studied in signatures [Des88, Fra90, DF90]. The goal is again to distribute the secret signing key to a group of multiple users, but typically with the goal that a certain threshold of distributed signers can get together to produce a signature σ on any message μ . In certain applications, it is also desirable to hide the group of signers who participate in the signature generation process. If we do not desired strong anonymity properties, then MAFE/MAABE encryption systems can be readily used to design distributed signatures for far more general policies beyond simple threshold policies. An added benefit of our design template is that it satisfies silent setup as well as each signer can generate their partial signatures completely asynchronously.

Our idea is to generalize the folklore Naor’s transformation (sketched in [BF01]) to the multi-authority setting. Each signing party simply samples MAABE master public-secret keys. In order to sign a message μ , the signer generates a secret key for $GID := \mu$ and attribute $x = 1$. Verification key is the set of all master public keys collected from these authorities. Using this verification key, depending on the threshold τ , we consider a predicate that checks if the number of 1s in the input is greater than τ . Using this ciphertext and a random message, we can verify if more than

τ many parties signed the same message μ . Security of this scheme can be reduced to MAABE security similar to Naor’s transformation². Indeed we get an additional benefit that in this scheme, each signing party can sign without requiring the signing keys of other parties. Moreover, we can also realize a stronger verification property which we call “predicated verification”. Essentially in this scheme, each signing party can additionally entertain an attribute x along with message m . Once all signatures are received for a message m , verification only succeeds if $P(x_1, \dots, x_n)$ for some arbitrary policy P specified by the system. For instance, in a private blockchain network, a transaction can only be approved by authorized members of the network. In such a scenario, predicated verification can be useful where an arbitrary policy can be specified by the network users. Our distributed threshold signature scheme as described above readily provides such a signature scheme.

Related work. Over the last three decades, the problem of reducing the trust in the central master key authority and solving key escrow in FE has received a lot of attention. Numerous approaches have been studied to weaken the trust model in FE and its various specializations [Sha85, Coc01, BF01, GPSW06, BSW11]. The most well-studied models include the multi-authority (studied in this work) [Cha07, CC09, LW11, BCG⁺17b], accountable-authority [Goy07, GLSW08], certificate-less encryption [ARP03, CCV04], anonymous encryption [BDOP04, Cho09, WQT18], and registration-based model [GHMR18, GHM⁺19, GV20]. Each model has its own benefits and limitations, and comparing different models is only feasible by comparing the advantages they provide in different application contexts.

In a concurrent and independent work, Cini-Lai-Woo [CLW24] constructed an MA-ABE scheme for circuits secure assuming under evasive LWE and tensor LWE in ROM. Their scheme only achieves very selective security similar to [WWW22].

2 Technical Overview

Reviewing MAFE. An MAFE scheme contains four main algorithms (ASetup, KGen, Enc, Dec). The authority setup algorithm ASetup is used by each authority to sample master public-secret key pair MPK, MSK on their own (i.e., without any interaction or synchronization). The key generation algorithm KGen algorithm is used by each authority to compute a partial secret key for a portion of the input x_i using their MSK_i . Enc and Dec are defined similar to FE, where the master public key consists of $\{MPK_i\}_i$ from all authorities, and the secret key for input $X = (x_i)_i$ is $\{SK_{i,x_i}\}_i$. To avoid mix-and-match attacks (by combining keys for two users from two different authorities), every partial secret key is embedded with a global user identifier GID. We refer the reader to prior works [LW11] to get a better understanding of GID. For this high level overview, we mostly ignore GIDs while explaining our main ideas.

In MAFE, there are two types of corruptions – (a) secret key corruptions, (b) authority corruptions. A type-(a) attacker can only query secret keys from each authority, but not obtain their master keys. Type-(a+b) is a stronger attacker, who can additionally corrupt any MSK_i of its choice. In this work, we study, both, the bounded collusion and general collusion-resistant models. In the bounded collusion setting, an attacker makes an a-priori fixed (say Q) number of key queries and security holds against a simulator Sim. In the collusion resistant setting, attacker makes an unbounded number of key queries and security is defined as an indistinguishability game.

²Security argument requires artificial abort techniques similar to [FGH⁺17]

2.1 MAFE with Malicious Authorities

As a starting point, let us consider an easier goal of designing MAFE with malicious authorities, where all malicious authorities are declared a-priori. That is, the MAFE system is initialized given a set of malicious authorities $\mathcal{K} \subset [n]$. While such “declared” malicious authorities do not capture any real-world attacks, we will show later how this can be used as stepping stone for handling arbitrary malicious authorities.

Consider $\mathcal{K} = \{1\}$, i.e. only first authority is malicious. Such a system will allow any attacker to learn $C(0, x_2, \dots, x_n)$ and $C(1, x_2, \dots, x_n)$ from a ciphertext CT and secret keys $\text{sk}_{2,x_2}, \dots, \text{sk}_{n,x_n}$ by honestly running KGen and Dec. Our intuition is to start with an MAFE that is secure against honest authorities, and once $\mathcal{K} = \{1\}$ is available during setup, then simply initialize two parallel (honest-authority) MAFE systems, $\text{MAFE}^{(0)}$, $\text{MAFE}^{(1)}$ that encrypt $C(0, \cdot)$ and $C(1, \cdot)$. And, each honest authority will generate secret keys for both instantiations. An attacker in possession of $\text{sk}_{2,x_2} = (\text{sk}_{2,x_2}^{(0)}, \text{sk}_{2,x_2}^{(1)})$, $\dots$ can learn $C(0, x_2, \dots, x_n)$ using $\text{MAFE}^{(0)}$ and $C(1, x_2, \dots, x_n)$ using $\text{MAFE}^{(1)}$ as desired. The above can be easily generalized for any set $\mathcal{K} \subset [n]$ as long as the total number of attribute bits that are associated with $\mathcal{K}$ are at most $O(\log \lambda)$.

One might wonder whether we could use complexity leveraging to guess $\mathcal{K}$, and then prove security of above approach under sub-exponential hardness assumptions. This would be wrong. Here we require $\mathcal{K}$ to be available even for setting up the MAFE system. Thus, description of the scheme reveals $\mathcal{K}$. This highlights why the assuming which authorities are malicious at setup time is meaningless. Regardless, it does serve as a good springboard as we show next.

Hiding $\mathcal{K}$ via HSS. Our idea is that to consider all feasible $\mathcal{K}$ and secret share the computation under them, rather than guessing $\mathcal{K}$. Let us elaborate.

Suppose k denotes the maximum number of authorities that can be malicious. Let $\mathcal{C}$ denote the set of all size- k subsets of $[n]$. Thus, $\mathcal{C}$ contains all possibilities for authorities who might be corrupt. Clearly, $|\mathcal{C}| = \binom{n}{k}$. We will sample $|\mathcal{C}|$ many MAFE systems with *declared malicious authorities*, $\text{MAFE}^{(0)}, \dots, \text{MAFE}^{(|\mathcal{C}|)}$. In $\text{MAFE}^{(i)}$, the malicious authority set $\mathcal{K}_i$ is the i -th entry in $\mathcal{C}$. During authority setup, the u -th authority samples fresh master public-secret key pairs from $\text{MAFE}^{(i)}$ if and only if $u \notin \mathcal{K}_i$. We will set $\text{MPK}_u = (\text{mpk}^{(i)})_{\{i: u \notin \mathcal{K}_i\}}$ and $\text{MSK}_u = (\text{msk}^{(i)})_{\{i: u \notin \mathcal{K}_i\}}$. Similarly, to generate a partial secret key, we will sample a partial secret key from each $\text{MAFE}^{(i)}$ such that $u \notin \mathcal{K}_i$ and $\text{SK}_{u,x_u} = (\text{sk}_{u,x_u}^{(i)})_{\{i: u \notin \mathcal{K}_i\}}$.

Now our idea is to rely on homomorphic secret sharing (HSS) [BGI16, BCG⁺17a, FGJS17] to perform encryption. In particular, we use an HSS to secret share C into $|\mathcal{C}|$ share and encrypt the corresponding share in each $\text{MAFE}^{(i)}$ and $\text{CT} = (\text{ct}^{(i)})_i$. To ensure decryption works correctly, we have to make mild adjustments, but the high level idea is simply that a decryptor will recover MAFE outputs for every $\mathcal{K} \in \mathcal{C}$, and later it combine them by using reconstruction feature of HSS.

To prove security, we need a strong simulation guarantee for HSS. In a few words, we require that an attacker can learn evaluated shares corresponding to various honest parties in addition to corrupting some original share holders. And, even then it still does not learn anything beyond the final reconstructed value. An HSS with strong simulation readily follows from multi-party reusable non-interactive MPC (mrNISC) [BL20]. Previously, this was studied under the name ‘Secret Sharing VBB’ and was built using any mrNISC scheme secure against semi-honest adversaries. We leave it as a very interesting open problem to construct HSS schemes with strong simulation from other standard assumptions (other than ones that imply mrNISC like SXDH or LWE) as it would readily give an MAFE scheme with malicious authorities using our approach.

Lastly, note that for efficiency we require $\binom{n}{k} = \text{poly}(\lambda)$, as otherwise the running time will be super-polynomial. For instance, if $n = \text{poly}(\lambda), k = O(1)$, $n = \text{poly}(\log \lambda), k = O(\log^{1-\xi} \lambda)$ for $\xi > 0$, or $n = O(\log \lambda), k = O(\log \lambda)$, then above construction could be efficiently instantiated. These parameter regimes are summarized in Figure 1.

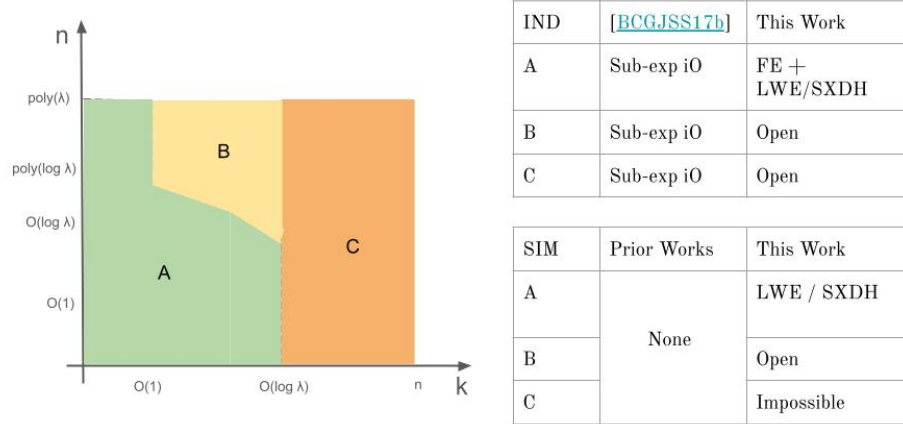


Figure 1: Parameter regimes for number of malicious authorities in our MAFE schemes. n is the number of authorities and k is the maximum number of malicious authorities. Although we do not have any results for the parameter regime in region B, it doesn't appear to require obfuscation. We leave it as an interesting open problem to design MAFE for this parameter range. Moreover, it would be interesting to design MAFE for region C from minimal assumptions in the collusion resistant setting.

Security of bounded collusion MAFE. Our design for MAFE with malicious authorities works almost generically in the bounded collusion model. We can readily take advantage of the simulation security provided by MAFE and only output evaluated shares of HSS to argue security. In a bit more detail, we guess an $i^* \in \mathcal{C}$ and assume that the attacker will corrupt authorities in $\mathcal{K}_{i^*}$. Note that we will only incur $1/|\mathcal{C}| = 1/\text{poly}(\lambda)$ loss in advantage. Then we will simulate $\text{MAFE}^{(i^*)}$ -th instantiation to argue security using strong simulation property of HSS. For more details, we refer the reader to Section 4.

Security of collusion resistant MAFE. In the bounded collusion setting, we can take advantage of simulation security and only rely on HSS evaluation outputs to argue security. However, in the collusion resistant setting, we don't have that facility due to known impossibilities [BSW11, AGVW13]. However, if we know the number of pre-challenge queries, say Q_1 , we can create Q_1 different slots in the ciphertext for evaluated HSS shares. In a bit more detail, for each GID query attacker makes before challenge ciphertext query, i -th authority for input x_i , additionally samples a random ρ_i . During encryption, we will add a trapdoor to the circuit being encrypted which if it detects the input as $(x_1, \rho_1, \dots, x_n, \rho_n)$, outputs the evaluated HSS share of $C(x_1, \dots, x_n)$. By encryption phase for challenge circuit, we know both C and all pre-challenge inputs $(x_1, \dots, x_n)$. So, this can be done by relying on indistinguishability of MAFE. Post-challenge queries can be simulated using SKE with pseudorandom ciphertexts. Thus, we can construct an MAFE scheme with unbounded post-challenge queries and a bounded (Q_1) pre-challenge queries using MAFE and

HSS. We leave it as an interesting open problem to remove the pre-challenge bound in MAFE with malicious authorities in the collusion resistant setting.

Lastly, we complement our positive results regarding authority corruptions with almost tight lower bounds. We discuss those briefly below.

Beyond $O(\log \lambda)$ -bit corruptions. In our design for MAFE with malicious authorities using HSS, we instantiated 2 MAFE schemes per one corrupted bit. Extending this generically, we would need 2^χ parallel compositions if an attacker can corrupt a maximum of χ bits. So, we ask the natural question — *can we achieve any slightly better efficiency?* In particular, can the parameters of our system grow sub-linearly in 2^χ , i.e, $2^{\chi(1-\xi)}$ where $\xi > 0$? We show that an MAFE scheme with such a guarantee can be transformed into an obfuscation scheme.

To provide an intuition for this result, recall [BV15, AJ15]. These results state that in a 1-authority FE scheme (1-FE) secure against Q secret-key corruptions, $|1\text{-FE.ct}| \geq Q \cdot \text{poly}(\lambda)$. Any sub-linearity in size i.e, $Q^{1-\xi}, \xi > 0$ implies iO (as long as 1-FE offers indistinguishability-based security). In Appendix O we show how to construct 1-FE with sub-linearity from an MAFE scheme with ‘selective’ corruptions³. At a high level, $1\text{-FE.Enc}(x) \equiv \text{MAFE.Enc}(F(x, \cdot, \cdot))$ where F is a circuit that uses j, C and outputs j -th bit of $C(x)$. We will corrupt MSK for slot j and set $1\text{-FE.sk}_C := (\text{MSK}, \text{mafe.sk}_C)$. By sub-linearity of MAFE, 1-FE.ct only grows in $Q^{1-\xi}$. In addition, when $Q = 1$, this will be a worst-case iO scheme (XiO) with sub-linear size, an object that in conjunction with LWE yields iO [LPST16]. Moreover, simulation-secure versions of above are impossible in the standard model [BSW11, AGVW13]. Thus, our MAFE schemes with malicious authorities are nearly optimal.

2.2 MAFE with Honest Authorities from Minimal Assumptions

Next, to instantiate our malicious authority MAFE schemes as discussed above, we need to design MAFE with honest authorities. As discussed in Section 1, even if all authorities behave honestly (but asynchronously), it is not known how to design MAFE from minimal assumptions. In this section, we address this problem in both, bounded collusion setting and collusion resistant setting.

2.2.1 Bounded Collusion MAFE from PKE

Our plan is to break down designing an n -authority MAFE scheme to composing n *single-authority* FE systems with certain special properties. At a high level, we introduce a new notion of MAFE system with *very weak security* below and show that it is a very important tool for designing MAFE.

MAFE with trusted setup. The new notion of MAFE we use is *MAFE with trusted setup* (tMAFE). In tMAFE, there is a *central trusted authority* that generates master public-secret keys for all authorities and distributes it to every authority. Thus, individual authorities do **not** sample any master public-secret keys. Essentially, instead of the **ASetup** algorithm, tMAFE has a global setup **GSetup** algorithm to generate $\{(\text{MPK}_i, \text{MSK}_i)\}_i$ for all authorities. Other than this, tMAFE is defined as its standard (decentralized) counterpart. That is, each authority generates partial secret keys in a decentralized and asynchronous manner as before. For security, we consider a nearly identical game, except the challenger generates all authority keys by running **GSetup** instead of **ASetup** n times. Next, we show how to generically bootstrap tMAFE to a standard (non-trusted) MAFE scheme.

³Attacker declares malicious authorities selectively before viewing public parameters. This is clearly stronger than declaring malicious authorities for a system.

tMAFE $\implies$ MAFE. Note that tMAFE already captures the desired functionality that we want from general MAFE, except the authorities need to get together to sample their individual master public-secret keys. Our idea is to delegate the computation of tMAFE’s global setup to encryption, by relying on another FE layer to handle the trusted setup problem. Suppose we have a single-authority FE scheme, let us call it 1-FE. Our plan is to use 1-FE to generate n independent 1-FE systems, one for each individual authority, and use these independent systems to unlock the tMAFE secret keys. Let us explain further.

In our MAFE systems, we ask the encryptor to run the global setup, GSetup , for tMAFE. This gives the encryptor a set of n master public-secret key pairs for tMAFE. Clearly, it can encrypt the predicate circuit (C) using tMAFE encryption. But, the question is *how to share the tMAFE authority keys with the “real” authorities*, or rather how to ensure an authorized user gets the right tMAFE key without talking to the encryptor? Our insight is that this can be achieved by delegating the tMAFE key generation. That is, we encrypt tMAFE.KGen circuit, with i -th tMAFE master key hardwired, under the i -th 1-FE system. Concretely, each authority samples 1-FE keys $(1\text{-fe.mpk}_i, 1\text{-fe.msk}_i)$, and uses 1-fe.msk_i to answer secret key queries as $\text{SK}_{i,x_i} \leftarrow 1\text{-FE.KGen}(1\text{-fe.msk}_i, x_i)$. During encryption, we run a fresh GSetup for tMAFE to generate n authority key pairs, $\{(\text{MPK}_i, \text{MSK}_i)\}_i$. Given $\{\text{MPK}_i\}_i$, encrypt C as $\text{tMAFE.Enc}(C)$. Moreover, delegate tMAFE key generation by creating n 1-FE ciphertexts as $1\text{-FE.Enc}(1\text{-fe.mpk}_i, F_i)$, where $F_i = \text{tMAFE.KGen}(\text{MSK}_i, \cdot)$. The resulting 1-FE ciphertexts $\{1\text{-fe.ct}_i\}_i$ and tMAFE ciphertext tMAFE.ct are jointly set as the full MAFE ciphertext. Clearly, decryption is done in two stages– (i) decrypt $\{1\text{-fe.ct}_i\}_i$ using $\{\text{SK}_{i,x_i}\}_i$ (resp.) to compute $\text{tMAFE.SK}_{i,x_i}$, and (ii) then decrypt tMAFE.ct using these to recover $C(x_1, \dots, x_n)$.

The security of the above template can be quite readily reduced to the n 1-FE instantiations and then tMAFE security. Our compiler is described in detail later in Section 3. We highlight that we do not need any synchronization among the authorities, they are even oblivious to the existence of each other. Next, we talk about constructing tMAFE.

Constructing tMAFE. Since 1-FE is already known from minimal assumptions [GVW12, AV19], thus we only need tMAFE under same set of assumptions to instantiate our compiler.

A natural question the reader might have is: *isn’t tMAFE already pretty close to a single-authority FE (1-FE) system?* Basically, since there is a trusted setup, then what prevents us from sampling just a 1-FE system during the global setup, GSetup , and later thresholdizing by relying on some threshold homomorphic encryption techniques [BGG⁺18]. While this will not give us the desired result from minimal assumptions, it does suggest that designing tMAFE might not be so hard after all. Although this is a good first idea, it does not work and highlights a key challenge towards MAFE.

In (t)MAFE, each authority only generates a partial key for *its portion of the attribute* without knowing any other authority’s inputs. That is, authority i only gets to see x_i , and not the full input $(x_1, \dots)$. Therefore, any approach to just use threshold FHE (or even 2-round MPC-techniques) fails. Our strategy is to start with prior 1-FE/MAFE constructions [SS10, GVW12, AV19, GY24], and generalize them to this new “trusted setup model”. In particular, we open up the recent (honest-authority) MAFE construction by Goyal-Yadugiri [GY24]. Their MAFE scheme could decentralize a 1-FE scheme [GVW12, AV19] as long as all authorities can make certain (randomized) choices to answer a partial key query for a GID *consistently across non-interacting authorities*. Essentially, they wanted each authority to sample the same random number for each GID non-interactively. To solve this with zero interaction, [GY24] took convoluted approaches involving non-interactive key

exchange or the ROM heuristic with sub-exponentially-hard assumptions.

Our insight is that the core technical hurdle (in the form of non-interactive random number sampling) faced by [GY24] can be rather easily solved in the *trusted* setting. Our idea is to have the trusted party sample a random PRF key K and hand the same key to each authority. Using the *same* key K , each authority samples a consistent random number per GID. While there are some additional technical nuances in executing the above idea, we show that tMAFE can indeed be designed from the minimal assumption of (poly-hard) PKE. We provide more details in Appendix C.

Interestingly, we show that our MAFE can be improved to resist attackers in the dynamic collusion model [AMVY21, GGLW22] without any additional effort. This demonstrates that (despite appearing meaningless at first) tMAFE is indeed the right abstraction for studying and designing MAFE systems. We discuss this further later in Appendices E and H.

2.2.2 Collusion Resistant MAFE from Functional Encryption

At a very high level, our core idea for designing collusion-resistant MAFE is the same as for our bounded-collusion construction. That is, to design an n -authority MAFE, we compose n distinct 1-authority functional encryption (FE) schemes⁴. However, we take a slightly different approach to combine them.

From FE to MAFE. In our scheme, each authority samples an FE public and secret key $(\text{fe.mpk}_i, \text{fe.msk}_i)$ and uses them as its own master public and secret keys. In order to encrypt a circuit C , the encryptor samples a PRF key K and encrypts the tuple (C, K) under each fe.mpk_i to generate fe.ct_i . Final ciphertext is a combination of all $(\text{fe.ct}_i)_i$. To generate a partial secret key for (GID, x_i) , the i -th authority samples a uniform random string ψ_i and generates FE secret key fe.sk_i for the following trapdoor circuit⁵:

- $\underline{F}$: If input is (C, K) , garble the circuit C using $\text{PRF}(K, \text{GID})$ as randomness and output the garbled circuit $\tilde{C}$ and i -th wire label for bit x_i ⁶, w_{i,x_i} . Otherwise, if input is ske.sk^* , decrypt ψ_i with ske.sk^* and output the message.

If all fe.sk_i are generated using same GID correctness will be immediate. Specifically, decryption proceeds in two steps (i) using fe.sk_i and fe.ct_i , we can obtain a $\tilde{C}, w_{i,x_i}$, (ii) using $(w_{i,x_i})_i$, we can evaluate $\tilde{C}$ to obtain $C(x_1, \dots, x_n)$ as randomness used for garbling is the same across all authorities. Now, we will argue security of our scheme by relying on security of FE.

Arguing selective security. We now show how our design achieves selective security. Recall that in the selective setting, the challenge circuits C_0, C_1 are chosen by the adversary before seeing $\{\text{MPK}_i\}_i$. Security follows in 4 steps. In step 1, we use a secret-key encryption scheme with pseudorandom ciphertexts (SKE) to replace each ψ_i sampled by each authority with an SKE ciphertext of $(\tilde{C}_0, w_{i,x_i})$ under some key ske.sk^* . This is done for every partial secret key query across all authorities. In step 2, we invoke the trapdoor circuit method [DIJ+13, GHRW14, BS18, ABSV15] to encrypt ske.sk^* under each fe.mpk_i , where indistinguishability holds since the outputs match for both inputs. Step 3 relies on PRF security to garble the circuit with uniform randomness, and in step 4 we simulate the garbling.

⁴We remark that FE is a weaker assumption than indistinguishability obfuscation (iO) as it does not require sub-exponential-hard assumptions.

⁵Counterintuitively, we use key-policy collusion resistant FE throughout this overview

⁶Throughout this overview, for ease of exposition, we assume that input to each authority is 1 bit. This can be generalized easily as we demonstrate in Section 5.3.

At first sight, step 4 seems to yield selective security since it depends only on $C_0(x_1, \dots, x_n)$, ensuring indistinguishability between C_0 and C_1 . However, this is not the case. Simulating the garbling requires entire input $(x_1, \dots, x_n)$, whereas each authority only observes its own input x_i . To resolve this mismatch, we argue security in a weaker model that we call *batched key generation*, where authorities generate keys jointly for a batch of inputs. This notion captures the security our construction achieves, and we expand on it below.

Interlude: Batched key generation. In an multi-authority system with batched key generation, the adversary submits inputs to all authorities as a batch, i.e. $(\text{GID}, x_1, \dots, x_n)$ and receives secret keys as a batch, i.e. $(\text{SK}_{1,x_1}, \dots, \text{SK}_{n,x_n})$. We stress that authorities can generate their own keys independently of others and can generate partial secret keys without interacting one another. However, they have to *synchronously*, in any order, generate partial secret keys for inputs corresponding to any GID. Although this hinders the asynchronous property of a multi-authority system, as we show later, any MAFE or MA-ABE scheme secure in the batched key generation setting can be compiled into a scheme with asynchronous partial key generation easily. We defer the details to the end of the section and for now proceed with strengthening selective to adaptive security.

Achieving adaptive security. In order to achieve adaptive security, we rely on the hybrid functional encryption idea from [ABSV15]. More specifically, we utilize a one-ciphertext *private-key* MAFE scheme (1-CT MAFE). A 1-CT MAFE scheme is defined similarly to a private-key MAFE scheme where encryption uses $\{\text{MSK}_i\}_i$ to generate ciphertext for circuit C . Security holds only against attackers that corrupt partial secret keys for any unbounded GID and at most *one* challenge ciphertext. Using this object in conjunction with widely-used techniques from FE domain such as trapdoor circuits, SKE with pseudorandom ciphertexts, etc., we can readily achieve an MAFE scheme with adaptive security in the batched key generation setting. For more details, we refer the reader to Section 5.4. We will now illustrate how to construct a 1-CT MAFE scheme.

Constructing 1-CT MAFE. At first glance, since we know how to construct MAFE schemes from minimal assumptions (Section 2.2.1) or from the public-key one-key MAFE scheme of [GY24], it may seem straightforward to derive 1-CT MAFE from one-way functions. However, this is not the case. In bounded-collusion MAFE, security holds only against an attacker that corrupts a bounded number of partial secret keys. In contrast, 1-CT MAFE enforces a bound on challenge encryptions, while allowing the adversary to corrupt *unbounded* partial secret keys. Hence, existing constructions do not yield a 1-CT MAFE scheme.

To overcome this, we start instead from *multi-client* functional encryption (MCFE) [GGG⁺14, CDSG⁺18]. Unlike MAFE, MCFE distributes encryption among clients, where each client uses its own encryption key ek_i , while key generation for a circuit C is performed using a single msk . If all clients encrypt inputs x_i under the same GID⁷, the ciphertexts can be combined to learn $C(x_1, \dots, x_n)$. Thus, if we have a MCFE secure against the corruption of a single circuit key, we can apply function-hiding techniques from [BS18] and swap the roles of encryption and key generation to obtain a 1-CT MAFE scheme. While general-circuit MCFE for $n = \text{poly}(\lambda)$ clients is only known from obfuscation [GGG⁺14], our setting requires 1-key MCFE in the weaker batched ciphertext generation model (analogous to batched key generation model), which is easier to construct. In particular, we can adapt ideas from our selectively secure MAFE design by composing n private-key 1-key FE schemes [SS10, GVW12], embedding the same PRF key in each ek_i . This yields a 1-CT MAFE construction from the minimal assumption of one-way functions. For further details,

⁷In the MCFE literature, these are called ‘labels’ or ‘tags’, but we use GIDs for notational uniformity.

check Appendix J. We leave constructing general MCFE from minimal assumptions as an interesting open problem. We will now proceed to deal with batched key generation restriction of our MAFE designs.

Removing batched key generation. As mentioned, batched key generation hinders the asynchronous nature of multi-authority systems. One idea to remove this is to use a MPC scheme between all authorities for identity function, i.e., $f(x_1, \dots, x_n) = (x_1, \dots, x_n)$. However, as multi-authority systems require no interaction, this method might not exactly work. In our work, we provide a more general solution using non-committing encryption. Recall that in a non-committing encryption scheme (NCE), is a public key encryption scheme with the additional property that we can “fake” a ciphertext and later reveal it to be encryption of message m by constructing a secret key for m .

For now, let’s start with a simpler goal with $n = 2$ and delaying generation of SK_{1,x_1} until second authority is queried. In order to do so, each authority samples two NCE instantiations ($\text{NCE}_1, \text{NCE}_2$). After generating SK_{1,x_1} using the multi-authority system, the first authority samples R_1, R_2 and sets its output as $((\text{NCE}_1.\text{Enc}(R_1), \text{NCE}_2.\text{Enc}(R_2)), \text{NCE}_1.\text{SK}, R_1 \oplus R_2 \oplus \text{SK}_{1,x_1})$. Only after second authority provides its own $\text{NCE}_2.\text{SK}$, SK_{1,x_1} is revealed. In fact assuming that the second authority queried later, first authority can set its output as $((\text{NCE}_1.\text{Enc}(R_1), \text{NCE}_2.\text{Fake}), \text{NCE}_1.\text{SK}, \tilde{R})$ for uniformly random $\tilde{R}$. When the second authority’s input is revealed, it will set $\text{NCE}_n.\text{SK}$ as a secret key for message $\tilde{R} \oplus R_1 \oplus \text{SK}_{1,x_1}$. This way, we can delay generation of SK_{1,x_1} until we receive second input. In order to do this for n authorities, each authority will need n instantiations of NCE. To handle unbounded GID queries, we can replace PKE-based NCE of [GVW12, HMNY22, HKM+24] with identity-based encryption (IBE). We call this scheme a vector non-committing encryption scheme (VNCE) and provide its definition and full construction in Appendix L.

Batched key generation: Alternate perspective. Another way of looking at our result is that in 1-authority FE and ABE systems, if we can generate a slight split in secret key generation such that secret key of input $x = (x_1, \dots, x_n)$ can be parsed as $\text{SK}_{1,x_1}, \dots, \text{SK}_{n,x_n}$, we can compile such a key generation to the multi-authority setting. Techniques such as these are implicit in [SS10, GVW12] which [GY24] explored to construct MAFE in the bounded collusion setting. We show that this technique is more general and can be applied in the collusion resistant MAFE and MA-ABE systems too.

2.3 Multi-Authority ABE from Witness Encryption

We begin by recalling a witness encryption scheme (WE) [GGSW13]. In a WE scheme encryption for a message m is performed w.r.t an instance ins of a language $\mathcal{L}$. Decryption on the other hand uses a witness wit and reveals the message m if and only if wit is a valid witness for ins . Security of WE states that a ciphertext generated for an $\text{ins} \notin \mathcal{L}$ will hide the m .

Now, we demonstrate how to construct an MA-ABE scheme using WE. An MA-ABE scheme is defined analogously to an MAFE scheme in the collusion resistant model except that we encrypt a polynomial-size predicate (or policy) P and a message m . First, we show a scheme secure when authorities are honest (type-(a) attackers). Our starting point is the 1-authority adaptively secure ABE scheme from [WW24].

Reviewing [WW24]. Here we provide a very high-level overview of the 1-authority ABE con-

struction of Waters and Wicks (WW) [WW24]. In their scheme, secret keys for an attribute x ⁸ are NIZK proofs for instances $(\text{com}, x, \text{tg}_x)$ in the OR language: “(1) com is a commitment to 0, (OR) (2) tg_x is a special garbling of the circuit F_x ”. Encryptions of a message m are witness encryption (WE) ciphertexts for the instance (P, tg_P) in the language that checks: “NIZK proof from secret key verifies, (AND) $P(x) = 1$, (AND) $\text{Garb.Eval}(\text{tg}_x, \text{tg}_P) = 0$ ”. Here, tg_x and tg_P are generated using a blind garbled circuit scheme [BLSV18]: tg_x is a garbled circuit of F_x that outputs 1 iff $P(x) = 1$, and tg_P consists of wire labels for P . Blind garbled circuits are a strengthening of standard garbled circuits in which the garbled circuit looks pseudorandom and reveals nothing beyond the evaluation result.

In the construction, the key-issuing authority samples tg_x as a uniformly random string and proves that com is a commitment of 0. Thus, $\text{Garb.Eval}(\text{tg}_x, \text{tg}_P) = 0$ with high probability and correctness holds. To argue security for a challenge predicate P^* and message m , one can switch tg_x from being uniformly random to being an actual garbling of F_x , thereby satisfying the second part of the OR statement. By the security of blind garbled circuits and zero-knowledge property of NIZKs, this change is not noticeable by attacker. WW refer to the case when tg_x is uniformly random as a dummy input tag and the actual garbling case as a smart input tag. Because both tg_x (in the smart case) and tg_{P^*} are generated using the same garbling scheme, together they reveal $F_x(P^*)$. Thus, when the NIZK verifies, it must be that both $\text{Garb.Eval}(\text{tg}_x, \text{tg}_P) = F_x(P^*) = 0$ and $P^*(x) = 1$, which is a contradiction. This makes the WE instance unsatisfiable, thus hiding m .

MA-ABE from WE. To construct an MA-ABE scheme from WE, we observe that the secret key generation procedure of WW’s ABE can naturally be divided among multiple authorities. Specifically, the i -th authority samples its own com_i and generates a NIZK proof for $(\text{GID}, x_i, \text{com}_i, \text{tg}_i)$ for a similar OR-language, where tg_i is a random string, and sets this as its partial secret key SK_{i,x_i} . Encryption then checks the following four conditions in the WE language: (1) each partial secret key’s NIZK proof verifies, (2) all proofs correspond to the same GID , (3) the predicate $P(x_1, \dots, x_n) = 1$, and (4) $\text{Garb.Eval}((\text{tg}_1, \dots, \text{tg}_n), \text{tg}_P) = 0$. This directly allows us to split secret key generation across multiple authorities, thereby realizing an MA-ABE scheme from WE. However, proving security requires additional care. Since the i -th authority only sees its own input x_i , it cannot directly generate the NIZK proof for the second part of the OR statement.

This difficulty can be addressed in the *batched key generation* setting. Here, the adversary submits $(\text{GID}, x_1, \dots, x_n)$ as a batch, and we generate a smart tag $\text{tg}_{(x_1, \dots, x_n)}$ as the garbling of circuit $F_{x_1, \dots, x_n}$, using it consistently across all NIZK proofs. As in Section 2.2.2, batched key generation can later be removed, so we first construct the MA-ABE scheme in this weaker model. Security in the batched setting can be argued analogously to WW’s construction, provided we can “distribute” input tag generation. To achieve this, we take a closer look at the functional tag system (FTS) of [WW24], which specifies how tg_x and tg_P are generated. We provide a short overview of FTS below.

Functional Tag System. In [WW24], smart input tag tg_x corresponds to $(\tilde{F}, a)$ where $\tilde{F}$ is garbling of circuit F using blind garbled circuit scheme. Here, $F = F_{x,a,r}$ is a circuit that on input policy P , outputs a if $P(x) = 1$ and r otherwise where a, r are uniformly random strings. Function tag tg_P corresponds to the wire labels for this garbled circuit. Dummy input tags are simply a uniformly random string of the same length as tg_x . By pseudorandomness property of blind garbled circuits, dummy and smart input tags are indistinguishable.

⁸We stick to ciphertext-policy ABE throughout this overview.

Distributed FTS. In order to use FTS in our MA-ABE scheme, we need a way to ‘distribute’ dummy tag generation $(\mathbf{tg}_{x_i})_i$ across several authorities. That is each authority uses dummy input tag generation $\text{DlnpTag}(i, x_i)$ to generate dummy tags $\mathbf{tg}_{x_i}$. Note that we do not need smart input tags to be distributed like dummy input tags. This is because in the security experiment, a single party uses $\text{SlnpTag}(x_1, \dots, x_n)$ to generate smart tags $\mathbf{tg}_{x_1}, \dots, \mathbf{tg}_{x_n}$ in one shot. In the NIZK’s OR language, we can verify that $\mathbf{tg}_{x_1}, \dots, \mathbf{tg}_{x_n}$ is the output of $\text{SlnpTag}(x_1, \dots, x_n)$ in every authority’s NIZK proof.

In order to distribute input tags, let us explain our idea to generate smart input tags. We additively secret share $\tilde{F}$ among all $\mathbf{tg}_{x_i}$. That is, we set smart tag $\mathbf{tg}_{x_i} = (\mathbf{sh}_i, a_i)$ where $\tilde{F} = \bigoplus_i \mathbf{sh}_i$ is the garbling of circuit $F_{(x_1, \dots, x_n), a, r}$. Here, r is a random string and $a = \bigoplus_i a_i$. As $\tilde{F}$ is a pseudorandom string, $\mathbf{sh}_i$ are also pseudorandom strings. So, the “distributed” dummy input tags can be uniformly random strings similar to FTS which can be generated independently by each authority. In order to evaluate, we add the shares and then perform garbled circuit evaluation.

Note that garbled circuits only provide one-time use. In order to make the garbling scheme reusable, we can use the reusable blind garbled circuit scheme from [WW24], and use somewhere-equivocal PRFs [HJO+16] to make distributed FTS adaptively secure. Recall that all of these techniques work in batched key generation setting which can be removed using techniques from Section 2.2.2. For full construction of the adaptively secure MA-ABE scheme we refer the reader to Section 7.

Handling malicious authorities. So far, we have constructed an MA-ABE scheme when all authorities are honest. The design with batched key generation is enough to construct an MA-ABE scheme with very selective security as defined in [WWW22] and can handle any number of malicious authorities⁹. However, we are interested in general corruptions where an attacker can corrupt any authority adaptively. We now show how to handle malicious authorities generically similar to our compiler for MAFE from Section 2.1. If the attacker corrupts k out of n authorities, let $\mathcal{C}$ denote the set of all possible authority corruptions. Clearly $|\mathcal{C}| = \binom{n}{k}$. As long as $|\mathcal{C}| = \text{poly}(\lambda)$ we can instantiate $|\mathcal{C}|$ many MA-ABE instantiations $\text{MA-ABE}^{(1)}, \dots, \text{MA-ABE}^{(|\mathcal{C}|)}$. As we do not hide policy P , we do not need HSS like in our MAFE systems. We can simply additively secret share message m across all of these $|\mathcal{C}|$ instantiations. Security holds because one of these instantiations remains secure which means one these shares are hidden.

3 MAFE from Public-Key Encryption

In this section, we provide the construction of an MAFE scheme for P/Poly circuits using a tMAFE scheme and a 1-authority MAFE (also called BFE) scheme for P/Poly circuits. In this section and Section 4, by MAFE we mean our bounded collusion simulation-secure version. Full definition is provided in Definition 3.1. We provide definition of MAFE in Section 3.1, required preliminaries in Section 3.2, and our construction in Section 3.3.

3.1 Definition

A multi-authority functional encryption (MAFE) scheme MAFE for circuit class $\mathcal{C} = \{\mathcal{C}_s\}_{s \in \mathbb{N}}$ and global identifier domain $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ consists of the following polynomial-time algorithms.

⁹A previous version of this work contained this result.

$\text{GSetup}(1^\lambda, 1^Q, 1^s) \rightarrow \text{CRS}$. The possibly randomized global setup algorithm takes as input the security parameter λ , the query bound Q , the maximum circuit size s , and outputs CRS . The following algorithms take CRS implicitly.

$\text{ASetup}(\text{id}, 1^{\ell_{\text{id}}}) \rightarrow (\text{MPK}_{\text{id}}, \text{MSK}_{\text{id}})$. The probabilistic authority setup algorithm takes as input the input-size for the id -th authority ℓ_{id} , and outputs master public and secret key pair $(\text{MPK}_{\text{id}}, \text{MSK}_{\text{id}})$.

$\text{KGen}(\text{id}, \text{MSK}_{\text{id}}, \text{GID}, x) \rightarrow \text{SK}_{\text{id}, \text{GID}, x}$. The possibly randomized key generation algorithm takes as input the id -th master secret key MSK_{id} , the global identifier for the user $\text{GID} \in \mathcal{GID}_\lambda$, an input $x \in \mathcal{X}_{\text{id}} = \{0, 1\}^{\ell_{\text{id}}}$, and outputs the secret key $\text{SK}_{\text{id}, \text{GID}, x}$.

$\text{Enc}(\{\text{MPK}_{\text{id}}\}_{\text{id} \in \mathcal{I}}, C) \rightarrow \text{CT}$. The probabilistic encryption algorithm takes as input the master public keys for authorities in $\mathcal{I}$, $\{\text{MPK}_{\text{id}}\}_{\text{id} \in \mathcal{I}}$, a circuit $C \in \mathcal{C}_\lambda$, and outputs ciphertext CT .

$\text{Dec}(\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in \mathcal{I}}, \text{CT}) \rightarrow y$. The deterministic decryption algorithm takes as input the secret keys from authorities in $\mathcal{I}$, $\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in \mathcal{I}}$, ciphertext CT , and outputs the value y .

Definition 3.1 (MAFE). An MAFE scheme $(\text{GSetup}, \text{ASetup}, \text{KGen}, \text{Enc}, \text{Dec})$ is said to be an MAFE scheme for circuit class $\mathcal{C}$ and $\mathcal{GID}$ if it satisfies the following properties.

Correctness. For any $\lambda \in \mathbb{N}$, $Q = Q(\lambda)$, $n = n(\lambda)$, $s = s(\lambda)$, $C \in \mathcal{C}_\lambda$, $\text{GID} \in \mathcal{GID}_\lambda$, $\forall \mathcal{I}, \ell_{\text{id}} = \ell_{\text{id}}(\lambda)$, $x_{\text{id}} \in \{0, 1\}^{\ell_{\text{id}}}$,

$$\Pr \left[\begin{array}{l} C(x_1, \dots, x_n) = \\ \text{Dec}(\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id}}, \text{CT}) \end{array} : \begin{array}{l} \text{CRS} \leftarrow \text{GSetup}(1^\lambda, 1^Q, 1^s), \forall \text{id} \in \mathcal{I}, \\ (\text{MPK}_{\text{id}}, \text{MSK}_{\text{id}}) \leftarrow \text{ASetup}(\text{id}, 1^{\ell_{\text{id}}}), \\ \text{SK}_{\text{id}, \text{GID}, x_{\text{id}}} \leftarrow \text{KGen}(\text{id}, \text{MSK}_{\text{id}}, \text{GID}, x_{\text{id}}), \\ \text{CT} \leftarrow \text{Enc}(\{\text{MPK}_{\text{id}}\}_{\text{id}}, C) \end{array} \right] = 1$$

Security. There exists a stateful simulator Sim such that for any admissible adversary $\mathcal{A}$, $\forall \lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} b \leftarrow \mathcal{A}^{\mathcal{O}_b(\cdot, \cdot)}(\text{CT}) \end{array} : \begin{array}{l} Q, s \leftarrow \mathcal{A}(1^\lambda), b \xleftarrow{\$} \{0, 1\}, \\ \text{CRS} \leftarrow \mathcal{O}_b(1^\lambda, 1^Q, 1^s), \\ (C, \mathcal{I}) \leftarrow \mathcal{A}^{\mathcal{O}_b(\cdot, \cdot)}(\text{CRS}), \\ \text{CT} \leftarrow \mathcal{O}_b(\mathcal{I}, C) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda)$$

where both $\mathcal{O}_0, \mathcal{O}_1$ are stateful oracles that respond to queries from $\mathcal{A}$ of the form

$(\text{ASetup}, (\text{id}, 1^{\ell_{\text{id}}}))$ $\mathcal{A}$ queries to setup id -th authority with input size ℓ_{id} . Gets MPK_{id} in response.

$(\text{KGen}, (\text{id}, \text{GID}, x))$ $\mathcal{A}$ queries to corrupt secret key for id -th authority for GID and input x such that $|x| = \ell_{\text{id}}$. Gets $\text{SK}_{\text{id}, \text{GID}, x}$ in response.

$\mathcal{O}_0$ uses $(\text{GSetup}, \text{ASetup}, \text{KGen}, \text{Enc})$ to respond to queries from $\mathcal{A}$. However, it maintains state to store public parameters for all authorities and secret keys generated thus far. $\mathcal{O}_1$ uses the simulator Sim to respond to these queries. $\mathcal{O}_1$ also provides Sim with the set $1^{|C|}, \mathcal{V}$ for ciphertext generation. Here, $\mathcal{V} = \{(\text{GID}, X = \{x_{\text{GID}, \text{id}}\}_{\text{id} \in \mathcal{I}}, C(X)) : \text{for every GID that was used to query all } \text{id} \in \mathcal{I} \text{ in pre-challenge query phase}\}$. In addition, $\mathcal{O}_1$ also provides $C(X)$ for every X completely queried in post-challenge phase to Sim .

A stateful PPT machine $\mathcal{A}$ is admissible if it makes polynomially many queries with the restriction that for the set of authorities submitted in challenge query phase, $\mathcal{I}$, it can make adaptively at most Q secret key queries across $|\mathcal{I}|$ authorities such that each authority is queried at most once per GID . In addition, $\mathcal{A}$ will make at most one ciphertext query with circuit $C \in \mathcal{C}_\lambda$ of size at most s adaptively. For simplicity, we assume that authorities will be created before secret key is queried for that authority. Thus, all $\text{id} \in \mathcal{I}$ will be created before challenge circuit is declared.

Definition 3.2 (MAFE with “bounded” authorities). An MAFE scheme ($\text{GSetup}, \text{ASetup}, \text{KGen}, \text{Enc}, \text{Dec}$) is said to be an MAFE scheme with bounded authorities if the number of authorities (n) are declared a-priori and GSetup takes 1^n as input and $\mathcal{I} = [n]$. In addition, KGen also takes master public keys from all n authorities, $\{\text{MPK}_{\text{id}_x}\}_{\text{id}_x \in [n]}$. The security definition is also slightly altered where all the authorities are initiated in the setup phase and master public keys for all authorities are provided along with CRS . That is, $\mathcal{O}_b$ is essentially a KGen oracle and $\mathcal{A}$ is only allowed to query it at most Q times.

3.2 Preliminaries

In this section, we provide the definition of a BFE scheme. We provide the definition and construction tMAFE scheme for P/Poly circuits in Appendix C from PKE.

Definition 3.3 (BFE). A BFE scheme is an MAFE scheme for a single authority (i.e, $n = 1$). Hence, we ignore CRS and $\mathcal{GID}$ as all key queries are answered by a single authority. We also alter the syntax where Setup is used instead of GSetup . id, GID are dropped from KGen . Our constructions require a stronger version of simulation that is implicitly satisfied by BFE schemes from [GVW12, AV19]. We define it as follows.

Strong Simulation Security. For any admissible adversary $\mathcal{A}$, there exists a stateful PPT simulator $\text{Sim} = (\mathcal{S}_0, \mathcal{S}_1, \mathcal{S}_2, \mathcal{S}_3)$ such that,

$$\begin{aligned} & \left\{ \mathcal{A}^{\text{KGen}(\text{MSK}, \cdot)}(\text{CT}) : \begin{array}{l} (Q, s) \leftarrow \mathcal{A}(1^\lambda), \\ (\text{MPK}, \text{MSK}) \leftarrow \text{Setup}(1^\lambda, 1^Q, 1^s), \\ C \leftarrow \mathcal{A}^{\text{KGen}(\text{MSK}, \cdot)}(\text{MPK}), \\ \text{CT} \leftarrow \text{Enc}(\text{MPK}, C) \end{array} \right\} \\ & \quad \equiv \\ & \left\{ \mathcal{A}^{\text{KGen}(\text{MSK}, \cdot)}(\text{CT}) : \begin{array}{l} (Q, s) \leftarrow \mathcal{A}(1^\lambda), \\ (\text{MPK}, \text{MSK}, \text{st}) \leftarrow \mathcal{S}_0(1^\lambda, 1^Q, 1^s), \\ C \leftarrow \mathcal{A}^{\mathcal{S}_1(\text{MSK}, \text{st}, \cdot)}(\text{MPK}), \\ \text{CT} \leftarrow \text{Enc}(\text{MPK}, C) \end{array} \right\} \\ & \quad \approx_c \\ & \left\{ \mathcal{A}^{\mathcal{S}_3^{C(\cdot)}(\text{st}, \cdot)}(\text{CT}) : \begin{array}{l} (Q, s) \leftarrow \mathcal{A}(1^\lambda), \\ (\text{MPK}, \text{MSK}, \text{st}) \leftarrow \mathcal{S}_0(1^\lambda, 1^Q, 1^s), \\ C \leftarrow \mathcal{A}^{\mathcal{S}_1(\text{MSK}, \text{st}, \cdot)}(\text{MPK}), \\ \text{CT} \leftarrow \mathcal{S}_2(\text{st}, 1^{|C|}, \mathcal{V}) \end{array} \right\} \end{aligned}$$

where $\mathcal{V} = \{(x, C(x)) : \text{for each } x \text{ queried in pre-challenge phase}\}$. A stateful PPT machine $\mathcal{A}$ is said to be admissible if

- It makes at most Q queries to the KGen oracle *and* one challenge circuit query (OR)
- Makes unbounded queries to KGen oracle and *does not* make any challenge query.

3.3 Construction

We now provide the construction of an MAFE scheme as follows.

Remark 3.4 (Deterministic KGen). In the construction below, we assume w.l.o.g that tMAFE.KGen is a deterministic algorithm. We can always de-randomize a probabilistic key generation algorithm by including a PRF key (for appropriate parameters) along with MSK as long as MSK is not corrupted.

Construction 3.5 (MAFE). We provide the construction of an MAFE scheme for P/Poly circuits and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ using a BFE scheme for P/Poly circuits (Definition 3.3) and a tMAFE scheme for P/Poly circuits and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ (Definition C.1) as follows.

GSetup($1^\lambda, 1^Q, 1^s$). Output $\text{CRS} := (\lambda, Q, s)$.

ASetup($\text{id}, 1^{\ell_{\text{id}}}$). Sample keys $(\text{bfe.mpk}, \text{bfe.msk}) \leftarrow \text{BFE.Setup}(1^\lambda, 1^Q, 1^{s'})$. Here $s' = \text{poly}(\lambda, Q, s, \ell_{\text{id}})$ is the maximum size of key generation circuit of tMAFE for these parameters. Output $\text{MPK} := (\ell_{\text{id}}, \text{bfe.mpk})$ and $\text{MSK} := (\ell_{\text{id}}, \text{bfe.msk})$.

KGen($\text{id}, \text{MSK}_{\text{id}}, \text{GID}, x$). Parse MSK_{id} as $(\ell_{\text{id}}, \text{bfe.msk}_{\text{id}})$. Sample $\text{bfe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{BFE.KGen}(\text{bfe.msk}_{\text{id}}, (\text{id}, \text{GID}, x))$. Output $\text{SK}_{\text{id}, \text{GID}, x} := \text{bfe.sk}_{\text{id}, \text{GID}, x}$.

Enc($\{\text{MPK}_{\text{id}}\}_{\text{id} \in \mathcal{I}}, C$). Parse MPK_{id} as $(\ell_{\text{id}}, \text{bfe.mpk}_{\text{id}})$ for each $\text{id} \in \mathcal{I}$. Let $n = |\mathcal{I}|$ and $\text{id}_1, \dots, \text{id}_n$ be the increasing order of elements in $\mathcal{I}$.

- Sample $\text{mafe.crs}, \{(\text{mafe.mpk}_i, \text{mafe.msk}_i)\}_i \leftarrow \text{tMAFE.GSetup}(1^\lambda, 1^Q, 1^n, 1^s, 1^{\ell_{\text{id}_1}}, \dots, 1^{\ell_{\text{id}_n}})$.
- Compute $\text{mafe.ct} \leftarrow \text{tMAFE.Enc}(\{\text{mafe.mpk}_i\}_i, C)$,
- $\forall i \in [n], \text{bfe.ct}_{\text{id}_i} \leftarrow \text{BFE.Enc}(\text{bfe.mpk}_{\text{id}_i}, \text{tMAFE.KGen}(\cdot, \text{mafe.msk}_i, \{\text{mafe.mpk}_{i'}\}_{i' \in [n]}, \cdot, \cdot))$.

Output $\text{CT} := (\text{mafe.ct}, \mathcal{I}, \{\text{bfe.ct}_{\text{id}}\}_{\text{id} \in \mathcal{I}})$.

Dec($\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in \mathcal{I}}, \text{CT}$). Parse $\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}$ as $\text{bfe.sk}_{\text{id}, \text{GID}, x_{\text{id}}}$ for each $\text{id} \in \mathcal{I}$ and $\text{CT} := (\text{mafe.ct}, \mathcal{I}', \{\text{bfe.ct}_{\text{id}}\}_{\text{id} \in \mathcal{I}'})$. If $\mathcal{I} \neq \mathcal{I}'$, abort and output $\perp$.

Otherwise, for each $\text{id} \in \mathcal{I}$, compute $\text{mafe.sk}_{\text{id}} = \text{BFE.Dec}(\text{bfe.sk}_{\text{id}, \text{GID}, x}, \text{bfe.ct}_{\text{id}})$. Output $y := \text{tMAFE.Dec}(\{\text{mafe.sk}_{\text{id}}\}_{\text{id} \in \mathcal{I}}, \text{mafe.ct})$.

Correctness. The correctness of the scheme follows from the correctness of BFE and tMAFE. Note that $\text{tMAFE.KGen}(\cdot, \text{mafe.msk}_i, \{\text{mafe.mpk}_{i'}\}_{i' \in [n]}, \cdot, \cdot)$ is a valid P/Poly circuit. Hence, by correctness of BFE, we have that $\text{mafe.sk}_{\text{id}} = \text{tMAFE.KGen}(\text{id}, \text{mafe.msk}_i, \{\text{mafe.mpk}_{i'}\}_{i' \in [n]}, \text{GID}, x_{\text{id}})$. Thus, $y = C(x_1, \dots, x_n)$ by correctness of tMAFE.

Theorem 3.6. If tMAFE is a tMAFE scheme (Definition C.1) for P/Poly circuits and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ and BFE is a BFE scheme for P/Poly circuits (Definition 3.3), then Construction 3.5 is an MAFE scheme for P/Poly circuits and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ (Definition 3.1).

Proof Sketch. We provide a sketch of the hybrid argument used to prove security of Construction 3.5 as follows.

Hybrid 0. This is the same as the real experiment from Definition 3.1. We use honest and stateless BFE and tMAFE algorithms to respond to $\mathcal{A}$.

Hybrid 1. In this hybrid, we will simulate the pre-challenge queries using $\text{BFE.S}_0, \text{BFE.S}_1$ for all queried authorities. That is, we will use $(\text{BFE.S}_0, \text{BFE.S}_1, \text{BFE.Enc}, \text{BFE.KGen})$ to respond to authority setup and secret key queries made to authorities in the pre-challenge query phase. We will still create authorities honestly in the post-challenge query phase. The output distribution will be identical to hybrid 0 from security of BFE.

Hybrid 2, j . for $j \in [n+1]$. In this hybrid, we simulate the BFE instantiation for $\text{id}_1, \dots, \text{id}_{j-1} \in \mathcal{I}$ using $\text{BFE.Sim} = (\text{BFE.S}_0, \text{BFE.S}_1, \text{BFE.S}_2, \text{BFE.S}_3)$. The indistinguishability between these hybrids is argued from the security of BFE instantiation.

Hybrid 3. In this hybrid, we will simulate the instantiation of tMAFE. Now that all $\text{id} \in \mathcal{I}$ are simulated, we will only be using secret keys and not the master secret keys of tMAFE. Hence, indistinguishability can be argued from security of tMAFE. This is the description of Sim from Definition 3.1.

We provide the full proof in Appendix D. □

Corollary 3.7. Assuming the existence of public-key encryption, there exists a MAFE scheme (with unbounded authorities) for P/Poly circuits and $\mathcal{GTD}$.

Proof. The proof of this corollary follows from Corollary C.4, and construction of BFE for P/Poly circuits assuming PKE from [AV19]. □

Remark 3.8. Assuming the existence of one-way functions, there exists a secret-key MAFE scheme (with unbounded authorities) for P/Poly circuits and $\mathcal{GTD}$ that satisfies full security, i.e, function- and message-privacy¹⁰.

4 Bounded Collusion MAFE with Malicious Authorities

In this section, we provide the definition and construction for an MAFE scheme (recall MAFE in bounded collusions and simulation secure, Definition 3.1) with malicious authorities. For an MAFE scheme with malicious authorities (cMAFE), the main difference is that it is secure even if some of the master secret keys for authorities are lost.

4.1 Definition

Definition 4.1 (cMAFE). A cMAFE scheme, $(\text{GSetup}, \text{ASetup}, \text{KGen}, \text{Enc}, \text{Dec})$ is said to be an MAFE scheme that supports χ corruptible bits for circuit class $\mathcal{C}$, and $\mathcal{GTD}$ if it supports adaptive corruptions of authority master secret keys where the cumulative sum of input sizes of corrupted authorities is at most χ . We take the number of authorities n and χ as inputs to GSetup and work

¹⁰As we work in the ciphertext-policy setting, message-privacy means that an adversary cannot distinguish the input used in KGen .

in the “bounded” authority setting of MAFE (Definition 3.2). Security of the scheme is defined analogously with the following important changes.

- $\mathcal{O}$ responds to KGen queries and in addition, responds to authority corruption queries where $\mathcal{A}$ sends (Corr, id) and receives MSK_{id} .
- Input for Sim in challenge query phase is defined as follows — $\mathcal{V} = \{(\text{GID}, X, C(X)) : X \in \mathbf{X}_{\text{GID},1} \times \dots \times \mathbf{X}_{\text{GID},n} \text{ for every GID that was used to query all authorities. If id-th authority corrupted, then } \mathbf{X}_{\text{GID},\text{id}} = \{0, 1\}^{\ell_{\text{id}}}\}.$ Similarly, the simulator Sim queries C for every $X \in \mathbf{X}_{\text{GID},1} \times \dots \times \mathbf{X}_{\text{GID},n}$ where for corrupted id, $\mathbf{X}_{\text{GID},\text{id}} = \{0, 1\}^{\ell_{\text{id}}}$ and a singleton $x_{\text{GID},\text{id}}$ otherwise.

A stateful PPT machine is admissible if it is an admissible adversary for MAFE. In addition, the adversary can corrupt authorities adaptively. For all corrupted authorities, $\sum_{\text{id}} \ell_{\text{id}} \leq \chi$ and $\chi(\lambda) = O(\log \lambda)$. Once id is corrupted, $\mathcal{A}$ should not issue for secret key queries to id.

4.2 MAFE with Adaptive Corruptions from HSS

In this section, we construct an MAFE scheme with adaptive authority corruptions using a homomorphic secret sharing scheme for P/Poly circuits. In particular, we construct a cMAFE scheme in which the number of authorities that can be corrupted, $k < n$ are known a-priori such that $\binom{n}{k} = \text{poly}(\lambda)$. For our construction, we use a homomorphic secret sharing scheme with strong simulation security (HSS). We provide definition of HSS in Section F. Below, we formally define and construct (n, k) -cMAFE.

Definition 4.2 ((n, k) -cMAFE). An (n, k) -cMAFE scheme for circuit class $\mathcal{C}$ and $\mathcal{GID}$ is defined similarly to Definition 4.1 and the maximum number of corruptible authorities, k are declared a-priori as an input to the GSetup algorithm. In addition, we require that $\binom{n}{k} = \text{poly}(\lambda)$. We satisfy the following notion of security in Construction 4.3. For any admissible adversary $\mathcal{A}$ that meets the admissibility criterion in Definition 4.1, there exists a PPT simulator Sim such that $\forall \lambda \in \mathbb{N}$,

$$\left\{ \begin{array}{l} Q, n, s, k, \chi, \text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id}}, \\ \{\text{MSK}_{\text{id}}\}_{\text{id} \in \mathcal{K}}, Q, \text{CT}, b \end{array} : \begin{array}{l} Q, n, k, \chi, s, \ell_1, \dots, \ell_n \leftarrow \mathcal{A}(1^\lambda), \\ \text{CRS} \leftarrow \text{GSetup}(1^\lambda, 1^Q, 1^n, 1^s, 1^k, 1^\chi), \\ \forall \text{id}, (\text{MPK}_{\text{id}}, \text{MSK}_{\text{id}}) \leftarrow \text{AShSetup}(\text{id}, 1^{\ell_{\text{id}}}), \\ C \leftarrow \mathcal{A}^{\mathcal{O}(\cdot, \cdot)}(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id}}), \\ \text{CT} \leftarrow \text{Enc}(\{\text{MPK}_{\text{id}}\}_{\text{id}}, C), \\ b \leftarrow \mathcal{A}^{\mathcal{O}(\cdot, \cdot)}(\text{CT}) \end{array} \right\} \\ \approx_c \\ \left\{ \text{Sim}^{\mathcal{A}, \mathcal{C}(\cdot)}(1^\lambda) \right\}$$

where $\mathcal{O}$ is a stateful oracle that responds to key generation using KGen and authority corruption queries. $\mathcal{O}$ adds the query and response to Q for each query made by $\mathcal{A}$. $\mathcal{K}$ is the set of all adaptively corrupted authorities.

Construction 4.3 ((n, k) -cMAFE). We provide the construction of (n, k) -cMAFE for P/Poly circuits and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ (Definition 4.2) using an MAFE scheme for P/Poly circuits and $\mathcal{GID}$ (Definition 3.2), a HSS scheme for P/Poly circuits (Definition F.1) as follows.

GSetup($1^\lambda, 1^Q, 1^n, 1^s, 1^k, 1^\chi$). Let $\mathbb{I} = \left[\binom{n}{k}\right]$ and $\mathbb{J} = [2^\chi]$. We will assume some fixed ordering of $2^{\binom{n}{k}}|_k$ ¹¹ and say $\mathcal{K}_i \in 2^{\binom{n}{k}}|_k$ for $i \in \mathbb{I}$. For each $i \in \mathbb{I}, j \in \mathbb{J}$, instantiate an MAFE,

$$\forall i \in \mathbb{I}, j \in \mathbb{J}, \text{crs}^{(i,j)} \leftarrow \text{MAFE.GSetup}(1^\lambda, 1^Q, 1^{n-k}, 1^s)$$

We assume w.l.o.g that for any $i \in \mathbb{I}$, the authority identifiers are elements of the set $[n] \setminus \mathcal{K}_i$ ¹², i.e, for the (i, j) -th instantiation of MAFE, $\text{id} \notin \mathcal{K}_i$ ¹³. Output $\text{CRS} := (\text{crs}^{(i,j)})_{i \in \mathbb{I}, j \in \mathbb{J}}$.

ASetup($\text{id}, 1^{\ell_{\text{id}}}$). For each $i \in \mathbb{I}, j \in \mathbb{J}$, generate master public and secret keys for the (i, j) -th instantiation of MAFE if $\text{id} \notin \mathcal{K}_i$, $(\text{mpk}^{(i,j)}, \text{msk}^{(i,j)}) \leftarrow \text{MAFE.ASetup}(\text{id}, 1^{\ell_{\text{id}}})$. Output $\text{MPK} := (\text{mpk}^{(i,j)})_{i,j}$ and $\text{MSK} := (\text{msk}^{(i,j)})_{i,j}$.

KGen($\text{id}, \text{MSK}_{\text{id}}, \{\text{MPK}_{\text{idx}}\}_{\text{idx} \in [n]}, \text{GID}, x$). Parse MSK_{id} as $(\text{mafe.msk}_{\text{id}}^{(i,j)})_{i \in \mathbb{I}, j \in \mathbb{J}}$. For each $\text{idx} \in [n]$, parse MPK_{idx} as $(\text{mafe.mpk}_{\text{idx}}^{(i,j)})_{i \in \mathbb{I}, j \in \mathbb{J}}$.

Note that each $\text{mafe.mpk}_{\text{idx}}^{(i,j)}$ implicitly consists of information about ℓ_{idx} . Using $(\ell_1, \dots, \ell_n)$, define a corruptible set $\mathcal{K}' \subseteq [n]$ such that $\ell_{\text{id}} = O(\log \lambda)$ for $\text{id} \in \mathcal{K}'$.

Let $\mathbb{I}' = \{i : i \in \mathbb{I}, \mathcal{K}_i \subseteq \mathcal{K}', \sum_{\text{id} \in \mathcal{K}_i} \ell_{\text{id}} \leq \chi\}$. Let $\mathbb{I}'_{\text{id}} = \{i : i \in \mathbb{I}', \text{id} \notin \mathcal{K}_i\}$. For every $i \in \mathbb{I}'_{\text{id}}, j \in \mathbb{J}$,

$$\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \text{MAFE.KGen}(\text{id}, \text{msk}_{\text{id}}^{(i,j)}, \{\text{mpk}_{\text{idx}}^{(i,j)}\}_{\text{idx}}, \text{GID}, x)$$

Output $\text{SK}_{\text{id}, \text{GID}, x} := (\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)})_{i,j}$.

Enc($\{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}, C$). Parse MPK_{id} as $(\text{mpk}_{\text{id}}^{(i,j)})_{i \in \mathbb{I}, j \in \mathbb{J}}$ for each $\text{id} \in [n]$. Parse $\mathcal{K}'$ similar to KGen.

Share the circuit C using HSS for all $i \in \mathbb{I}'$, $\{\tilde{C}_i\}_{i \in \mathbb{I}'} \leftarrow \text{HSS.Share}(1^\lambda, 1^Q, 1^{|\mathbb{I}'|}, C)$. For each $i \in \mathbb{I}'$, encrypt the circuit F_{ij} where this circuit takes $(\ell_1 + \dots + \ell_n) - \chi$ bits as input and sets $\{x_{\text{id}}\}_{\text{id} \in \mathcal{K}_i} = j$ (truncations done appropriately) and computes $\text{HSS.Eval}(i, \tilde{C}_i, x)$.

That is, for every $i \in \mathbb{I}', j \in \mathbb{J}$, $\text{ct}^{(i,j)} \leftarrow \text{MAFE.Enc}(\{\text{mpk}_{\text{id}}^{(i,j)}\}_{\text{id} \notin \mathcal{K}_i}, F_{ij})$. Output $\text{CT} := (\text{ct}^{(i,j)})_{i \in \mathbb{I}', j \in \mathbb{J}}$.

Dec($\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}, \text{CT}$). Parse CT as $(\text{mafe.ct}^{(i,j)})_{i \in \mathbb{I}', j \in \mathbb{J}}$ and $\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}$ as $(\text{mafe.sk}_{\text{id}, \text{GID}, x_{\text{id}}}^{(i,j)})_{i,j}$ for each $\text{id} \in [n]$. Let $x = (x_1, \dots, x_n)$. For each $i \in \mathbb{I}'$, partition x as $\{x_{\text{id}}\}_{\text{id} \in \mathcal{K}_i}$ and $\{x_{\text{id}}\}_{\text{id} \notin \mathcal{K}_i}$. Observe that we have secret keys for all $\{x_{\text{id}}\}_{\text{id} \notin \mathcal{K}_i}$. Decrypt the $(i, j = \{x_{\text{id}}\}_{\text{id} \notin \mathcal{K}_i})$ -th instantiation of MAFE, $y_{i,j} = \text{MAFE.Dec}(\{\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)}\}_{\text{id} \notin \mathcal{K}_i}, \text{ct}^{(i,j)})$. Output $y := \text{HSS.Retrieve}(\{y_{i,j}\}_{i \in \mathbb{I}'})$.

Correctness. The correctness of the scheme follows from the correctness of MAFE and HSS schemes. In particular, $\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}} = (\text{sk}_{\text{id}, \text{GID}, x_{\text{id}}}^{(i,j)})_{i \in \mathbb{I}'_{\text{id}}, j \in \mathbb{J}}$ and $\text{CT} = \{\text{MAFE.Enc}(\{\text{mpk}_{\text{id}}^{(i,j)}\}_{\text{id} \notin \mathcal{K}_i}, \text{HSS.Eval}(i, \tilde{C}_i, \cdot, j))\}_{i \in \mathbb{I}', j \in \mathbb{J}}$. By correctness of $(i, j = \{x_{\text{id}}\}_{\text{id} \notin \mathcal{K}_i})$ -th instantiation of MAFE, $y_{i,j} = \text{MAFE.Dec}(\{\text{sk}_{\text{id}, \text{GID}, x_{\text{id}}}^{(i,j)}\}_{\text{id} \notin \mathcal{K}_i}, \text{ct}^{(i,j)}) = \text{HSS.Eval}(i, \tilde{C}_i, x = (x_1, \dots, x_n))$. Thus, by correctness of HSS, $y = C(x_1, \dots, x_n)$.

¹¹Recall from Appendix A that $2^{\mathbf{A}}|_a$ is the set of all subsets of set $\mathbf{A}$ of size a .

¹²We use $\text{id} \notin \mathcal{K}_i$ and $\text{id} \in [n] \setminus \mathcal{K}_i$ interchangeably.

¹³This is done to avoid complicated notation required in the subsequent algorithms. We use this fact implicitly.

Theorem 4.4. If HSS is a secure HSS scheme for P/Poly circuits (Definition F.1), MAFE is a secure MAFE scheme for P/Poly circuits and $\mathcal{GID}$ (Definition 3.2), then Construction 4.3 is a secure (n, k) -cMAFE scheme for P/Poly circuits and $\mathcal{GID}$.

Proof Sketch. We provide a sketch of the hybrid argument used to prove security of Construction 4.3 as follows.

Hybrid 0. This is the real experiment from Definition 4.2. We use the stateless MAFE and HSS algorithms to respond to queries from adversary.

Hybrid 1, κ . For $\kappa \in [|\mathbb{J}| + 1]$. In this hybrid, we will randomly guess $i^* \xleftarrow{\$} \mathbb{I}$ and simulate the $(i^*, 1), \dots, (i^*, \kappa - 1)$ instantiations of MAFE. The indistinguishability of hybrids can be argued using the security of MAFE. In addition, if the adversary corrupts any id that is not in the i^* -th element of $\mathbb{I}$, we abort. The indistinguishability is argued using a two fold argument – if we correctly guess the malicious authority set correctly, the challenger does not abort with $1/|\mathbb{I}|$ probability which by design is $1/\binom{n}{k} = 1/\text{poly}(\lambda)$. If this happens, we then proceed to argue indistinguishability from MAFE security.

Hybrid 2. In this hybrid, we will simulate the HSS instantiation. Now that all MAFE instantiations for i^* (after correctly guessing at the beginning) are simulated, we would only be using the outputs of share evaluation. Thus, we can readily rely on the strong security of HSS to show indistinguishability. This is the description of Sim from Definition 4.2.

The full proof is in Appendix G. □

Remark 4.5 (Corruptions & dynamic collusions). We remark that Construction 4.3 is agnostic to the query bound Q . That is, even if we provide the query bound during encryption, the same construction works *mutatis mutandis*. This is because of the black-box usage of MAFE. Hence, a similar construction yields an (n, k) -cMAFE scheme with dynamic collusions.

5 Collusion Resistant MAFE from Functional Encryption

In this section, we construct a collusion-resistant MAFE scheme for any class of P/Poly circuits from the minimal assumption of an FE scheme for any class of P/Poly circuits. In this section and Section 6, for simplicity, by MAFE we mean collusion-resistant version with indistinguishability security. We define this Definition 5.1. Our construction proceeds in three steps.

1. We construct a weaker version of MAFE scheme that is *selectively* secure and with *batched key generation*. We formally define these notions in Section 5.3.
2. We then generically bootstrap any selectively secure MAFE scheme to an adaptively secure version with batched key generation using a private-key one-ciphertext MAFE scheme.
3. We then use *vector* non-committing encryption to realize adaptively secure MAFE as per Definition 5.1.

5.1 Definition

We now formally define our MAFE scheme.

Definition 5.1 (MAFE). An MAFE scheme $(\text{GSetup}, \text{ASetup}, \text{KGen}, \text{Enc}, \text{Dec})$ is said to be an MAFE scheme if there is no a-priori bound on the number of GID's an adversary can corrupt. Syntactically, we drop the bound 1^Q from GSetup . Note that this is a strengthening of Definition 3.1. MAFE in the bounded authorities model is defined similarly to Definition 3.2 with the aforementioned strengthening. From henceforth, whenever we refer to MAFE, we refer to the bounded authority version. Security definition is altered to a weaker indistinguishability notion defined below.

IND Security. For any PPT admissible adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} (n, s, \ell_1, \dots, \ell_n) \leftarrow \mathcal{A}(1^\lambda), L = \max_{\text{id}} \{\ell_{\text{id}}\} \\ \text{CRS} \leftarrow \text{GSetup}(1^\lambda, 1^n, 1^s, 1^L), \\ \forall \text{id} \in [n], (\text{MPK}_{\text{id}}, \text{MSK}_{\text{id}}) \leftarrow \text{ASetup}(\text{id}, 1^{\ell_{\text{id}}}), \\ (C_0, C_1) \leftarrow \mathcal{A}^{\mathcal{O}(\cdot, \cdot, \cdot)}(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}), \\ b \xleftarrow{\$} \{0, 1\}, \text{CT} \leftarrow \text{Enc}(\{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}, C_b) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda)$$

where $\mathcal{O}$ is a key-generation oracle that takes as input $(\text{id}, \text{GID}, x_{\text{id}}) \in [n] \times \mathcal{GID} \times \{0, 1\}^{\ell_{\text{id}}}$ and outputs $\text{SK}_{\text{id}, \text{GID}, x} \leftarrow \text{KGen}(\text{id}, \text{MSK}_{\text{id}}, \{\text{MPK}_{\text{id}_x}\}_{\text{id}_x \in [n]}, \text{GID}, x_{\text{id}})$. A stateful PPT machine $\mathcal{A}$ is said to be an admissible adversary if for each $\text{GID} \in \mathcal{GID}$ queried by $\mathcal{A}$ with the corresponding queries $(1, \text{GID}, x_1), \dots, (n, \text{GID}, x_n)$, $C_0(x_1, \dots, x_n) = C_1(x_1, \dots, x_n)$. Moreover, $C_0, C_1 \in \mathcal{C}_\lambda, |C_0| = |C_1|$ and are of size at most s .

5.2 Preliminaries

We provide the definition of single-authority MAFE scheme, FE.

Definition 5.2 (FE). An FE scheme $(\text{Setup}, \text{KGen}, \text{Enc}, \text{Dec})$ is said to be (key-policy) FE scheme if there is no a-priori bound on the number of secret-keys an adversary can corrupt. Specifically, the syntax is similar to Definition 3.3 except we drop 1^Q from Setup . We rely on the key-policy variant in this work where KGen takes circuits $C \in \mathcal{C}_\lambda$ as input and Enc takes inputs x . Security definition is altered to a weaker indistinguishability notion defined below.

IND Security. For any PPT admissible adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} s \leftarrow \mathcal{A}(1^\lambda), \\ (\text{MPK}, \text{MSK}) \leftarrow \text{Setup}(1^\lambda, 1^s), \\ (x_0, x_1) \leftarrow \mathcal{A}^{\text{KGen}(\text{MSK}, \cdot)}(\text{MPK}), \\ b \xleftarrow{\$} \{0, 1\}, \text{CT} \leftarrow \text{Enc}(\text{MPK}, x_b) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda)$$

A PPT machine $\mathcal{A}$ is said to be admissible if each $C \in \mathcal{C}_\lambda$ queried by $\mathcal{A}$ is at most size s , $C(x_0) = C(x_1)$ and $|x_0| = |x_1|$.

5.3 Selectively Secure MAFE with Batched Key Generation

In this section, we provide the definition and construction of a selectively-secure MAFE with batched key generation from a (key-policy) FE scheme (Definition 5.2). Firstly, we define an MAFE scheme with batched key generation in Definition 5.3 and then define selectively secure version as a natural weakening of this definition. At a high level, batched key generation security requires that the adversary produce all the input $x_1, \dots, x_n$ for n authorities for the same GID at once as a “batch” of inputs. Note that KGen of each $\text{id} \in [n]$ is still independent of other.

Definition 5.3. An abMAFE scheme $(\text{GSetup}, \text{ASetup}, \text{KGen}, \text{Enc}, \text{Dec})$ is said to be an MAFE with batched key generation if it satisfies the correctness property from Definition 5.1 and following (weaker) security definition.

IND Security with Batched Key Generation. For any PPT admissible adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\left| \Pr \left[\begin{array}{l} b \leftarrow \mathcal{A}^{\mathcal{O}(\cdot, \cdot)}(\text{CT}) : \\ \begin{array}{l} (n, s, \ell_1, \dots, \ell_n) \leftarrow \mathcal{A}(1^\lambda), L = \max_{\text{id}} \{\ell_{\text{id}}\}, \\ \text{CRS} \leftarrow \text{GSetup}(1^\lambda, 1^n, 1^s, 1^L), \\ \forall \text{id} \in [n], (\text{MPK}_{\text{id}}, \text{MSK}_{\text{id}}) \leftarrow \text{ASetup}(\text{id}, 1^{\ell_{\text{id}}}), \\ (C_0, C_1) \leftarrow \mathcal{A}^{\mathcal{O}(\cdot, \cdot)}(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}), \\ b \xleftarrow{\$} \{0, 1\}, \text{CT} \leftarrow \text{Enc}(\{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}, C_b) \end{array} \end{array} \right] - \frac{1}{2} \right| \leq \text{negl}(\lambda)$$

where $\mathcal{O}$ is a key-generation oracle that takes as input $(\text{GID}, (x_1, \dots, x_n)) \in \mathcal{GID} \times \{0, 1\}^{\ell_1} \times \dots \times \{0, 1\}^{\ell_n}$ and outputs $(\text{SK}_{1, \text{GID}, x_1}, \dots, \text{SK}_{n, \text{GID}, x_n})$ where $\text{SK}_{\text{id}, \text{GID}, x} \leftarrow \text{KGen}(\text{id}, \text{MSK}_{\text{id}}, \{\text{MPK}_{\text{id}_x}\}_{\text{id}_x}, \text{GID}, x_{\text{id}})$ for each $\text{id} \in [n]$. A stateful PPT machine $\mathcal{A}$ is said to be an admissible adversary if for each $\text{GID} \in \mathcal{GID}$ queried by $\mathcal{A}$ with the corresponding query $(\text{GID}, x_1, \dots, x_n)$, $C_0(x_1, \dots, x_n) = C_0(x_1, \dots, x_n)$. Moreover, $C_0, C_1 \in \mathcal{C}_\lambda, |C_0| = |C_1|$ and are of size at most s .

Definition 5.4. An sbMAFE scheme $(\text{GSetup}, \text{ASetup}, \text{KGen}, \text{Enc}, \text{Dec})$ is said to be a *selectively* secure MAFE with batched key generation if in the above security game, the circuits C_0, C_1 are declared selectively by $\mathcal{A}$, even before receiving $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id}})$. $\mathcal{A}$ then receives CT along with $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id}})$.

Now, we construct an sbMAFE scheme satisfying Definition 5.4.

Construction 5.5 (sbMAFE). We provide the construction of a selectively-secure MAFE scheme with batched key generation for P/Poly circuits and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ using an FE scheme for P/Poly circuits (Definition 5.2), pseudorandom function $\text{PRF} : \{0, 1\}^\lambda \times \mathcal{GID} \rightarrow \{0, 1\}^\lambda$ (Definition A.3), an SKE with pseudorandom ciphertexts (Definition A.6), and garbled circuit scheme $(\text{Garb}, \text{Eval})$ for P/Poly circuits (Definition A.1) as follows.

$\text{GSetup}(1^\lambda, 1^n, 1^L, 1^s)$. Output $\text{CRS} := (\lambda, n, L, s)$.

$\text{ASetup}(\text{id}, 1^{\ell_{\text{id}}})$. Given $\text{id} \in [n], \ell_{\text{id}} \leq L$, sample FE keys, $(\text{fe.mpk}, \text{fe.msk}) \leftarrow \text{FE.Setup}(1^\lambda, 1^{s'})$ where $s' = \text{poly}(\lambda, n, L, s)$ is the maximum size of the circuit from KGen . Output $\text{MPK} := (\text{id}, \ell_{\text{id}}, \text{fe.mpk})$ and $\text{MSK} := (\text{id}, \text{fe.msk})$.

KGen(id, MSK_{id}, {MPK_{idx}}_{idx ∈ [n]}, GID, x). Parse MSK_{id} as (id, fe.msk_{id}) and for each idx ∈ [n], MPK_{idx}

as (idx, ℓ_{idx}, fe.mpk_{idx}). Sample ske.ct* $\xleftarrow{\$}$ {0, 1}^{ℓ_{id}*} where ℓ_{id}* = poly(λ, n, L, s) is the size of the garbled circuit for s-size circuits and ℓ_{id}-many wire labels padded appropriately. Create the circuit *F* as described in Figure 2.

Sample an FE secret key fe.sk_{*F*} ← FE.KGen(fe.msk_{id}, *F*). Output SK_{id,GID,x} := fe.sk_{*F*}.

Enc({MPK_{id}}_{id ∈ [n]}, *C*). Parse MPK_{id} as (id, ℓ_{id}, fe.mpk_{id}) for each id ∈ [n].

Sample key for PRF, *K* $\xleftarrow{\$}$ {0, 1}^λ. Define *n* sets of indices ℐ₁, ..., ℐ_{*n*} where ℐ₁ := {1, ..., ℓ₁} and ℐ_{id} = { (∑_{i=1}^{id-1} ℓ_i) + 1, ..., (∑_{i=1}^{id} ℓ_i) } for id ∈ [2, *n*]. Similarly, define indices α₁, ..., α_{*n*} to be the numerically first indices in sets ℐ₁, ..., ℐ_{*n*} respectively.

For each id ∈ [n], sample an FE ciphertext, fe.ct_{id} ← FE.Enc(fe.mpk_{id}, (*C*, *K*, α_{id}, ℐ_{id}, 0, ⊥)).

Output CT := (fe.ct_{id})_{id ∈ [n]}.

Dec({SK_{id,GID,x_{id}}}_{id ∈ [n]}, CT). Parse SK_{id,GID,x_{id}} as fe.sk_{*F_{id}*} for each id ∈ [n] and CT as (fe.ct_{id})_{id ∈ [n]}.

Compute (C̃_{id}, w̄_{id}) = FE.Dec(fe.sk_{*F_{id}*}, fe.ct_{id}). If all C̃_{id} are not the same, abort and output ⊥. Otherwise, let C̃ = C̃₁ and output *y* = Eval(C̃, (w̄_{id})_{id ∈ [n]}).

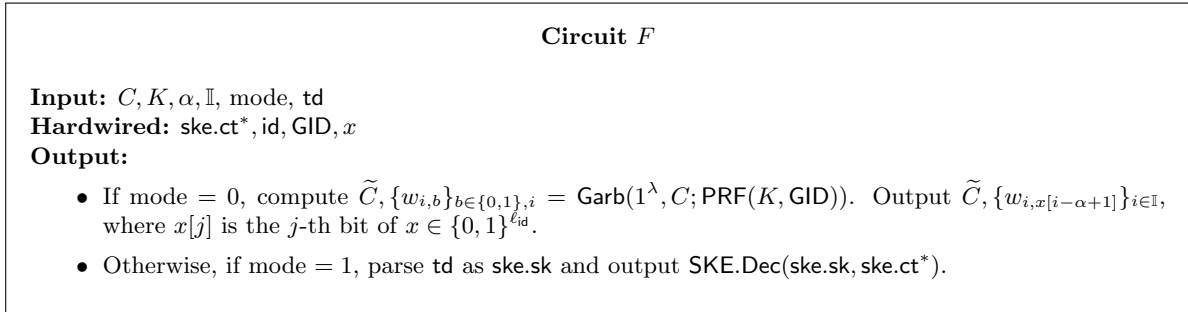


Figure 2: Description of circuit *F*

Correctness. The correctness of Construction 5.5 follows from correctness of FE, garbling scheme, and deterministic nature of PRF. By correctness of FE, we have that for each id ∈ [n], (C̃_{id}, w̄_{id}) = *F_{id}*(*C*, *K*, α_{id}, ℐ_{id}, 0, ⊥). By Figure 2, we have that as mode = 0, C̃_{id} is the garbling of *C* and w̄_{id} is the set of wire labels for the id-th input *x_{id}* ∈ {0, 1}^{ℓ_{id}}. For any id ∈ [n] these are generated using the randomness PRF(*K*, GID). Hence, if all GID are the same, all C̃_{id} are the same and w̄_{id} are the wire labels for the input (*x*₁, ..., *x_n*) to the circuit *C*. Hence, *y* = *C*(*x*₁, ..., *x_n*).

Lemma 5.6. If FE is an FE scheme for P/Poly circuits (Definition 5.2), PRF : {0, 1}^λ × GID_λ → {0, 1}^λ is a PRF scheme (Definition A.3), SKE is an SKE scheme with pseudorandom ciphertexts (Definition A.6), and (Garb, Eval) is a garbling scheme for P/Poly circuits (Definition A.1), then Construction 5.5 is a selectively secure MAFE scheme with batched key generation (Definition 5.4) for P/Poly circuits and GID.

Proof Sketch. We provide a sketch of the hybrid argument used to prove security of Construction 5.5 as follows.

Hybrid 0. This is the real experiment from Definition 5.4 with $b = 0/1$.

Hybrid 1. In this experiment, we will sample $\text{ske.sk} \leftarrow \text{SKE.Setup}(1^\lambda)$ and set $\text{ske.ct}_{\text{id}, \text{GID}}^*$ to be encryptions of the string $(\tilde{C}, \{w_{i, x_{\text{id}}[i - \alpha_{\text{id}}]}\}_{i \in \mathbb{I}_{\text{id}}})$ for each $\text{id} \in [n]$ and each GID queried by $\mathcal{A}$ where $(\tilde{C}, \{w_{i, b}\}_{i, b}) = \text{Garb}(1^\lambda, C_b; \text{PRF}(K, \text{GID}))$. Note that this is the same as the output of F_{id} when $\text{mode} = 0$. Hybrids 0 and 1 are indistinguishable by security of SKE which has pseudorandom ciphertexts.

Hybrid 2, j . for $j \in [n+1]$. In this hybrid, we will set the input to the first $j-1$ FE encryptions as $(\bar{0}, \text{mode} = 1, \text{ske.sk})$ ¹⁴. When $\text{mode} = 1$, the output is the plaintext of $\text{ske.ct}_{\text{id}}^*$. By previous hybrid, we set it to be the same as output when $\text{mode} = 0$. Thus, indistinguishability follows by FE security.

Hybrid 3. In this hybrid, we sample a uniformly random string $R_{\text{GID}} \xleftarrow{\$} \{0, 1\}^\lambda$ for each GID instead of using $\text{PRF}(K, \cdot)$. Hybrid 2, $n+1$ and 3 are indistinguishable by PRF security.

Hybrid 4, j . for $j \in [Q+1]$. Here, $Q = \text{poly}(\lambda)$ is the number of queries made by $\mathcal{A}$ to the challenger. In this hybrid, we simulate the first $j-1$ garblings of circuit C_b . As we know C_0, C_1 selectively, we can use $C_b(x_1, \dots, x_n)$ for each GID to perform this operation. Thus, indistinguishability follows by Garb security. At the end of Hybrid 4, $Q+1$, challenger only relies on $C_b(x_1, \dots, x_n)$. As $C_0(x_1, \dots, x_n) = C_1(x_1, \dots, x_n)$, Hybrid 4, $Q+1$ is independent of bit b and advantage of any adversary in this experiment is exactly $1/2$.

We provide full proof in Appendix I. □

5.4 Adaptively Secure MAFE with Batched Key Generation

In this section, we construct an adaptively secure MAFE scheme that satisfies Definition 5.3. For our construction, we require a private-key one-ciphertext MAFE scheme with IND security which we define in Definition J.1.

Construction 5.7 (abMAFE). We provide the construction of an MAFE scheme with batched key generation for P/Poly circuits and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ using a selectively-secure MAFE scheme for P/Poly circuits (Definition 5.4) and $\mathcal{GID}$, an SKE with pseudorandom ciphertexts (Definition A.6), pseudorandom function $\text{PRF} : \{0, 1\}^\lambda \times \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$ (Definition A.3), and a one-ciphertext private-key MAFE scheme for P/Poly circuits and $\mathcal{GID}$ (Definition J.1) as follows.

GSetup($1^\lambda, 1^n, 1^L, 1^s$). Sample $\text{selmafe.crs} \leftarrow \text{sbMAFE.GSetup}(1^\lambda, 1^n, 1^L, 1^s)$. Output $\text{CRS} := \text{selmafe.crs}$.

ASetup($\text{id}, 1^{\ell_{\text{id}}}$). Given $\text{id} \in [n]$, $\ell_{\text{id}} \leq L$, sample sbMAFE keys, $(\text{selmafe.mpk}, \text{selmafe.msk}) \leftarrow \text{sbMAFE.ASetup}(\text{id}, 1^{\ell_{\text{id}}})$ where $\ell_{\text{id}}' = \text{poly}(\lambda, n, \ell_1, \dots, \ell_n, s)$ is the input size for the id -th authority from the circuit in Figure 3. Output $\text{MPK} := \text{selmafe.mpk}$ and $\text{MSK} := \text{selmafe.msk}$.

¹⁴By $\bar{0}$ we mean an all zero string of sufficient length.

KGen(id, MSK_{id}, {MPK_{idx}}_{idx ∈ [n]}, GID, x). Parse MSK_{id} as selmafe.msk_{id} and for each idx ∈ [n], MPK_{idx} as selmafe.mpk_{idx}. Sample $\rho \xleftarrow{\$} \{0, 1\}^\lambda$, $\text{ske.ct}^* \xleftarrow{\$} \{0, 1\}^{\ell_{id}^*}$ where $\ell_{id}^* = \text{poly}(\lambda, n, \ell_1, \dots, \ell_n, s)$ is the size of the opMAFE secret keys for the parameters specified.

Sample an sbMAFE secret key $\text{selmafe.sk}_{id, \text{GID}, x} \leftarrow \text{sbMAFE.KGen}(\text{id}, \text{selmafe.msk}_{id}, \{\text{selmafe.mpk}_{idx}\}_{idx \in [n]}, \text{GID}, (\text{id}, \text{GID}, x, \rho, \text{ske.ct}^*))$. Output $\text{SK}_{id, \text{GID}, x} := \text{selmafe.sk}_{id, \text{GID}, x}$.

Enc({MPK_{id}}_{id ∈ [n]}, C). Parse MPK_{id} as (id, ℓ_{id} , fe.mpk_{id}) for each id ∈ [n].

Sample an opMAFE instantiation $(\text{pmafe.crs}, \{\text{pmafe.msk}_{id}\}_{id \in [n]}) \leftarrow \text{opMAFE.GSetup}(1^\lambda, 1^n, 1^s, 1^{\ell_1}, \dots, 1^{\ell_n})$ and PRF key, $K \xleftarrow{\$} \{0, 1\}^\lambda$. Construct the circuit G specified in Figure 3.

Sample opMAFE ciphertext for C , $\text{pmafe.ct} \leftarrow \text{opMAFE.Enc}(\{\text{pmafe.msk}_{id}\}_{id \in [n]}, C)$ and sbMAFE ciphertext for circuit G , $\text{selmafe.ct} \leftarrow \text{sbMAFE.Enc}(\{\text{selmafe.mpk}_{id}\}_{id \in [n]}, G)$.

Output $\text{CT} := (\text{pmafe.ct}, \text{selmafe.ct})$.

Dec({SK_{id, GID, x_{id}}}_{id ∈ [n]}, CT). Parse $\text{SK}_{id, \text{GID}, x_{id}}$ as $\text{selmafe.sk}_{id, \text{GID}, x_{id}}$ for each id ∈ [n] and CT as $(\text{pmafe.ct}, \text{selmafe.ct})$.

Compute $(\text{pmafe.sk}_{id, \text{GID}, x_{id}})_{id \in [n]} = \text{sbMAFE.Dec}(\text{selmafe.sk}_{id, \text{GID}, x_{id}}, \text{selmafe.ct})$. If this aborts, then abort and output $\perp$.

Otherwise, compute and output $y = \text{opMAFE.Dec}(\{\text{pmafe.sk}_{id, \text{GID}, x_{id}}\}_{id \in [n]}, \text{pmafe.ct})$.

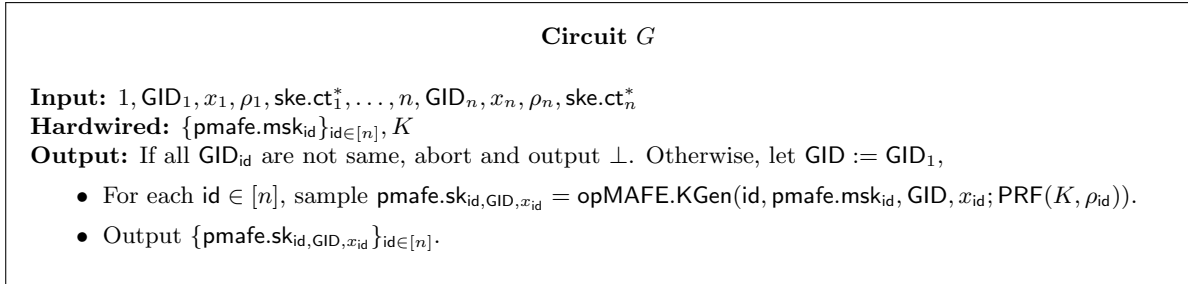


Figure 3: Description of circuit G

Correctness. The correctness of Construction 5.7 follows from correctness of sbMAFE and opMAFE. By correctness of sbMAFE, we have that for each $id \in [n]$, $\text{pmafe.sk}_{id, \text{GID}, x_{id}} = \text{opMAFE.KGen}(\text{id}, \text{pmafe.msk}_{id}, \text{GID}, x_{id}; \text{PRF}(K, \rho_{id}))$. Then, by correctness of opMAFE, if all GID are the same, $y = C(x_1, \dots, x_n)$.

Lemma 5.8. If sbMAFE is a selectively-secure MAFE scheme for P/Poly circuits and $\mathcal{GID}$ (Definition 5.4), PRF : $\{0, 1\}^\lambda \times \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$ is a PRF scheme (Definition A.3), SKE is an SKE scheme with pseudorandom ciphertexts (Definition A.6), and opMAFE is a one-ciphertext private-key MAFE scheme for P/Poly circuits and $\mathcal{GID}$ (Definition J.1), then Construction 5.7 is an MAFE scheme with batched key generation (Definition 5.3) for P/Poly circuits and $\mathcal{GID}$.

Proof Sketch. We provide a sketch of the hybrid argument used to prove security of Construction 5.7 as follows.

Hybrid 0. This is the real experiment from Definition 5.4 with $b = 0/1$.

Hybrid 1. In this experiment, we will sample $\text{ske.sk} \leftarrow \text{SKE.Setup}(1^\lambda)$ and set $\text{ske.ct}_{\text{id},\text{GID}}^*$ to be encryptions of the string $\text{pmafe.sk}_{\text{id},\text{GID},x}$ for each $\text{id} \in [n]$ and each GID queried by $\mathcal{A}$ where $\text{pmafe.sk}_{\text{id},\text{GID},x} = \text{opMAFE.KGen}(\text{id}, \text{pmafe.msk}_{\text{id}}, \text{GID}, x; \text{PRF}(K, \rho_{\text{id},\text{GID}}))$. Note that this is the same as one of the output of G . To do so, we will sample $K, \{\text{pmafe.msk}_{\text{id}}\}_{\text{id} \in [n]}$ early which doesn't change the distribution of ciphertext in Hybrid 0. Hybrid 0 and 1 are indistinguishable by security of SKE which has pseudorandom ciphertexts.

Hybrid 2. In this hybrid, we will change the circuit G to G' which has the same size as G , but instead of hardwiring $\text{pmafe.msk}_{\text{id}}$'s and K , we hardwire ske.sk in G' . We also change the functionality of G' to simply decrypt and output $\text{ske.ct}_{\text{id},\text{GID}}^*$. By previous hybrid, we set $\text{ske.ct}_{\text{id},\text{GID}}^*$ to be the same as output of G . Thus, indistinguishability follows by sbMAFE security as we can determine G and G' at the beginning of the experiment.

Hybrid 3. In this hybrid, we sample a uniformly random string $R_{\text{id},\text{GID}} \xleftarrow{\$} \{0,1\}^\lambda$ for each GID query instead of using $\text{PRF}(K, \rho_{\text{id},\text{GID}})$. Hybrid 2 and 3 are indistinguishable by PRF security.

Hybrid 4. In this hybrid, we use opMAFE security to switch pmafe.ct from an encryption of C_b to C_{1-b} . Thus, indistinguishability follows by opMAFE security. From here, we can follow the same sequence of hybrids to reach Hybrid 1 with $1 - b$.

We provide full proof in Appendix K. □

5.5 MAFE from FE

In this section, we construct a collusion resistant MAFE scheme satisfying Definition 5.1. We will use an MAFE scheme with batched key generation (Definition 5.3) and VNCE (Definition L.1). The idea essentially is that using VNCE, we can “flatten” the key generation queries as batch and rely on batched key generation security.

Construction 5.9. We provide the construction of an MAFE scheme for P/Poly circuits and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ using an MAFE scheme with batched key generation for P/Poly circuits (Definition 5.3) and $\mathcal{GID}$, and a VNCE scheme for $\mathcal{GID}$ (Definition L.1) as follows.

GSetup($1^\lambda, 1^n, 1^L, 1^s$). Sample $\text{mafe.crs} \leftarrow \text{abMAFE.GSetup}(1^\lambda, 1^n, 1^L, 1^s)$ and $\text{ncc.crs} \leftarrow \text{VNCE.GSetup}(1^\lambda, 1^n, 1^{L'})$ where $L' = \text{poly}(\lambda, n, s, L)$ is the maximum length of abMAFE secret keys for any authority. Output $\text{CRS} := (\text{mafe.crs}, \text{ncc.crs})$.

ASetup($\text{id}, 1^{\ell_{\text{id}}}$). Given $\text{id} \in [n], \ell_{\text{id}} \leq L$, sample abMAFE keys, $(\text{mafe.mpk}, \text{mafe.msk}) \leftarrow \text{abMAFE.ASetup}(\text{id}, 1^{\ell_{\text{id}}})$ and VNCE keys $(\text{ncc.mpk}, \text{ncc.msk}) \leftarrow \text{VNCE.ASetup}(\text{id})$. Output $\text{MPK} := (\text{mafe.mpk}, \text{ncc.mpk})$ and $\text{MSK} := (\text{mafe.msk}, \text{ncc.msk})$.

KGen($\text{id}, \text{MSK}_{\text{id}}, \{\text{MPK}_{\text{id}_x}\}_{\text{id}_x \in [n]}, \text{GID}, x$). Parse MSK_{id} as $(\text{mafe.msk}_{\text{id}}, \text{ncc.msk}_{\text{id}})$ and for each $\text{id}_x \in [n]$, MPK_{id_x} as $(\text{mafe.mpk}_{\text{id}_x}, \text{ncc.mpk}_{\text{id}_x})$.

Sample an abMAFE secret key $\text{mafe.sk}_{\text{id},\text{GID},x} \leftarrow \text{abMAFE.KGen}(\text{id}, \text{mafe.msk}_{\text{id}}, \{\text{mafe.mpk}_{\text{id}_x}\}_{\text{id}_x}, \text{GID}, x)$. Now, encrypt this string under VNCE, $\text{ncc.sk}_{\text{id},\text{GID},x} \leftarrow \text{VNCE.Enc}(\text{id}, \text{ncc.msk}_{\text{id}}, \{\text{ncc.mpk}_{\text{id}_x}\}_{\text{id}_x \in [n]}, \text{GID}, \text{mafe.sk}_{\text{id},\text{GID},x})$.

Output $\text{SK}_{\text{id},\text{GID},x} := \text{ncc.sk}_{\text{id},\text{GID},x}$.

Enc($\{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}, C$). Parse MPK_{id} as $(\text{mafe.mpk}_{\text{id}}, \text{nce.mpk}_{\text{id}})$ for each $\text{id} \in [n]$.

Sample an abMAFE ciphertext for C , $\text{mafe.ct} \leftarrow \text{abMAFE.Enc}(\{\text{mafe.mpk}_{\text{id}}\}_{\text{id} \in [n]}, C)$.

Output $\text{CT} := \text{mafe.ct}$.

Dec($\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}, \text{CT}$). Parse $\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}$ as $\text{nce.sk}_{\text{id}, \text{GID}, x_{\text{id}}}$ for each $\text{id} \in [n]$ and CT as mafe.ct .

Compute $\{\text{mafe.sk}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]} = \text{VNCE.Dec}(\{\text{nce.sk}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]})$. If this aborts, then abort and output $\perp$.

Otherwise, compute and output $y = \text{abMAFE.Dec}(\{\text{mafe.sk}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}, \text{mafe.ct})$.

Correctness. Correctness of Construction 5.9 follows from correctness of abMAFE and VNCE.

Theorem 5.10. If abMAFE is an MAFE scheme with batched key generation for P/Poly circuits and $\mathcal{GID}$ (Definition 5.3), and VNCE is a vector non-committing encryption scheme for $\mathcal{GID}$ (Definition L.1), then Construction 5.9 is an MAFE scheme (Definition 5.1) for P/Poly circuits and $\mathcal{GID}$.

Proof Sketch. We provide a sketch of the hybrid argument used to prove security of Construction 5.9 as follows.

Hybrid 0. This is the honest experiment with challenger with $b0/1$.

Hybrid 1. In this hybrid, we simulate the VNCE instantiation. This way, all secret keys will only be revealed after all the authorities are queried. Hybrids 0 and 1 are indistinguishable by VNCE security. item[Hybrid 2.] In this hybrid, we encrypt C_{1-b} using abMAFE. Indistinguishability of Hybrids 1 and 2 can be argued by adaptive batched key generation security of abMAFE.

Hybrid 3. This is honest experiment with $1 - b$. Hybrid 2 and 3 are indistinguishable by VNCE security.

We provide full proof in Appendix M. □

6 Collusion Resistant MAFE with Malicious Authorities

In this section we construct an MAFE scheme (recall by MAFE, we mean MAFE in the collusion-resistant setting) with malicious authorities. We consider the setting where k out of n authorities are malicious such that $\binom{n}{k} = \text{poly}(\lambda)$. Moreover, in our construction, we require the pre-challenge secret key queries to be bounded. We provide full definition in Section 6.1. Our construction uses HSS with strong simulation security and is provided in Section 6.2

6.1 Definition

We formally define an MAFE with malicious authorities where at most k out of n authorities can be malicious $((n, k)\text{-cMAFE})$ below.

Definition 6.1. An (n, k) -cMAFE scheme is syntactically almost same as MAFE scheme. Except, we now take a *pre-challenge collusion bound* Q , maximum number of malicious authorities k , maximum number of corrupt bits χ as input to the GSetup algorithm. Security is altered to accommodate malicious corruptions as follows.

IND Security. For any PPT admissible adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\left| \Pr \left[\begin{array}{l} b \leftarrow \mathcal{A}^{\mathcal{O}(\cdot)}(\text{CT}) : \\ \begin{array}{l} (Q, n, s, k, \chi, \ell_1, \dots, \ell_n) \leftarrow \mathcal{A}(1^\lambda), L = \max_{\text{id}} \{\ell_{\text{id}}\} \\ \text{CRS} \leftarrow \text{GSetup}(1^\lambda, 1^Q, 1^n, 1^s, 1^k, 1^\chi, 1^L), \\ \forall \text{id} \in [n], (\text{MPK}_{\text{id}}, \text{MSK}_{\text{id}}) \leftarrow \text{ASetup}(\text{id}, 1^{\ell_{\text{id}}}), \\ (C_0, C_1) \leftarrow \mathcal{A}^{\mathcal{O}(\cdot)}(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}), \\ b \stackrel{\$}{\leftarrow} \{0, 1\}, \text{CT} \leftarrow \text{Enc}(\{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}, C_b) \end{array} \end{array} \right] - \frac{1}{2} \right| \leq \text{negl}(\lambda)$$

where $\mathcal{O}$ is a stateful oracle that responds to two types of queries from $\mathcal{A}$

(KGen, (id, GID, x)). $\mathcal{A}$ sends $(\text{id}, \text{GID}, x) \in [n] \times \mathcal{GID} \times \{0, 1\}^{\ell_{\text{id}}}$ and receives $\text{SK}_{\text{id}, \text{GID}, x} \leftarrow \text{KGen}(\text{id}, \text{MSK}_{\text{id}}, \{\text{MPK}_{\text{id}_x}\}_{\text{id}_x \in [n]}, \text{GID}, x)$ from $\mathcal{O}$.

(Corr, id). $\mathcal{A}$ sends $\text{id} \in [n]$. $\mathcal{O}$ responds with MSK_{id} .

A stateful PPT machine $\mathcal{A}$ is said to be an admissible adversary if $|C_0| = |C_1|$ and are both of size at most s and for each $(\text{GID}, X) \in \mathcal{V}$, $C_0(X) = C_1(X)$. Here $\mathcal{V} = \{(\text{GID}, X) : X \in \mathbf{X}_{\text{GID}, 1} \times \dots \times \mathbf{X}_{\text{GID}, n} \text{ for every GID that was used to query all authorities. If id-th authority corrupted, then } \mathbf{X}_{\text{GID}, \text{id}} = \{0, 1\}^{\ell_{\text{id}}}. \text{ Otherwise, } \mathbf{X}_{\text{GID}, \text{id}} \text{ is a singleton } x \text{ corresponding to the query } (\text{KGen}, (\text{id}, \text{GID}, x))\}. \text{ Moreover, } \mathcal{V}_{\text{pre}} \subseteq \mathcal{V} \text{ is the set of pre-challenge queries for a particular GID for which all non-malicious authorities are queried. Then, } |\mathcal{V}_{\text{pre}}| \leq Q^{15}.$

6.2 Construction

Construction 6.2. We provide the construction of a (n, k) -cMAFE scheme for P/Poly circuits and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ (Definition 6.1) using MAFE scheme for P/Poly circuits and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ (Definition 5.1) and a HSS scheme with strong security for P/Poly circuits (Definition F.1), and a SKE scheme (Definition A.6) as follows.

GSetup($1^\lambda, 1^Q, 1^n, 1^s, 1^k, 1^\chi, 1^L$). Let $\mathbb{I} = \left[\binom{n}{k}\right]$ and $\mathbb{J} = [2^\chi]$. We will assume some fixed ordering of $2^{\left[\binom{n}{k}\right]}_k$ and say $\mathcal{K}_i \in 2^{\left[\binom{n}{k}\right]}_k$ for $i \in \mathbb{I}$. Let ℓ_{out} represent the SKE encryption size of a message of output length of $\mathcal{C}_s$. For each $i \in \mathbb{I}, j \in \mathbb{J}$, instantiate an MAFE,

$$\forall i \in \mathbb{I}, j \in \mathbb{J}, \text{crs}^{(i, j)} \leftarrow \text{MAFE.GSetup}(1^\lambda, 1^{n-k}, 1^s, 1^{L+\lambda+\ell_{\text{out}}})$$

We assume w.l.o.g that for any $i \in \mathbb{I}$, the authority identifiers are elements of the set $[n] \setminus \mathcal{K}_i$, i.e, for the (i, j) -th instantiation of MAFE, $\text{id} \notin \mathcal{K}_i$. Output $\text{CRS} := (\text{crs}^{(i, j)})_{i \in \mathbb{I}, j \in \mathbb{J}}$.

ASetup(id, $1^{\ell_{\text{id}}}$). For each $i \in \mathbb{I}, j \in \mathbb{J}$, generate master public and secret keys for the (i, j) -th instantiation of MAFE if $\text{id} \notin \mathcal{K}_i$, $(\text{mpk}^{(i, j)}, \text{msk}^{(i, j)}) \leftarrow \text{MAFE.ASetup}(\text{id}, 1^{\ell_{\text{id}}+\lambda+\ell_{\text{out}}})$. Output $\text{MPK} := (\text{mpk}^{(i, j)})_{i, j}$ and $\text{MSK} := (\text{msk}^{(i, j)})_{i, j}$.

¹⁵Note that $\mathcal{A}$ can still make unbounded pre-challenge queries. It just makes Q queries with a unique GID each such that for this GID all non-malicious authorities are queried.

KGen(id, MSK_{id}, {MPK_{idx}}_{idx ∈ [n]}, GID, x). Parse MSK_{id} as (mafe.msk_{id}^(i,j))_{i ∈ ℐ, j ∈ ℐ}. For each idx ∈ [n],

parse MPK_{idx} as (mafe.mpk_{idx}^(i,j))_{i ∈ ℐ, j ∈ ℐ}.

Note that each mafe.mpk_{idx}^(i,j) implicitly consists of information about ℓ_{idx} . Using $(\ell_1, \dots, \ell_n)$, define a corruptible set $\mathcal{K}' \subseteq [n]$ such that $\ell_{\text{id}} = O(\log \lambda)$ for $\text{id} \in \mathcal{K}'$.

Let $\mathbb{I}' = \{i : i \in \mathbb{I}, \mathcal{K}_i \subseteq \mathcal{K}', \sum_{\text{id} \in \mathcal{K}_i} \ell_{\text{id}} \leq \chi\}$. Let $\mathbb{I}'_{\text{id}} = \{i : i \in \mathbb{I}', \text{id} \notin \mathcal{K}_i\}$. For every $i \in \mathbb{I}'_{\text{id}}, j \in \mathbb{J}$ sample $r_{i,j} \xleftarrow{\$} \{0, 1\}^\lambda$, $\text{ske.ct}_{ij}^* \xleftarrow{\$} \{0, 1\}^{\ell_{\text{out}}}$ where ℓ_{out} is the output size of circuits in $\mathcal{C}_s$ and

$$\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \text{MAFE.KGen}(\text{id}, \text{msk}_{\text{id}}^{(i,j)}, \{\text{mpk}_{\text{idx}}^{(i,j)}\}_{\text{idx}}, \text{GID}, (x, r_{i,j}, \text{ske.ct}_{ij}^*))$$

Output $\text{SK}_{\text{id}, \text{GID}, x} := (\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)})_{i,j}$.

Enc({MPK_{id}}_{id ∈ [n]}, C). Parse MPK_{id} as (mpk_{id}^(i,j))_{i ∈ ℐ, j ∈ ℐ} for each id ∈ [n]. Parse $\mathcal{K}'$ similar to KGen.

Share the circuit C using HSS for all $i \in \mathbb{I}'$, $\{\tilde{C}_i\}_{i \in \mathbb{I}'} \leftarrow \text{HSS.Share}(1^\lambda, 1^{|\mathbb{I}'|}, C)$. For each $i \in \mathbb{I}'$, encrypt the circuit F_{ij} from Figure 4 after padding it to the size of circuit from Figure 10.

That is, for every $i \in \mathbb{I}', j \in \mathbb{J}$, $\text{ct}^{(i,j)} \leftarrow \text{MAFE.Enc}(\{\text{mpk}_{\text{id}}^{(i,j)}\}_{\text{id} \notin \mathcal{K}_i}, F_{ij})$. Output $\text{CT} := (\text{ct}^{(i,j)})_{i \in \mathbb{I}', j \in \mathbb{J}}$.

Dec({SK_{id, GID, x_{id}}}_{id ∈ [n]}, CT). Parse CT as (mafe.ct^(i,j))_{i ∈ ℐ', j ∈ ℐ} and $\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}$ as (mafe.sk_{id, GID, x_{id}}^(i,j))_{i,j}

for each id ∈ [n]. Let $x = (x_1, \dots, x_n)$. For each $i \in \mathbb{I}'$, partition x as $\{x_{\text{id}}\}_{\text{id} \in \mathcal{K}_i}$ and $\{x_{\text{id}}\}_{\text{id} \notin \mathcal{K}_i}$. Observe that we have secret keys for all $\{x_{\text{id}}\}_{\text{id} \notin \mathcal{K}_i}$. Decrypt the $(i, j = \{x_{\text{id}}\}_{\text{id} \notin \mathcal{K}_i})$ -th instantiation of MAFE, $y_{i,j} = \text{MAFE.Dec}(\{\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)}\}_{\text{id} \notin \mathcal{K}_i}, \text{ct}^{(i,j)})$. Output $y := \text{HSS.Retrieve}(\{y_{i,j}\}_{i \in \mathbb{I}'})$.

Circuit F_{ij}
Input: $x_1, r_1, \text{ske.ct}_1^*, \dots, x_{n-k}, r_{n-k}, \text{ske.ct}_{n-k}^*$ Hardwired: $i, j, \tilde{C}_i$ Output: Parse $x_1, \dots, x_{n-k}$ as $(\ell_1 + \dots + \ell_n) - \chi$ bits of input for $\{x_{\text{id}}\}_{\text{id} \notin \mathcal{K}_i}$ and set $\{x_{\text{id}}\}_{\text{id} \in \mathcal{K}_i} = j$ (truncations done appropriately). Compute and output $\text{HSS.Eval}(i, \tilde{C}_i, x_1, \dots, x_n)$.

Figure 4: Description of circuit F_{ij}

Correctness. The correctness of the scheme follows from the correctness of MAFE and HSS schemes. In particular, $\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}} = (\text{sk}_{\text{id}, \text{GID}, x_{\text{id}}}^{(i,j)})_{i \in \mathbb{I}'_{\text{id}}, j \in \mathbb{J}}$ and $\text{CT} = \{\text{MAFE.Enc}(\{\text{mpk}_{\text{id}}^{(i,j)}\}_{\text{id} \notin \mathcal{K}_i}, \text{HSS.Eval}(i, \tilde{C}_i, \cdot, j))\}_{i \in \mathbb{I}', j \in \mathbb{J}}$. By correctness of $(i, j = \{x_{\text{id}}\}_{\text{id} \notin \mathcal{K}_i})$ -th instantiation of MAFE, $y_{i,j} = \text{MAFE.Dec}(\{\text{sk}_{\text{id}, \text{GID}, x_{\text{id}}}^{(i,j)}\}_{\text{id} \notin \mathcal{K}_i}, \text{ct}^{(i,j)}) = \text{HSS.Eval}(i, \tilde{C}_i, x = (x_1, \dots, x_n))$. Thus, by correctness of HSS, $y = C(x_1, \dots, x_n)$.

Theorem 6.3. If HSS is a secure HSS scheme for P/Poly circuits (Definition F.1), MAFE is a secure MAFE scheme for P/Poly circuits and $\mathcal{GID}$ (Definition 5.1), SKE with pseudorandom ciphertexts (Definition A.6), then Construction 6.2 is a secure (n, k) -cMAFE scheme for P/Poly circuits and $\mathcal{GID}$ (Definition 6.1).

Proof Sketch. We provide a sketch of the hybrid argument used to prove security of Construction 6.2 as follows.

Hybrid 0. This is the honest experiment from Definition 6.1 with $b = 0/1$.

Hybrid 1. In this hybrid, we sample $i^* \xleftarrow{\$} \mathbb{I}'$ and abort if any $\text{id} \notin \mathcal{K}_{i^*}$ is corrupted.

Hybrid 2. In this hybrid, for each GID query, we will sample ρ_{i^*j} and set $r_{i^*j}^{(\text{id})}$ used in secret key generation for id -th authority to be such that $\bigoplus_{\text{id}} r_{i^*j}^{(\text{id})} = \rho_{i^*j}$ and also ensure ρ_{i^*j} for each GID and j are unique.

Hybrid 3. In this hybrid, we will set $\text{ske.ct}_{i^*j}^*$ for each id to be encryptions of additive secret shares of $\text{HSS.Eval}(i^*, \tilde{C}_{i^*}, \{\{x_{\text{id}}\}_{\text{id} \notin \mathcal{K}_{i^*}}, j\})$. That is, for a specific GID, $\text{ske.ct}_1^*, \dots, \text{ske.ct}_n^*$ are encryptions of $\text{sh}_1, \dots, \text{sh}_n$ such that $\bigoplus_{\text{id}} \text{sh}_{\text{id}} = \text{HSS.Eval}(i^*, \tilde{C}_{i^*}, \{\{x_{\text{id}}\}_{\text{id} \notin \mathcal{K}_{i^*}}, j\})$. Indistinguishability follows by SKE security.

Hybrid 4, κ . For $\kappa \in [|\mathbb{J}| + 1]$. In this hybrid, we will encrypt a circuit F'_{i^*j} for $j \in [\kappa - 1]$ instantiations of MAFE. Here F'_{i^*j} decrypts ske.ct^* and uses r_{id} to output values instead of using $\tilde{C}_{i^*}$. The indistinguishability of hybrids can be argued using the security of MAFE.

Hybrid 5. In this hybrid, we will simulate the HSS instantiation. Now that all MAFE instantiations for i^* (after correctly guessing at the beginning) do not use $\tilde{C}_{i^*}$, we would only be using the outputs of share evaluation. Thus, we can readily rely on the strong security of HSS to show indistinguishability. In this hybrid, $\mathcal{A}$ has 0 advantage.

The full proof is in Appendix N. □

7 MA-ABE from Witness Encryption

In this section, we will show how to generically transform the 1-authority ABE scheme for general predicates from [WW24] which was constructed using witness encryption [GGSW13] into an MA-ABE scheme. We provide the definitions for preliminaries used in our construction, WE, COM, and NIZK in Section A and construct the scheme in Section 7.2.

7.1 Definitions

In this section, we provide definitions of various flavors MA-ABE schemes.

Definition 7.1 (MA-ABE). An MA-ABE scheme is an MAFE scheme for the circuit class $\mathcal{C} = \{\mathcal{P}_s\}_{s \in \mathbb{N}} \times \{0, 1\}^*$. That is, encryption is done w.r.to a policy $P \in \mathcal{P}_s$ and a message $m \in \{0, 1\}^*$ where the circuit's output for input $(x_1, \dots, x_n)$ is m if and only if $P(x_1, \dots, x_n) = 1$. Security requires to hide m for all inputs such $P(x_1, \dots, x_n) = 1$. Specifically, a MA-ABE scheme $(\text{GSetup}, \text{ASetup}, \text{KGen}, \text{Enc}, \text{Dec})$ is secure if for any PPT admissible adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\left| \Pr \left[\begin{array}{l} (n, s, L) \leftarrow \mathcal{A}(1^\lambda), L = \max_{\text{id}} \{\ell_{\text{id}}\} \\ \text{CRS} \leftarrow \text{GSetup}(1^\lambda, 1^n, 1^s, 1^L), \\ b \leftarrow \mathcal{A}^{\mathcal{O}(\cdot, \cdot)}(\text{CT}) : \forall \text{id} \in [n], (\text{MPK}_{\text{id}}, \text{MSK}_{\text{id}}) \leftarrow \text{ASetup}(\text{id}, 1^{\ell_{\text{id}}}), \\ (P, m_0, m_1) \leftarrow \mathcal{A}^{\mathcal{O}(\cdot, \cdot)}(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}), \\ b \stackrel{\$}{\leftarrow} \{0, 1\}, \text{CT} \leftarrow \text{Enc}(\{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}, P, m_b) \end{array} \right] - \frac{1}{2} \right| \leq \text{negl}(\lambda)$$

where $\mathcal{O}$ is a stateful oracle that responds to two types of queries from $\mathcal{A}$

(KGen, (id, GID, x)). $\mathcal{A}$ sends $(\text{id}, \text{GID}, x) \in [n] \times \mathcal{GID} \times \{0, 1\}^{\ell_{\text{id}}}$ and receives $\text{SK}_{\text{id}, \text{GID}, x} \leftarrow \text{KGen}(\text{id}, \text{MSK}_{\text{id}}, \{\text{MPK}_{\text{id}_x}\}_{\text{id}_x \in [n]}, \text{GID}, x)$ from $\mathcal{O}$.

(Corr, id). $\mathcal{A}$ sends $\text{id} \in [n]$. $\mathcal{O}$ responds with MSK_{id} .

A stateful PPT machine $\mathcal{A}$ is said to be an admissible adversary if $P \in \mathcal{P}_s, |m_0| = |m_1|$ and for each $(\text{GID}, X) \in \mathcal{V}$, $P(X) = 0$. Here $\mathcal{V} = \{(\text{GID}, X) : X \in \mathbf{X}_{\text{GID}, 1} \times \dots \times \mathbf{X}_{\text{GID}, n} \text{ for every GID that was used to query all authorities. If id-th authority corrupted, then } \mathbf{X}_{\text{GID}, \text{id}} = \{0, 1\}^{\ell_{\text{id}}}. \text{ Otherwise, } \mathbf{X}_{\text{GID}, \text{id}} \text{ is a singleton } x \text{ corresponding to the query } (\text{KGen}, (\text{id}, \text{GID}, x))\}.$

Definition 7.2 (MA-ABE without malicious authorities). An MA-ABE scheme $(\text{GSetup}, \text{ASetup}, \text{KGen}, \text{Enc}, \text{Dec})$ is said to be an MA-ABE scheme without malicious authorities if in Definition 7.1, the oracle $\mathcal{O}$ responds with $\perp$ for any (Corr, id) queries from $\mathcal{A}$. That is, no malicious authorities are allowed.

Definition 7.3 (MA-ABE with batched key generation). An MA-ABE scheme $(\text{GSetup}, \text{ASetup}, \text{KGen}, \text{Enc}, \text{Dec})$ is said to be an MA-ABE with batched key generation setting if Definition 7.2 is secure in the relaxed batched key generation security defined analogously to Definition 5.3.

7.2 MA-ABE with Batched Key Generation

Construction 7.4. We provide the construction of an MA-ABE scheme for any policy class $\{\mathcal{P}_s\}_{s \in \mathbb{N}}$ and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ (Definition 7.3) using a NIZK scheme (Definition A.10) for $\mathcal{L}_{\text{NIZK}}$ (Figure 5), a WE scheme (Definition A.9) for $\mathcal{L}_{\text{WE}}$ (Figure 6), and a statistically binding commitment scheme COM (Definition A.8), and a distributed functional tag system dFTS for policy class $\mathcal{P}$ and $\mathcal{GID}$ (Definition P.1) as follows.

GSetup($1^\lambda, 1^n, 1^s, 1^L$). Sample a crs for COM, $\text{com.crs} \leftarrow \text{COM.Setup}(1^\lambda)^{16}$ and a crs for dFTS, $\text{fts.crs} \leftarrow \text{dFTS.GSetup}(1^\lambda, 1^n, 1^s, 1^L)$.

Output $\text{CRS} = (n, s, L, \text{com.crs}, \text{fts.crs})$.

ASetup(id, $1^{\ell_{\text{id}}}$). Sample a commitment for 0 with randomness $r^{(0)}$, $\text{com}^{(0)} \leftarrow \text{COM.Com}(1^\lambda, 0; r^{(0)})$ and commitment for 0^λ with randomness $r^{(1)}$, $\text{com}^{(1)} \leftarrow \text{Com}(1^\lambda, 0^\lambda; r^{(1)})$. Sample a crs for NIZK for language $\mathcal{L}_{\text{NIZK}}$ (Figure 5), $\text{nizk.crs} \leftarrow \text{NIZK.Setup}(1^\lambda)$.

Output $\text{MPK} := (\text{com}^{(0)}, \text{com}^{(1)}, \text{nizk.crs})$ and $\text{MSK} := r^{(0)}$.

¹⁶We remark that having a common commitment crs is not required for our construction. This choice is used to reduce the complexity of security hybrids. Indeed each authority can sample it's own commitment crs and the scheme still works.

Language $\mathcal{L}_{\text{NIZK}}$
Instance: $n, s, L, \text{com.crs}, \text{fts.crs}, \text{id}, \text{GID}, x, \text{com}^{(0)}, \text{com}^{(1)}, \text{tg}_{\text{id}}$ Witness: $(r^{(0)}, r^{(1)}, r^*, r_{\text{tg}}, \{(\text{idx}, \text{GID}_{\text{idx}}, x_{\text{idx}}, \text{tg}_{\text{idx}})\}_{\text{idx} \neq \text{id}})$ Relation: Output 1 if and only if $\phi_1 \vee (\phi_2 \wedge \phi_3 \wedge \phi_4) = 1$ where, $\phi_1 = \text{Com}(\text{com.crs}, 0; r^{(0)}) \stackrel{?}{=} \text{com}^{(0)}$ $\phi_2 = \text{Com}(\text{com.crs}, r^*; r^{(1)}) \stackrel{?}{=} \text{com}^{(1)}$ $\phi_3 = \text{GID}$ and $\{\text{GID}_{\text{idx}}\}_{\text{idx} \neq \text{id}}$ are all same. $\phi_4 = (\text{tg}_i)_{i \in [n]} \stackrel{?}{=} \text{dFTS.SlnpTag}(\text{dFTS.KGen}(\text{fts.crs}; r^*), (i, \text{GID}, x_i)_{i \in [n]}; r_{\text{tg}})$ Here, F is a circuit that checks if all GID_{idx} are same as GID and then executes $P(x_1, \dots, x_n)$.

Figure 5: Description of $\mathcal{L}_{\text{NIZK}}$

Language $\mathcal{L}_{\text{WE}}$
Instance: $n, s, L, \text{com.crs}, \text{fts.crs}, P, \text{tg}_P, (\text{com}_1^{(0)}, \text{com}_1^{(1)}, \text{nizk.crs}_1), \dots, (\text{com}_n^{(0)}, \text{com}_n^{(1)}, \text{nizk.crs}_n)$ Witness: $\{(\text{id}, \text{GID}_{\text{id}}, x_{\text{id}}, \text{tg}_{\text{id}}, \pi_{\text{id}})\}_{\text{id} \in [n]}$ Relation: Output 1 if and only if $\forall \text{id} \in [n], \text{NIZK.Verify}(\text{nizk.crs}_{\text{id}}, (n, s, L, \text{com.crs}, \text{fts.crs}, \text{id}, \text{GID}_{\text{id}}, x_{\text{id}}, \text{com}_{\text{id}}^{(0)}, \text{com}_{\text{id}}^{(1)}, \text{tg}_{\text{id}}), \pi_{\text{id}}) = 1$ GID_{id} is same for all $\text{id} \in [n]$. $P(x_1, \dots, x_n) = 1$ $\text{dFTS.Trig}(\text{tg}_P, (\text{tg}_{\text{id}})_{\text{id} \in [n]}) = 0$.

Figure 6: Description of $\mathcal{L}_{\text{WE}}$

$\text{KGen}(\text{id}, \text{MSK}_{\text{id}}, \{\text{MPK}_{\text{idx}}\}_{\text{idx} \in [n]}, \text{GID}, x)$. Parse MSK_{id} as $r_{\text{id}}^{(0)}$ and MPK_{idx} as $(\text{com}_{\text{idx}}^{(0)}, \text{com}_{\text{idx}}^{(1)}, \text{nizk.crs}_{\text{idx}})$ for each $\text{idx} \in [n]$.

Generate dummy tag, $\text{tg}_{\text{id}, \text{GID}, x} \leftarrow \text{dFTS.DlnpTag}(\text{id}, \text{GID}, x)$. Generate a NIZK proof, $\pi_{\text{id}} \leftarrow \text{NIZK.Prove}(\text{nizk.crs}_{\text{id}}, (\text{CRS}, \text{id}, \text{GID}, x, \text{com}^{(0)}, \text{com}^{(1)}), (r_{\text{id}}^{(0)}, \bar{0}))^{17}$.

Output $\text{SK}_{\text{id}, \text{GID}, x} := (\text{id}, \text{GID}, x, \text{tg}_{\text{id}, \text{GID}, x}, \pi_{\text{id}})$.

$\text{Enc}(\{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}, P, m)$. Parse MPK_{id} as $(\text{com}_{\text{id}}^{(0)}, \text{com}_{\text{id}}^{(1)}, \text{nizk.crs}_{\text{id}})$ for each $\text{id} \in [n]$. Sample dummy function tag $\text{tg}_P \leftarrow \text{dFTS.DFuncTag}(P)$.

Generate a WE ciphertext for language $\mathcal{L}_{\text{WE}}$ (Figure 6), message m , $\text{we.ct} \leftarrow \text{WE.Enc}(1^\lambda, (\text{CRS}, P, \text{tg}_P, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}), m)$. Output $\text{CT} := (\text{we.ct}, P, \text{tg}_P)$.

$\text{Dec}(\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}\}_{\text{id} \in [n]}, \text{CT})$. Parse CT as $(\text{we.ct}, P, \text{tg}_P)$ and $\forall \text{id} \in [n]$, $\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}$ as $(\text{id}, \text{GID}_{\text{id}}, x_{\text{id}}, \text{tg}_{\text{id}}, \pi_{\text{id}})$.

If GID_{id} are not same for $\text{id} \in [n]$ or $P(x_1, \dots, x_n) = 0$, abort and output $\perp$. Otherwise, decrypt using WE, $m' = \text{WE.Dec}(\text{we.ct}, \{(\text{id}, \text{GID}_{\text{id}}, x_{\text{id}}, \text{tg}_{\text{id}}, \pi_{\text{id}})\}_{\text{id}})$. Output m' .

Correctness. The correctness of Construction 7.4 follows from the correctness of WE, completeness of NIZK, dummy correctness of dFTS, and COM. In particular, from the completeness of COM

¹⁷By $\bar{0}$, we denote an all zero string of sufficient length.

and NIZK, we have that $\forall \text{id} \in [n]$, $\text{NIZK.Verify}(\text{nizk.crs}_{\text{id}}, (\text{CRS}, \text{id}, \text{GID}_{\text{id}}, x_{\text{id}}, \text{tg}_{\text{id}}, \text{com}_{\text{id}}^{(0)}, \text{com}_{\text{id}}^{(1)}), \pi_{\text{id}}) = 1$ as $\text{com}_{\text{id}}^{(0)} = \text{COM.Com}(\text{com.crs}, 0; r_{\text{id}}^{(0)})$. Additionally, when all secret keys correspond to the same GID and by dummy correctness of dFTS, $\text{dFTS.Trig}(\text{tg}_P, (\text{tg}_1, \dots, \text{tg}_n)) = 0$. So, $(\text{CRS}, P, \text{tg}_P, \text{MPK}_1, \dots, \text{MPK}_n)$ is a valid instance of $\mathcal{L}_{\text{WE}}$. Hence, by the correctness of WE, $m' = m$.

Theorem 7.5. If NIZK is a NIZK scheme for language $\mathcal{L}_{\text{NIZK}}$ (Definition A.10), COM is a statistically binding commitment scheme (Definition A.8), dFTS is a distributed functional tag system for $\mathcal{P}, \mathcal{GID}$ (Definition P.1) and WE is a witness encryption scheme for language $\mathcal{L}_{\text{WE}}$ (Definition A.9), then Construction 7.4 is an MA-ABE scheme (Definition 7.3) for $\mathcal{P} = \{\mathcal{P}_s\}_{s \in \mathbb{N}}, \mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ with batched key generation.

Proof Sketch. We provide a sketch of the hybrid argument used to prove security of Construction 7.4 as follows.

Hybrid 0. This is the honest experiment with challenger with $b = 0/1$.

Hybrid 1. In this hybrid, we will switch from using dummy tags to smart tags for the distributed functional tag system. Indistinguishability can be argued using mode indistinguishability property of dFTS.

Hybrid 2 In this hybrid, we commit to fts.msk sampled in the previous hybrid in all of $\text{com}_{\text{id}}^{(1)}$. Hybrids 1 and 2 are indistinguishable by hiding property of COM.

Hybrid 3, j . for $j \in [n+1]$. In this hybrid, we simulate all NIZK proofs generated in partial secret key generation for all $\text{id} < j$. Indistinguishability is argued by NIZK zero-knowledge property.

Hybrid 4, j for $j \in [n+1]$. In this hybrid, we switch to using alternate witnesses all NIZK proofs generated in partial secret key generation for all $\text{id} < j$, i.e, use $(r^{(1)}, r^*, r_{\text{tg}}, \dots)$ as witness in each NIZK proof. Indistinguishability is argued by NIZK zero-knowledge property.

Hybrid 5. In this hybrid, we set $\text{com}_{\text{id}}^{(0)}$ to commit to 1 for each authority. Hybrid 4, $n+1$ and Hybrid 5 are indistinguishable by commitment hiding property.

Hybrid 6. Now, WE instance is unsatisfiable. So, we can commit an all zero string. $\mathcal{A}$ has 0 advantage in this experiment.

We provide full proof in Appendix Q. □

7.3 MA-ABE from WE

In this section, we remove the batched key generation restriction from previous section and construct an MA-ABE scheme that satisfies Definition 7.2 from witness encryption and NIZKs. We remark that the compiler use VNCE and resembles the compiler from Section 5.5. We provide full construction in Appendix R. Proof of the following theorem is immediate and provided in Appendix R.

Theorem 7.6. If abMA-ABE is an MA-ABE scheme with batched key generation for P/Poly circuits and $\mathcal{GID}$ (Definition 7.3), and VNCE is a vector non-committing encryption scheme for $\mathcal{GID}$ (Definition L.1), then Construction R.1 is an MA-ABE scheme (Definition 7.2) for P/Poly circuits and $\mathcal{GID}$.

7.4 MA-ABE with Malicious Authorities

In this section, we construct an MA-ABE scheme in which the adversary can adaptively corrupt k out of n authorities as long as $\binom{n}{k} = \text{poly}(\lambda)$ and the number of corrupted bits are $O(\log \lambda)$. The idea is to use the same compiler as our **abe** schemes. However, we do not need to rely on HSS with strong simulation security as we do not argue for hiding the policy P . We simply encrypt additive secret shares of message m and as one of the shares remain hidden (by MA-ABE security), we can hide m easily. Full construction is provided below.

Construction 7.7. We provide the construction of an MA-ABE scheme for P/Poly circuits and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ (Definition 7.1) using an MA-ABE scheme for P/Poly circuits and $\mathcal{GID}$ (Definition 7.2) as follows.

GSetup($1^\lambda, 1^n, 1^s, 1^k, 1^\chi, 1^L$). Let $\mathbb{I} = \left[\binom{n}{k}\right]$ and $\mathbb{J} = [2^\chi]$. We will assume some fixed ordering of $2^{[n]}|_k$ and say $\mathcal{K}_i \in 2^{[n]}|_k$ for $i \in \mathbb{I}$. For each $i \in \mathbb{I}, j \in \mathbb{J}$, instantiate an MA-ABE,

$$\forall i \in \mathbb{I}, j \in \mathbb{J}, \text{crs}^{(i,j)} \leftarrow \text{MA-ABE.GSetup}(1^\lambda, 1^{n-k}, 1^{s'}, 1^L)$$

We assume w.l.o.g that for any $i \in \mathbb{I}$, the authority identifiers are elements of the set $[n] \setminus \mathcal{K}_i$, i.e, for the (i, j) -th instantiation of MA-ABE, $\text{id} \notin \mathcal{K}_i$. Output $\text{CRS} := (\text{crs}^{(i,j)})_{i \in \mathbb{I}, j \in \mathbb{J}}$.

ASetup($\text{id}, 1^{\ell_{\text{id}}}$). For each $i \in \mathbb{I}, j \in \mathbb{J}$, generate master public and secret keys for the (i, j) -th instantiation of MA-ABE if $\text{id} \notin \mathcal{K}_i$, $(\text{mpk}^{(i,j)}, \text{msk}^{(i,j)}) \leftarrow \text{MA-ABE.ASetup}(\text{id}, 1^{\ell_{\text{id}}})$. Output $\text{MPK} := (\text{mpk}^{(i,j)})_{i,j}$ and $\text{MSK} := (\text{msk}^{(i,j)})_{i,j}$.

KGen($\text{id}, \text{MSK}_{\text{id}}, \{\text{MPK}_{\text{id}_x}\}_{\text{id}_x \in [n]}, \text{GID}, x$). Parse MSK_{id} as $(\text{msk}_{\text{id}}^{(i,j)})_{i \in \mathbb{I}, j \in \mathbb{J}}$. For each $\text{id}_x \in [n]$, parse MPK_{id_x} as $(\text{mpk}_{\text{id}_x}^{(i,j)})_{i \in \mathbb{I}, j \in \mathbb{J}}$.

Note that each $\text{abe.mpk}_{\text{id}_x}^{(i,j)}$ implicitly consists of information about ℓ_{id_x} . Using $(\ell_1, \dots, \ell_n)$, define a corruptible set $\mathcal{K}' \subseteq [n]$ such that $\ell_{\text{id}} = O(\log \lambda)$ for $\text{id} \in \mathcal{K}'$.

Let $\mathbb{I}' = \{i : i \in \mathbb{I}, \mathcal{K}_i \subseteq \mathcal{K}', \sum_{\text{id} \in \mathcal{K}_i} \ell_{\text{id}} \leq \chi\}$. Let $\mathbb{I}'_{\text{id}} = \{i : i \in \mathbb{I}', \text{id} \notin \mathcal{K}_i\}$. For every $i \in \mathbb{I}'_{\text{id}}, j \in \mathbb{J}$,

$$\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \text{MA-ABE.KGen}(\text{id}, \text{msk}_{\text{id}}^{(i,j)}, \{\text{mpk}_{\text{id}_x}^{(i,j)}\}_{\text{id}_x}, \text{GID}, x)$$

Output $\text{SK}_{\text{id}, \text{GID}, x} := (\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)})_{i,j}$.

Enc($\{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}, P, m$). Parse MPK_{id} as $(\text{mpk}_{\text{id}}^{(i,j)})_{i \in \mathbb{I}, j \in \mathbb{J}}$ for each $\text{id} \in [n]$. Parse $\mathcal{K}'$ similar to

KGen. Sample $\iota \xleftarrow{\$} [\mathbb{I}']$, and $\{\text{sh}_\alpha\}_{\alpha \in [\mathbb{I}'], \alpha \neq \iota} \xleftarrow{\$} \{0, 1\}^{|m|}$ and set $\text{sh}_\iota = (\bigoplus_\alpha \text{sh}_\alpha) \oplus m$. For each $i \in \mathbb{I}'$, encrypt the circuit F_{ij} where this circuit takes $(\ell_1 + \dots + \ell_n) - \chi$ bits as input and sets $\{x_{\text{id}}\}_{\text{id} \in \mathcal{K}_i} = j$ (truncations done appropriately) and computes $P(x_1, \dots, x_n)$.

That is, for every $i \in \mathbb{I}', j \in \mathbb{J}$, $\text{ct}^{(i,j)} \leftarrow \text{MA-ABE.Enc}(\{\text{mpk}_{\text{id}}^{(i,j)}\}_{\text{id} \notin \mathcal{K}_i}, F_{ij}, \text{sh}_i)$. Output $\text{CT} := (\text{ct}^{(i,j)})_{i \in \mathbb{I}', j \in \mathbb{J}}$.

Dec($\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}, \text{CT}$). Parse CT as $(\text{ct}^{(i,j)})_{i \in \mathbb{I}', j \in \mathbb{J}}$ and $\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}$ as $(\text{sk}_{\text{id}, \text{GID}, x_{\text{id}}}^{(i,j)})_{i,j}$ for each $\text{id} \in [n]$. Let $x = (x_1, \dots, x_n)$. For each $i \in \mathbb{I}'$, partition x as $\{x_{\text{id}}\}_{\text{id} \in \mathcal{K}_i}$ and $\{x_{\text{id}}\}_{\text{id} \notin \mathcal{K}_i}$. Observe that we have secret keys for all $\{x_{\text{id}}\}_{\text{id} \notin \mathcal{K}_i}$. Decrypt the $(i, j = \{x_{\text{id}}\}_{\text{id} \notin \mathcal{K}_i})$ -th instantiation of MA-ABE, $\text{sh}_{i,j} = \text{MA-ABE.Dec}(\{\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)}\}_{\text{id} \notin \mathcal{K}_i}, \text{ct}^{(i,j)})$. Output $m := \bigoplus_{i \in \mathbb{I}'} \text{sh}_{i,j}$.

Correctness. The correctness of the scheme follows from the correctness of MA-ABE. In particular, $\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}} = (\text{sk}_{\text{id}, \text{GID}, x_{\text{id}}}^{(i,j)})_{i \in \mathbb{I}'_{\text{id}}, j \in \mathbb{J}}$ and $\text{CT} = \{\text{MA-ABE.Enc}(\{\text{mpk}_{\text{id}}^{(i,j)}\}_{\text{id} \notin \mathcal{K}_i}, F_{ij}, \text{sh}_i)\}_{i \in \mathbb{I}', j \in \mathbb{J}}$. By correctness of $(i, j = \{x_{\text{id}}\}_{\text{id} \notin \mathcal{K}_i})$ -th instantiation of MA-ABE, $\text{sh}_{i,j} = \text{MA-ABE.Dec}(\{\text{sk}_{\text{id}, \text{GID}, x_{\text{id}}}^{(i,j)}\}_{\text{id} \notin \mathcal{K}_i}, \text{ct}^{(i,j)}) = \text{sh}_i$ if $P(x_1, \dots, x_n) = 1$. Thus, $m = \bigoplus_{i \in \mathbb{I}'} \text{sh}_i$.

Theorem 7.8. If MA-ABE is a secure MA-ABE scheme for P/Poly circuits and $\mathcal{GID}$ with honest authorities (Definition 7.2), then Construction 7.7 is a secure MA-ABE scheme for P/Poly circuits and $\mathcal{GID}$ with malicious authorities (Definition 7.1).

Proof Sketch. We provide a sketch of the hybrid argument used to prove security of Construction 7.7 as follows.

Hybrid 0. This is the honest experiment from Definition 7.1 with $b = 0/1$.

Hybrid 1, κ . For $\kappa \in [|\mathbb{J}| + 1]$. In this hybrid, we will randomly guess $i^* \xleftarrow{\$} \mathbb{I}$ and encrypt a random string in the $(i^*, 1), \dots, (i^*, \kappa - 1)$ instantiations of MA-ABE. The indistinguishability of hybrids can be argued using the security of MA-ABE. In addition, if the adversary corrupts any id that is not in the i^* -th element of $\mathbb{I}$, we abort. The indistinguishability is argued using a two fold argument – if we correctly guess the malicious authority set, the challenger does not abort with $1/|\mathbb{I}|$ probability which by design is $1/\binom{n}{k} = 1/\text{poly}(\lambda)$. If this happens, we then proceed to argue indistinguishability from MA-ABE security.

Hybrid 2. In this hybrid, we additively secret share a random string during encryption. Now that all MA-ABE instantiations for i^* (after correctly guessing at the beginning) are set to random strings, message m_b is not revealed. Thus, we can readily set each share to be a random string which equates to encrypting a random string. Advantage of $\mathcal{A}$ in this hybrid is 0.

The full proof is in Appendix S. □

References

- [ABSV15] Prabhanjan Ananth, Zvika Brakerski, Gil Segev, and Vinod Vaikuntanathan. From selective to adaptive security in functional encryption. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *Advances in Cryptology – CRYPTO 2015, Part II*, volume 9216 of *Lecture Notes in Computer Science*, pages 657–677, Santa Barbara, CA, USA, August 16–20, 2015. Springer Berlin Heidelberg, Germany.
- [AG21] Miguel Ambrona and Romain Gay. Multi-authority abe, revisited. *Cryptology ePrint Archive*, 2021.
- [AGT21] Shweta Agrawal, Rishab Goyal, and Junichi Tomida. Multi-party functional encryption. In *Theory of Cryptography: 19th International Conference, TCC 2021, Raleigh, NC, USA, November 8–11, 2021, Proceedings, Part II*, pages 224–255. Springer, 2021.
- [AGVW13] Shweta Agrawal, Sergey Gorbunov, Vinod Vaikuntanathan, and Hoeteck Wee. Functional encryption: New perspectives and lower bounds. In *Advances in Cryptology – CRYPTO 2013: 33rd Annual Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2013. Proceedings, Part II*, pages 500–518. Springer, 2013.

- [AJ15] Prabhanjan Ananth and Abhishek Jain. Indistinguishability obfuscation from compact functional encryption. In *Annual Cryptology Conference*, pages 308–326. Springer, 2015.
- [AJJM21] Prabhanjan Ananth, Abhishek Jain, Zhengzhong Jin, and Giulio Malavolta. Unbounded multi-party computation from learning with errors. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 754–781. Springer, 2021.
- [AMVY21] Shweta Agrawal, Monosij Maitra, Narasimha Sai Vempati, and Shota Yamada. Functional encryption for turing machines with dynamic bounded collusion from lwe. In *Advances in Cryptology–CRYPTO 2021: 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16–20, 2021, Proceedings, Part IV 41*, pages 239–269. Springer, 2021.
- [ARP03] Sattam S Al-Riyami and Kenneth G Paterson. Certificateless public key cryptography. In *International conference on the theory and application of cryptology and information security*, pages 452–473. Springer, 2003.
- [ARS24] Damiano Abram, Lawrence Roy, and Peter Scholl. Succinct homomorphic secret sharing. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 301–330. Springer, 2024.
- [AV19] Prabhanjan Ananth and Vinod Vaikuntanathan. Optimal bounded-collusion secure functional encryption. In *Theory of Cryptography Conference*, pages 174–198. Springer, 2019.
- [BCG⁺17a] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, and Michele Orrù. Homomorphic secret sharing: optimizations and applications. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 2105–2122, 2017.
- [BCG⁺17b] Zvika Brakerski, Nishanth Chandran, Vipul Goyal, Aayush Jain, Amit Sahai, and Gil Segev. Hierarchical functional encryption. In *8th Innovations in Theoretical Computer Science Conference (ITCS 2017)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, 2017.
- [BDOP04] Dan Boneh, Giovanni Di Crescenzo, Rafail Ostrovsky, and Giuseppe Persiano. Public key encryption with keyword search. In Christian Cachin and Jan Camenisch, editors, *Advances in Cryptology – EUROCRYPT 2004*, volume 3027 of *Lecture Notes in Computer Science*, pages 506–522, Interlaken, Switzerland, May 2–6, 2004. Springer Berlin Heidelberg, Germany.
- [BF01] Dan Boneh and Matt Franklin. Identity-based encryption from the weil pairing. In *Annual international cryptology conference*, pages 213–229. Springer, 2001.
- [BGG⁺18] Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter M. R. Rasmussen, and Amit Sahai. Threshold cryptosystems from threshold fully homomorphic encryption. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in*

- Cryptology – CRYPTO 2018, Part I*, volume 10991 of *Lecture Notes in Computer Science*, pages 565–596, Santa Barbara, CA, USA, August 19–23, 2018. Springer, Cham, Switzerland.
- [BGI⁺12] Boaz Barak, Oded Goldreich, Russell Impagliazzo, Steven Rudich, Amit Sahai, Salil P. Vadhan, and Ke Yang. On the (im)possibility of obfuscating programs. *J. ACM*, 59(2):6, 2012.
 - [BGI16] Elette Boyle, Niv Gilboa, and Yuval Ishai. Breaking the circuit size barrier for secure computation under ddh. In *Annual International Cryptology Conference*, pages 509–539. Springer, 2016.
 - [BHR12] Mihir Bellare, Viet Tung Hoang, and Phillip Rogaway. Foundations of garbled circuits. In *Proceedings of the 2012 ACM conference on Computer and communications security*, pages 784–796, 2012.
 - [BIOW20] Ohad Barta, Yuval Ishai, Rafail Ostrovsky, and David J. Wu. On succinct arguments and witness encryption from groups. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology – CRYPTO 2020, Part I*, volume 12170 of *Lecture Notes in Computer Science*, pages 776–806, Santa Barbara, CA, USA, August 17–21, 2020. Springer, Cham, Switzerland.
 - [BJKL21] Fabrice Benhamouda, Aayush Jain, Ilan Komargodski, and Huijia Lin. Multiparty reusable non-interactive secure computation from LWE. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 724–753. Springer, 2021.
 - [BL20] Fabrice Benhamouda and Huijia Lin. Mr NISC: Multiparty reusable non-interactive secure computation. In Rafael Pass and Krzysztof Pietrzak, editors, *Theory of Cryptography*, pages 349–378, Cham, 2020. Springer International Publishing.
 - [BLM⁺24] Pedro Branco, Russell W. F. Lai, Monosij Maitra, Giulio Malavolta, Ahmadreza Rahimi, and Ivy K. Y. Woo. Traitor tracing without trusted authority from registered functional encryption. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024, Part III*, volume 15486 of *Lecture Notes in Computer Science*, pages 33–66, Kolkata, India, December 9–13, 2024. Springer, Singapore, Singapore.
 - [BLSV18] Zvika Brakerski, Alex Lombardi, Gil Segev, and Vinod Vaikuntanathan. Anonymous IBE, leakage resilience and circular security from new assumptions. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2018, Part I*, volume 10820 of *Lecture Notes in Computer Science*, pages 535–564, Tel Aviv, Israel, April 29 – May 3, 2018. Springer, Cham, Switzerland.
 - [BS18] Zvika Brakerski and Gil Segev. Function-private functional encryption in the private-key setting. *Journal of Cryptology*, 31:202–225, 2018.
 - [BSW06] Dan Boneh, Amit Sahai, and Brent Waters. Fully collusion resistant traitor tracing with short ciphertexts and private keys. In Serge Vaudenay, editor, *Advances in Cryptology – EUROCRYPT 2006*, volume 4004 of *Lecture Notes in Computer Science*, pages

573–592, St. Petersburg, Russia, May 28 – June 1, 2006. Springer Berlin Heidelberg, Germany.

- [BSW11] Dan Boneh, Amit Sahai, and Brent Waters. Functional encryption: Definitions and challenges. In *Theory of Cryptography: 8th Theory of Cryptography Conference, TCC 2011, Providence, RI, USA, March 28-30, 2011. Proceedings 8*, pages 253–273. Springer, 2011.
- [BV15] Nir Bitansky and Vinod Vaikuntanathan. Indistinguishability obfuscation from functional encryption. In *FOCS*, 2015.
- [BW07] Dan Boneh and Brent Waters. Conjunctive, subset, and range queries on encrypted data. In *Theory of Cryptography: 4th Theory of Cryptography Conference, TCC 2007, Amsterdam, The Netherlands, February 21-24, 2007. Proceedings 4*, pages 535–554. Springer, 2007.
- [CC09] Melissa Chase and Sherman S. M. Chow. Improving privacy and security in multi-authority attribute-based encryption. In *ACM Conference on Computer and Communications Security*, pages 121–130, 2009.
- [CCV04] Zhaohui Cheng, Richard Comley, and Luminita Vasiu. Remove key escrow from the identity-based encryption system. In *Exploring New Frontiers of Theoretical Informatics: IFIP 18th World Computer Congress TC1 3rd International Conference on Theoretical Computer Science (TCS2004) 22–27 August 2004 Toulouse, France*, pages 37–50. Springer, 2004.
- [CDSG⁺18] Jérémy Chotard, Edouard Dufour Sans, Romain Gay, Duong Hieu Phan, and David Pointcheval. Decentralized multi-client functional encryption for inner product. In *Advances in Cryptology—ASIACRYPT 2018: 24th International Conference on the Theory and Application of Cryptology and Information Security, Brisbane, QLD, Australia, December 2–6, 2018, Proceedings, Part II 24*, pages 703–732. Springer, 2018.
- [CFGN96] Ran Canetti, Uriel Feige, Oded Goldreich, and Moni Naor. Adaptively secure multi-party computation. In *28th Annual ACM Symposium on Theory of Computing*, pages 639–648, Philadelphia, PA, USA, May 22–24, 1996. ACM Press.
- [CFN94] Benny Chor, Amos Fiat, and Moni Naor. Tracing traitors. In Yvo Desmedt, editor, *Advances in Cryptology – CRYPTO’94*, volume 839 of *Lecture Notes in Computer Science*, pages 257–270, Santa Barbara, CA, USA, August 21–25, 1994. Springer Berlin Heidelberg, Germany.
- [Cha07] Melissa Chase. Multi-authority attribute based encryption. In *Theory of Cryptography: 4th Theory of Cryptography Conference, TCC 2007, Amsterdam, The Netherlands, February 21-24, 2007. Proceedings 4*, pages 515–534. Springer, 2007.
- [Cho09] Sherman S. M. Chow. Removing escrow from identity-based encryption. In Stanislaw Jarecki and Gene Tsudik, editors, *PKC 2009: 12th International Conference on Theory and Practice of Public Key Cryptography*, volume 5443 of *Lecture Notes in Computer Science*, pages 256–276, Irvine, CA, USA, March 18–20, 2009. Springer Berlin Heidelberg, Germany.

- [CLW24] Valerio Cini, Russell W. F. Lai, and Ivy K. Y. Woo. Lattice-based multi-authority/client attribute-based encryption for circuits. *IACR Communications in Cryptology (CiC)*, 1(4):1, 2024.
- [CM21] Geoffroy Couteau and Pierre Meyer. Breaking the circuit size barrier for secure computation under quasi-polynomial lpn. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 842–870. Springer, 2021.
- [Coc01] Clifford Cocks. An identity based encryption scheme based on quadratic residues. In *Cryptography and Coding: 8th IMA International Conference Cirencester, UK, December 17–19, 2001 Proceedings* 8, pages 360–363. Springer, 2001.
- [CVW⁺18] Yilei Chen, Vinod Vaikuntanathan, Brent Waters, Hoeteck Wee, and Daniel Wichs. Traitor-tracing from lwe made simple and attribute-based. In *Theory of Cryptography: 16th International Conference, TCC 2018, Panaji, India, November 11–14, 2018, Proceedings, Part II* 16, pages 341–369. Springer, 2018.
- [Des88] Yvo Desmedt. Society and group oriented cryptography: A new concept. In Carl Pomerance, editor, *Advances in Cryptology – CRYPTO’87*, volume 293 of *Lecture Notes in Computer Science*, pages 120–127, Santa Barbara, CA, USA, August 16–20, 1988. Springer Berlin Heidelberg, Germany.
- [DF90] Yvo Desmedt and Yair Frankel. Threshold cryptosystems. In Gilles Brassard, editor, *Advances in Cryptology – CRYPTO’89*, volume 435 of *Lecture Notes in Computer Science*, pages 307–315, Santa Barbara, CA, USA, August 20–24, 1990. Springer, New York, USA.
- [DIJ⁺13] Angelo De Caro, Vincenzo Iovino, Abhishek Jain, Adam O’Neill, Omer Paneth, and Giuseppe Persiano. On the achievability of simulation-based security for functional encryption. In Ran Canetti and Juan A. Garay, editors, *Advances in Cryptology – CRYPTO 2013, Part II*, volume 8043 of *Lecture Notes in Computer Science*, pages 519–535, Santa Barbara, CA, USA, August 18–22, 2013. Springer Berlin Heidelberg, Germany.
- [DIJL23] Quang Dao, Yuval Ishai, Aayush Jain, and Huijia Lin. Multi-party homomorphic secret sharing and sublinear mpc from sparse lpn. In *Annual International Cryptology Conference*, pages 315–348. Springer, 2023.
- [DJWW25] Lalita Devadas, Abhishek Jain, Brent Waters, and David J Wu. Succinct witness encryption for batch languages and applications. *To appear in Asiacrypt*, 2025.
- [DKW21] Pratish Datta, Ilan Komargodski, and Brent Waters. Decentralized multi-authority abe for dnf s from lwe. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 177–209. Springer, 2021.
- [DKW23] Pratish Datta, Ilan Komargodski, and Brent Waters. Decentralized multi-authority abe for nc 1 from bdh. *Journal of Cryptology*, 36(2):6, 2023.

- [DKXY02] Yevgeniy Dodis, Jonathan Katz, Shouhuai Xu, and Moti Yung. Key-insulated public key cryptosystems. In *International Conference on the Theory and Applications of Cryptographic Techniques*, 2002.
- [DP23] Pratish Datta and Tapas Pal. Decentralized multi-authority attribute-based inner-product fe: Large universe and unbounded. In *IACR International Conference on Public-Key Cryptography*, pages 587–621. Springer, 2023.
- [FGH⁺17] Cody Freitag, Rishab Goyal, Susan Hohenberger, Venkata Koppula, Eysa Lee, Tatsuaki Okamoto, Jordan Tran, and Brent Waters. Signature schemes with randomized verification. In *International Conference on Applied Cryptography and Network Security*, pages 373–389. Springer, 2017.
- [FGJS17] Nelly Fazio, Rosario Gennaro, Tahereh Jafarikhah, and William E Skeith. Homomorphic secret sharing from paillier encryption. In *Provable Security: 11th International Conference, ProvSec 2017, Xi’an, China, October 23-25, 2017, Proceedings 11*, pages 381–399. Springer, 2017.
- [Fra90] Yair Frankel. A practical protocol for large group oriented networks. In Jean-Jacques Quisquater and Joos Vandewalle, editors, *Advances in Cryptology – EUROCRYPT’89*, volume 434 of *Lecture Notes in Computer Science*, pages 56–61, Houthalen, Belgium, April 10–13, 1990. Springer Berlin Heidelberg, Germany.
- [GGG⁺14] Shafi Goldwasser, S. Dov Gordon, Vipul Goyal, Abhishek Jain, Jonathan Katz, Feng-Hao Liu, Amit Sahai, Elaine Shi, and Hong-Sheng Zhou. Multi-input functional encryption. In Phong Q. Nguyen and Elisabeth Oswald, editors, *Advances in Cryptology – EUROCRYPT 2014*, volume 8441 of *Lecture Notes in Computer Science*, pages 578–602, Copenhagen, Denmark, May 11–15, 2014. Springer Berlin Heidelberg, Germany.
- [GGH⁺16] Sanjam Garg, Craig Gentry, Shai Halevi, Mariana Raykova, Amit Sahai, and Brent Waters. Candidate indistinguishability obfuscation and functional encryption for all circuits. *SIAM Journal on Computing*, 45(3):882–929, 2016.
- [GGL24] Rachit Garg, Rishab Goyal, and George Lu. Dynamic collusion functional encryption and multi-authority attribute-based encryption. In *IACR International Conference on Public-Key Cryptography*, pages 69–104. Springer, 2024.
- [GGLW22] Rachit Garg, Rishab Goyal, George Lu, and Brent Waters. Dynamic collusion bounded functional encryption from identity-based encryption. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 736–763. Springer, 2022.
- [GGSW13] Sanjam Garg, Craig Gentry, Amit Sahai, and Brent Waters. Witness encryption and its applications. In Dan Boneh, Tim Roughgarden, and Joan Feigenbaum, editors, *45th Annual ACM Symposium on Theory of Computing*, pages 467–476, Palo Alto, CA, USA, June 1–4, 2013. ACM Press.
- [GHM⁺19] Sanjam Garg, Mohammad Hajiabadi, Mohammad Mahmoody, Ahmadreza Rahimi, and Sruthi Sekar. Registration-based encryption from standard assumptions. In *IACR international workshop on public key cryptography*, pages 63–93. Springer, 2019.

- [GHMR18] Sanjam Garg, Mohammad Hajiabadi, Mohammad Mahmoody, and Ahmadreza Rahimi. Registration-based encryption: removing private-key generator from ibe. In *Theory of Cryptography: 16th International Conference, TCC 2018, Panaji, India, November 11–14, 2018, Proceedings, Part I 16*, pages 689–718. Springer, 2018.
- [GHRW14] Craig Gentry, Shai Halevi, Mariana Raykova, and Daniel Wichs. Outsourcing private RAM computation. In *55th Annual Symposium on Foundations of Computer Science*, pages 404–413, Philadelphia, PA, USA, October 18–21, 2014. IEEE Computer Society Press.
- [GKPW24] Sanjam Garg, Dimitris Kolonelos, Guru-Vamsi Policharla, and Mingyuan Wang. Threshold encryption with silent setup. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024, Part VII*, volume 14926 of *Lecture Notes in Computer Science*, pages 352–386, Santa Barbara, CA, USA, August 18–22, 2024. Springer, Cham, Switzerland.
- [GKW17] Rishab Goyal, Venkata Koppula, and Brent Waters. Lockable obfuscation. In *58th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2017*, pages 612–621, 2017.
- [GKW18] Rishab Goyal, Venkata Koppula, and Brent Waters. Collusion resistant traitor tracing from learning with errors. In *Proceedings of the 50th Annual ACM SIGACT Symposium on Theory of Computing*, pages 660–670, 2018.
- [GLSW08] Vipul Goyal, Steve Lu, Amit Sahai, and Brent Waters. Black-box accountable authority identity-based encryption. In Peng Ning, Paul F. Syverson, and Somesh Jha, editors, *ACM CCS 2008: 15th Conference on Computer and Communications Security*, pages 427–436, Alexandria, Virginia, USA, October 27–31, 2008. ACM Press.
- [GLW12] Shafi Goldwasser, Allison Lewko, and David A Wilson. Bounded-collusion ibe from key homomorphism. In *Theory of Cryptography: 9th Theory of Cryptography Conference, TCC 2012, Taormina, Sicily, Italy, March 19-21, 2012. Proceedings 9*, pages 564–581. Springer, 2012.
- [Goy07] Vipul Goyal. Reducing trust in the pkg in identity based cryptosystems. In *Annual International Cryptology Conference*, pages 430–447. Springer, 2007.
- [GPSW06] Vipul Goyal, Omkant Pandey, Amit Sahai, and Brent Waters. Attribute-based encryption for fine-grained access control of encrypted data. In *Proceedings of the 13th ACM conference on Computer and communications security*, pages 89–98, 2006.
- [GV20] Rishab Goyal and Satyanarayana Vusirikala. Verifiable registration-based encryption. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology – CRYPTO 2020, Part I*, volume 12170 of *Lecture Notes in Computer Science*, pages 621–651, Santa Barbara, CA, USA, August 17–21, 2020. Springer, Cham, Switzerland.
- [GVW12] Sergey Gorbunov, Vinod Vaikuntanathan, and Hoeteck Wee. Functional encryption with bounded collusions via multi-party computation. In *Advances in Cryptology–CRYPTO 2012: 32nd Annual Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2012. Proceedings*, pages 162–179. Springer, 2012.

- [GY24] Rishab Goyal and Saikumar Yadugiri. Multi-authority functional encryption with bounded collusions from standard assumptions. In Elette Boyle and Mohammad Mahmoody, editors, *TCC 2024: 22nd Theory of Cryptography Conference, Part III*, volume 15366 of *Lecture Notes in Computer Science*, pages 3–30, Milan, Italy, December 2–6, 2024. Springer, Cham, Switzerland.
- [HJO⁺16] Brett Hemenway, Zahra Jafargholi, Rafail Ostrovsky, Alessandra Scafuro, and Daniel Wichs. Adaptively secure garbled circuits from one-way functions. In Matthew Robshaw and Jonathan Katz, editors, *Advances in Cryptology – CRYPTO 2016, Part III*, volume 9816 of *Lecture Notes in Computer Science*, pages 149–178, Santa Barbara, CA, USA, August 14–18, 2016. Springer Berlin Heidelberg, Germany.
- [HKM⁺24] Taiga Hiroka, Fuyuki Kitagawa, Tomoyuki Morimae, Ryo Nishimaki, Tapas Pal, and Takashi Yamakawa. Certified everlasting secure collusion-resistant functional encryption, and more. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 434–456. Springer, 2024.
- [HMNY22] Taiga Hiroka, Tomoyuki Morimae, Ryo Nishimaki, and Takashi Yamakawa. Certified everlasting functional encryption. *arXiv preprint arXiv:2207.13878*, 2022.
- [Kim19] Sam Kim. Multi-authority attribute-based encryption from lwe in the ot model. *Cryptology ePrint Archive*, 2019.
- [KSW08] Jonathan Katz, Amit Sahai, and Brent Waters. Predicate encryption supporting disjunctions, polynomial equations, and inner products. In *Advances in Cryptology – EUROCRYPT 2008: 27th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Istanbul, Turkey, April 13-17, 2008. Proceedings 27*, pages 146–162. Springer, 2008.
- [LPST16] Huijia Lin, Rafael Pass, Karn Seth, and Sidharth Telang. Indistinguishability obfuscation with non-trivial efficiency. In *Public-Key Cryptography–PKC 2016*, pages 447–462. Springer, 2016.
- [Luo24] Ji Luo. Ad hoc broadcast, trace, and revoke: Plus time-space trade-offs for attribute-based encryption. *IACR Communications in Cryptology (CiC)*, 1(2):15, 2024.
- [LW11] Allison B. Lewko and Brent Waters. Decentralizing attribute-based encryption. In *EUROCRYPT*, pages 568–588, 2011.
- [MJ18] Yan Michalevsky and Marc Joye. Decentralized policy-hiding abe with receiver privacy. In *Computer Security: 23rd European Symposium on Research in Computer Security, ESORICS 2018, Barcelona, Spain, September 3-7, 2018, Proceedings, Part II 23*, pages 548–567. Springer, 2018.
- [OT20] Tatsuaki Okamoto and Katsuyuki Takashima. Decentralized attribute-based encryption and signatures. *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, 103(1):41–73, 2020.
- [Rog15] Phillip Rogaway. The moral character of cryptographic work. *Cryptology ePrint Archive*, Paper 2015/1162, 2015.

- [Sha85] Adi Shamir. Identity-based cryptosystems and signature schemes. In *Advances in Cryptology: Proceedings of CRYPTO 84* 4, pages 47–53. Springer, 1985.
- [Shi22] Sina Shiehian. mnrisc from lwe with polynomial modulus. In *International Conference on Security and Cryptography for Networks*, pages 481–493. Springer, 2022.
- [SS10] Amit Sahai and Hakan Seyalioglu. Worry-free encryption: functional encryption with public keys. In *Proceedings of the 17th ACM conference on Computer and communications security*, pages 463–472, 2010.
- [SW05] Amit Sahai and Brent Waters. Fuzzy identity-based encryption. In *Advances in Cryptology–EUROCRYPT 2005: 24th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Aarhus, Denmark, May 22-26, 2005. Proceedings 24*, pages 457–473. Springer, 2005.
- [Tsa22] Rotem Tsabary. Candidate witness encryption from lattice techniques. In *Annual International Cryptology Conference*, pages 535–559. Springer, 2022.
- [VWW22] Vinod Vaikuntanathan, Hoeteck Wee, and Daniel Wichs. Witness encryption and null-io from evasive lwe. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 195–221. Springer, 2022.
- [Wat05] Brent Waters. Efficient identity-based encryption without random oracles. In *Advances in Cryptology–EUROCRYPT 2005: 24th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Aarhus, Denmark, May 22-26, 2005. Proceedings 24*, pages 114–127. Springer, 2005.
- [WFL19] Zhedong Wang, Xiong Fan, and Feng-Hao Liu. Fe for inner products and its application to decentralized abe. In *IACR international workshop on public key cryptography*, pages 97–127. Springer, 2019.
- [WQT18] Quanyun Wei, Fang Qi, and Zhe Tang. Remove key escrow from the bf and gentry identity-based encryption with non-interactive key generation. *Telecommunication Systems*, 69(2):253–262, 2018.
- [WW24] Brent Waters and Daniel Wichs. Adaptively secure attribute-based encryption from witness encryption. In Elette Boyle and Mohammad Mahmoody, editors, *TCC 2024: 22nd Theory of Cryptography Conference, Part III*, volume 15366 of *Lecture Notes in Computer Science*, pages 65–90, Milan, Italy, December 2–6, 2024. Springer, Cham, Switzerland.
- [WWW22] Brent Waters, Hoeteck Wee, and David J Wu. Multi-authority abe from lattices without random oracles. In *Theory of Cryptography Conference*, pages 651–679. Springer, 2022.
- [WZ17] Daniel Wichs and Giorgos Zirdelis. Obfuscating compute-and-compare programs under LWE. In *58th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2017*, pages 600–611, 2017.
- [Yao86] Andrew Chi-Chih Yao. How to generate and exchange secrets. In *27th annual symposium on foundations of computer science (Sfcs 1986)*, pages 162–167. IEEE, 1986.

A Additional Preliminaries

Notation. By PPT we denote probabilistic polynomial-time. We denote the security parameter by λ and the set of positive integers by $\mathbb{N}$. For any $a, b \in \{0\} \cup \mathbb{N}$, $a \leq b$, we denote by $[a, b]$, the set of all integers from a to b including a and b . In other words, $[a, b] = \{a, \dots, b\}$. We denote by $[n] := [1, n]$. We denote by $x \xleftarrow{\$} \mathcal{X}$, the process of sampling an element x from the set $\mathcal{X}$, with uniform probability. Similarly, for any PPT algorithm $\mathcal{A}$, $x \leftarrow \mathcal{A}(y)$ denotes the process of sampling x from the output distribution of $\mathcal{A}$ when run on y . By $\text{negl}(\cdot)$, we denote negligible functions. By poly , we denote positive polynomials. All of our logarithms $\log$ are in base 2. We denote the power set A by 2^A and the set of all subsets of A of size a by $2^A|_a$.

We say that two efficiently samplable probability distributions $\mathcal{X} = \{\mathcal{X}_\lambda\}_{\lambda \in \mathbb{N}}$ and $\mathcal{Y} = \{\mathcal{Y}_\lambda\}_{\lambda \in \mathbb{N}}$ are computationally indistinguishable if for any non-uniform PPT distinguisher $\mathcal{D} = \{\mathcal{D}_\lambda\}_{\lambda \in \mathbb{N}}$, and for large enough $\lambda \in \mathbb{N}$,

$$\left| \Pr_{\alpha \leftarrow \mathcal{X}_\lambda} [1 \leftarrow \mathcal{D}(1^\lambda, \alpha)] - \Pr_{\alpha \leftarrow \mathcal{Y}_\lambda} [1 \leftarrow \mathcal{D}(1^\lambda, \alpha)] \right| \leq \text{negl}(\lambda)$$

We denote statistical indistinguishability by $\approx_s$ and identical distributions by $\equiv$.

We recall the definition of garbled circuits [Yao86] which is derived from [BHR12].

Definition A.1 (Garb). A Garb scheme (Garble, Eval) is said to be a garbling scheme for P/Poly circuits if satisfies the following properties.

Correctness. For any $\lambda \in \mathbb{N}$, n -ary P/Poly circuit C ,

$$\Pr \left[\begin{array}{l} y = \\ C(x_1, \dots, x_n) \end{array} : \begin{array}{l} (\tilde{C}, \{w_{i,b}\}_{i \in [n], b \in \{0,1\}}) \leftarrow \text{Garble}(1^\lambda, C) \\ y = \text{Eval}(\tilde{C}, \{w_{i,x_i}\}_{i \in [n]}) \end{array} \right] = 1$$

Security. For any PPT adversary $\mathcal{A}$, there exists a simulator Sim , $\forall \lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} (C, x) \leftarrow \mathcal{A}(1^\lambda), b \xleftarrow{\$} \{0, 1\}, \\ (\tilde{C}^{(0)}, \{w_{i,\beta}^{(0)}\}_{i \in [n], \beta \in \{0,1\}}) \leftarrow \text{Garble}(1^\lambda, C) \\ (\tilde{C}^{(1)}, \{w_{i,x_i}^{(1)}\}_{i \in [n]}) \leftarrow \text{Sim}(1^\lambda, 1^n, 1^{|C|}, C(x)) \\ b' \leftarrow \mathcal{A}(\tilde{C}^{(b)}, \{w_{i,x_i}^{(b)}\}_{i \in [n]}) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda)$$

Remark A.2 ([Yao86]). Assuming the existence of one-way functions, garbled circuits for any family of P/Poly circuits exist.

Definition A.3 (PRF). A secure pseudorandom function PRF is a deterministic polynomial-time function that takes inputs from $\mathcal{X} = \{\mathcal{X}_\lambda\}_{\lambda \in \mathbb{N}}$ and a key $K \in \{0, 1\}^\lambda$ and outputs values in $\mathcal{R} = \{\mathcal{R}_\lambda\}_{\lambda \in \mathbb{N}}$. We say that the PRF is a secure pseudorandom function if it satisfies the following properties.

Pseudorandomness. For any stateful PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that $\forall \lambda \in \mathbb{N}$,

$$\Pr \left[b \leftarrow \mathcal{A}^{\mathcal{O}_b(\cdot)}(1^\lambda) : b \xleftarrow{\$} \{0, 1\}, K \xleftarrow{\$} \{0, 1\}^\lambda \right] \leq \frac{1}{2} + \text{negl}(\lambda)$$

where $\mathcal{O}_0(\cdot) = \text{PRF}(K, \cdot)$ and $\mathcal{O}_1(\cdot)$ generates consistent random strings in $\mathcal{R}_\lambda$.

Definition A.4 (IBE). An IBE scheme $(\text{Setup}, \text{KGen}, \text{Enc}, \text{Dec})$ is said to be an Identity-Based Encryption scheme (IBE) for identity space $\mathcal{I} = \{\mathcal{I}_\lambda\}_{\lambda \in \mathbb{N}}$ if it satisfies the following properties.

Correctness. For any $\lambda \in \mathbb{N}$, for any identity $\text{ibID} \in \mathcal{I}_\lambda, |\text{ibID}| = z = z(\lambda)$, for any message $m \in \{0, 1\}^*$ ¹⁸,

$$\Pr \left[m = \text{Dec}(\text{SK}_{\text{ibID}}, \text{CT}) \quad : \quad \begin{array}{l} (\text{MPK}, \text{MSK}) \leftarrow \text{Setup}(1^\lambda, 1^z), \\ \text{SK}_{\text{ibID}} \leftarrow \text{KGen}(\text{MSK}, \text{ibID}), \\ \text{CT} \leftarrow \text{Enc}(\text{MPK}, \text{ibID}, m) \end{array} \right] = 1$$

Security. For any admissible stateful PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for any $\lambda \in \mathbb{N}$,

$$\Pr \left[b \leftarrow \mathcal{A}^{\mathcal{O}(\cdot)}(\text{CT}) \quad : \quad \begin{array}{l} (\text{MPK}, \text{MSK}) \leftarrow \text{Setup}(1^\lambda, 1^z), \\ (\text{ibID}^*, m) \leftarrow \mathcal{A}^{\mathcal{O}(\cdot)}(1^\lambda, 1^z, \text{MPK}), \\ b \xleftarrow{\$} \{0, 1\}, m_0 := m, m_1 := 0^{|m|}, \\ \text{CT} \leftarrow \text{Enc}(\text{MPK}, \text{ibID}^*, m_b) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda)$$

where $\mathcal{O}(\text{ibID})$ outputs $\text{KGen}(\text{MSK}, \text{ibID})$ and an admissible adversary queries the oracle $\mathcal{O}(\cdot)$ with $\text{ibID} \neq \text{ibID}^*$.

Remark A.5 (Multi-challenge security). Although Definition A.4 is defined for a single challenge identity, the same holds for multiple challenge queries by a simple hybrid argument. We will use this version in this work.

Definition A.6 (SKE with Pseudorandom Ciphertexts). An SKE scheme $(\text{Setup}, \text{Enc}, \text{Dec})$ is said to be a Secret-Key Encryption scheme (SKE) with pseudorandom ciphertexts if it satisfies the following properties.

Correctness. For any $\lambda \in \mathbb{N}$, for any message $m \in \{0, 1\}^*$ ¹⁹,

$$\Pr \left[m = \text{Dec}(\text{SK}, \text{CT}) \quad : \quad \begin{array}{l} \text{SK} \leftarrow \text{Setup}(1^\lambda), \\ \text{CT} \leftarrow \text{Enc}(\text{SK}, m) \end{array} \right] = 1$$

Pseudorandom Ciphertext Security. For any admissible stateful PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for any $\lambda \in \mathbb{N}$,

$$\Pr \left[b \leftarrow \mathcal{A}^{\mathcal{O}_b(\cdot)}(1^\lambda) \quad : \quad b \xleftarrow{\$} \{0, 1\}, \text{SK} \leftarrow \text{Setup}(1^\lambda) \right] \leq \frac{1}{2} + \text{negl}(\lambda)$$

where $\mathcal{O}_0$ takes as input $m \in \{0, 1\}^*$ and samples $\text{CT} \leftarrow \text{Enc}(\text{SK}, m)$ and outputs CT. On the other hand, $\mathcal{O}_1$ on input $m \in \{0, 1\}^*$ samples uniform random string of length $|\text{Enc}(\text{SK}, m; r)|$ for some $r \xleftarrow{\$} \{0, 1\}^\lambda$. Note that this definition implies standard IND-CPA security for SKE.

Remark A.7. Assuming the existence of one-way functions, there exists an SKE scheme.

¹⁸Recall that IBE can encrypt messages of arbitrary length using hybrid encryption.

¹⁹Recall that SKE can encrypt messages of arbitrary length using hybrid encryption.

Statistically Binding Commitments. A statistically binding commitment scheme (COM) consists of the following polynomial time algorithms.

$\text{Setup}(1^\lambda) \rightarrow \text{crs}$. The probabilistic setup algorithm takes as input security parameter λ and outputs a common reference string crs . The following algorithms take crs implicitly.

$\text{Com}(\mu; r) \rightarrow \text{com}$. The probabilistic commitment algorithm takes as input message μ , randomness r , and outputs commitment value com .

Definition A.8 (COM). A COM scheme $(\text{Setup}, \text{Com})$ is said to be a COM scheme if it satisfies the following properties.

Computational Hiding. For any stateful PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that $\forall \lambda \in \mathbb{N}$,

$$\left| \Pr \left[0 \leftarrow \mathcal{A}^{\mathcal{O}_0(\cdot, \cdot)}(\text{crs}) \right] - \Pr \left[0 \leftarrow \mathcal{A}^{\mathcal{O}_1(\cdot, \cdot)}(\text{crs}) \right] \right| \leq \text{negl}(\lambda)$$

where $\text{crs} \leftarrow \text{Setup}(1^\lambda)$ and $\mathcal{O}_b(\mu_0, \mu_1)$ responds with $\text{com} \leftarrow \text{Com}(\text{crs}, \mu_b)$ for $b \in \{0, 1\}$.

Statistical Binding. There exists $\text{negl}(\cdot)$ such that $\forall \lambda \in \mathbb{N}$,

$$\Pr \left[\text{com} = \text{Com}(\mu_0; r_0) \wedge \text{com} = \text{Com}(\mu_1; r_1) \quad : \quad \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda), \\ \exists (\text{com}, \mu_0, \mu_1, r_0, r_1) \end{array} \right] \leq \text{negl}(\lambda)$$

where probability is taken on the randomness of crs .

Witness Encryption. A witness encryption (WE) scheme for language $\mathcal{L} = \{x : \exists w, R(x, w) = 1\}$ consists of the following polynomial time algorithms.

$\text{Enc}(1^\lambda, x, \mu) \rightarrow \text{CT}$. The probabilistic encryption algorithm takes as input the security parameter λ , language index s , instance $x \in \mathcal{L}_s$, a message μ , and outputs a ciphertext CT .

$\text{Dec}(\text{CT}, w) \rightarrow \mu'$. The deterministic decryption algorithm takes as input a ciphertext CT , a witness w , and outputs a message μ' .

Definition A.9 (WE). A WE scheme (Enc, Dec) is said to be a WE scheme for language $\mathcal{L}_s$ if it satisfies the following properties.

Correctness. $\forall \lambda \in \mathbb{N}, \mu, x \in \mathcal{L}$, if $R(x, w) = 1$, $\Pr[\mu = \text{Dec}(\text{CT}, w) : \text{CT} \leftarrow \text{Enc}(1^\lambda, x, \mu)] = 1$.

Semantic Security. For any stateful PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that $\forall \lambda \in \mathbb{N}, x \notin \mathcal{L}, |\mu_0| = |\mu_1|$,

$$\Pr \left[b \leftarrow \mathcal{A}(\text{CT}) \quad : \quad \begin{array}{l} (\mu_0, \mu_1) \leftarrow \mathcal{A}(1^\lambda), b \xleftarrow{\$} \{0, 1\}, \\ \text{CT} \leftarrow \text{Enc}(1^\lambda, x, \mu_b) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda)$$

NIZK. A non-interactive zero-knowledge proof system (NIZK) for $\mathcal{L} = \{x : \exists w, R(x, w) = 1\}$ consists of the following polynomial time algorithms.

Setup(1^λ) $\rightarrow$ **crs**. The probabilistic setup algorithm takes as input security parameter λ , language index s , and outputs a common reference string **crs**. The following algorithms take **crs** implicitly.

Prove(x, w) $\rightarrow$ π . The probabilistic proving algorithm takes as input instance $x \in \mathcal{L}$, witness w , and outputs proof π .

Verify(x, π) $\rightarrow$ 0/1. The deterministic verification algorithm takes as input instance x , proof π , and outputs 0 (reject) or 1 (accept).

Definition A.10 (NIZK). A NIZK scheme (**Setup**, **Prove**, **Verify**) is said to be a NIZK scheme for language $\mathcal{L}$ if it satisfies the following properties.

Correctness. $\forall \lambda \in \mathbb{N}, x \in \mathcal{L}, \Pr[1 = \text{Verify}(x, \pi) : \text{crs} \leftarrow \text{Setup}(1^\lambda), \pi \leftarrow \text{Prove}(x, w)] = 1$.

Statistical Soundness. There exists a negligible function $\text{negl}(\cdot)$ such that $\forall \lambda \in \mathbb{N}, x \notin \mathcal{L}, \forall \pi, \Pr[1 = \text{Verify}(x, \pi) : \text{crs} \leftarrow \text{Setup}(1^\lambda), \exists \pi] \leq \text{negl}(\lambda)$.

Computational Zero Knowledge. For any admissible adversary $\mathcal{A}$, there exists a stateful simulator **Sim** such that $\forall \lambda \in \mathbb{N}$,

$$\left| \Pr \left[1 \leftarrow \mathcal{A}^{\mathcal{O}_0(\cdot, \cdot)}(1^\lambda, \text{crs}) : \text{crs} \leftarrow \text{Setup}(1^\lambda) \right] - \Pr \left[1 \leftarrow \mathcal{A}^{\mathcal{O}_1^{\text{Sim}(\cdot)}(\cdot, \cdot)}(1^\lambda, \text{crs}) : \text{crs} \leftarrow \text{Sim}(1^\lambda) \right] \right| \leq \text{negl}(\lambda)$$

where $\mathcal{O}_0$ uses $\pi \leftarrow \text{Prove}(x, w)$ and $\mathcal{O}_1$ uses $\pi \leftarrow \text{Sim}(x)$ to respond to $\mathcal{A}$'s queries of the form (x, w) . A stateful PPT machine is said to be admissible if for each of its queries, $R(x, w) = 1$.

B Relevant Material from [GY24]

In this section, we provide definitions and results from [GY24] that will be useful for the reader.

B.1 Distributed Client-Server Framework

Syntax. A distributed client-server framework (dCSF) for circuit class $\mathcal{C} = \{\mathcal{C}_\lambda\}_{\lambda \in \mathbb{N}}$ and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ consists of the following polynomial time algorithms.

ServEnc($1^\lambda, 1^Q, 1^n, 1^s, \text{id}, \text{GID}, x, \Delta$) $\rightarrow \{\hat{x}_{\text{id}, \text{GID}}^u\}_{u \in [N]}$. The deterministic server encoding algorithm takes as input the query bound Q , number of authorities n , maximum size of circuit s , authority identifier id , user global identifier $\text{GID} \in \mathcal{GID}_\lambda$, input x , cover-free set Δ , and outputs input encodings $\{\hat{x}_{\text{id}, \text{GID}}^u\}_{u \in [N]}$.

CliEnc($1^\lambda, 1^Q, 1^n, 1^s, C$) $\rightarrow \{\hat{C}^u\}_{u \in [N]}$. The probabilistic client encoding algorithm takes as input query bound Q , number of authorities n , maximum size of circuit s , circuit $C \in \mathcal{C}_\lambda$, and outputs client encodings $\{\hat{C}^u\}_{u \in [N]}$.

$\text{UserComp}(\{\hat{x}_{\text{id}, \text{GID}}^u\}_{\text{id} \in [n]}, \hat{C}^u) \rightarrow \hat{y}_{\text{GID}}^u$. The deterministic user computation algorithm takes the input encodings for a $\text{GID} \in \mathcal{GID}_\lambda$ from all authorities, $\{\hat{x}_{\text{id}, \text{GID}}^u\}_{\text{id} \in [n]}$, client encoding $\hat{C}^u$, and computes the output encoding $\hat{y}_{\text{GID}}^u$.

$\text{Decode}(\{\hat{y}_{\text{GID}}^u\}_{u \in \mathbf{S}}, \mathbf{S}) \rightarrow y$. The deterministic decoding algorithm takes as input output encodings from users in $\mathbf{S} \subseteq [N]$, $\{\hat{y}_{\text{GID}}^u\}_{u \in \mathbf{S}}$ and outputs the value y .

Definition B.1 (dCSF). A dCSF scheme (ServEnc , CliEnc , UserComp , Decode) is said to be a dCSF scheme for circuit class $\mathcal{C}$ and $\mathcal{GID}$ if it satisfies the following properties.

Correctness. For any $\lambda \in \mathbb{N}$, $Q = Q(\lambda)$, $n = n(\lambda)$, $s = s(\lambda)$, $\text{id} \in [n]$, $C \in \mathcal{C}_\lambda$, $\text{GID} \in \mathcal{GID}_\lambda$, $u \in \mathbf{S} \subseteq [N]$, $|\mathbf{S}| = D$,

$$\Pr \left[\begin{array}{l} C(x_1, \dots, x_n) = \\ \text{Decode}(\{\hat{y}_{\text{GID}}^u\}_{u \in \mathbf{S}}, \mathbf{S}) \end{array} : \begin{array}{l} \{\hat{x}_{\text{id}, \text{GID}}^u\}_{u \in [N]} = \text{ServEnc}(1^\lambda, 1^Q, 1^n, \\ 1^s, \text{id}, \text{GID}, x, \mathbf{\Delta}), \\ \{\hat{C}^u\}_{u \in [N]} \leftarrow \text{CliEnc}(1^\lambda, 1^Q, 1^n, 1^s, C), \\ \hat{y}_{\text{GID}}^u \leftarrow \text{UserComp}(\{\hat{x}_{\text{id}, \text{GID}}^u\}_{\text{id}}, \hat{C}^u) \end{array} \right] = 1$$

Security. For any admissible adversary $\mathcal{A}$, there exists a stateful simulator Sim such that $\forall \lambda \in \mathbb{N}$,

$$\left\{ \text{dCSF-Expt}_{\mathcal{A}, 0}^{\text{Chal}}(1^\lambda) \right\} \approx_c \left\{ \text{dCSF-Expt}_{\mathcal{A}, 1}^{\text{Sim}}(1^\lambda) \right\}$$

where definitions of $\text{dCSF-Expt}_{\mathcal{A}, 0}^{\text{Chal}}(1^\lambda)$, $\text{dCSF-Expt}_{\mathcal{A}, 1}^{\text{Sim}}(1^\lambda)$, and admissible adversary are as follows — A stateful PPT machine $\mathcal{A}$ is admissible if

- $\mathcal{A}$ makes at most Q server encoding queries across n authorities such that each authority is queried at most once per GID .
- For all server encoding queries, $\mathbf{S}_{\text{GID}}$, $\mathbf{\Delta}_{\text{GID}}$ should be unique for a GID . And all $\{\mathbf{S}_{\text{GID}}\}$ and $\{\mathbf{\Delta}_{\text{GID}}\}$ satisfy Lemma B.2 and Lemma B.3 respectively. $\tilde{\mathbf{S}} = [N] \setminus (\bigcup_{\text{GID} \neq \text{GID}'} \mathbf{S}_{\text{GID}} \cap \mathbf{S}_{\text{GID}'})$ and j_{GID}^* is the unique index in $\mathbf{\Delta}_{\text{GID}}$ for every GID .

$\text{dCSF-Expt}_{\mathcal{A}, 0}^{\text{Chal}}(1^\lambda)$.

Setup. $\mathcal{A}$ sends Q, n, s , set of non-corrupted users $\tilde{\mathbf{S}}$.

Pre-challenge Queries. $\mathcal{A}$ queries for server encodings with $((\text{id}, \text{GID}, x, \mathbf{\Delta}_{\text{GID}}), \mathbf{S}_{\text{GID}})$. Chal runs $\{\hat{x}_{\text{id}, \text{GID}}^u\}_{u \in [N]} = \text{ServEnc}(1^\lambda, 1^Q, 1^n, 1^s, \text{id}, \text{GID}, x, \mathbf{\Delta}_{\text{GID}})$, and sends $\{\hat{x}_{\text{id}, \text{GID}}^u\}_{u \in \mathbf{S}_{\text{GID}}}$.

Challenge Query. $\mathcal{A}$ sends circuit C , and $\{j_{\text{GID}}^*\}_{\text{GID}}$ for each completed GID in the previous phase. Chal samples $\{\hat{C}^u\}_{u \in [N]} \leftarrow \text{CliEnc}(1^\lambda, 1^Q, 1^n, 1^s, C)$ and for each GID , $\{\{\hat{y}_{\text{GID}}^u\} \leftarrow \text{UserComp}(\{\hat{x}_{\text{id}, \text{GID}}^u\}_{\text{id} \in [n]}, \hat{C}^u)\}_{u \in \mathbf{S}_{\text{GID}}}$ and sends $\{\hat{C}^u\}_{u \notin \tilde{\mathbf{S}}}$ and $\{\hat{y}_{\text{GID}}^u\}_{\text{GID}, u \in \mathbf{S}_{\text{GID}}}$ to $\mathcal{A}$.

Post-challenge Queries. This is similar to pre-challenge query phase, except that $\mathcal{A}$ sends an additional input j_{GID}^* when for this GID , this is the last authority to be queried. In turn, Chal responds additionally with $\{\hat{y}_{\text{GID}}^u\}_{u \in \mathbf{S}_{\text{GID}}}$ that it computes using UserComp .

Guess. Output whatever $\mathcal{A}$ outputs.

dCSF-Expt $_{\mathcal{A},1}^{\text{Sim}}(1^\lambda)$.

Setup. $\mathcal{A}$ sends Q, n, s , set of non-corrupted users $\tilde{\mathbf{S}}$. Initiate $\text{Sim}(1^\lambda, 1^Q, 1^n, 1^s, \tilde{\mathbf{S}})$.

Pre-challenge Queries. $\mathcal{A}$ queries for server encodings with $((\text{id}, \text{GID}, x, \Delta_{\text{GID}}), \mathbf{S}_{\text{GID}})$. Respond with $\{\hat{x}_{\text{id}, \text{GID}}^u\}_{u \in \mathbf{S}_{\text{GID}}} = \text{Sim}(\text{id}, \text{GID}, x, \Delta_{\text{GID}})$.

Challenge Query. $\mathcal{A}$ sends circuit C , and $\{j_{\text{GID}}^*\}_{\text{GID}}$ for each completed GID in the previous phase. Respond with $\{\hat{C}^u\}_{u \in \tilde{\mathbf{S}}}, \{\hat{y}_{\text{GID}}^u\}_{\text{GID}, u \in \mathbf{S}_{\text{GID}}} \leftarrow \text{Sim}(1^{|C|}, \mathcal{V}, \{j_{\text{GID}}^*\}_{\text{GID}})$. Here, $\mathcal{V} = \{(\text{GID}, X = \{x_{\text{id}, \text{GID}}\}_{\text{id} \in [n]}, C(X)) \text{ for all GID that was used to query all authorities in pre-challenge query phase}\}$.

Post-challenge Queries. This is similar to pre-challenge query phase, except that $\mathcal{A}$ sends an additional input j_{GID}^* when for this GID, this is the last authority to be queried. Respond with $(\{\hat{x}_{\text{id}, \text{GID}}^u\}, \{\hat{y}_{\text{GID}}^u\})_{u \in \mathbf{S}_{\text{GID}}} \leftarrow \text{Sim}^{C(\cdot)}(\text{id}, \text{GID}, x, \Delta_{\text{GID}}, j_{\text{GID}}^*)$. Here, Sim queries $C(\cdot)$ with $X = (x_{1, \text{GID}}, \dots, x_{n, \text{GID}})$ and receives $C(X)$.

Guess. Output whatever $\mathcal{A}$ outputs.

B.2 Augmented Statistical Lemmas

We provide the following lemmas from [GY24] used in Section C without proofs.

Lemma B.2 (Small Pairwise Intersection). Let $P = P(\lambda), Q = Q(\lambda)$ such that $P \geq Q$. Let $t = \Theta(Q\lambda), D = \Theta(t), N = \Theta(Q^2t)$ such that $D > 3t$. Let $\mathbf{S}_1, \dots, \mathbf{S}_P$ be independently and uniformly drawn sets from $[N]$ of size D . Then for any $\mathcal{R} \subseteq [P], |\mathcal{R}| = Q$, with all but negligible probability,

$$\left| \bigcup_{i,j \in \mathcal{R}, i \neq j} \mathbf{S}_i \cap \mathbf{S}_j \right| \leq t$$

Lemma B.3 (Cover-freeness). Let $P = P(\lambda), Q = Q(\lambda)$ such that $P \geq Q$. Let $v = \Theta(Q^2\lambda), T = vQ$. Let $\Delta_1, \dots, \Delta_P$ be independently and uniformly drawn sets from $[T]$ of size v . Then for any $\mathcal{R} \subseteq [P], |\mathcal{R}| = Q, i \in \mathcal{R}$, with all but negligible probability,

$$\Delta_i \setminus \bigcup_{j \neq i, j \in \mathcal{R}} \Delta_j \neq \emptyset$$

C MAFE with Trusted Setup

In this section, we define and show the construction of an important variant of MAFE, *MAFE with trusted setup* (tMAFE). In a tMAFE scheme, there is central authority that initiates the system by generating CRS and master public-secret key pairs for all authorities. So, we forgo the ASetup algorithm in this scheme and consider GSetup to output all the required parameters for setting up the system. In the next section, we show that this weakening of MAFE is enough to construct general MAFE from minimal assumptions. We formally define this object below and provide the construction.

Definition C.1 (tMAFE). A tMAFE scheme $(\text{GSetup}, \text{KGen}, \text{Enc}, \text{Dec})$ is said to be an MAFE scheme with trusted setup (tMAFE) for circuit class $\mathcal{C}$ and $\mathcal{GITD}$ if there is a central authority that

generates all system parameters (i.e, $\text{CRS}, \text{MPK}_{\text{id}}, \text{MSK}_{\text{id}}$ for all id). To reflect this, we ignore the ASetup algorithm and change GSetup algorithm as follows.

$\text{GSetup}(1^\lambda, 1^Q, 1^n, 1^s, 1^{\ell_1}, \dots, 1^{\ell_n}) \rightarrow (\text{CRS}, \{(\text{MPK}_{\text{id}}, \text{MSK}_{\text{id}})\}_{\text{id} \in [n]})$. The possibly randomized global setup takes the number of authorities n and input sizes for all authorities, $\ell_1, \dots, \ell_n$ as additional input, and outputs CRS , master public-secret key pairs $\{(\text{MPK}_{\text{id}}, \text{MSK}_{\text{id}})\}_{\text{id} \in [n]}$ for all authorities.

The security of the scheme is defined analogously.

C.1 Construction

We provide the construction of a tMAFE scheme for P/Poly circuits and $\mathcal{GID}$ as follows. At a high level, in [GY24] constructs a *statically* secure MAFE scheme from the minimal assumption of one-way functions using a PRF key embedded as part of the CRS . This is already an optimal construction if we are limited to static security. However, they use additional assumptions to realize an adaptively secure MAFE scheme such as random oracle, non-interactive key exchange, etc.

As explained in overview, our observation, if we restrict ourselves to the trusted setup model, we can embed the same PRF key as part of master secret key for each authorities. Since there is a centralized authority that distributes these keys and we do not consider authority corruptions, this key remains hidden and we can realize adaptive security easily. In Section 3 we show how we can use tMAFE in conjunction with BFE to construct a general MAFE scheme. Parts of the following construction are taken verbatim from [GY24].

Construction C.2 (tMAFE). We provide the construction of a tMAFE scheme for P/Poly circuits and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ using an MAFE for P/Poly circuits for *one* $\mathcal{GID}$. ([GY24]), a dCSF scheme for P/Poly circuits and $\mathcal{GID}$ (Definition B.1), and $\text{PRF} : \{0, 1\}^\lambda \times \mathcal{GID}_\lambda \rightarrow 2^{[N]}_D \times 2^{[T]}_v$ (Definition A.3) as follows. Here N, D, T, v are from Lemma B.2 and Lemma B.3.

$\text{GSetup}(1^\lambda, 1^Q, 1^n, 1^s, 1^{\ell_1}, \dots, 1^{\ell_n})$. $\forall u \in [N]$, sample $\text{mafe.crs}_u \leftarrow \text{1MAFE.GSetup}(1^\lambda, 1^n, 1^s)$. Set $\text{CRS} := (\text{mafe.crs}_u)_{u \in [N]}$. Sample $K \xleftarrow{\$} \{0, 1\}^\lambda$. $\forall u \in [N]$, $(\text{mafe.mpk}_{u,\text{id}}, \text{mafe.msk}_{u,\text{id}}) \leftarrow \text{1MAFE.ASetup}(\text{id}, 1^{\ell_{\text{id}}})$. Set $\text{MPK}_{\text{id}} := (\text{mafe.mpk}_{u,\text{id}})_{u \in [N]}$, and $\text{MSK}_{\text{id}} := (K, (\text{mafe.msk}_{u,\text{id}})_u)$. Output $\text{CRS}, \{(\text{MPK}_{\text{id}}, \text{MSK}_{\text{id}})\}_{\text{id} \in [n]}$.

$\text{KGen}(\text{id}, \text{MSK}_{\text{id}}, \{\text{MPK}_{\text{idx}}\}_{\text{idx} \in [n]}, \text{GID}, x)$. Parse MPK_{idx} as $(\text{mafe.mpk}_{u,\text{idx}})_{u \in [N]}$ for each $\text{idx} \in [n]$ and MSK_{id} as $(K, (\text{mafe.msk}_{u,\text{id}})_{u \in [N]})$. Compute $(\mathbf{S}, \mathbf{\Delta}) = \text{PRF}(K, \text{GID})$.

- $\{\hat{x}_{\text{GID},\text{id}}^u\}_{u \in [N]} = \text{dCSF.ServEnc}(1^\lambda, 1^Q, 1^n, 1^s, \text{GID}, \text{id}, x, \mathbf{\Delta})$.
- $\forall u \in \mathbf{S}$, $\text{mafe.sk}_u = \text{1MAFE.KGen}(\text{id}, \text{mafe.msk}_{u,\text{id}}, \{\text{mafe.mpk}_{u,\text{idx}}\}_{\text{idx}}, \hat{x}_{\text{GID},\text{id}}^u)$.

Output $\text{SK}_{\text{id},\text{GID},x} := (\mathbf{S}, \{\text{mafe.sk}_u\}_{u \in \mathbf{S}})$.

$\text{Enc}(\{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}, C)$. Parse MPK_{id} as $(\text{mafe.mpk}_{u,\text{id}})_{u \in [N]}$ for each $\text{id} \in [n]$. Sample $\{\hat{C}^u\}_{u \in [N]} \leftarrow \text{dCSF.CliEnc}(1^\lambda, 1^Q, 1^n, 1^s, C)$ and with $\hat{F}^u = \text{dCSF.UserComp}(\cdot, \dots, \cdot, \hat{C}^u)$, $\text{mafe.ct}_u \leftarrow \text{1MAFE.Enc}(\{\text{mafe.mpk}_{u,\text{id}}\}_{\text{id} \in [n]}, \hat{F}^u)$. Output $\text{CT} := (\text{mafe.ct}_u)_{u \in [N]}$.

$\text{Dec}(\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}, \text{CT})$. Parse $\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}$ as $(\mathbf{S}_{\text{id}}, \{\text{mafe.sk}_{u, \text{id}}\}_{u \in \mathbf{S}_{\text{id}}})$ and CT as $(\text{mafe.ct}_u)_{u \in [N]}$.
 If all $\mathbf{S}_{\text{id}}$'s are not the same, abort and output $\perp$.
 Otherwise, let $\mathbf{S} = \mathbf{S}_1$ and for each $u \in \mathbf{S}$, $\hat{y}_{\text{GID}}^u = \text{1MAFE.Dec}(\{\text{mafe.sk}_{u, \text{id}}\}_{\text{id}}, \text{mafe.ct}_u)$.
 Output $y = \text{dCSF.Decode}(\{\hat{y}_{\text{GID}}^u\}_{u \in \mathbf{S}}, \mathbf{S})$.

Theorem C.3. If 1MAFE is a 1-GID MAFE scheme for P/Poly circuits, dCSF is a secure dCSF scheme for P/Poly circuits and $\mathcal{GID}$ (Definition B.1), and $\text{PRF} : \{0, 1\}^\lambda \times \mathcal{GID}_\lambda \rightarrow 2^{[N]}_D \times 2^{[T]}_v$ is a secure PRF (Definition A.3), then Construction C.2 is a tMAFE scheme for P/Poly circuits and $\mathcal{GID}$.

Proof. The proof of this theorem follows closely the structure of proofs from [GY24]. We provide a sketch of the hybrid argument here.

Hybrid 0. This is the real experiment with honest challenger. We use GSetup, KGen, Enc algorithms to respond to adversary's queries.

Hybrid 1. In this hybrid, we will sample random $(\mathbf{S}, \Delta)$ for each of the Q GID queries during setup. As the PRF key is not leaked to the adversary, this change remains unnoticeable by the security of PRF.

Hybrid 2. In this hybrid, we will check if $\{\mathbf{S}_{\text{GID}}\}_{\text{GID}}$ obey Lemma B.2 and $\{\Delta_{\text{GID}}\}_{\text{GID}}$ obey Lemma B.3. Hybrids 1 and 2 are statistically indistinguishable due to those lemmas. We will also compute the set of corrupted users and set of unique indices in each Δ_{GID} .

Hybrid 3, j . for $j \in [N + 1]$. In this hybrid, we will simulate the first $j - 1$ 1MAFE instantiations if the corresponding u is not corrupted. The indistinguishability between hybrids can be argued from the security of 1MAFE.

Hybrid 4. In this hybrid, we know the set of corrupted users, unique indices and all the non-corrupted $u \in [N]$ only use outputs from dCSF. Hence, we will simulate dCSF. The indistinguishability of hybrids can be argued by security of the same. This is description of tMAFE.Sim.

The full is similar to the non-interactive key exchange based construction of MAFE from [GY24] and will be provided in the full version. $\square$

Corollary C.4 ([GY24]). Assuming the existence of public-key encryption, there exists a tMAFE scheme for P/Poly circuits and $\mathcal{GID}$.

Proof. As shown in [GY24], we can construct a 1MAFE scheme for P/Poly circuits and dCSF for P/Poly circuits and $\mathcal{GID}$ from PKE and one-way functions respectively. Hence, the proof of this corollary follows immediately. $\square$

Remark C.5. Assuming the existence of one-way functions, there exists a secret-key tMAFE scheme for P/Poly circuits and $\mathcal{GID}$.

D Proofs from Section 3

In this section, we provide full proof of Theorem 3.6.

Hyb₀^A(1^λ). This is the real experiment with Chal.

Setup. $\mathcal{A}$ sends Q, s to Chal. Chal sends $\text{CRS} := (\lambda, Q, s)$ to $\mathcal{A}$.

Auth Setup Queries. $\mathcal{A}$ sends $(\text{id}, \ell_{\text{id}})$. Chal samples $(\text{bfe.mpk}_{\text{id}}, \text{bfe.msk}_{\text{id}}) \leftarrow \text{BFE.Setup}(1^\lambda, 1^Q, 1^{s'})$. Set $\text{MPK}_{\text{id}} = (\ell_{\text{id}}, \text{bfe.mpk}_{\text{id}})$, and $\text{MSK}_{\text{id}} = (\ell_{\text{id}}, \text{bfe.msk}_{\text{id}})$. Chal sends MPK_{id} to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$. Chal samples $\text{bfe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{BFE.KGen}(\text{bfe.msk}_{\text{id}}, (\text{id}, \text{GID}, x))$ and sends $\text{SK}_{\text{id}, \text{GID}, x} := \text{bfe.sk}_{\text{id}, \text{GID}, x}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends circuit $\mathcal{I}, C$ to Chal. Let $n = |\mathcal{I}|$ and $\text{id}_1, \dots, \text{id}_n$ be the increasing ordering of elements in $\mathcal{I}$.

- Chal samples $\text{mafe.crs}, \{(\text{mafe.mpk}_i, \text{mafe.msk}_i)\}_{i \in [n]} \leftarrow \text{tMAFE.GSetup}(1^\lambda, 1^Q, 1^n, 1^s, 1^{\ell_{\text{id}_1}}, \dots, 1^{\ell_{\text{id}_n}})$.
- $\forall i \in [n], \text{bfe.ct}_{\text{id}_i} \leftarrow \text{BFE.Enc}(\text{bfe.mpk}_{\text{id}_i}, \text{tMAFE.KGen}(\cdot, \text{mafe.msk}_i, \{\text{mafe.mpk}_{i'}\}_{i', \cdot, \cdot}))$.
- Sample $\text{mafe.ct} \leftarrow \text{tMAFE.Enc}(\{\text{mafe.mpk}_i\}_{i \in [n]}, C)$.

Send $\text{CT} := (\text{mafe.ct}, \mathcal{I}, \{\text{bfe.ct}_{\text{id}}\}_{\text{id} \in \mathcal{I}})$ to $\mathcal{A}$.

Auth Setup Queries. $\mathcal{A}$ sends $(\text{id}, \ell_{\text{id}})$. Chal samples $(\text{bfe.mpk}_{\text{id}}, \text{bfe.msk}_{\text{id}}) \leftarrow \text{BFE.Setup}(1^\lambda, 1^Q, 1^{s'})$. Set $\text{MPK}_{\text{id}} = (\ell_{\text{id}}, \text{bfe.mpk}_{\text{id}})$, and $\text{MSK}_{\text{id}} = (\ell_{\text{id}}, \text{bfe.msk}_{\text{id}})$. Chal sends MPK_{id} to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$. Chal samples $\text{bfe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{BFE.KGen}(\text{bfe.msk}_{\text{id}}, (\text{id}, \text{GID}, x))$ and sends $\text{SK}_{\text{id}, \text{GID}, x} := \text{bfe.sk}_{\text{id}, \text{GID}, x}$ to $\mathcal{A}$.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb₁^A(1^λ). In this hybrid, we will simulate the pre-challenge queries using (S_0, S_1) of BFE instantiations. The changes are highlighted in **red**.

Setup. $\mathcal{A}$ sends Q, s to Chal. Chal sends $\text{CRS} := (\lambda, Q, s)$ to $\mathcal{A}$.

Auth Setup Queries. $\mathcal{A}$ sends $(\text{id}, \ell_{\text{id}})$. Chal samples $(\text{bfe.mpk}_{\text{id}}, \text{bfe.msk}_{\text{id}}, \text{bfe.st}_{\text{id}}) \leftarrow \text{BFE.S}_0(1^\lambda, 1^Q, 1^{\lambda + |\text{GID}| + \ell_{\text{id}}}, 1^{\kappa_{\text{id}}})$. Set $\text{MPK}_{\text{id}} = (\ell_{\text{id}}, \text{bfe.mpk}_{\text{id}})$, and $\text{MSK}_{\text{id}} = (\ell_{\text{id}}, \text{bfe.msk}_{\text{id}})$. Chal sends MPK_{id} to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$. Chal samples $\text{bfe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{BFE.S}_1(\text{bfe.mpk}_{\text{id}}, \text{bfe.msk}_{\text{id}}, \text{bfe.st}_{\text{id}}, (\text{id}, \text{GID}, x))$ and sends $\text{SK}_{\text{id}, \text{GID}, x} := \text{bfe.sk}_{\text{id}, \text{GID}, x}$ to $\mathcal{A}$.

Challenge Query, Post-challenge Queries, Guess Phase. Do this exactly like **Hyb₀^A(1^λ)**.

Hyb_{2,j}^A(1^λ). for $j \in [n+1]$. In this hybrid, we will simulate BFE instantiations of $\text{id}_1, \dots, \text{id}_{j-1} \in \mathcal{I}$. The changes are highlighted in **red**.

Setup. $\mathcal{A}$ sends Q, s . Send $\text{CRS} := (\lambda, Q, s)$ to $\mathcal{A}$.

Auth Setup Queries. $\mathcal{A}$ sends $(\text{id}, \ell_{\text{id}})$. Sample $(\text{bfe.mpk}_{\text{id}}, \text{bfe.msk}_{\text{id}}, \text{bfe.st}_{\text{id}}) \leftarrow \text{BFE.S}_0(1^\lambda, 1^Q, 1^{\lambda + |\text{GID}| + \ell_{\text{id}}}, 1^{\kappa_{\text{id}}})$. Set $\text{MPK}_{\text{id}} = (\ell_{\text{id}}, \text{bfe.mpk}_{\text{id}})$, and $\text{MSK}_{\text{id}} = (\ell_{\text{id}}, \text{bfe.msk}_{\text{id}})$. Send MPK_{id} to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$. Sample $\text{bfe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{BFE.S}_1(\text{bfe.mpk}_{\text{id}}, \text{bfe.msk}_{\text{id}}, \text{bfe.st}_{\text{id}}, (\text{id}, \text{GID}, x))$ and send $\text{SK}_{\text{id}, \text{GID}, x} := \text{bfe.sk}_{\text{id}, \text{GID}, x}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends circuit $\mathcal{I}, C$. Let $n = |\mathcal{I}|$ and $\text{id}_1, \dots, \text{id}_n$ be the increasing ordering of elements in $\mathcal{I}$.

- Sample $\text{mafe.crs}, \{(\text{mafe.mpk}_i, \text{mafe.msk}_i)\}_{i \in [n]} \leftarrow \text{tMAFE.GSetup}(1^\lambda, 1^Q, 1^n, 1^s, 1^{\ell_{\text{id}_1}}, \dots, 1^{\ell_{\text{id}_n}})$.
- $\forall i \in [n]$, if $i \geq j$, $\text{bfe.ct}_{\text{id}_i} \leftarrow \text{BFE.Enc}(\text{bfe.mpk}_{\text{id}_i}, F_i)$. Otherwise, $\text{bfe.ct}_{\text{id}_i} \leftarrow \text{BFE.S}_2(\text{st}_{\text{id}_i}, 1^{|F_i|}, \mathcal{V}_i)$. Here $F_i = \text{tMAFE.KGen}(\cdot, \text{mafe.msk}_i, \{\text{mafe.mpk}_{i'}\}_{i' \neq i}, \cdot, \cdot)$ and $\mathcal{V}_i = \{(X, F_i(X)) : X = (\text{id}_i, \text{GID}, x_{\text{GID}, \text{id}_i}) \text{ for all GID queries in previous phase}\}$.
- Sample $\text{mafe.ct} \leftarrow \text{tMAFE.Enc}(\{\text{mafe.mpk}_i\}_{i \in [n]}, C)$.

Send $\text{CT} := (\text{mafe.ct}, \mathcal{I}, \{\text{bfe.ct}_{\text{id}}\}_{\text{id} \in \mathcal{I}})$ to $\mathcal{A}$.

Auth Setup Queries. $\mathcal{A}$ sends $(\text{id}, \ell_{\text{id}})$. Sample $(\text{bfe.mpk}_{\text{id}}, \text{bfe.msk}_{\text{id}}) \leftarrow \text{BFE.Setup}(1^\lambda, 1^Q, 1^{\lambda+|\text{GID}|+\ell_{\text{id}}}, 1^{\kappa_{\text{id}}})$. Set $\text{MPK}_{\text{id}} = (\ell_{\text{id}}, \text{bfe.mpk}_{\text{id}})$, $\text{MSK}_{\text{id}} := (\ell_{\text{id}}, \text{bfe.msk}_{\text{id}})$. Send MPK_{id} to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$. If $\text{id} \in \{\text{id}_1, \dots, \text{id}_{j-1}\}$, sample $\text{bfe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{BFE.S}_3^{F_{\text{id}}(\cdot)}(\text{st}_{\text{id}}, (\text{id}, \text{GID}, x))$. Otherwise, $\text{bfe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{BFE.KGen}(\text{bfe.msk}_{\text{id}}, (\text{id}, \text{GID}, x))$. Send $\text{bfe.sk}_{\text{id}, \text{GID}, x}$ to $\mathcal{A}$.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb₃^A(1^λ). In this hybrid, we will simulate tMAFE. The changes are highlighted in red.

Setup. $\mathcal{A}$ sends Q, s . Send $\text{CRS} := (\lambda, Q, s)$ to $\mathcal{A}$.

Auth Setup Queries. $\mathcal{A}$ sends $(\text{id}, \ell_{\text{id}})$. Sample $(\text{bfe.mpk}_{\text{id}}, \text{bfe.msk}_{\text{id}}, \text{bfe.st}_{\text{id}}) \leftarrow \text{BFE.S}_0(1^\lambda, 1^Q, 1^{\lambda+|\text{GID}|+\ell_{\text{id}}}, 1^{\kappa_{\text{id}}})$. Set $\text{MPK}_{\text{id}} = (\ell_{\text{id}}, \text{bfe.mpk}_{\text{id}})$, and $\text{MSK}_{\text{id}} = (\ell_{\text{id}}, \text{bfe.msk}_{\text{id}})$. Send MPK_{id} to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$. Sample $\text{bfe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{BFE.S}_1(\text{bfe.mpk}_{\text{id}}, \text{bfe.msk}_{\text{id}}, \text{bfe.st}_{\text{id}}, (\text{id}, \text{GID}, x))$ and send $\text{SK}_{\text{id}, \text{GID}, x} := \text{bfe.sk}_{\text{id}, \text{GID}, x}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends circuit $\mathcal{I}, C$. Let $n = |\mathcal{I}|$ and $\text{id}_1, \dots, \text{id}_n$ be the increasing ordering of elements in $\mathcal{I}$.

- Sample $\text{mafe.crs}, \{\text{mafe.mpk}_{\text{id}}\}_{\text{id} \in [n]} \leftarrow \text{tMAFE.Sim}(1^\lambda, 1^Q, 1^n, 1^s, 1^{\ell_{\text{id}_1}}, \dots, 1^{\ell_{\text{id}_n}})$.
- $\forall i \in [n]$, $\text{bfe.ct}_{\text{id}_i} \leftarrow \text{BFE.S}_2(\text{st}_{\text{id}_i}, 1^{|F_i|}, \mathcal{V}_i)$. Here $\mathcal{V}_i = \{(X, \text{tMAFE.Sim}(\text{id}_i, \text{GID}, x_{\text{GID}, \text{id}_i})) : \text{for all GID queries in previous phase}\}$.
- Sample $\text{mafe.ct} \leftarrow \text{tMAFE.Sim}(1^{|C|}, \mathcal{V}_C)$ where $\mathcal{V}_C$ is as defined in Definition C.1.

Send $\text{CT} := (\text{mafe.ct}, \mathcal{I}, \{\text{bfe.ct}_{\text{id}}\}_{\text{id} \in \mathcal{I}})$ to $\mathcal{A}$.

Auth Setup Queries. $\mathcal{A}$ sends $(\text{id}, \ell_{\text{id}})$. Sample $(\text{bfe.mpk}_{\text{id}}, \text{bfe.msk}_{\text{id}}) \leftarrow \text{BFE.Setup}(1^\lambda, 1^Q, 1^{\lambda+|\text{GID}|+\ell_{\text{id}}}, 1^{\kappa_{\text{id}}})$. Set $\text{MPK}_{\text{id}} := (\ell_{\text{id}}, \text{bfe.mpk}_{\text{id}})$, $\text{MSK}_{\text{id}} := (\ell_{\text{id}}, \text{bfe.msk}_{\text{id}})$. Send MPK_{id} to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$. If $\text{id} \in \mathcal{I}$, sample $\text{bfe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{BFE.S}_3^{F'_{\text{id}}(\cdot)}(\text{st}_{\text{id}}, (\text{id}, \text{GID}, x))$. Otherwise, sample $\text{bfe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{BFE.KGen}(\text{bfe.msk}_{\text{id}}, (\text{id}, \text{GID}, x))$ and send $\text{bfe.sk}_{\text{id}, \text{GID}, x}$ to $\mathcal{A}$. Here $F'_{\text{id}}(\cdot)$ responds with $\text{tMAFE.Sim}(\text{id}, \text{GID}, x, V_C)$ where V_C is as defined in Definition C.1.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Lemma D.1. $\text{Hyb}_0^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_1^{\mathcal{A}}(1^\lambda)$ are identical.

Proof. This follows from the security of BFE where pre-challenge simulator $(\text{BFE.S}_0, \text{BFE.S}_1)$ behave exactly as $\text{BFE.Setup}, \text{BFE.KGen}$ respectively. $\square$

Lemma D.2. $\text{Hyb}_1^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_{2,1}^{\mathcal{A}}(1^\lambda)$ are identical.

Proof. As we are not simulating any instantiations of BFE in $\text{Hyb}_{2,1}^{\mathcal{A}}(1^\lambda)$, these hybrids are identically distributed. $\square$

Lemma D.3. Assuming the security of BFE, $\text{Hyb}_{2,j}^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_{2,j+1}^{\mathcal{A}}(1^\lambda)$ for $j \in [n]$ are computationally indistinguishable.

Proof. Assume that there exists a PPT adversary that can distinguish between $\text{Hyb}_{2,j}^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_{2,j+1}^{\mathcal{A}}(1^\lambda)$ with non-negligible advantage $\epsilon(\lambda)$, i.e.,

$$\left| \Pr \left[1 \leftarrow \text{Hyb}_{2,j}^{\mathcal{A}}(1^\lambda) \right] - \Pr \left[1 \leftarrow \text{Hyb}_{2,j+1}^{\mathcal{A}}(1^\lambda) \right] \right| > \epsilon(\lambda)$$

We will construct a reduction adversary $\mathcal{B}$ that can break the security of BFE with non-negligible probability. The description of $\mathcal{B}^{\mathcal{O}}$ is as follows.

Setup. $\mathcal{A}$ sends Q, s . Send $\text{CRS} := (\lambda, Q, s)$ to $\mathcal{A}$. As $\mathcal{A}$ runs in polynomial time, it makes $q_{\text{auth}} = \text{poly}(\lambda)$ many authority setup queries. Choose $N^* \xleftarrow{\$} [q_{\text{auth}}]$.

Auth Setup Queries. $\mathcal{A}$ sends $(\text{id}, \ell_{\text{id}})$. If this is the N^* -th authority setup query, receive $\text{bfe.mpk}_{\text{id}} \leftarrow \mathcal{O}(1^Q, 1^{\lambda+|\text{GID}|+\ell_{\text{id}}}, 1^{\kappa_{\text{id}}})$ and set $\text{id}^* = \text{id}$. Otherwise, sample $(\text{bfe.mpk}_{\text{id}}, \text{bfe.msk}_{\text{id}}, \text{bfe.st}_{\text{id}}) \leftarrow \text{BFE.S}_0(1^\lambda, 1^Q, 1^{\lambda+|\text{GID}|+\ell_{\text{id}}}, 1^{\kappa_{\text{id}}})$. Set $\text{MPK}_{\text{id}} = (\ell_{\text{id}}, \text{bfe.mpk}_{\text{id}})$, and $\text{MSK}_{\text{id}} = (\ell_{\text{id}}, \text{bfe.msk}_{\text{id}})$. Send MPK_{id} to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$. If $\text{id}^* = \text{id}$, $\text{bfe.sk}_{\text{id}, \text{GID}, x} \leftarrow \mathcal{O}(\text{id}, \text{GID}, x)$. Sample $\text{bfe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{BFE.S}_1(\text{bfe.mpk}_{\text{id}}, \text{bfe.msk}_{\text{id}}, \text{bfe.st}_{\text{id}}, (\text{id}, \text{GID}, x))$ and send $\text{SK}_{\text{id}, \text{GID}, x} := \text{bfe.sk}_{\text{id}, \text{GID}, x}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends circuit $\mathcal{I}, C$. Let $n = |\mathcal{I}|$ and $\text{id}_1, \dots, \text{id}_n$ be the increasing ordering of elements in $\mathcal{I}$. If $\text{id}_j \neq \text{id}^*$, abort and output $b' \xleftarrow{\$} \{0, 1\}$. Otherwise,

- Sample $\text{mafe.crs}, \{(\text{mafe.mpk}_i, \text{mafe.msk}_i)\}_{i \in [n]} \leftarrow \text{tMAFE.GSetup}(1^\lambda, 1^Q, 1^n, 1^s, 1^{\ell_{\text{id}_1}}, \dots, 1^{\ell_{\text{id}_n}})$.
- $\forall i \in [n]$, if $i > j$, $\text{bfe.ct}_{\text{id}_i} \leftarrow \text{BFE.Enc}(\text{bfe.mpk}_{\text{id}_i}, F_i)$. Otherwise, if $i = j$, $\text{bfe.ct}_{\text{id}_i} \leftarrow \mathcal{O}(F_i)$. Otherwise, $\text{bfe.ct}_{\text{id}_i} \leftarrow \text{BFE.S}_2(\text{st}_{\text{id}_i}, 1^{|F_i|}, \mathcal{V}_i)$. Here $F_i = \text{tMAFE.KGen}(\cdot, \text{mafe.msk}_i, \{\text{mafe.mpk}_{i'}\}_{i' \neq i}, \cdot, \cdot)$ and $\mathcal{V}_i = \{(X, F_i(X)) : \forall X = (\text{id}_i, \text{GID}, x_{\text{GID}, \text{id}_i})\}$.

- Sample $\text{mafe.ct} \leftarrow \text{tMAFE.Enc}(\{\text{mafe.mpk}_i\}_{i \in [n]}, C)$.

Send $\text{CT} := (\text{mafe.ct}, \mathcal{I}, \{\text{bfe.ct}_{\text{id}}\}_{\text{id} \in \mathcal{I}})$ to $\mathcal{A}$.

Auth Setup Queries. $\mathcal{A}$ sends $(\text{id}, \ell_{\text{id}})$. Sample $(\text{bfe.mpk}_{\text{id}}, \text{bfe.msk}_{\text{id}}) \leftarrow \text{BFE.Setup}(1^\lambda, 1^Q, 1^{s'})$. Set $\text{MPK}_{\text{id}} = (\ell_{\text{id}}, \text{bfe.mpk}_{\text{id}})$, $\text{MSK}_{\text{id}} = (\ell_{\text{id}}, \text{bfe.msk}_{\text{id}})$. Send MPK_{id} to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$. If $\text{id} \in \{\text{id}_1, \dots, \text{id}_{j-1}\}$, $\text{bfe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{BFE.S}_3^{F_{\text{id}}(\cdot)}(\text{st}_{\text{id}}, (\text{id}, \text{GID}, x))$. Otherwise, if $\text{id} = \text{id}_j$, $\text{bfe.sk}_{\text{id}, \text{GID}, x} \leftarrow \mathcal{O}(\text{id}, \text{GID}, x)$. Otherwise, $\text{bfe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{BFE.KGen}(\text{bfe.msk}_{\text{id}}, (\text{id}, \text{GID}, x))$ and send $\text{bfe.sk}_{\text{id}, \text{GID}, x}$ to $\mathcal{A}$.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

The probability that $\mathcal{B}$ doesn't abort is at most $1/q_{\text{auth}}$. In that case, the running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$ and λ . If $\mathcal{O}$ is $(\text{BFE.S}_0, \text{BFE.S}_1, \text{Enc}, \text{KGen})$ for BFE, $\mathcal{B}$ behaves as $\text{Hyb}_{2,j}^{\mathcal{A}}(1^\lambda)$ and if $\mathcal{O}$ is a BFE.Sim, $\mathcal{B}$ behaves like $\text{Hyb}_{2,j+1}^{\mathcal{A}}(1^\lambda)$. Hence, $\mathcal{B}$ is a valid adversary against the security of BFE that can break its security with non-negligible advantage, ϵ/q_{auth} . Thus, $\text{Hyb}_{2,j}^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_{2,j+1}^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable. $\square$

Lemma D.4. Assuming the security of tMAFE, $\text{Hyb}_{2,n+1}^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_3^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable.

Proof. Assume that there exists a PPT adversary that can distinguish between $\text{Hyb}_{2,n+1}^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_3^{\mathcal{A}}(1^\lambda)$ with non-negligible advantage $\epsilon(\lambda)$, i.e.,

$$\left| \Pr \left[1 \leftarrow \text{Hyb}_{2,n+1}^{\mathcal{A}}(1^\lambda) \right] - \Pr \left[1 \leftarrow \text{Hyb}_3^{\mathcal{A}}(1^\lambda) \right] \right| > \epsilon(\lambda)$$

We will construct a reduction adversary $\mathcal{B}$ that can break the security of tMAFE with non-negligible probability. The description of $\mathcal{B}^{\mathcal{O}}$ is as follows.

Setup. $\mathcal{A}$ sends Q, s . Send $\text{CRS} := (\lambda, Q, s)$ to $\mathcal{A}$.

Auth Setup Queries. $\mathcal{A}$ sends $(\text{id}, \ell_{\text{id}})$. Sample $(\text{bfe.mpk}_{\text{id}}, \text{bfe.msk}_{\text{id}}, \text{bfe.st}_{\text{id}}) \leftarrow \text{BFE.S}_0(1^\lambda, 1^Q, 1^{\lambda + |\text{GID}| + \ell_{\text{id}}}, 1^{\kappa_{\text{id}}})$. Set $\text{MPK}_{\text{id}} = (\ell_{\text{id}}, \text{bfe.mpk}_{\text{id}})$, and $\text{MSK}_{\text{id}} = (\ell_{\text{id}}, \text{bfe.msk}_{\text{id}})$. Send MPK_{id} to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$. Sample $\text{bfe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{BFE.S}_1(\text{bfe.mpk}_{\text{id}}, \text{bfe.msk}_{\text{id}}, \text{bfe.st}_{\text{id}}, (\text{id}, \text{GID}, x))$ and send $\text{SK}_{\text{id}, \text{GID}, x} := \text{bfe.sk}_{\text{id}, \text{GID}, x}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends circuit $\mathcal{I}, C$. Let $n = |\mathcal{I}|$ and $\text{id}_1, \dots, \text{id}_n$ be the increasing ordering of elements in $\mathcal{I}$.

- Receive $\text{mafe.crs}, \{\text{mafe.mpk}_{\text{id}}\}_{\text{id} \in [n]} \leftarrow \mathcal{O}(1^Q, 1^n, 1^s, 1^{\ell_{\text{id}_1}}, \dots, 1^{\ell_{\text{id}_n}})$.
- $\forall i \in [n]$, $\text{bfe.ct}_{\text{id}_i} \leftarrow \text{BFE.S}_2(\text{st}_{\text{id}_i}, 1^{|\mathcal{F}_i|}, \mathcal{V}_i)$. Here $\mathcal{V}_i = \{(X, \mathcal{O}(\text{id}_i, \text{GID}, x_{\text{GID}, \text{id}_i})) : \text{for all GID queries in previous phase}\}$.
- Sample $\text{mafe.ct} \leftarrow \mathcal{O}(C)$.

Send $\text{CT} := (\text{mafe.ct}, \mathcal{I}, \{\text{bfe.ct}_{\text{id}}\}_{\text{id} \in \mathcal{I}})$ to $\mathcal{A}$.

Auth Setup Queries. $\mathcal{A}$ sends $(\text{id}, \ell_{\text{id}})$. Sample $(\text{bfe.mpk}_{\text{id}}, \text{bfe.msk}_{\text{id}}) \leftarrow \text{BFE.Setup}(1^\lambda, 1^Q, 1^{\lambda+|\text{GID}|+\ell_{\text{id}}}, 1^{\kappa_{\text{id}}})$. Set $\text{MPK}_{\text{id}} := (\ell_{\text{id}}, \text{bfe.mpk}_{\text{id}})$, $\text{MSK}_{\text{id}} := (\ell_{\text{id}}, \text{bfe.msk}_{\text{id}})$. Send MPK_{id} to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$. If $\text{id} \in \mathcal{I}$, sample $\text{bfe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{BFE.S}_3^{F'_{\text{id}}(\cdot)}(\text{st}_{\text{id}}, (\text{id}, \text{GID}, x))$. Otherwise, sample $\text{bfe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{BFE.KGen}(\text{bfe.msk}_{\text{id}}, (\text{id}, \text{GID}, x))$ and send $\text{bfe.sk}_{\text{id}, \text{GID}, x}$ to $\mathcal{A}$. Here $F'_{\text{id}}(\cdot)$ responds with $\mathcal{O}(\text{id}, \text{GID}, x, V_C)$.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

And, the running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$ and λ . If $\mathcal{O}$ is an honest challenger for tMAFE, $\mathcal{B}$ behaves as $\text{Hyb}_{2,n+1}^{\mathcal{A}}(1^\lambda)$ and if $\mathcal{O}$ is a simulator, $\mathcal{B}$ behaves like $\text{Hyb}_3^{\mathcal{A}}(1^\lambda)$. Hence, $\mathcal{B}$ is a valid adversary against the security of tMAFE that can break its security with non-negligible advantage. Thus, $\text{Hyb}_{2,n+1}^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_3^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable. $\square$

E MAFE with Dynamic Collusions

Dynamic collusion model as introduced by [AMVY21, GGLW22] is a strengthening of the bounded query model [DKXY02, SS10, GLW12]. In this model encryptor can choose the collusion bound for a ciphertext. This allows the encryptor to set a desired collusion bound and provide more fine-grained control per encryption rather than adhere to strict collusion bound set during the system initiation. We refer the reader to [AMVY21, GGLW22] for an in-depth discussion on why the dynamic collusion model is much stronger and useful for BFE systems.

In this section, we construct an MAFE scheme in which an encryptor can dynamically set the query bound during the encryption and thus usher in all the advantages of the dynamic collusion setting to the multi-authority model for functional encryption systems. Concretely, the idea is that the GSetup, ASetup, and KGen algorithms of an MAFE scheme should be independent of the query bound Q and only the Enc and Dec algorithms should grow polynomially in Q . We formally define an MAFE scheme with dynamic collusions as follows.

E.1 Definitions

Definition E.1 (MAFE with Dynamic Collusions). MAFE with dynamic collusions is an MAFE scheme where GSetup, ASetup, and KGen algorithms are independent of query bound Q . Enc takes the query bound 1^Q as input. The security definition is also defined similarly to Definition 3.1 where $\mathcal{A}$ specifies the query bound 1^Q during challenge query phase.

Taking a closer look at Construction 3.5, as we use a BFE scheme in ASetup and KGen. Hence, if we switch to BFE with dynamic collusions, we should realize a dynamic query MAFE scheme readily. As we are initiating tMAFE in Enc, we can depend polynomially in Q and proceed as before. However, there is a technical subtlety that avoids this straight forward construction.

The issue is that BFE scheme is instantiated with circuit size $s' = \text{poly}(\lambda, Q, \ell)$ and depends on the query bound Q . This is because the output of tMAFE.KGen grows polynomially in Q . The main idea in this section is that if we can construct a tMAFE scheme where the GSetup, KGen and secret keys grow polynomially in $\lceil \log Q \rceil$, we can readily use BFE scheme with dynamic collusions to construct an MAFE scheme with dynamic collusions. More details are provided in Section E.4.

We call this object a “weakly optimal” tMAFE scheme (wotMAFE). This is analogous to weakly optimal BFE constructed in [GGLW22]. Definition of wotMAFE is as follows.

Definition E.2 (wotMAFE). A wotMAFE scheme ($\text{GSetup}, \text{KGen}, \text{Enc}, \text{Dec}$) is said to be a weakly optimal trusted MAFE scheme (wotMAFE) for circuit class $\mathcal{C} = \{\mathcal{C}_\lambda\}_{\lambda \in \mathbb{N}}$ and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ if the running time of GSetup and KGen is upper bounded by $\text{poly}(\lambda, s, \ell_1, \dots, \ell_n, \lceil \log Q \rceil)$. Running time of Enc can grow with Q . Hence, we alter the syntax of GSetup and Enc where GSetup takes the query bound Q in binary whereas Enc takes the query bound in unary, 1^Q .

E.2 Preliminaries

Our construction depends on garbled circuits (Definition A.1), pseudorandom functions (Definition A.3), Identity-Based Encryption (IBE) (Definition A.4). We also use dynamic BFE which is defined below.

Definition E.3 (Dynamic BFE). A BFE scheme ($\text{Setup}, \text{KGen}, \text{Enc}, \text{Dec}$) is said to be a dynamic query bounded BFE scheme for circuit class $\mathcal{C} = \{\mathcal{C}_\lambda : \{0, 1\}^{\ell_{\text{in}}} \rightarrow \{0, 1\}^{\ell_{\text{out}}}\}_{\lambda \in \mathbb{N}}$ if it satisfies Definition 3.3 and Setup no longer takes the query bound Q as input and instead Enc takes it as input. This also implies that the running times of Setup and KGen are independent of Q and are polynomial in λ, s and running time of Enc is polynomial in λ, Q, s . Security of the scheme is defined accordingly.

Remark E.4 ([AMVY21, GGLW22, GGL24]). A dynamic query bounded (ciphertext-policy) BFE scheme for any class of P/Poly circuits can be constructed from the minimal assumption of Identity-Based Encryption.

E.3 Weakly Optimal tMAFE from IBE and tMAFE

We construct a wotMAFE scheme using IBE and tMAFE below. The techniques used in Construction E.5 are similar to the construction of weakly optimal BFE from IBE as seen in [GGL24, GGLW22]. In addition, as noted by [AMVY21], IBE is the minimal assumption for wotMAFE.

Construction E.5 (wotMAFE). We provide the construction of a wotMAFE scheme for P/Poly circuits and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ using a tMAFE scheme for P/Poly circuits (Definition E.2), $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ (Definition C.1), an IBE scheme for identity space $[Q] \times \{0, 1\}^{O(\log \lambda)} \times \{0, 1\}$, garbling scheme ($\text{Garble}, \text{Eval}$) for P/Poly circuits (Definition A.1), and pseudorandom functions $\text{PRF}_0 : \{0, 1\}^\lambda \times \mathcal{GID} \rightarrow [Q]$, $\text{PRF}_1 : \{0, 1\}^\lambda \times [Q] \rightarrow \{0, 1\}^\lambda$ (Definition A.3) as follows.

$\text{GSetup}(1^\lambda, 1^n, 1^s, 1^{\ell_1}, \dots, 1^{\ell_n}, Q)$. Set $\text{CRS} := (\lambda, Q, n, s)$. Sample $K^{(0)}, K^{(1)} \xleftarrow{\$} \{0, 1\}^\lambda$. For each $\text{id} \in [n]$, $(\text{ibe.mpk}_{\text{id}}, \text{ibe.msk}_{\text{id}}) \leftarrow \text{IBE.Setup}(1^\lambda, 1^z)$ where $z = \lceil \log Q \rceil + \lceil \log L_{\text{id}} \rceil + 1$ (L_{id} defined in KGen).

Set $\text{MPK}_{\text{id}} := (\ell_{\text{id}}, \text{ibe.mpk}_{\text{id}})$, $\text{MSK}_{\text{id}} := (\ell_{\text{id}}, \text{ibe.msk}_{\text{id}}, K^{(0)}, K^{(1)})$. Output $(\text{CRS}, \{(\text{MPK}_{\text{id}}, \text{MSK}_{\text{id}})\}_{\text{id} \in [n]})$.

$\text{KGen}(\text{id}, \text{MSK}_{\text{id}}, \{\text{MPK}_{\text{id}_x}\}_{\text{id}_x \in [n]}, \text{GID}, x)$. Parse MSK_{id} as $(\ell_{\text{id}}, \text{ibe.msk}_{\text{id}}, K^{(0)}, K^{(1)})$ and $\forall \text{id}_x \in [n]$, MPK_{id_x} as $(\ell_{\text{id}_x}, \text{ibe.mpk}_{\text{id}_x})$. Compute $\text{Tag} = \text{PRF}_0(K^{(0)}, \text{GID})$ and $R = \text{PRF}_1(K^{(1)}, \text{Tag})$.

Using R deterministically sample $\text{mafe.crs}, \{(\text{mafe.mpk}_{\text{id}_x}, \text{mafe.msk}_{\text{id}_x})\}_{\text{id}_x} \leftarrow \text{tMAFE.GSetup}(1^\lambda, 1^q, 1^n, 1^s, 1^{\ell_1}, \dots, 1^{\ell_n}; R)$ where $q = \lambda$.

Sample secret key, $\text{mafe.sk}_{\text{id}, \text{GID}, x} = \text{tMAFE.KGen}(\text{id}, \text{mafe.msk}_{\text{id}}, \{\text{mafe.mpk}_{\text{id}_x}\}_{\text{id}_x}, \text{GID}, x)$.

Let $L_{\text{id}} = |\text{mafe.mpk}_{\text{id}}|$. $\forall i \in [L_{\text{id}}]$, $\text{ibe.sk}_{\text{Tag}, i} \leftarrow \text{IBE.KGen}(\text{ibe.msk}_{\text{id}}, (\text{Tag}, i, \text{mafe.mpk}_{\text{id}}[i]))^{20}$.

Output $\text{SK}_{\text{id}, \text{GID}, x} := (\text{Tag}, \text{mafe.mpk}_{\text{id}}, \text{mafe.sk}_{\text{id}, \text{GID}, x}, \{\text{ibe.sk}_{\text{Tag}, i}\}_{i \in [L_{\text{id}}]})$.

$\text{Enc}(1^Q, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}, C)$. Parse MPK_{id} as $(\ell_{\text{id}}, \text{ibe.mpk}_{\text{id}})$ for each $\text{id} \in [n]$. Let F be circuit that takes as input $\{\text{mafe.mpk}_{\text{id}_x}\}_{\text{id}_x \in [n]}$ and outputs $\text{tMAFE.Enc}(\{\text{mafe.mpk}_{\text{id}_x}\}_{\text{id}_x}, C; R)$ for some hardwired randomness R . $\forall \text{Tag} \in [Q]$,

- Sample $R^{\text{Tag}} \xleftarrow{\$} \{0, 1\}^\lambda$, garble $F^{\text{Tag}}, (\tilde{F}^{\text{Tag}}, \{w_{\text{id}, i, b}^{\text{Tag}}\}) \leftarrow \text{Garble}(1^\lambda, F^{\text{Tag}})$ where $\text{id} \in [n], i \in [L_{\text{id}}], b \in \{0, 1\}^{21}$.
- $\forall \text{id} \in [n]$, $\text{ibe.ct}_{\text{id}, i, b}^{\text{Tag}} \leftarrow \text{IBE.Enc}(\text{ibe.mpk}_{\text{id}}, (\text{Tag}, i, b), w_{\text{id}, i, b}^{\text{Tag}})$.

Output $\text{CT} := (\tilde{F}^{\text{Tag}}, \{\text{ibe.ct}_{\text{id}, i, b}^{\text{Tag}}\}_{\text{id}, i, b})_{\text{Tag} \in [Q]}$.

$\text{Dec}(\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}, \text{CT})$. Parse $\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}$ as $(\text{Tag}_{\text{id}}, \text{mafe.mpk}_{\text{id}}, \text{mafe.sk}_{\text{id}, \text{GID}, x_{\text{id}}}, \{\text{ibe.sk}_{\text{Tag}_{\text{id}}, i}\}_i)$ for each $\text{id} \in [n], i \in [L_{\text{id}}]$. If all Tag_{id} are not the same, abort and output $\perp$. Otherwise, set $\text{Tag} = \text{Tag}_1$.

Parse CT accordingly and use the Tag -th component, $\tilde{F}^{\text{Tag}}, \{\text{ibe.ct}_{\text{id}, i, b}^{\text{Tag}}\}$.

Perform IBE decryption $w_{\text{id}, i, \text{mafe.mpk}_{\text{id}}[i]}^{\text{Tag}} = \text{IBE.Dec}(\text{ibe.sk}_{\text{Tag}, i}, \text{ibe.ct}_{\text{id}, i, \text{mafe.mpk}_{\text{id}}[i]}^{\text{Tag}})$. Evaluate garbled circuit, $\text{mafe.ct} = \text{Eval}(\tilde{F}^{\text{Tag}}, \{w_{\text{id}, i, \text{mafe.mpk}_{\text{id}}[i]}^{\text{Tag}}\}_{\text{id}, i \in [L_{\text{id}}]})$.

Output $y := \text{tMAFE.Dec}(\{\text{mafe.sk}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}, \text{mafe.ct})$.

Correctness. The correctness of the scheme follows from the correctness of tMAFE , IBE , Garb , and deterministic nature of PRF . Concretely, by correctness of IBE , we get the wire labels for the $w_{\text{id}, i, \text{mafe.mpk}_{\text{id}}[i]}^{\text{Tag}}$ as we have secret keys for the identities $(\text{Tag}, i, \text{mafe.mpk}_{\text{id}}[i])$. Thus, by correctness of Garb , we have that $\text{mafe.ct} = \text{tMAFE.Enc}(\{\text{mafe.mpk}_{\text{id}}\}_{\text{id} \in [n]}, C; R^{\text{Tag}})$. Finally, by correctness of tMAFE , we have $y = C(x_1, \dots, x_n)$.

Efficiency. Note that running times of GSetup and KGen are only dependent on $\lceil \log Q \rceil$ as query bound q is set to λ and $[Q] \subseteq \{0, 1\}^{\lceil \log Q \rceil}$. The running times of Enc and Dec are polynomial in Q .

Theorem E.6. If tMAFE is a tMAFE scheme (Definition C.1) for P/Poly circuits, $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$, IBE is an IBE scheme (Definition A.4) for identity space $[Q] \times \{0, 1\}^{O(\log \lambda)} \times \{0, 1\}$, $(\text{Garble}, \text{Eval})$ is a garbling scheme (Definition A.1) for P/Poly circuits, and $\text{PRF}_0, \text{PRF}_1$ are pseudorandom functions (Definition A.3), then Construction E.5 is a wotMAFE scheme for P/Poly circuits, $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$.

Proof. We prove this theorem using the following hybrids and lemmas.

²⁰Here, $\text{mafe.mpk}_{\text{id}}[i]$ is the i -th bit of $\text{mafe.mpk}_{\text{id}}$.

²¹For notational convenience, we assume that wires are pre-partitioned this way.

Hyb₀^A(1^λ). This is the real experiment from Definition E.2.

Setup. $\mathcal{A}$ sends $(Q, n, s, \ell_1, \dots, \ell_n)$. Set $\text{CRS} := \text{mafe.crs}$. Sample $K^{(0)}, K^{(1)} \xleftarrow{\$} \{0, 1\}^\lambda$ and for each $\text{id} \in [n]$, $(\text{ibe.mpk}_{\text{id}}, \text{ibe.msk}_{\text{id}}) \leftarrow \text{IBE.Setup}(1^\lambda, 1^z)$. Set $\text{MPK}_{\text{id}} := (\ell_{\text{id}}, \text{ibe.mpk}_{\text{id}})$, and $\text{MSK}_{\text{id}} := (\ell_{\text{id}}, \text{ibe.msk}_{\text{id}}, K^{(0)}, K^{(1)})$. Send $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]})$ to $\mathcal{A}$.

Pre-challenge Queries. $\mathcal{A}$ makes Q_1 queries of the form $(\text{id}, \text{GID}, x)$. Chal computes $\text{Tag} = \text{PRF}_0(K^{(0)}, \text{GID})$, $R = \text{PRF}_1(K^{(1)}, \text{Tag})$, and $\text{mafe.crs}, \{(\text{mafe.mpk}_{\text{idx}}, \text{mafe.msk}_{\text{idx}})\}_{\text{idx}} \leftarrow \text{tMAFE.GSetup}(1^\lambda, 1^\lambda, 1^n, 1^s, 1^{\ell_1}, \dots, 1^{\ell_n}; R)$. For each $i \in [L_{\text{id}}]$, sample $\text{ibe.sk}_{\text{Tag}, i} \leftarrow \text{IBE.KGen}(\text{ibe.msk}_{\text{id}}, (\text{Tag}, i, \text{mafe.mpk}_{\text{id}}[i]))$. And, $\text{mafe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{tMAFE.KGen}(\text{id}, \text{mafe.msk}_{\text{id}}, \{\text{mafe.mpk}_{\text{idx}}\}_{\text{idx}}, \text{GID}, x)$. Send $\text{SK}_{\text{id}, \text{GID}, x} := (\text{Tag}, \text{mafe.mpk}_{\text{id}}, \text{mafe.sk}_{\text{id}, \text{GID}, x}, \{\text{ibe.sk}_{\text{Tag}, i}\}_{i \in [L_{\text{id}}]})$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends circuit C to Chal. For each $\text{Tag} \in [Q]$,

- Sample $R^{\text{Tag}} \leftarrow \{0, 1\}^\lambda$, $(\tilde{F}^{\text{Tag}}, \{w_{\text{id}, i, b}\}_{\text{id} \in [n], i \in [L_{\text{id}}], b \in \{0, 1\}}) \leftarrow \text{Garble}(1^\lambda, F)$.
- For each $\text{id} \in [n]$, $\text{ibe.ct}_{\text{id}, i, b}^{\text{Tag}} \leftarrow \text{IBE.Enc}(\text{ibe.mpk}_{\text{id}}, (\text{Tag}, i, b), w_{\text{id}, i, b}^{\text{Tag}})$.

Send $\text{CT} := (\tilde{F}^{\text{Tag}}, \{\text{ibe.ct}_{\text{id}, i, b}^{\text{Tag}}\}_{\text{id} \in [n], i \in [L_{\text{id}}], b \in \{0, 1\}})_{\text{Tag} \in [Q]}$ to $\mathcal{A}$.

Post-challenge Queries. $\mathcal{A}$ makes $Q - Q_1$ queries of the form $(\text{id}, \text{GID}, x)$. Chal computes $\text{Tag} = \text{PRF}_0(K^{(0)}, \text{GID})$, $R = \text{PRF}_1(K^{(1)}, \text{Tag})$, and $\text{mafe.crs}, \{(\text{mafe.mpk}_{\text{idx}}, \text{mafe.msk}_{\text{idx}})\}_{\text{idx}} \leftarrow \text{tMAFE.GSetup}(1^\lambda, 1^\lambda, 1^n, 1^s, 1^{\ell_1}, \dots, 1^{\ell_n}; R)$. For each $i \in [L_{\text{id}}]$, sample $\text{ibe.sk}_{\text{Tag}, i} \leftarrow \text{IBE.KGen}(\text{ibe.msk}_{\text{id}}, (\text{Tag}, i, \text{mafe.mpk}_{\text{id}}[i]))$. And, $\text{mafe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{tMAFE.KGen}(\text{id}, \text{mafe.msk}_{\text{id}}, \{\text{mafe.mpk}_{\text{idx}}\}_{\text{idx}}, \text{GID}, x)$. Send $\text{SK}_{\text{id}, \text{GID}, x} := (\text{Tag}, \text{mafe.mpk}_{\text{id}}, \text{mafe.sk}_{\text{id}, \text{GID}, x}, \{\text{ibe.sk}_{\text{Tag}, i}\}_{i \in [L_{\text{id}}]})$ to $\mathcal{A}$.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb₁^A(1^λ). In this hybrid, we will sample all the Tags used in key generation early. The changes are highlighted in red.

Setup. $\mathcal{A}$ sends $(Q, n, s, \ell_1, \dots, \ell_n)$. Set $\text{CRS} := \text{mafe.crs}$. Sample $K^{(0)}, K^{(1)} \xleftarrow{\$} \{0, 1\}^\lambda$ and for each $\text{id} \in [n]$, $(\text{ibe.mpk}_{\text{id}}, \text{ibe.msk}_{\text{id}}) \leftarrow \text{IBE.Setup}(1^\lambda, 1^z)$. Set $\text{MPK}_{\text{id}} := (\ell_{\text{id}}, \text{ibe.mpk}_{\text{id}})$, and $\text{MSK}_{\text{id}} := (\ell_{\text{id}}, \text{ibe.msk}_{\text{id}}, K^{(0)}, K^{(1)})$. Send $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]})$ to $\mathcal{A}$. **Sample $\text{Tag}_1, \dots, \text{Tag}_Q \xleftarrow{\$} [Q]$. Initiate counter $j = 1$ and dictionary $\mathcal{T}$.**

Pre-challenge Queries. $\mathcal{A}$ makes Q_1 queries of the form $(\text{id}, \text{GID}, x)$. **If $\text{GID} \in \mathcal{T}$, set $\text{Tag} = \mathcal{T}[\text{GID}]$. Otherwise, set $\mathcal{T}[\text{GID}] := \text{Tag}_j$, $\text{Tag} = \mathcal{T}[\text{GID}]$ and increment j .** Compute $R = \text{PRF}_1(K^{(1)}, \text{Tag})$, and $\text{mafe.crs}, \{(\text{mafe.mpk}_{\text{idx}}, \text{mafe.msk}_{\text{idx}})\}_{\text{idx}} \leftarrow \text{tMAFE.GSetup}(1^\lambda, 1^\lambda, 1^n, 1^s, 1^{\ell_1}, \dots, 1^{\ell_n}; R)$. For each $i \in [L_{\text{id}}]$, sample $\text{ibe.sk}_{\text{Tag}, i} \leftarrow \text{IBE.KGen}(\text{ibe.msk}_{\text{id}}, (\text{Tag}, i, \text{mafe.mpk}_{\text{id}}[i]))$. And, $\text{mafe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{tMAFE.KGen}(\text{id}, \text{mafe.msk}_{\text{id}}, \{\text{mafe.mpk}_{\text{idx}}\}_{\text{idx}}, \text{GID}, x)$. Send $\text{SK}_{\text{id}, \text{GID}, x} := (\text{Tag}, \text{mafe.mpk}_{\text{id}}, \text{mafe.sk}_{\text{id}, \text{GID}, x}, \{\text{ibe.sk}_{\text{Tag}, i}\}_{i \in [L_{\text{id}}]})$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends circuit C . For each $\text{Tag} \in [Q]$,

- Sample $R^{\text{Tag}} \leftarrow \{0, 1\}^\lambda$, $(\tilde{F}^{\text{Tag}}, \{w_{\text{id}, i, b}\}_{\text{id} \in [n], i \in [L_{\text{id}}], b \in \{0, 1\}}) \leftarrow \text{Garble}(1^\lambda, F)$.
- For each $\text{id} \in [n]$, $\text{ibe.ct}_{\text{id}, i, b}^{\text{Tag}} \leftarrow \text{IBE.Enc}(\text{ibe.mpk}_{\text{id}}, (\text{Tag}, i, b), w_{\text{id}, i, b}^{\text{Tag}})$.

Send $\text{CT} := (\tilde{F}^{\text{Tag}}, \{\text{ibe.ct}_{\text{id}, i, b}^{\text{Tag}}\}_{\text{id} \in [n], i \in [L_{\text{id}}], b \in \{0, 1\}})_{\text{Tag} \in [Q]}$ to $\mathcal{A}$.

Post-challenge Queries. $\mathcal{A}$ makes $Q - Q_1$ queries of the form $(\text{id}, \text{GID}, x)$. If $\text{GID} \in \mathcal{T}$, set $\text{Tag} = \mathcal{T}[\text{GID}]$. Otherwise, set $\mathcal{T}[\text{GID}] := \text{Tag}_j$, $\text{Tag} = \mathcal{T}[\text{GID}]$ and increment j . Compute $R = \text{PRF}_1(K^{(1)}, \text{Tag})$, and $\text{mafe.crs}, \{(\text{mafe.mpk}_{\text{id}_x}, \text{mafe.msk}_{\text{id}_x})\}_{\text{id}_x} \leftarrow \text{tMAFE.GSetup}(1^\lambda, 1^\lambda, 1^n, 1^s, 1^{\ell_1}, \dots, 1^{\ell_n}; R)$. For each $i \in [L_{\text{id}}]$, sample $\text{ibe.sk}_{\text{Tag}, i} \leftarrow \text{IBE.KGen}(\text{ibe.msk}_{\text{id}}, (\text{Tag}, i, \text{mafe.mpk}_{\text{id}}[i]))$. And, $\text{mafe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{tMAFE.KGen}(\text{id}, \text{mafe.msk}_{\text{id}}, \{\text{mafe.mpk}_{\text{id}_x}\}_{\text{id}_x}, \text{GID}, x)$. Send $\text{SK}_{\text{id}, \text{GID}, x} := (\text{Tag}, \text{mafe.mpk}_{\text{id}}, \text{mafe.sk}_{\text{id}, \text{GID}, x}, \{\text{ibe.sk}_{\text{Tag}, i}\}_{i \in [L_{\text{id}}]})$ to $\mathcal{A}$.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb₂^A(1^λ). In this hybrid, we check if any $\text{Tag} \in [Q]$ was sampled more than λ many times in previous hybrid. The changes are highlighted in red.

Setup. $\mathcal{A}$ sends $(Q, n, s, \ell_1, \dots, \ell_n)$. Set $\text{CRS} := \text{mafe.crs}$. Sample $K^{(1)} \xleftarrow{\$} \{0, 1\}^\lambda$ and for each $\text{id} \in [n]$, $(\text{ibe.mpk}_{\text{id}}, \text{ibe.msk}_{\text{id}}) \leftarrow \text{IBE.Setup}(1^\lambda, 1^z)$. Set $\text{MPK}_{\text{id}} := (\ell_{\text{id}}, \text{ibe.mpk}_{\text{id}})$, and $\text{MSK}_{\text{id}} := (\ell_{\text{id}}, \text{ibe.msk}_{\text{id}}, K^{(1)})$. Send $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]})$ to $\mathcal{A}$. Sample $\text{Tag}_1, \dots, \text{Tag}_Q \xleftarrow{\$} [Q]$. Initiate counter $j = 1$ and dictionary $\mathcal{T}$. If there exists a $\text{Tag} \in [Q]$ and indices $i_1^*, \dots, i_q^*$ such that $q > \lambda$ and $\text{Tag}_{i_1^*}, \dots, \text{Tag}_{i_q^*}$ are same as Tag , then abort and output $\perp$.

Pre-challenge, Challenge, Post-challenge Queries, Guess Phase. Same as **Hyb₁^A(1^λ)**.

Hyb₃^A(1^λ). In this hybrid, we will sample random strings R and thus random tMAFE instantiations instead of using PRF_1 . The changes are highlighted in red.

Setup. $\mathcal{A}$ sends $(Q, n, s, \ell_1, \dots, \ell_n)$. Set $\text{CRS} := \text{mafe.crs}$. Sample $K^{(1)} \xleftarrow{\$} \{0, 1\}^\lambda$ and for each $\text{id} \in [n]$, $(\text{ibe.mpk}_{\text{id}}, \text{ibe.msk}_{\text{id}}) \leftarrow \text{IBE.Setup}(1^\lambda, 1^z)$. Set $\text{MPK}_{\text{id}} := (\ell_{\text{id}}, \text{ibe.mpk}_{\text{id}})$, and $\text{MSK}_{\text{id}} := (\ell_{\text{id}}, \text{ibe.msk}_{\text{id}}, K^{(1)})$. Send $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]})$ to $\mathcal{A}$. Sample $\text{Tag}_1, \dots, \text{Tag}_Q \xleftarrow{\$} [Q]$. Initiate counter $j = 1$ and dictionary $\mathcal{T}$. If there exists a $\text{Tag} \in [Q]$ and indices $i_1^*, \dots, i_q^*$ such that $q > \lambda$ and $\text{Tag}_{i_1^*}, \dots, \text{Tag}_{i_q^*}$ are same as Tag , then abort and output $\perp$. Sample $R^{(1)}, \dots, R^{(Q)} \xleftarrow{\$} \{0, 1\}^\lambda$.

Pre-challenge Queries. $\mathcal{A}$ makes Q_1 queries of the form $(\text{id}, \text{GID}, x)$. If $\text{GID} \in \mathcal{T}$, set $\text{Tag} = \mathcal{T}[\text{GID}]$. Otherwise, set $\mathcal{T}[\text{GID}] := \text{Tag}_j$, $\text{Tag} = \mathcal{T}[\text{GID}]$. Increment j . Sample $\text{mafe.crs}, \{(\text{mafe.mpk}_{\text{id}_x}, \text{mafe.msk}_{\text{id}_x})\}_{\text{id}_x} \leftarrow \text{tMAFE.GSetup}(1^\lambda, 1^\lambda, 1^n, 1^s, 1^{\ell_1}, \dots, 1^{\ell_n}; R^{(\text{Tag})})$. For each $i \in [L_{\text{id}}]$, sample $\text{ibe.sk}_{\text{Tag}, i} \leftarrow \text{IBE.KGen}(\text{ibe.msk}_{\text{id}}, (\text{Tag}, i, \text{mafe.mpk}_{\text{id}}[i]))$. And, $\text{mafe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{tMAFE.KGen}(\text{id}, \text{mafe.msk}_{\text{id}}, \{\text{mafe.mpk}_{\text{id}_x}\}_{\text{id}_x}, \text{GID}, x)$. Send $\text{SK}_{\text{id}, \text{GID}, x} := (\text{Tag}, \text{mafe.mpk}_{\text{id}}, \text{mafe.sk}_{\text{id}, \text{GID}, x}, \{\text{ibe.sk}_{\text{Tag}, i}\}_{i \in [L_{\text{id}}]})$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends circuit C . For each $\text{Tag} \in [Q]$,

- Sample $R^{\text{Tag}} \leftarrow \{0, 1\}^\lambda$, $(\tilde{F}^{\text{Tag}}, \{w_{\text{id}, i, b}\}_{\text{id} \in [n], i \in [L_{\text{id}}], b \in \{0, 1\}}) \leftarrow \text{Garble}(1^\lambda, F)$.
- For each $\text{id} \in [n]$, $\text{ibe.ct}_{\text{id}, i, b}^{\text{Tag}} \leftarrow \text{IBE.Enc}(\text{ibe.mpk}_{\text{id}}, (\text{Tag}, i, b), w_{\text{id}, i, b}^{\text{Tag}})$.

Send $\text{CT} := (\tilde{F}^{\text{Tag}}, \{\text{ibe.ct}_{\text{id}, i, b}^{\text{Tag}}\}_{\text{id} \in [n], i \in [L_{\text{id}}], b \in \{0, 1\}})$ to $\mathcal{A}$.

Post-challenge Queries. $\mathcal{A}$ makes $Q - Q_1$ queries of the form $(\text{id}, \text{GID}, x)$. If $\text{GID} \in \mathcal{T}$, set $\text{Tag} = \mathcal{T}[\text{GID}]$. Otherwise, set $\mathcal{T}[\text{GID}] := \text{Tag}_j$, $\text{Tag} = \mathcal{T}[\text{GID}]$ and increment j . **Sample** $\text{mafe.crs}, \{(\text{mafe.mpk}_{\text{id}_x}, \text{mafe.msk}_{\text{id}_x})\}_{\text{id}_x} \leftarrow \text{tMAFE.GSetup}(1^\lambda, 1^\lambda, 1^n, 1^s, 1^{\ell_1}, \dots, 1^{\ell_n}; R^{(\text{Tag})})$. For each $i \in [L_{\text{id}}]$, sample $\text{ibe.sk}_{\text{Tag}, i} \leftarrow \text{IBE.KGen}(\text{ibe.msk}_{\text{id}}, (\text{Tag}, i, \text{mafe.mpk}_{\text{id}}[i]))$. And, $\text{mafe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{tMAFE.KGen}(\text{id}, \text{mafe.msk}_{\text{id}}, \{\text{mafe.mpk}_{\text{id}_x}\}_{\text{id}_x}, \text{GID}, x)$. Send $\text{SK}_{\text{id}, \text{GID}, x} := (\text{Tag}, \text{mafe.mpk}_{\text{id}}, \text{mafe.sk}_{\text{id}, \text{GID}, x}, \{\text{ibe.sk}_{\text{Tag}, i}\}_{i \in [L_{\text{id}}]})$ to $\mathcal{A}$.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb₄^A(1^λ). In this hybrid, we will all Q many tMAFE instantiations early and use the appropriate keys to generate secret keys. The changes are highlighted in **red**.

Setup. $\mathcal{A}$ sends $(Q, n, s, \ell_1, \dots, \ell_n)$. Set $\text{CRS} := \text{mafe.crs}$. Sample $K^{(1)} \xleftarrow{\$} \{0, 1\}^\lambda$ and for each $\text{id} \in [n]$, $(\text{ibe.mpk}_{\text{id}}, \text{ibe.msk}_{\text{id}}) \leftarrow \text{IBE.Setup}(1^\lambda, 1^z)$. Set $\text{MPK}_{\text{id}} := (\ell_{\text{id}}, \text{ibe.mpk}_{\text{id}})$, and $\text{MSK}_{\text{id}} := (\ell_{\text{id}}, \text{ibe.msk}_{\text{id}}, K^{(1)})$. Send $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]})$ to $\mathcal{A}$. Sample $\text{Tag}_1, \dots, \text{Tag}_Q \xleftarrow{\$} [Q]$. Initiate counter $j = 1$ and dictionary $\mathcal{T}$. If there exists a $\text{Tag} \in [Q]$ and indices $i_1^*, \dots, i_q^*$ such that $q > \lambda$ and $\text{Tag}_{i_1^*}, \dots, \text{Tag}_{i_q^*}$ are same as Tag , then abort and output $\perp$. **For each** $\text{Tag} \in [Q]$, **mafe.crs^{Tag}, $\{(\text{mafe.mpk}_{\text{id}_x}^{\text{Tag}}, \text{mafe.msk}_{\text{id}_x}^{\text{Tag}})\}_{\text{id}_x} \leftarrow \text{tMAFE.GSetup}(1^\lambda, 1^\lambda, 1^n, 1^s, 1^{\ell_1}, \dots, 1^{\ell_n})$.**

Pre-challenge Queries. $\mathcal{A}$ makes Q_1 queries of the form $(\text{id}, \text{GID}, x)$. If $\text{GID} \in \mathcal{T}$, set $\text{Tag} = \mathcal{T}[\text{GID}]$. Otherwise, set $\mathcal{T}[\text{GID}] := \text{Tag}_j$, $\text{Tag} = \mathcal{T}[\text{GID}]$ and increment j . For each $i \in [L_{\text{id}}]$, **sample** $\text{ibe.sk}_{\text{Tag}, i} \leftarrow \text{IBE.KGen}(\text{ibe.msk}_{\text{id}}, (\text{Tag}, i, \text{mafe.mpk}_{\text{id}}^{\text{Tag}}[i]))$. **And,** $\text{mafe.sk}_{\text{id}, \text{GID}, x}^{\text{Tag}} \leftarrow \text{tMAFE.KGen}(\text{id}, \text{mafe.msk}_{\text{id}}^{\text{Tag}}, \{\text{mafe.mpk}_{\text{id}_x}^{\text{Tag}}\}_{\text{id}_x}, \text{GID}, x)$. Send $\text{SK}_{\text{id}, \text{GID}, x} := (\text{Tag}, \text{mafe.mpk}_{\text{id}}^{\text{Tag}}, \text{mafe.sk}_{\text{id}, \text{GID}, x}^{\text{Tag}}, \{\text{ibe.sk}_{\text{Tag}, i}\}_i)$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends circuit C . For each $\text{Tag} \in [Q]$,

- Sample $R^{\text{Tag}} \leftarrow \{0, 1\}^\lambda$, $(\tilde{F}^{\text{Tag}}, \{w_{\text{id}, i, b}\}_{\text{id} \in [n], i \in [L_{\text{id}}], b \in \{0, 1\}}) \leftarrow \text{Garble}(1^\lambda, F)$.
- For each $\text{id} \in [n]$, $\text{ibe.ct}_{\text{id}, i, b}^{\text{Tag}} \leftarrow \text{IBE.Enc}(\text{ibe.mpk}_{\text{id}}, (\text{Tag}, i, b), w_{\text{id}, i, b}^{\text{Tag}})$.

Send $\text{CT} := (\tilde{F}^{\text{Tag}}, \{\text{ibe.ct}_{\text{id}, i, b}^{\text{Tag}}\}_{\text{id}, i, b})_{\text{Tag} \in [Q]}$ to $\mathcal{A}$.

Post-challenge Queries. $\mathcal{A}$ makes $Q - Q_1$ queries of the form $(\text{id}, \text{GID}, x)$. If $\text{GID} \in \mathcal{T}$, set $\text{Tag} = \mathcal{T}[\text{GID}]$. Otherwise, set $\mathcal{T}[\text{GID}] := \text{Tag}_j$, $\text{Tag} = \mathcal{T}[\text{GID}]$ and increment j . For each $i \in [L_{\text{id}}]$, **sample** $\text{ibe.sk}_{\text{Tag}, i} \leftarrow \text{IBE.KGen}(\text{ibe.msk}_{\text{id}}, (\text{Tag}, i, \text{mafe.mpk}_{\text{id}}^{\text{Tag}}[i]))$. **And,** $\text{mafe.sk}_{\text{id}, \text{GID}, x}^{\text{Tag}} \leftarrow \text{tMAFE.KGen}(\text{id}, \text{mafe.msk}_{\text{id}}^{\text{Tag}}, \{\text{mafe.mpk}_{\text{id}_x}^{\text{Tag}}\}_{\text{id}_x}, \text{GID}, x)$. Send $\text{SK}_{\text{id}, \text{GID}, x} := (\text{Tag}, \text{mafe.mpk}_{\text{id}}^{\text{Tag}}, \text{mafe.sk}_{\text{id}, \text{GID}, x}^{\text{Tag}}, \{\text{ibe.sk}_{\text{Tag}, i}\}_{i \in [L_{\text{id}}]})$ to $\mathcal{A}$.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb_{5,k}^A(1^λ). for $k \in [n + 1]$. In this hybrid, we will set half of the IBE encryptions to all zero strings of appropriate length for the first $k - 1$ authorities. The changes are highlighted in **red**.

Setup, Pre-challenge Queries. Same as **Hyb₄^A(1^λ)**.

Challenge Query. $\mathcal{A}$ sends circuit C . For each $\text{Tag} \in [Q]$,

- Sample $R^{\text{Tag}} \leftarrow \{0, 1\}^\lambda$, $(\tilde{F}^{\text{Tag}}, \{w_{\text{id},i,b}\}_{\text{id} \in [n], i \in [L_{\text{id}]}, b \in \{0,1\}}) \leftarrow \text{Garble}(1^\lambda, F)$.
- For each $\text{id} \in [n]$,
 - If $\text{id} < k$, $i \in [L_{\text{id}}]$, let $\beta = \text{mafe.mpk}_{\text{id}}^{\text{Tag}}[i]$, $\text{ibe.ct}_{\text{id},i,\beta}^{\text{Tag}} \leftarrow \text{IBE.Enc}(\text{ibe.mpk}_{\text{id}}, (\text{Tag}, i, \beta), w_{\text{id},i,\beta}^{\text{Tag}})$ and $\text{ibe.ct}_{\text{id},i,1-\beta}^{\text{Tag}} \leftarrow \text{IBE.Enc}(\text{ibe.mpk}_{\text{id}}, (\text{Tag}, i, 1-\beta), 0^{|w_{\text{id},i,1-\beta}^{\text{Tag}}|})$.
 - Otherwise, for $i \in [L_{\text{id}}]$, $b \in \{0, 1\}$, $\text{ibe.ct}_{\text{id},i,b}^{\text{Tag}} \leftarrow \text{IBE.Enc}(\text{ibe.mpk}_{\text{id}}, (\text{Tag}, i, b), w_{\text{id},i,b}^{\text{Tag}})$

Send $\text{CT} := (\tilde{F}^{\text{Tag}}, \{\text{ibe.ct}_{\text{id},i,b}^{\text{Tag}}\}_{\text{id},i,b})_{\text{Tag} \in [Q]}$ to $\mathcal{A}$.

Post-challenge Queries, Guess Phase. Same as $\text{Hyb}_4^A(1^\lambda)$.

$\text{Hyb}_{6,t}^A(1^\lambda)$. for $t \in [Q+1]$. In this hybrid, we will simulate the garbling of F for each $\text{Tag} < t$. The changes are highlighted in **red**.

Setup, Pre-challenge Queries. Same as $\text{Hyb}_{5,n+1}^A(1^\lambda)$.

Challenge Query. $\mathcal{A}$ sends circuit C . For each $\text{Tag} \in [Q]$,

- If $\text{Tag} < t$ ($\tilde{F}^{\text{Tag}}, \{w_{\text{id},i,\text{mafe.mpk}_{\text{id}}^{\text{Tag}}[i]}\}_{\text{id},i} \leftarrow \text{Garb.Sim}(1^\lambda, 1^L, 1^{|F|}, \text{mafe.ct}^{\text{Tag}})$ where $L = L_1 + \dots + L_n$ and $\text{mafe.ct}^{\text{Tag}} \leftarrow \text{tMAFE.Enc}(\{\text{mafe.mpk}_{\text{id}}^{\text{Tag}}\}_{\text{id} \in [n]}, C)$).
- Otherwise, $R^{\text{Tag}} \leftarrow \{0, 1\}^\lambda$, $(\tilde{F}^{\text{Tag}}, \{w_{\text{id},i,b}\}_{\text{id} \in [n], i \in [L_{\text{id}]}, b \in \{0,1\}}) \leftarrow \text{Garble}(1^\lambda, F)$.
- $\forall \text{id} \in [n]$, $i \in [L_{\text{id}}]$, let $\beta = \text{mafe.mpk}_{\text{id}}^{\text{Tag}}[i]$, $\text{ibe.ct}_{\text{id},i,\beta}^{\text{Tag}} \leftarrow \text{IBE.Enc}(\text{ibe.mpk}_{\text{id}}, (\text{Tag}, i, \beta), w_{\text{id},i,\beta}^{\text{Tag}})$ and $\text{ibe.ct}_{\text{id},i,1-\beta}^{\text{Tag}} \leftarrow \text{IBE.Enc}(\text{ibe.mpk}_{\text{id}}, (\text{Tag}, i, 1-\beta), 0^{|w_{\text{id},i,1-\beta}^{\text{Tag}}|})$.

Send $\text{CT} := (\tilde{F}^{\text{Tag}}, \{\text{ibe.ct}_{\text{id},i,b}^{\text{Tag}}\}_{\text{id},i,b})_{\text{Tag} \in [Q]}$ to $\mathcal{A}$.

Post-challenge Queries, Guess Phase. Same as $\text{Hyb}_{5,n+1}^A(1^\lambda)$.

$\text{Hyb}_{7,t}^A(1^\lambda)$. for $t \in [Q+1]$. In this hybrid, we will simulate the Tag -th tMAFE instantiations for $\text{Tag} < t$. The changes are highlighted in **red**. Here, $\mathcal{V}^{\text{Tag}}$ is the set of all input and outputs relations for C as defined in Definition C.1 except we only consider the queries where Tag is used.

Setup. $\mathcal{A}$ sends $(Q, n, s, \ell_1, \dots, \ell_n)$. Set $\text{CRS} := \text{mafe.crs}$. Sample $K^{(1)} \xleftarrow{\$} \{0, 1\}^\lambda$ and for each $\text{id} \in [n]$, $(\text{ibe.mpk}_{\text{id}}, \text{ibe.msk}_{\text{id}}) \leftarrow \text{IBE.Setup}(1^\lambda, 1^z)$. Set $\text{MPK}_{\text{id}} := (\ell_{\text{id}}, \text{ibe.mpk}_{\text{id}})$, and $\text{MSK}_{\text{id}} := (\ell_{\text{id}}, \text{ibe.msk}_{\text{id}}, K^{(1)})$. Send $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]})$ to $\mathcal{A}$. Sample $\text{Tag}_1, \dots, \text{Tag}_Q \xleftarrow{\$} [Q]$. Initiate counter $j = 1$ and dictionary $\mathcal{T}$. If there exists a $\text{Tag} \in [Q]$ and indices $i_1^*, \dots, i_q^*$ such that $q > \lambda$ and $\text{Tag}_{i_1^*}, \dots, \text{Tag}_{i_q^*}$ are same as Tag , then abort and output $\perp$. **For each $\text{Tag} \in [Q]$, if $\text{Tag} < t$, $\text{mafe.crs}^{\text{Tag}}, \{\text{mafe.mpk}_{\text{id}}^{\text{Tag}}\}_{\text{id}} \leftarrow \text{tMAFE.Sim}_{\text{Tag}}(1^\lambda, 1^\lambda, 1^n, 1^s, 1^{\ell_1}, \dots, 1^{\ell_n})$. Otherwise, $\text{mafe.crs}^{\text{Tag}}, \{(\text{mafe.mpk}_{\text{id}}^{\text{Tag}}, \text{mafe.msk}_{\text{id}}^{\text{Tag}})\}_{\text{id}} \leftarrow \text{tMAFE.GSetup}(1^\lambda, 1^\lambda, 1^n, 1^s, 1^{\ell_1}, \dots, 1^{\ell_n})$.**

Pre-challenge Queries. $\mathcal{A}$ makes Q_1 queries of the form $(\text{id}, \text{GID}, x)$. If $\text{GID} \in \mathcal{T}$, set $\text{Tag} = \mathcal{T}[\text{GID}]$. Otherwise, set $\mathcal{T}[\text{GID}] := \text{Tag}_j$, $\text{Tag} = \mathcal{T}[\text{GID}]$ and increment j . For each $i \in [L_{\text{id}}]$, sample $\text{ibe.sk}_{\text{Tag},i} \leftarrow \text{IBE.KGen}(\text{ibe.msk}_{\text{id}}, (\text{Tag}, i, \text{mafe.mpk}_{\text{id}}^{\text{Tag}}[i]))$. And, if $\text{Tag} < t$, $\text{mafe.sk}_{\text{id}, \text{GID}, x}^{\text{Tag}} \leftarrow \text{tMAFE.Sim}_{\text{Tag}}(\text{id}, \text{GID}, x)$. Otherwise, $\text{mafe.sk}_{\text{id}, \text{GID}, x}^{\text{Tag}} \leftarrow \text{tMAFE.KGen}(\text{id}, \text{mafe.msk}_{\text{id}}^{\text{Tag}}, \{\text{mafe.mpk}_{\text{id}, x}^{\text{Tag}}\}_{\text{id}, x}, \text{GID}, x)$. Send $\text{SK}_{\text{id}, \text{GID}, x} := (\text{Tag}, \text{mafe.mpk}_{\text{id}}^{\text{Tag}}, \text{mafe.sk}_{\text{id}, \text{GID}, x}^{\text{Tag}}, \{\text{ibe.sk}_{\text{Tag}, i}\}_{i \in [L_{\text{id}}]})$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends circuit C . For each $\text{Tag} \in [Q]$,

- $(\tilde{F}^{\text{Tag}}, \{w_{\text{id}, i, \text{mafe.mpk}_{\text{id}}^{\text{Tag}}[i]}\}_{\text{id}, i}) \leftarrow \text{Garb.Sim}(1^\lambda, 1^L, 1^{|F|}, \text{mafe.ct}^{\text{Tag}})$ where $L = L_1 + \dots + L_n$ and if $\text{Tag} < t$, $\text{mafe.ct}^{\text{Tag}} \leftarrow \text{tMAFE.Sim}(1^{|C|}, \mathcal{V}^{\text{Tag}})$. Otherwise, $\text{mafe.ct}^{\text{Tag}} \leftarrow \text{tMAFE.Enc}(\{\text{mafe.mpk}_{\text{id}, x}^{\text{Tag}}\}_{\text{id}, x \in [n]}, C)$.
- $\forall \text{id} \in [n], i \in [L_{\text{id}}]$, let $\beta = \text{mafe.mpk}_{\text{id}}^{\text{Tag}}[i]$, $\text{ibe.ct}_{\text{id}, i, \beta}^{\text{Tag}} \leftarrow \text{IBE.Enc}(\text{ibe.mpk}_{\text{id}}, (\text{Tag}, i, \beta), w_{\text{id}, i, \beta}^{\text{Tag}})$ and $\text{ibe.ct}_{\text{id}, i, 1-\beta}^{\text{Tag}} \leftarrow \text{IBE.Enc}(\text{ibe.mpk}_{\text{id}}, (\text{Tag}, i, 1-\beta), 0^{|w_{\text{id}, i, 1-\beta}^{\text{Tag}}|})$.

Send $\text{CT} := (\tilde{F}^{\text{Tag}}, \{\text{ibe.ct}_{\text{id}, i, b}^{\text{Tag}}\}_{\text{id}, i, b})_{\text{Tag} \in [Q]}$ to $\mathcal{A}$.

Post-challenge Queries. $\mathcal{A}$ makes $Q - Q_1$ queries of the form $(\text{id}, \text{GID}, x)$. If $\text{GID} \in \mathcal{T}$, set $\text{Tag} = \mathcal{T}[\text{GID}]$. Otherwise, set $\mathcal{T}[\text{GID}] := \text{Tag}_j$, $\text{Tag} = \mathcal{T}[\text{GID}]$ and increment j . For each $i \in [L_{\text{id}}]$, sample $\text{ibe.sk}_{\text{Tag}, i} \leftarrow \text{IBE.KGen}(\text{ibe.msk}_{\text{id}}, (\text{Tag}, i, \text{mafe.mpk}_{\text{id}}^{\text{Tag}}[i]))$. And, if $\text{Tag} < t$, $\text{mafe.sk}_{\text{id}, \text{GID}, x}^{\text{Tag}} \leftarrow \text{tMAFE.Sim}_{\text{Tag}}^{C(\cdot)}(\text{id}, \text{GID}, x)$. Otherwise, $\text{mafe.sk}_{\text{id}, \text{GID}, x}^{\text{Tag}} \leftarrow \text{tMAFE.KGen}(\text{id}, \text{mafe.msk}_{\text{id}}^{\text{Tag}}, \{\text{mafe.mpk}_{\text{id}, x}^{\text{Tag}}\}_{\text{id}, x}, \text{GID}, x)$. Send $\text{SK}_{\text{id}, \text{GID}, x} := (\text{Tag}, \text{mafe.mpk}_{\text{id}}^{\text{Tag}}, \text{mafe.sk}_{\text{id}, \text{GID}, x}^{\text{Tag}}, \{\text{ibe.sk}_{\text{Tag}, i}\}_{i \in [L_{\text{id}}]})$ to $\mathcal{A}$.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Lemma E.7. Assuming the pseudorandomness of PRF_0 , $\text{Hyb}_0^A(1^\lambda)$ and $\text{Hyb}_1^A(1^\lambda)$ are computationally indistinguishable.

Proof. Assume that there exists a PPT adversary that can distinguish between $\text{Hyb}_0^A(1^\lambda)$ and $\text{Hyb}_1^A(1^\lambda)$ with non-negligible advantage $\epsilon(\lambda)$, i.e,

$$\left| \Pr \left[1 \leftarrow \text{Hyb}_0^A(1^\lambda) \right] - \Pr \left[1 \leftarrow \text{Hyb}_1^A(1^\lambda) \right] \right| > \epsilon(\lambda)$$

We will construct a reduction adversary $\mathcal{B}$ that can break the pseudorandomness of PRF_0 with non-negligible probability. The description of $\mathcal{B}^\mathcal{O}$ is as follows.

Setup. $\mathcal{A}$ sends $(Q, n, s, \ell_1, \dots, \ell_n)$. Set $\text{CRS} := \text{mafe.crs}$. Sample $K^{(0)}, K^{(1)} \xleftarrow{\$} \{0, 1\}^\lambda$ and for each $\text{id} \in [n]$, $(\text{ibe.mpk}_{\text{id}}, \text{ibe.msk}_{\text{id}}) \leftarrow \text{IBE.Setup}(1^\lambda, 1^z)$. Set $\text{MPK}_{\text{id}} := (\ell_{\text{id}}, \text{ibe.mpk}_{\text{id}})$, and $\text{MSK}_{\text{id}} := (\ell_{\text{id}}, \text{ibe.msk}_{\text{id}}, K^{(0)}, K^{(1)})$. Send $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]})$ to $\mathcal{A}$.

Pre-challenge Queries. $\mathcal{A}$ makes Q_1 queries of the form $(\text{id}, \text{GID}, x)$. Let $\text{Tag} \leftarrow \mathcal{O}(\text{GID})$. Compute $R = \text{PRF}_1(K^{(1)}, \text{Tag})$, and $\text{mafe.crs}, \{(\text{mafe.mpk}_{\text{id}, x}, \text{mafe.msk}_{\text{id}, x})\}_{\text{id}, x} \leftarrow \text{tMAFE.GSetup}(1^\lambda, 1^\lambda, 1^n, 1^s, 1^{\ell_1}, \dots, 1^{\ell_n}; R)$. For each $i \in [L_{\text{id}}]$, sample $\text{ibe.sk}_{\text{Tag}, i} \leftarrow \text{IBE.KGen}(\text{ibe.msk}_{\text{id}}, (\text{Tag}, i, \text{mafe.mpk}_{\text{id}}[i]))$. And, $\text{mafe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{tMAFE.KGen}(\text{id}, \text{mafe.msk}_{\text{id}}, \{\text{mafe.mpk}_{\text{id}, x}\}_{\text{id}, x}, \text{GID}, x)$. Send $\text{SK}_{\text{id}, \text{GID}, x} := (\text{Tag}, \text{mafe.mpk}_{\text{id}}, \text{mafe.sk}_{\text{id}, \text{GID}, x}, \{\text{ibe.sk}_{\text{Tag}, i}\}_{i \in [L_{\text{id}}]})$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends circuit C . For each $\text{Tag} \in [Q]$,

- Sample $R^{\text{Tag}} \leftarrow \{0, 1\}^\lambda$, $(\tilde{F}^{\text{Tag}}, \{w_{\text{id},i,b}\}_{\text{id} \in [n], i \in [L_{\text{id}]}, b \in \{0,1\}}) \leftarrow \text{Garble}(1^\lambda, F)$.
- For each $\text{id} \in [n]$, $\text{ibe.ct}_{\text{id},i,b}^{\text{Tag}} \leftarrow \text{IBE.Enc}(\text{ibe.mpk}_{\text{id}}, (\text{Tag}, i, b), w_{\text{id},i,b}^{\text{Tag}})$.

Send $\text{CT} := (\tilde{F}^{\text{Tag}}, \{\text{ibe.ct}_{\text{id},i,b}^{\text{Tag}}\}_{\text{id},i,b})_{\text{Tag} \in [Q]}$ to $\mathcal{A}$.

Post-challenge Queries. $\mathcal{A}$ makes $Q - Q_1$ queries of the form $(\text{id}, \text{GID}, x)$. Let $\text{Tag} \leftarrow \mathcal{O}(\text{GID})$. Compute $R = \text{PRF}_1(K^{(1)}, \text{Tag})$, and $\text{mafe.crs}, \{(\text{mafe.mpk}_{\text{id}_x}, \text{mafe.msk}_{\text{id}_x})\}_{\text{id}_x} \leftarrow \text{tMAFE.GSetup}(1^\lambda, 1^\lambda, 1^n, 1^s, 1^{\ell_1}, \dots, 1^{\ell_n}; R)$. For each $i \in [L_{\text{id}}]$, sample $\text{ibe.sk}_{\text{Tag},i} \leftarrow \text{IBE.KGen}(\text{ibe.msk}_{\text{id}}, (\text{Tag}, i, \text{mafe.mpk}_{\text{id}}[i]))$. And, $\text{mafe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{tMAFE.KGen}(\text{id}, \text{mafe.msk}_{\text{id}}, \{\text{mafe.mpk}_{\text{id}_x}\}_{\text{id}_x}, \text{GID}, x)$. Send $\text{SK}_{\text{id}, \text{GID}, x} := (\text{Tag}, \text{mafe.mpk}_{\text{id}}, \text{mafe.sk}_{\text{id}, \text{GID}, x}, \{\text{ibe.sk}_{\text{Tag},i}\}_{i \in [L_{\text{id}}]})$ to $\mathcal{A}$.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

As we can see, the running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$ and λ . If $\mathcal{O}$ is a honest challenger for PRF_0 , $\mathcal{B}$ behaves as $\mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)$ and if $\mathcal{O}$ samples outputs randomly, $\mathcal{B}$ behaves like $\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$. Hence, $\mathcal{B}$ is a valid adversary against the pseudorandomness of PRF_0 that can breaks its security with non-negligible advantage. Thus, $\mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable. $\square$

Lemma E.8. $\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ are statistically indistinguishable.

Proof. The proof of this lemma follows from Lemma 5.2 of [GGLW22] or Lemma 3.4 of [AMVY21] or Claim 1 of [AV19]. $\square$

Lemma E.9. Assuming the pseudorandomness of PRF_1 , $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable.

Proof. Assume that there exists a PPT adversary that can distinguish between $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$ with non-negligible advantage $\epsilon(\lambda)$, i.e.,

$$\left| \Pr \left[1 \leftarrow \mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda) \right] - \Pr \left[1 \leftarrow \mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda) \right] \right| > \epsilon(\lambda)$$

We will construct a reduction adversary $\mathcal{B}$ that can break the pseudorandomness of PRF_1 with non-negligible probability. The description of $\mathcal{B}^{\mathcal{O}}$ is as follows.

Setup. $\mathcal{A}$ sends $(Q, n, s, \ell_1, \dots, \ell_n)$. Set $\text{CRS} := \text{mafe.crs}$. Sample $K^{(1)} \xleftarrow{\$} \{0, 1\}^\lambda$ and for each $\text{id} \in [n]$, $(\text{ibe.mpk}_{\text{id}}, \text{ibe.msk}_{\text{id}}) \leftarrow \text{IBE.Setup}(1^\lambda, 1^z)$. Set $\text{MPK}_{\text{id}} := (\ell_{\text{id}}, \text{ibe.mpk}_{\text{id}})$, and $\text{MSK}_{\text{id}} := (\ell_{\text{id}}, \text{ibe.msk}_{\text{id}}, K^{(1)})$. Send $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]})$ to $\mathcal{A}$. Sample $\text{Tag}_1, \dots, \text{Tag}_Q \xleftarrow{\$} [Q]$. Initiate counter $j = 1$ and dictionary $\mathcal{T}$. If there exists a $\text{Tag} \in [Q]$ and indices $i_1^*, \dots, i_q^*$ such that $q > \lambda$ and $\text{Tag}_{i_1^*}, \dots, \text{Tag}_{i_q^*}$ are same as Tag , then abort and output $\perp$.

Pre-challenge Queries. $\mathcal{A}$ makes Q_1 queries of the form $(\text{id}, \text{GID}, x)$. If $\text{GID} \in \mathcal{T}$, set $\text{Tag} = \mathcal{T}[\text{GID}]$. Otherwise, set $\mathcal{T}[\text{GID}] := \text{Tag}_j$, $\text{Tag} = \mathcal{T}[\text{GID}]$ and increment j . Sample $\text{mafe.crs}, \{(\text{mafe.mpk}_{\text{id}_x}, \text{mafe.msk}_{\text{id}_x})\}_{\text{id}_x} \leftarrow \text{tMAFE.GSetup}(1^\lambda, 1^\lambda, 1^n, 1^s, 1^{\ell_1}, \dots, 1^{\ell_n}; \mathcal{O}(\text{Tag}))$. For each $i \in [L_{\text{id}}]$, sample $\text{ibe.sk}_{\text{Tag},i} \leftarrow \text{IBE.KGen}(\text{ibe.msk}_{\text{id}}, (\text{Tag}, i, \text{mafe.mpk}_{\text{id}}[i]))$. And, $\text{mafe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{tMAFE.KGen}(\text{id}, \text{mafe.msk}_{\text{id}}, \{\text{mafe.mpk}_{\text{id}_x}\}_{\text{id}_x}, \text{GID}, x)$. Send $\text{SK}_{\text{id}, \text{GID}, x} := (\text{Tag}, \text{mafe.mpk}_{\text{id}}, \text{mafe.sk}_{\text{id}, \text{GID}, x}, \{\text{ibe.sk}_{\text{Tag},i}\}_{i \in [L_{\text{id}}]})$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends circuit C . For each $\text{Tag} \in [Q]$,

- Sample $R^{\text{Tag}} \leftarrow \{0, 1\}^\lambda$, $(\tilde{F}^{\text{Tag}}, \{w_{\text{id}, i, b}\}_{\text{id} \in [n], i \in [L_{\text{id}]}, b \in \{0, 1\}}) \leftarrow \text{Garble}(1^\lambda, F)$.
- For each $\text{id} \in [n]$, $\text{ibe.ct}_{\text{id}, i, b}^{\text{Tag}} \leftarrow \text{IBE.Enc}(\text{ibe.mpk}_{\text{id}}, (\text{Tag}, i, b), w_{\text{id}, i, b}^{\text{Tag}})$.

Send $\text{CT} := (\tilde{F}^{\text{Tag}}, \{\text{ibe.ct}_{\text{id}, i, b}^{\text{Tag}}\}_{\text{id}, i, b})_{\text{Tag} \in [Q]}$ to $\mathcal{A}$.

Post-challenge Queries. $\mathcal{A}$ makes $Q - Q_1$ queries of the form $(\text{id}, \text{GID}, x)$. If $\text{GID} \in \mathcal{T}$, set $\text{Tag} = \mathcal{T}[\text{GID}]$. Otherwise, set $\mathcal{T}[\text{GID}] := \text{Tag}_j$, $\text{Tag} = \mathcal{T}[\text{GID}]$ and increment j . Sample $\text{mafe.crs}, \{(\text{mafe.mpk}_{\text{id}_x}, \text{mafe.msk}_{\text{id}_x})\}_{\text{id}_x} \leftarrow \text{tMAFE.GSetup}(1^\lambda, 1^\lambda, 1^n, 1^s, 1^{\ell_1}, \dots, 1^{\ell_n}; \mathcal{O}(\text{Tag}))$. For each $i \in [L_{\text{id}}]$, sample $\text{ibe.sk}_{\text{Tag}, i} \leftarrow \text{IBE.KGen}(\text{ibe.msk}_{\text{id}}, (\text{Tag}, i, \text{mafe.mpk}_{\text{id}}[i]))$. And, $\text{mafe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{tMAFE.KGen}(\text{id}, \text{mafe.msk}_{\text{id}}, \{\text{mafe.mpk}_{\text{id}_x}\}_{\text{id}_x}, \text{GID}, x)$. Send $\text{SK}_{\text{id}, \text{GID}, x} := (\text{Tag}, \text{mafe.mpk}_{\text{id}}, \text{mafe.sk}_{\text{id}, \text{GID}, x}, \{\text{ibe.sk}_{\text{Tag}, i}\}_{i \in [L_{\text{id}}]})$ to $\mathcal{A}$.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

As we can see, the running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$ and λ . If $\mathcal{O}$ is a honest challenger for PRF_1 , $\mathcal{B}$ behaves as $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ and if $\mathcal{O}$ samples outputs randomly, $\mathcal{B}$ behaves like $\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$. Hence, $\mathcal{B}$ is a valid adversary against the pseudorandomness of PRF_1 that can breaks its security with non-negligible advantage. Thus, $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable. $\square$

Lemma E.10. $\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_4^{\mathcal{A}}(1^\lambda)$ are identical.

Proof. As we are sampling $R^{(\text{Tag})}$ uniformly in both the hybrids (explicitly in $\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$ and implicitly in $\mathbf{Hyb}_4^{\mathcal{A}}(1^\lambda)$), it follows from the output distribution that both of these hybrids are identical. $\square$

Lemma E.11. $\mathbf{Hyb}_4^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_{5,1}^{\mathcal{A}}(1^\lambda)$ are identical.

Proof. As we are not substituting any encryptions of IBE with zero strings in $\mathbf{Hyb}_{5,1}^{\mathcal{A}}(1^\lambda)$ for any authorities, both these hybrids remain identically distributed. $\square$

Lemma E.12. Assuming the multi-challenge message security of IBE, $\mathbf{Hyb}_{5,k}^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_{5,k+1}^{\mathcal{A}}(1^\lambda)$ for any $k \in [n]$ are computationally indistinguishable.

Proof. Note that the only difference between $\mathbf{Hyb}_{5,k}^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_{5,k+1}^{\mathcal{A}}(1^\lambda)$ is that for the k -th authority we generate ciphertexts differently in $\mathbf{Hyb}_{5,k+1}^{\mathcal{A}}(1^\lambda)$. Assume that there exists a PPT adversary that can distinguish between $\mathbf{Hyb}_{5,k}^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_{5,k+1}^{\mathcal{A}}(1^\lambda)$ with non-negligible advantage $\epsilon(\lambda)$, i.e.,

$$\left| \Pr \left[1 \leftarrow \mathbf{Hyb}_{5,k}^{\mathcal{A}}(1^\lambda) \right] - \Pr \left[1 \leftarrow \mathbf{Hyb}_{5,k+1}^{\mathcal{A}}(1^\lambda) \right] \right| > \epsilon(\lambda)$$

We will construct a reduction adversary $\mathcal{B}$ that can break the multi-challenge security of IBE with non-negligible probability. The description of $\mathcal{B}^{\mathcal{O}}$ is as follows.

Setup, Pre-challenge Queries. Same as $\mathbf{Hyb}_{5,k}^{\mathcal{A}}(1^\lambda)$ except that set ibe.mpk_k that is received from $\mathcal{O}$. And $\text{ibe.msk}_k = \perp$. For pre-challenge queries, when we need secret keys for k -th authority, query $\mathcal{O}((\text{Tag}, i, \text{mafe.mpk}_k^{\text{Tag}}[i]))$ for $i \in [L_k]$.

Challenge Query. $\mathcal{A}$ sends circuit C . For each $\text{Tag} \in [Q]$,

- Sample $R^{\text{Tag}} \leftarrow \{0, 1\}^\lambda$, $(\tilde{F}^{\text{Tag}}, \{w_{\text{id},i,b}\}_{\text{id} \in [n], i \in [L_{\text{id}}], b \in \{0,1\}}) \leftarrow \text{Garble}(1^\lambda, F)$.
- For each $\text{id} \in [n]$,
 - If $\text{id} < k, i \in [L_{\text{id}}]$, let $\beta = \text{mafe.mpk}_{\text{id}}^{\text{Tag}}[i]$, $\text{ibe.ct}_{\text{id},i,\beta}^{\text{Tag}} \leftarrow \text{IBE.Enc}(\text{ibe.mpk}_{\text{id}}, (\text{Tag}, i, \beta), w_{\text{id},i,\beta}^{\text{Tag}})$ and $\text{ibe.ct}_{\text{id},i,1-\beta}^{\text{Tag}} \leftarrow \text{IBE.Enc}(\text{ibe.mpk}_{\text{id}}, (\text{Tag}, i, 1-\beta), 0^{|w_{\text{id},i,1-\beta}^{\text{Tag}}|})$.
 - Otherwise, if $\text{id} = k, i \in [L_{\text{id}}]$, let $\beta = \text{mafe.mpk}_{\text{id}}^{\text{Tag}}[i]$, $\text{ibe.ct}_{\text{id},i,\beta}^{\text{Tag}} \leftarrow \text{IBE.Enc}(\text{ibe.mpk}_{\text{id}}, (\text{Tag}, i, \beta), w_{\text{id},i,\beta}^{\text{Tag}})$ and $\text{ibe.ct}_{\text{id},i,1-\beta}^{\text{Tag}} \leftarrow \mathcal{O}((\text{Tag}, i, 1-\beta), w_{\text{id},i,1-\beta}^{\text{Tag}})$.
 - Otherwise, for $i \in [L_{\text{id}}], b \in \{0, 1\}$, $\text{ibe.ct}_{\text{id},i,b}^{\text{Tag}} \leftarrow \text{IBE.Enc}(\text{ibe.mpk}_{\text{id}}, (\text{Tag}, i, b), w_{\text{id},i,b}^{\text{Tag}})$

Send $\text{CT} := (\tilde{F}^{\text{Tag}}, \{\text{ibe.ct}_{\text{id},i,b}^{\text{Tag}}\}_{\text{id},i,b})_{\text{Tag} \in [Q]}$ to $\mathcal{A}$.

Post-challenge Queries, Guess Phase. Same as $\text{Hyb}_{5,k}^A(1^\lambda)$ except for post-challenge queries, when we need secret keys for k -th authority, query $\mathcal{O}((\text{Tag}, i, \text{mafe.mpk}_k^{\text{Tag}}[i]))$ for $i \in [L_k]$.

Note that the identities queried by $\mathcal{B}$ during encryption are not the same as any identity that $\mathcal{B}$ queried a secret key for. And, the running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$ and λ . If $\mathcal{O}$ encrypts for all $\text{Tag} \in [Q]$, $w_{\text{id},i,1-\beta[i]}^{\text{Tag}}$ in $\text{ibe.ct}_{\text{id},i,1-\beta[i]}^{\text{Tag}}$, $\mathcal{B}$ behaves as $\text{Hyb}_{5,k}^A(1^\lambda)$ and if $\mathcal{O}$ encrypts all zero string, $\mathcal{B}$ behaves like $\text{Hyb}_{5,k+1}^A(1^\lambda)$. Hence, $\mathcal{B}$ is a valid adversary against the security of IBE that can break its multi-challenge security with non-negligible advantage. Thus, $\text{Hyb}_{5,k}^A(1^\lambda)$ and $\text{Hyb}_{5,k+1}^A(1^\lambda)$ are computationally indistinguishable. $\square$

Lemma E.13. $\text{Hyb}_{5,n+1}^A(1^\lambda)$ and $\text{Hyb}_{6,1}^A(1^\lambda)$ are identical.

Proof. As we are not simulating any garbled circuits for any Tags $\text{Hyb}_{5,1}^A(1^\lambda)$, these two hybrids remain identically distributed. $\square$

Lemma E.14. Assuming the security of Garb , $\text{Hyb}_{6,t}^A(1^\lambda)$ and $\text{Hyb}_{6,t+1}^A(1^\lambda)$ for any $t \in [Q]$ are computationally indistinguishable.

Proof. Assume that there exists a PPT adversary that can distinguish between $\text{Hyb}_{6,t}^A(1^\lambda)$ and $\text{Hyb}_{6,t+1}^A(1^\lambda)$ with non-negligible advantage $\epsilon(\lambda)$, i.e.,

$$\left| \Pr \left[1 \leftarrow \text{Hyb}_{6,t}^A(1^\lambda) \right] - \Pr \left[1 \leftarrow \text{Hyb}_{6,t+1}^A(1^\lambda) \right] \right| > \epsilon(\lambda)$$

We will construct a reduction adversary $\mathcal{B}$ that can break the security of Garb with non-negligible probability. The description of $\mathcal{B}^\mathcal{O}$ is as follows.

Setup, Pre-challenge Queries. Same as $\text{Hyb}_{5,n+1}^A(1^\lambda)$.

Challenge Query. $\mathcal{A}$ sends circuit C . For each $\text{Tag} \in [Q]$,

- If $\text{Tag} < t$ $(\tilde{F}^{\text{Tag}}, \{w_{\text{id},i,\text{mafe.mpk}_{\text{id}}^{\text{Tag}}[i]}\}_{\text{id},i}) \leftarrow \text{Garb.Sim}(1^\lambda, 1^L, 1^{|F|}, \text{mafe.ct}^{\text{Tag}})$ where $L = L_1 + \dots + L_n$ and $\text{mafe.ct}^{\text{Tag}} \leftarrow \text{tMAFE.Enc}(\{\text{mafe.mpk}_{\text{id}_x}^{\text{Tag}}\}_{\text{id}_x \in [n]}, C)$.

- Otherwise, $R^{\text{Tag}} \leftarrow \{0, 1\}^\lambda$, if $\text{Tag} = t$, $(\tilde{F}^{\text{Tag}}, \{w_{\text{id}, i, \text{mafe.mpk}_{\text{id}}^{\text{Tag}}[i]}^{\text{Tag}}\}_{\text{id}, i}) \leftarrow \mathcal{O}(F^{\text{Tag}}, \{\text{mafe.mpk}_{\text{id}}^{\text{Tag}}\}_{\text{id}})$.
- Otherwise, $R^{\text{Tag}} \leftarrow \{0, 1\}^\lambda$, $(\tilde{F}^{\text{Tag}}, \{w_{\text{id}, i, b}\}_{\text{id} \in [n], i \in [L_{\text{id}}], b \in \{0, 1\}}) \leftarrow \text{Garble}(1^\lambda, F)$.
- For each $\text{id} \in [n]$, $i \in [L_{\text{id}}]$, let $\beta = \text{mafe.mpk}_{\text{id}}^{\text{Tag}}[i]$, $\text{ibe.ct}_{\text{id}, i, \beta}^{\text{Tag}} \leftarrow \text{IBE.Enc}(\text{ibe.mpk}_{\text{id}}, (\text{Tag}, i, \beta), w_{\text{id}, i, \beta}^{\text{Tag}})$ and $\text{ibe.ct}_{\text{id}, i, 1-\beta}^{\text{Tag}} \leftarrow \text{IBE.Enc}(\text{ibe.mpk}_{\text{id}}, (\text{Tag}, i, 1-\beta), 0^{|w_{\text{id}, i, 1-\beta}^{\text{Tag}}|})$.

Send $\text{CT} := (\tilde{F}^{\text{Tag}}, \{\text{ibe.ct}_{\text{id}, i, b}^{\text{Tag}}\}_{\text{id}, i, b})_{\text{Tag} \in [Q]}$ to $\mathcal{A}$.

Post-challenge Queries, Guess Phase. Same as $\mathbf{Hyb}_{5, n+1}^A(1^\lambda)$.

And, the running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$ and λ . If $\mathcal{O}$ is an honest challenger for Garb , $\mathcal{B}$ behaves as $\mathbf{Hyb}_{6, t}^A(1^\lambda)$ and if $\mathcal{O}$ simulated the garbled circuit, $\mathcal{B}$ behaves like $\mathbf{Hyb}_{6, t+1}^A(1^\lambda)$. Hence, $\mathcal{B}$ is a valid adversary against the security of Garb that can break its security with non-negligible advantage. Thus, $\mathbf{Hyb}_{6, t}^A(1^\lambda)$ and $\mathbf{Hyb}_{6, t+1}^A(1^\lambda)$ are computationally indistinguishable. $\square$

Lemma E.15. $\mathbf{Hyb}_{6, Q+1}^A(1^\lambda)$ and $\mathbf{Hyb}_{7, 1}^A(1^\lambda)$ are identical.

Proof. As we are not simulating any tMAFE instantiations for any Tags in $\mathbf{Hyb}_{7, 1}^A(1^\lambda)$, these two hybrids remain identically distributed. $\square$

Lemma E.16. Assuming the adaptive security of tMAFE, $\mathbf{Hyb}_{7, t}^A(1^\lambda)$ and $\mathbf{Hyb}_{7, t+1}^A(1^\lambda)$ for any $t \in [Q]$ are computationally indistinguishable.

Proof. Assume that there exists a PPT adversary that can distinguish between $\mathbf{Hyb}_{7, t}^A(1^\lambda)$ and $\mathbf{Hyb}_{7, t+1}^A(1^\lambda)$ with non-negligible advantage $\epsilon(\lambda)$, i.e.,

$$\left| \Pr \left[1 \leftarrow \mathbf{Hyb}_{7, t}^A(1^\lambda) \right] - \Pr \left[1 \leftarrow \mathbf{Hyb}_{7, t+1}^A(1^\lambda) \right] \right| > \epsilon(\lambda)$$

We will construct a reduction adversary $\mathcal{B}$ that can break the security of tMAFE with non-negligible probability. The description of $\mathcal{B}^\mathcal{O}$ is as follows.

Setup. $\mathcal{A}$ sends $(Q, n, s, \ell_1, \dots, \ell_n)$. Set $\text{CRS} := \text{mafe.crs}$. Sample $K^{(1)} \xleftarrow{\$} \{0, 1\}^\lambda$ and for each $\text{id} \in [n]$, $(\text{ibe.mpk}_{\text{id}}, \text{ibe.msk}_{\text{id}}) \leftarrow \text{IBE.Setup}(1^\lambda, 1^z)$. Set $\text{MPK}_{\text{id}} := (\ell_{\text{id}}, \text{ibe.mpk}_{\text{id}})$, and $\text{MSK}_{\text{id}} := (\ell_{\text{id}}, \text{ibe.msk}_{\text{id}}, K^{(1)})$. Send $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]})$ to $\mathcal{A}$. Sample $\text{Tag}_1, \dots, \text{Tag}_Q \xleftarrow{\$} [Q]$. Initiate counter $j = 1$ and dictionary $\mathcal{T}$. If there exists a $\text{Tag} \in [Q]$ and indices $i_1^*, \dots, i_q^*$ such that $q > \lambda$ and $\text{Tag}_{i_1^*}, \dots, \text{Tag}_{i_q^*}$ are same as Tag , then abort and output $\perp$. For each $\text{Tag} \in [Q]$, if $\text{Tag} < t$, $\text{mafe.crs}^{\text{Tag}}, \{\text{mafe.mpk}_{\text{id}}^{\text{Tag}}\}_{\text{id}} \leftarrow \text{tMAFE.Sim}_{\text{Tag}}(1^\lambda, 1^\lambda, 1^n, 1^s, 1^{\ell_1}, \dots, 1^{\ell_n})$. Otherwise, if $\text{Tag} = t$, $\text{mafe.crs}^{\text{Tag}}, \{\text{mafe.mpk}_{\text{id}}^{\text{Tag}}\}_{\text{id}} \leftarrow \mathcal{O}(1^\lambda, 1^\lambda, 1^n, 1^s, 1^{\ell_1}, \dots, 1^{\ell_n})$. Otherwise, $\text{mafe.crs}^{\text{Tag}}, \{(\text{mafe.mpk}_{\text{id}}^{\text{Tag}}, \text{mafe.msk}_{\text{id}}^{\text{Tag}})\}_{\text{id}} \leftarrow \text{tMAFE.GSetup}(1^\lambda, 1^\lambda, 1^n, 1^s, 1^{\ell_1}, \dots, 1^{\ell_n})$.

Pre-challenge Queries. $\mathcal{A}$ makes Q_1 queries of the form $(\text{id}, \text{GID}, x)$. If $\text{GID} \in \mathcal{T}$, set $\text{Tag} = \mathcal{T}[\text{GID}]$. Otherwise, set $\mathcal{T}[\text{GID}] := \text{Tag}_j$, $\text{Tag} = \mathcal{T}[\text{GID}]$ and increment j . For each $i \in [L_{\text{id}}]$, sample $\text{ibe.sk}_{\text{Tag}, i} \leftarrow \text{IBE.KGen}(\text{ibe.msk}_{\text{id}}, (\text{Tag}, i, \text{mafe.mpk}_{\text{id}}^{\text{Tag}}[i]))$. And, if $\text{Tag} < t$,

$\text{mafe.sk}_{\text{id,GID},x}^{\text{Tag}} \leftarrow \text{tMAFE.Sim}_{\text{Tag}}(\text{id}, \text{GID}, x)$. Otherwise, if $\text{Tag} = t$, $\text{mafe.sk}_{\text{id,GID},x}^{\text{Tag}} \leftarrow \mathcal{O}(\text{id}, \text{GID}, x)$. Otherwise, $\text{mafe.sk}_{\text{id,GID},x}^{\text{Tag}} \leftarrow \text{tMAFE.KGen}(\text{id}, \text{mafe.msk}_{\text{id}}^{\text{Tag}}, \{\text{mafe.mpk}_{\text{id}_x}^{\text{Tag}}\}_{\text{id}_x}, \text{GID}, x)$. Send $\text{SK}_{\text{id,GID},x} := (\text{Tag}, \text{mafe.mpk}_{\text{id}}^{\text{Tag}}, \text{mafe.sk}_{\text{id,GID},x}^{\text{Tag}}, \{\text{ibe.sk}_{\text{Tag},i}\}_{i \in [L_{\text{id}}]})$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends circuit C . For each $\text{Tag} \in [Q]$,

- $(\tilde{F}^{\text{Tag}}, \{w_{\text{id},i,\text{mafe.mpk}_{\text{id}}^{\text{Tag}}[i]}\}_{\text{id},i}) \leftarrow \text{Garb.Sim}(1^\lambda, 1^L, 1^{|F|}, \text{mafe.ct}^{\text{Tag}})$ where $L = L_1 + \dots + L_n$ and if $\text{Tag} < t$, $\text{mafe.ct}^{\text{Tag}} \leftarrow \text{tMAFE.Sim}(1^{|C|}, \nu^{\text{Tag}})$. Otherwise, if $\text{Tag} = t$, $\text{mafe.ct}^{\text{Tag}} \leftarrow \mathcal{O}(C)$. Otherwise, $\text{mafe.ct}^{\text{Tag}} \leftarrow \text{tMAFE.Enc}(\{\text{mafe.mpk}_{\text{id}_x}^{\text{Tag}}\}_{\text{id}_x \in [n]}, C)$.
- For each $\text{id} \in [n]$, $i \in [L_{\text{id}}]$, let $\beta = \text{mafe.mpk}_{\text{id}}^{\text{Tag}}[i]$, $\text{ibe.ct}_{\text{id},i,\beta}^{\text{Tag}} \leftarrow \text{IBE.Enc}(\text{ibe.mpk}_{\text{id}}, (\text{Tag}, i, \beta), w_{\text{id},i,\beta}^{\text{Tag}})$ and $\text{ibe.ct}_{\text{id},i,1-\beta}^{\text{Tag}} \leftarrow \text{IBE.Enc}(\text{ibe.mpk}_{\text{id}}, (\text{Tag}, i, 1-\beta), 0^{|w_{\text{id},i,1-\beta}^{\text{Tag}}|})$.

Send $\text{CT} := (\tilde{F}^{\text{Tag}}, \{\text{ibe.ct}_{\text{id},i,b}^{\text{Tag}}\}_{\text{id},i,b})_{\text{Tag} \in [Q]}$ to $\mathcal{A}$.

Post-challenge Queries. $\mathcal{A}$ makes $Q - Q_1$ queries of the form $(\text{id}, \text{GID}, x)$. If $\text{GID} \in \mathcal{T}$, set $\text{Tag} = \mathcal{T}[\text{GID}]$. Otherwise, set $\mathcal{T}[\text{GID}] := \text{Tag}_j$, $\text{Tag} = \mathcal{T}[\text{GID}]$ and increment j . For each $i \in [L_{\text{id}}]$, sample $\text{ibe.sk}_{\text{Tag},i} \leftarrow \text{IBE.KGen}(\text{ibe.msk}_{\text{id}}, (\text{Tag}, i, \text{mafe.mpk}_{\text{id}}^{\text{Tag}}[i]))$. And, if $\text{Tag} < t$, $\text{mafe.sk}_{\text{id,GID},x}^{\text{Tag}} \leftarrow \text{tMAFE.Sim}_{\text{Tag}}^{C(\cdot)}(\text{id}, \text{GID}, x)$. Otherwise, if $\text{Tag} = t$, $\text{mafe.sk}_{\text{id,GID},x}^{\text{Tag}} \leftarrow \mathcal{O}(\text{id}, \text{GID}, x)$. Otherwise, $\text{mafe.sk}_{\text{id,GID},x}^{\text{Tag}} \leftarrow \text{tMAFE.KGen}(\text{id}, \text{mafe.msk}_{\text{id}}^{\text{Tag}}, \{\text{mafe.mpk}_{\text{id}_x}^{\text{Tag}}\}_{\text{id}_x}, \text{GID}, x)$. Send $\text{SK}_{\text{id,GID},x} := (\text{Tag}, \text{mafe.mpk}_{\text{id}}^{\text{Tag}}, \text{mafe.sk}_{\text{id,GID},x}^{\text{Tag}}, \{\text{ibe.sk}_{\text{Tag},i}\}_{i \in [L_{\text{id}}]})$ to $\mathcal{A}$.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

And, the running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$ and λ . If $\mathcal{O}$ is an honest challenger for MAFE, $\mathcal{B}$ behaves as $\text{Hyb}_{7,t}^{\mathcal{A}}(1^\lambda)$ and if $\mathcal{O}$ is a simulator, $\mathcal{B}$ behaves like $\text{Hyb}_{7,t+1}^{\mathcal{A}}(1^\lambda)$. Hence, $\mathcal{B}$ is a valid adversary against the security of MAFE that can break its security with non-negligible advantage. Thus, $\text{Hyb}_{7,t}^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_{7,t+1}^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable. $\square$

$\square$

E.4 Dynamic MAFE from Minimal Assumptions

As discussed, essentially the same construction as Construction 3.5 when BFE is replaced with a dynamic query bounded BFE scheme for P/Poly circuits (Definition E.3) and tMAFE by wotMAFE scheme for P/Poly circuits and $\mathcal{GID}$ (Definition E.2) yields a dynamic query bounded MAFE scheme for P/Poly circuits and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$. Although the output size of wotMAFE.KGen grows with $\lceil \log Q \rceil$, we can set $Q = 2^\lambda$ while initiating dynamic BFE and pad the output of wotMAFE secret keys to this length. Thus, we can eliminate the dependency of Q on GSetup, ASetup, KGen completely. On the other hand, in Enc, we get Q in unary and can proceed with instantiating wotMAFE as normal.

Construction E.17 (MAFE with Dynamic Collusions). We provide the construction of an MAFE scheme with dynamic collusions for P/Poly circuits and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ (Definition E.1) using a BFE scheme with dynamic collusions for P/Poly circuits (Definition E.3) and a wotMAFE scheme for P/Poly circuits and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ (Definition E.2) as follows.

GSetup($1^\lambda, 1^s$). Output $\text{CRS} := (\lambda, s)$.

ASetup($\text{id}, 1^{\ell_{\text{id}}}$). Sample keys $(\text{bfe.mpk}, \text{bfe.msk}) \leftarrow \text{dBFE.Setup}(1^\lambda, 1^{s'})$. Here $s' \leq \text{poly}(\lambda, |\text{GID}|, \ell_{\text{id}})$ is the maximum size of key generation algorithm of wotMAFE for these parameters with $\lceil \log Q \rceil = \lambda$. Output $\text{MPK} := (\ell_{\text{id}}, \text{bfe.mpk})$ and $\text{MSK} := (\ell_{\text{id}}, \text{bfe.msk})$.

KGen($\text{id}, \text{MSK}_{\text{id}}, \text{GID}, x$). Parse MSK_{id} as $(\ell_{\text{id}}, \text{bfe.msk}_{\text{id}})$. Sample $\text{bfe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{dBFE.KGen}(\text{bfe.msk}_{\text{id}}, (\text{id}, \text{GID}, x))$. Output $\text{SK}_{\text{id}, \text{GID}, x} := \text{bfe.sk}_{\text{id}, \text{GID}, x}$.

Enc($1^Q, \{\text{MPK}_{\text{id}}\}_{\text{id} \in \mathcal{I}}, C$). Parse MPK_{id} as $(\ell_{\text{id}}, \text{bfe.mpk}_{\text{id}})$ for each $\text{id} \in \mathcal{I}$. Let $n = |\mathcal{I}|$ and $\text{id}_1, \dots, \text{id}_n$ be the increasing order of elements in $\mathcal{I}$.

- $\text{mafe.crs}, \{(\text{mafe.mpk}_i, \text{mafe.msk}_i)\}_{i \in [n]} \leftarrow \text{wotMAFE.GSetup}(1^\lambda, 1^n, 1^s, 1^{\ell_{\text{id}_1}}, \dots, 1^{\ell_{\text{id}_n}}, Q)$.
- Compute $\text{mafe.ct} \leftarrow \text{wotMAFE.Enc}(1^Q, \{\text{mafe.mpk}_i\}_i, C)$,
- $\forall i \in [n], \text{bfe.ct}_{\text{id}_i} \leftarrow \text{dBFE.Enc}(\text{bfe.mpk}_{\text{id}_i}, \text{wotMAFE.KGen}(\cdot, \text{mafe.msk}_i, \{\text{mafe.mpk}_{i'}\}_{i' \in [n]}, \cdot, \cdot))$.

Output $\text{CT} := (\text{mafe.ct}, \mathcal{I}, \{\text{bfe.ct}_{\text{id}}\}_{\text{id} \in \mathcal{I}})$.

Dec($\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in \mathcal{I}}, \text{CT}$). Parse $\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}$ as $\text{bfe.sk}_{\text{id}, \text{GID}, x_{\text{id}}}$ for each $\text{id} \in \mathcal{I}$ and $\text{CT} := (\text{mafe.ct}, \mathcal{I}', \{\text{bfe.ct}_{\text{id}}\}_{\text{id} \in \mathcal{I}'})$. If $\mathcal{I} \neq \mathcal{I}'$, abort and output $\perp$.

Otherwise, for each $\text{id} \in \mathcal{I}$, compute $\text{mafe.sk}_{\text{id}} = \text{dBFE.Dec}(\text{bfe.sk}_{\text{id}, \text{GID}, x}, \text{bfe.ct}_{\text{id}})$. Output $y := \text{wotMAFE.Dec}(\{\text{mafe.sk}_{\text{id}}\}_{\text{id} \in \mathcal{I}}, \text{mafe.ct})$.

Correctness. The correctness of the scheme follows from the correctness of dBFE and wotMAFE . The proof of the following theorem is immediate.

Theorem E.18 (MAFE with Dynamic Collusions). If there exists a BFE scheme for P/Poly circuits with dynamic collusions (Definition E.3) and a wotMAFE scheme for P/Poly circuits and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ (Definition E.2), there exists an MAFE scheme with dynamic collusions for P/Poly circuits and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ (Definition E.1).

Corollary E.19. Assuming the existence of IBE, there exists a general MAFE scheme (with unbounded authorities) for P/Poly circuits and $\mathcal{GID}$.

Proof. As IBE implies PKE and one-way functions, this corollary follows from Theorem E.6 and Theorem E.18. $\square$

Remark E.20. Assuming the existence of one-way functions, there exists a secret-key general MAFE scheme (with unbounded authorities) for P/Poly circuits and $\mathcal{GID}$ with full security i.e, function- and message-privacy.

F Homomorphic Secret Sharing with Strong Security

In this section, we provide the definition and construction of an HSS scheme for P/Poly circuits that obeys a stronger version of security than what is found in the present literature. Specifically, we construct an HSS scheme in which the attacker not only corrupts shares of any party but also corrupts evaluations of honest shares. We formulate our security using a simulator that only requires $C(x)$ to simulate the honest evaluated shares.

Syntax. An all-but-one homomorphic secret sharing (HSS) scheme for the circuit class $\mathcal{C} = \{\mathcal{C}_\lambda\}_{\lambda \in \mathbb{N}}$ consists of the following polynomial-time algorithms.

$\text{Share}(1^\lambda, 1^K, C) \rightarrow \{\tilde{C}_i\}_{i \in [K]}$. The probabilistic sharing algorithm takes as input security parameter λ , number of shares K , description of a circuit $C \in \mathcal{C}_\lambda$, and outputs the shares $\{\tilde{C}_i\}_{i \in [K]}$.

$\text{Eval}(i, \tilde{C}_i, x) \rightarrow y_i$. The deterministic evaluation algorithm takes as input the i -th share $\tilde{C}_i$, input x , and outputs the output share y_i .

$\text{Retrieve}(y_1, \dots, y_K) \rightarrow y$. The deterministic retrieval algorithm takes as input the output shares $y_1, \dots, y_K$ and outputs the output y .

Definition F.1 (HSS). A HSS scheme ($\text{Share}, \text{Eval}, \text{Retrieve}$) is said to be an all-but-one HSS scheme for the circuit class $\mathcal{C} = \{\mathcal{C}_\lambda\}_{\lambda \in \mathbb{N}}$ if it satisfies the following properties.

Correctness. For any $\lambda \in \mathbb{N}$, and any $C \in \mathcal{C}_\lambda$,

$$\Pr \left[C(x) = \text{Retrieve}(y_1, \dots, y_K) \quad : \quad \begin{array}{l} \{\tilde{C}_i\}_{i \in [K]} \leftarrow \text{Share}(1^\lambda, 1^K, C), \\ \forall i \in [K], y_i \leftarrow \text{Eval}(i, \tilde{C}_i, x) \end{array} \right] = 1$$

Strong Security. There exists a stateful PPT simulator Sim for any stateful PPT adversary $\mathcal{A}$ such that $\forall \lambda \in \mathbb{N}$,

$$\begin{aligned} & \left\{ \mathcal{A}^{\text{Eval}(i^*, \tilde{C}_{i^*, \cdot})}(\{\tilde{C}_i\}_{i \neq i^*}) : \begin{array}{l} (K, C, i^* \in [K]) \leftarrow \mathcal{A}(1^\lambda), \\ \{\tilde{C}_i\}_{i \in [K]} \leftarrow \text{Share}(1^\lambda, 1^K, C) \end{array} \right\} \\ & \qquad \qquad \qquad \approx_c \\ & \left\{ \mathcal{A}^{\text{Sim}^{C(\cdot)}(\cdot)}(\{\tilde{C}_i\}_{i \neq i^*}) : \begin{array}{l} (K, C, i^* \in [K]) \leftarrow \mathcal{A}(1^\lambda), \\ \{\tilde{C}_i\}_{i \neq i^*} \leftarrow \text{Sim}(1^\lambda, 1^K, 1^{|C|}, i^*) \end{array} \right\} \end{aligned}$$

Remark F.2 (HSS with indistinguishability). We remark that a weaker version of HSS where the security is guaranteed when an adversary gets no evaluations on shares of either x_0 or x_1 can be constructed from a multitude of standard assumptions (cf. [ARS24, DIJL23, CM21, BCG⁺17a, FGJS17, BGI16] and the references therein). However, these are insufficient for our construction and we require the strong security property as described above.

Theorem F.3 ([BL20, BJKL21, AJJM21, Shi22]). Assuming SXDH or LWE, there exists a HSS scheme with strong security for any class of P/Poly circuits.

Proof. As shown in [BL20], any semi-honestly secure multi-party reusable non-interactive secure computation (MrNISC) scheme for P/Poly circuits implies an HSS scheme with strong security for any class of P/Poly circuits. We remark that in [BL20], this notion was referred to this notion as Secret Sharing VBB. $\square$

G Proofs of Theorem 4.4

In this section, we provide full proof of Theorem 4.4.

Hyb₀^A(1^λ). This is the real experiment with Chal.

Setup. $\mathcal{A}$ sends $Q, n, s, k, \chi, \ell_1, \dots, \ell_n$ to Chal. For each $i \in \mathbb{I}, j \in \mathbb{J}$, Chal samples $\text{crs}^{(i,j)} \leftarrow \text{MAFE.GSetup}(1^\lambda, 1^Q, 1^{n-k}, 1^s)$. In addition, sample for each $\text{id} \in [n]$, $(\text{mpk}_{\text{id}}^{(i,j)}, \text{msk}_{\text{id}}^{(i,j)}) \leftarrow \text{MAFE.ASetup}(\text{id}, 1^{\ell_{\text{id}}})$ for i such that $\text{id} \notin \mathcal{K}_i$. Set $\text{CRS} := (\text{crs}^{(i,j)})_{i,j}$, $\text{MPK}_{\text{id}} := (\text{mpk}_{\text{id}}^{(i,j)})_{i,j}$, and $\text{MSK}_{\text{id}} := (\text{msk}_{\text{id}}^{(i,j)})_{i,j}$ and send $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]})$ to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$ for $q \in [Q_1]$ to Chal such that id is not corrupted. For each $i \in \mathbb{I}'_{\text{id}}, j \in \mathbb{J}$, sample $\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \text{MAFE.KGen}(\text{id}, \text{msk}_{\text{id}}^{(i,j)}, \{\text{mpk}_{\text{id}_x}^{(i,j)}\}_{\text{id}_x}, \text{GID}, x)$. Set $\text{SK}_{\text{id}, \text{GID}, x} = (\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)})_{i \in \mathbb{I}'_{\text{id}}, j}$ and send it to $\mathcal{A}$.

Corruption Queries. $\mathcal{A}$ sends (Corr, id) to Chal. Send $(\text{msk}_{\text{id}}^{(i,j)})_{i,j}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends circuit C to Chal. Chal samples $\{\tilde{C}_i\}_{i \in \mathbb{I}'} \leftarrow \text{HSS.Share}(1^\lambda, 1^{|\mathbb{I}'|}, C)$. For each $i \in \mathbb{I}', j \in \mathbb{J}$, sample $\text{ct}^{(i,j)} \leftarrow \text{MAFE.Enc}(\{\text{mpk}_{\text{id}}^{(i,j)}\}_{\text{id} \notin \mathcal{K}_i}, F_{ij})$. Set $\text{CT} = (\text{ct}^{(i,j)})_{i,j}$ and send it to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$ for $q \in [Q_1 + 1, Q]$ to Chal such that id is not corrupted. For each $i \in \mathbb{I}'_{\text{id}}, j \in \mathbb{J}$, sample $\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \text{MAFE.KGen}(\text{id}, \text{msk}_{\text{id}}^{(i,j)}, \{\text{mpk}_{\text{id}_x}^{(i,j)}\}_{\text{id}_x}, \text{GID}, x)$. Set $\text{SK}_{\text{id}, \text{GID}, x} = (\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)})_{i,j}$ and send it to $\mathcal{A}$.

Corruption Queries. $\mathcal{A}$ sends (Corr, id) to Chal. Send $(\text{msk}_{\text{id}}^{(i,j)})_{i,j}$ to $\mathcal{A}$.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb_{1,κ}^A(1^λ). For $\kappa \in [2^\chi + 1]$. In this hybrid, we randomly select $i^* \leftarrow \mathbb{I}$ and simulate the $(i^*, 1), \dots, (i^*, \kappa - 1)$ instantiations of MAFE. The changes as highlighted in **red**.

Setup. $\mathcal{A}$ sends $Q, n, s, k, \chi, \ell_1, \dots, \ell_n$. Randomly choose $i^* \leftarrow \mathbb{I}$. For $i \in \mathbb{I}, j \in \mathbb{J}$,

- If $i \neq i^*$ or $i = i^*$ and $j \geq \kappa$, $\text{crs}^{(i,j)} \leftarrow \text{MAFE.GSetup}(1^\lambda, 1^Q, 1^{n-k}, 1^s)$. Sample, for $\text{id} \in [n], \text{id} \notin \mathcal{K}_i$, $\text{mpk}_{\text{id}}^{(i,j)} \leftarrow \text{MAFE.ASetup}(\text{id}, 1^{\ell_{\text{id}}})$.
- **Otherwise, sample $\text{crs}^{(i,j)} \leftarrow \text{MAFE.Sim}(1^\lambda, 1^Q, 1^{n-k}, 1^s)$ and for $\text{id} \notin \mathcal{K}_i$, $(\text{mpk}_{\text{id}}^{(i,j)}, \text{msk}_{\text{id}}^{(i,j)}) \leftarrow \text{MAFE.Sim}(\text{id}, 1^{\ell_{\text{id}}})$.**

Set $\text{CRS} := (\text{crs}^{(i,j)})_{i,j}$, $\text{MPK}_{\text{id}} := (\text{mpk}_{\text{id}}^{(i,j)})_{i,j}$, and **$\text{MSK}_{\text{id}} := ((\text{msk}_{\text{id}}^{(i,j)})_{i \neq i^*, j}, (\text{msk}_{\text{id}}^{(i,j)})_{i = i^*, j \geq \kappa})$** and send $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]})$ to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$ for $q \in [Q_1]$ such that id is not corrupted. For each $i \in \mathbb{I}'_{\text{id}}, j \in \mathbb{J}$,

- If $i \neq i^*$ or $i = i^*$ and $j \geq \kappa$, $\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \text{MAFE.KGen}(\text{id}, \text{msk}_{\text{id}}^{(i,j)}, \{\text{mpk}_{\text{id}_x}^{(i,j)}\}_{\text{id}_x}, \text{GID}, x)$.
- **Otherwise, sample $\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \text{MAFE.Sim}(\text{id}, \text{GID}, x)$.**

Set $\text{SK}_{\text{id}, \text{GID}, x} = (\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)})_{i,j}$ and send it to $\mathcal{A}$.

Corruption Queries. $\mathcal{A}$ sends (Corr, id) . If $\text{id} \notin \mathcal{K}_{i^*}$, **abort**. Otherwise, send $(\text{msk}_{\text{id}}^{(i,j)})_{i,j}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends circuit C . Sample $\{\tilde{C}_i\}_{i \in \mathbb{I}'} \leftarrow \text{HSS.Share}(1^\lambda, 1^{|\mathbb{I}'|}, C)$. For each $i \in \mathbb{I}'$, $j \in \mathbb{J}$, sample

- If $i \neq i^*$ or $i = i^*$ and $j \geq \kappa$, $\text{ct}^{(i,j)} \leftarrow \text{MAFE.Enc}(\{\text{mpk}_{\text{id}}^{(i)}\}_{\text{id} \notin \mathcal{K}_i}, F_{ij})$.
- Otherwise, sample $\text{ct}^{(i,j)} \leftarrow \text{MAFE.Sim}(1^{|F_{ij}|}, \mathcal{V}_{ij})$ where $\mathcal{V}_{ij}$ is as defined in Definition 3.1 for the circuit F_{ij} .

Set $\text{CT} = (\text{ct}^{(i,j)})_{i,j}$ and send it to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$ for $q \in [Q_1 + 1, Q]$ such that id is not corrupted. For each $i \in \mathbb{I}'_{\text{id}}$, $j \in \mathbb{J}$,

- If $i \neq i^*$ or $i = i^*$ and $j \geq \kappa$, $\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \text{MAFE.KGen}(\text{id}, \text{msk}_{\text{id}}^{(i,j)}, \{\text{mpk}_{\text{id}_x}^{(i,j)}\}_{\text{id}_x}, \text{GID}, x)$.
- Otherwise, sample $\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \text{MAFE.Sim}(\text{id}, \text{GID}, x, \mathbf{V})$ where $\mathbf{V}$ is as defined in Definition 3.1 for the circuit F_{ij} .

Set $\text{SK}_{\text{id}, \text{GID}, x} = (\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)})_{i,j}$ and send it to $\mathcal{A}$.

Corruption Queries. $\mathcal{A}$ sends (Corr, id) . If $\text{id} \notin \mathcal{K}_{i^*}$, **abort**. Otherwise, send $(\text{msk}_{\text{id}}^{(i,j)})_{i,j}$ to $\mathcal{A}$.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb₂^A(1^λ). In this hybrid, we simulate HSS. The changes are highlighted in red.

Setup. $\mathcal{A}$ sends $Q, n, s, k, \chi, \ell_1, \dots, \ell_n$. Randomly choose $i^* \leftarrow \mathbb{I}$. For $i \in \mathbb{I}$, $j \in \mathbb{J}$,

- If $i \neq i^*$, $\text{crs}^{(i,j)} \leftarrow \text{MAFE.GSetup}(1^\lambda, 1^Q, 1^{n-k}, 1^s)$. Sample, for $\text{id} \in [n]$, $\text{id} \notin \mathcal{K}_i$, $\text{mpk}_{\text{id}}^{(i,j)} \leftarrow \text{MAFE.Sim}(\text{id}, 1^{\ell_{\text{id}}})$.
- Otherwise, $\text{crs}^{(i,j)} \leftarrow \text{MAFE.Sim}(1^\lambda, 1^Q, 1^{n-k}, 1^s)$ and for $\text{id} \notin \mathcal{K}_i$, $(\text{mpk}_{\text{id}}^{(i,j)}, \text{msk}_{\text{id}}^{(i,j)}) \leftarrow \text{MAFE.ASetup}(\text{id}, 1^{\ell_{\text{id}}})$.

Set $\text{CRS} := (\text{crs}^{(i,j)})_{i,j}$, $\text{MPK}_{\text{id}} := (\text{mpk}_{\text{id}}^{(i,j)})_{i,j}$, and $\text{MSK}_{\text{id}} := (\text{msk}_{\text{id}}^{(i,j)})_{i \neq i^*, j}$ and send $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]})$ to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$ for $q \in [Q_1]$ such that id is not corrupted. For each $i \in \mathbb{I}'_{\text{id}}$, $j \in \mathbb{J}$,

- If $i \neq i^*$, sample $\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \text{MAFE.KGen}(\text{id}, \text{msk}_{\text{id}}^{(i,j)}, \{\text{mpk}_{\text{id}_x}^{(i,j)}\}_{\text{id}_x}, \text{GID}, x)$.
- Otherwise, sample $\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \text{MAFE.Sim}(\text{id}, \text{GID}, x)$.

Set $\text{SK}_{\text{id}, \text{GID}, x} = (\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)})_{i,j}$ and send it to $\mathcal{A}$.

Corruption Queries. $\mathcal{A}$ sends (Corr, id) . If $\text{id} \notin \mathcal{K}_{i^*}$, **abort**. Otherwise, send $(\text{msk}_{\text{id}}^{(i,j)})_{i,j}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends circuit C . Sample $\{\tilde{C}_i\}_{i \neq i^*} \leftarrow \text{HSS.Sim}(1^\lambda, 1^{|\mathbb{I}'|}, 1^{|C|}, i^*)$. For each $i \in \mathbb{I}', j \in \mathbb{J}$, sample

- If $i \neq i^*$, $\text{ct}^{(i,j)} \leftarrow \text{MAFE.Enc}(\{\text{mpk}_{\text{id}}^{(i,j)}\}_{\text{id} \notin \mathcal{K}_i}, F_{ij})$.
- Otherwise, sample $\text{ct}^{(i,j)} \leftarrow \text{MAFE.Sim}(1^{|F_{ij}|}, \mathcal{V}_{ij})$ where $\mathcal{V}_{ij}$ is as defined in Definition 3.1 for the circuit F_{ij} . **However, we will set the output as $\text{HSS.Sim}(X, C(X))$ for $X \in \mathbf{X}_1 \dots, \mathbf{X}_n$.**

Set $\text{CT} = (\text{ct}^{(i,j)})_{i,j}$ and send it to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$ for $q \in [Q_1 + 1, Q]$ such that id is not corrupted. For each $i \in \mathbb{I}'_{\text{id}}, j \in \mathbb{J}$,

- If $i \neq i^*$, sample $\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \text{MAFE.KGen}(\text{id}, \text{msk}_{\text{id}}^{(i,j)}, \{\text{mpk}_{\text{id}_x}^{(i,j)}\}_{\text{id}_x}, \text{GID}, x)$.
- Otherwise, sample $\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \text{MAFE.Sim}(\text{id}, \text{GID}, x, \mathbf{V})$ where $\mathbf{V}$ is as defined in Definition 3.1 for the circuit F_{ij} . **However, we will set the output as $\text{HSS.Sim}(X, C(X))$ for $X \in \mathbf{X}_1 \dots, \mathbf{X}_n$.**

Set $\text{SK}_{\text{id}, \text{GID}, x} = (\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)})_{i,j}$ and send it to $\mathcal{A}$.

Corruption Query. $\mathcal{A}$ sends (Corr, id) . If $\text{id} \notin \mathcal{K}_{i^*}$, abort. Otherwise, send $(\text{msk}_{\text{id}}^{(i,j)})_{i,j}$ to $\mathcal{A}$.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Lemma G.1. $\text{Hyb}_0^{\mathcal{A}}(1^\lambda)$, $\text{Hyb}_{1,1}^{\mathcal{A}}(1^\lambda)$ are identically distributed.

Proof. As we are not simulating any MAFE instantiations in $\text{Hyb}_{1,1}^{\mathcal{A}}(1^\lambda)$, we are only sampling i^* . So, these hybrids are identically distributed. $\square$

Lemma G.2. Assuming the security of MAFE, $\text{Hyb}_{1,\kappa}^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_{1,\kappa+1}^{\mathcal{A}}(1^\lambda)$ for $\kappa \in [|\mathbb{J}|]$ are computationally indistinguishable.

Proof. Note that the probability that $\text{Hyb}_{1,\kappa}^{\mathcal{A}}(1^\lambda)$ does not abort is $1/|\mathbb{J}| = 1/\text{poly}(\lambda)$. Let us denote this event by $\overline{\text{Abort}}$. Assume that there exists an adversary $\mathcal{A}$ that can distinguish between $\text{Hyb}_{1,\kappa}^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_{1,\kappa+1}^{\mathcal{A}}(1^\lambda)$ with non-negligible probability $\epsilon(\lambda)$, i.e.,

$$\left| \Pr \left[1 \leftarrow \text{Hyb}_{1,\kappa}^{\mathcal{A}}(1^\lambda) \right] - \Pr \left[1 \leftarrow \text{Hyb}_{1,\kappa+1}^{\mathcal{A}}(1^\lambda) \right] \right| > \epsilon(\lambda)$$

We will construct a reduction adversary $\mathcal{B}$ that can break the security of fcMAFE with non-negligible probability. The description of $\mathcal{B}^{\mathcal{O}}$ is as follows.

Setup. $\mathcal{A}$ sends $Q, n, s, k, \chi, \ell_1, \dots, \ell_n$. Randomly choose $i^* \leftarrow \mathbb{I}$. For $i \in \mathbb{I}, j \in \mathbb{J}$,

- If $i \neq i^*$ or $i = i^*$ and $j > \kappa$, $\text{crs}^{(i,j)} \leftarrow \text{MAFE.GSetup}(1^\lambda, 1^Q, 1^{n-k}, 1^s)$. Sample, for $\text{id} \in [n], \text{id} \notin \mathcal{K}_i$, $\text{mpk}_{\text{id}}^{(i,j)} \leftarrow \text{MAFE.ASetup}(\text{id}, 1^{\ell_{\text{id}}})$. Otherwise, if $i = i^*, j = \kappa$, $\text{crs}^{(i,j)}, \{\text{mpk}_{\text{id}}^{(i,j)}\}_{\text{id}} \leftarrow \mathcal{O}(Q, n - k, s, \ell_1, \dots, \ell_n)$.
- Otherwise, $\text{crs}^{(i,j)} \leftarrow \text{MAFE.Sim}(1^\lambda, 1^Q, 1^{n-k}, 1^s)$ and for $\text{id} \notin \mathcal{K}_i$, $(\text{mpk}_{\text{id}}^{(i,j)}, \text{msk}_{\text{id}}^{(i,j)}) \leftarrow \text{MAFE.Sim}(\text{id}, 1^{\ell_{\text{id}}})$.

Set $\text{CRS} := (\text{crs}^{(i,j)})_{i,j}$, $\text{MPK}_{\text{id}} := (\text{mpk}_{\text{id}}^{(i,j)})_{i,j}$, and $\text{MSK}_{\text{id}} := ((\text{msk}_{\text{id}}^{(i,j)})_{i \neq i^*, j}, (\text{msk}_{\text{id}}^{(i,j)})_{i=i^*, j \geq \kappa})$ and send $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]})$ to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$ for $q \in [Q_1]$ such that id is not corrupted. For each $i \in \mathbb{I}'_{\text{id}}, j \in \mathbb{J}$,

- If $i \neq i^*$ or $i = i^*$ and $j > \kappa$, $\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \text{MAFE.KGen}(\text{id}, \text{msk}_{\text{id}}^{(i,j)}, \{\text{mpk}_{\text{id}_x}^{(i,j)}\}_{\text{id}_x}, \text{GID}, x)$.
Otherwise, if $i = i^*, j = \kappa$, $\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \mathcal{O}(\text{id}, \text{GID}, x)$.
- Otherwise, sample $\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \text{MAFE.Sim}(\text{id}, \text{GID}, x)$.

Set $\text{SK}_{\text{id}, \text{GID}, x} = (\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)})_{i,j}$ and send it to $\mathcal{A}$.

Corruption Queries. $\mathcal{A}$ sends (Corr, id) . If $\text{id} \notin \mathcal{K}_{i^*}$, abort. Otherwise, send $(\text{msk}_{\text{id}}^{(i,j)})_{i,j}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends circuit C . Sample $\{\tilde{C}_i\}_{i \in \mathbb{I}'} \leftarrow \text{HSS.Share}(1^\lambda, 1^{|\mathbb{I}'|}, C)$. For each $i \in \mathbb{I}', j \in \mathbb{J}$, sample

- If $i \neq i^*$ or $i = i^*$ and $j > \kappa$, $\text{ct}^{(i,j)} \leftarrow \text{MAFE.Enc}(\{\text{mpk}_{\text{id}}^{(i)}\}_{\text{id} \notin \mathcal{K}_i}, F_{ij})$. Otherwise, if $i = i^*, j = \kappa$, $\text{ct}^{(i,j)} \leftarrow \mathcal{O}(F_{ij})$.
- Otherwise, sample $\text{ct}^{(i,j)} \leftarrow \text{MAFE.Sim}(1^{|F_{ij}|}, \mathcal{V}_{ij})$ where $\mathcal{V}_{ij}$ is as defined in Definition 3.1 for the circuit F_{ij} .

Set $\text{CT} = (\text{ct}^{(i,j)})_{i,j}$ and send it to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$ for $q \in [Q_1 + 1, Q]$ such that id is not corrupted. For each $i \in \mathbb{I}'_{\text{id}}, j \in \mathbb{J}$,

- If $i \neq i^*$ or $i = i^*$ and $j > \kappa$, $\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \text{MAFE.KGen}(\text{id}, \text{msk}_{\text{id}}^{(i,j)}, \{\text{mpk}_{\text{id}_x}^{(i,j)}\}_{\text{id}_x}, \text{GID}, x)$.
Otherwise, if $i = i^*, j = \kappa$, $\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \mathcal{O}(\text{id}, \text{GID}, x)$.
- Otherwise, sample $\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \text{MAFE.Sim}(\text{id}, \text{GID}, x, \mathbf{V})$ where $\mathbf{V}$ is as defined in Definition 3.1 for the circuit F_{ij} .

Set $\text{SK}_{\text{id}, \text{GID}, x} = (\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)})_{i,j}$ and send it to $\mathcal{A}$.

Corruption Queries. $\mathcal{A}$ sends (Corr, id) . If $\text{id} \notin \mathcal{K}_{i^*}$, abort. Otherwise, send $(\text{msk}_{\text{id}}^{(i,j)})_{i,j}$ to $\mathcal{A}$.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

As we can see, running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$ and λ . In addition, the probability that $\mathcal{B}$ does not abort is exactly $\Pr[\overline{\text{Abort}}]$. Hence,

$$\begin{aligned} \text{Adv}_{\mathcal{B}} &= \left| \Pr[\text{Abort}] \frac{1}{2} + \Pr[\overline{\text{Abort}}] \Pr[\mathcal{A} \text{ wins}] - \frac{1}{2} \right| \\ &= \left| \left(1 - \frac{1}{|\mathbb{I}|} \right) \frac{1}{2} + \frac{1}{|\mathbb{I}|} \Pr[\mathcal{A} \text{ wins}] - \frac{1}{2} \right| \\ &= \frac{\text{Adv}_{\mathcal{A}}}{|\mathbb{I}|} \end{aligned}$$

If $\mathcal{O}$ is a simulator, $\mathcal{B}$ behaves like $\mathbf{Hyb}_{1,\kappa+1}^{\mathcal{A}}(1^\lambda)$ and if $\mathcal{O}$ is an honest challenger, $\mathcal{B}$ behaves like $\mathbf{Hyb}_{1,\kappa}^{\mathcal{A}}(1^\lambda)$. In addition, as long as $|\mathbb{I}| = \text{poly}(\lambda)$, we have that $\mathcal{B}$ is a successful reduction adversary that can break the security of MAFE with non-negligible probability. $\square$

Lemma G.3. Assuming the security of HSS, $\mathbf{Hyb}_{1,|\mathbb{J}|+1}^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable.

Proof. Assume that there exists a PPT adversary $\mathcal{A}$ that can distinguish between $\mathbf{Hyb}_{1,|\mathbb{J}|+1}^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ with non-negligible advantage, $\epsilon(\lambda)$, i.e.,

$$\left| \Pr \left[1 \leftarrow \mathbf{Hyb}_{1,|\mathbb{J}|+1}^{\mathcal{A}}(1^\lambda) \right] - \Pr \left[1 \leftarrow \mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda) \right] \right| > \epsilon(\lambda)$$

We will construct a reduction adversary $\mathcal{B}$ that can break the security of HSS with non-negligible probability. The description of $\mathcal{B}^{\mathcal{O}}$ is as follows.

Setup. $\mathcal{A}$ sends $Q, n, s, k, \chi, \ell_1, \dots, \ell_n$. Randomly choose $i^* \leftarrow \mathbb{I}$. For $i \in \mathbb{I}, j \in \mathbb{J}$,

- If $i \neq i^*$, $\text{crs}^{(i,j)} \leftarrow \text{MAFE.GSetup}(1^\lambda, 1^Q, 1^{n-k}, 1^s)$. Sample, for $\text{id} \in [n], \text{id} \notin \mathcal{K}_i$, $\text{mpk}_{\text{id}}^{(i,j)} \leftarrow \text{MAFE.Sim}(\text{id}, 1^{\ell_{\text{id}}})$.
- Otherwise, $\text{crs}^{(i,j)} \leftarrow \text{MAFE.Sim}(1^\lambda, 1^Q, 1^{n-k}, 1^s)$ and for $\text{id} \notin \mathcal{K}_i$, $(\text{mpk}_{\text{id}}^{(i,j)}, \text{msk}_{\text{id}}^{(i,j)}) \leftarrow \text{MAFE.ASetup}(\text{id}, 1^{\ell_{\text{id}}})$.

Set $\text{CRS} := (\text{crs}^{(i,j)})_{i,j}$, $\text{MPK}_{\text{id}} := (\text{mpk}_{\text{id}}^{(i,j)})_{i,j}$, and $\text{MSK}_{\text{id}} := (\text{msk}_{\text{id}}^{(i,j)})_{i \neq i^*, j}$ and send $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]})$ to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$ for $q \in [Q_1]$ such that id is not corrupted. For each $i \in \mathbb{I}'_{\text{id}}, j \in \mathbb{J}$,

- If $i \neq i^*$, sample $\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \text{MAFE.KGen}(\text{id}, \text{msk}_{\text{id}}^{(i,j)}, \{\text{mpk}_{\text{id}_x}^{(i,j)}\}_{\text{id}_x}, \text{GID}, x)$.
- Otherwise, sample $\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \text{MAFE.Sim}(\text{id}, \text{GID}, x)$.

Set $\text{SK}_{\text{id}, \text{GID}, x} = (\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)})_{i,j}$ and send it to $\mathcal{A}$.

Corruption Queries. $\mathcal{A}$ sends (Corr, id) . If $\text{id} \notin \mathcal{K}_{i^*}$, abort. Otherwise, send $(\text{msk}_{\text{id}}^{(i,j)})_{i,j}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends circuit C . Sample $\{\tilde{C}_i\}_{i \neq i^*} \leftarrow \mathcal{O}(C, |\mathbb{I}'|, i^*)$. For each $i \in \mathbb{I}', j \in \mathbb{J}$, sample

- If $i \neq i^*$, $\text{ct}^{(i,j)} \leftarrow \text{MAFE.Enc}(\{\text{mpk}_{\text{id}}^{(i,j)}\}_{\text{id} \notin \mathcal{K}_i}, F_{ij})$.
- Otherwise, sample $\text{ct}^{(i,j)} \leftarrow \text{MAFE.Sim}(1^{|F_{ij}|}, \mathcal{V}_{ij})$ where $\mathcal{V}_{ij}$ is as defined in Definition 3.1 for the circuit F_{ij} . However, we will set the output as $\mathcal{O}(X)$ for $X \in \mathbf{X}_1 \dots, \mathbf{X}_n$.

Set $\text{CT} = (\text{ct}^{(i,j)})_{i,j}$ and send it to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$ for $q \in [Q_1 + 1, Q]$ such that id is not corrupted. For each $i \in \mathbb{I}'_{\text{id}}, j \in \mathbb{J}$,

- If $i \neq i^*$, sample $\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \text{MAFE.KGen}(\text{id}, \text{msk}_{\text{id}}^{(i,j)}, \{\text{mpk}_{\text{id}_x}^{(i,j)}\}_{\text{id}_x}, \text{GID}, x)$.

- Otherwise, sample $\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \text{MAFE.Sim}(\text{id}, \text{GID}, x, V)$ where V is as defined in Definition 3.1 for the circuit F_{ij} . However, we will set the output as $\mathcal{O}(X)$ for $X \in \mathbf{X}_1 \dots, \mathbf{X}_n$.

Set $\text{SK}_{\text{id}, \text{GID}, x} = (\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)})_{i,j}$ and send it to $\mathcal{A}$.

Corruption Queries. $\mathcal{A}$ sends (Corr, id) . If $\text{id} \notin \mathcal{K}_{i^*}$, abort. Otherwise, send $(\text{msk}_{\text{id}}^{(i,j)})_{i,j}$ to $\mathcal{A}$.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

As we can see, the running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$, λ . If $\mathcal{O}$ is a honest challenger for HSS, $\mathcal{B}$ behaves like $\text{Hyb}_{1, |\mathbb{J}|+1}^{\mathcal{A}}(1^\lambda)$ and if $\mathcal{O}$ is a simulator for HSS, $\mathcal{B}$ behaves like $\text{Hyb}_2^{\mathcal{A}}(1^\lambda)$. Hence, $\mathcal{B}$ is a valid adversary that break the security of HSS with non-negligible probability. Thus, $\text{Hyb}_{1, |\mathbb{J}|+1}^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_2^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable. $\square$

H Generic Compilers for Authority Corruptions

In this section, we show relations between various notions of authority corruptions in MAFE as defined in Definition 4.1. In particular, we show that selective corruptions are good enough to achieve non-adaptive corruptions generically and adaptive corruptions via complexity leveraging. MAFE with selective and non-adaptive authority corruptions are defined as follows.

Definition H.1 (sel-cMAFE). A MAFE scheme with selectively corruptible authorities (sel-cMAFE) scheme for circuit class $\mathcal{C}$ and $\mathcal{GID}$ is defined similarly to Definition 4.1 except that the adversary will declare all the corruptions a-priori. That is, the adversary will send the set of corrupted authorities, $\mathcal{K}$ at the beginning and receive $\{\text{MSK}_{\text{id}}\}_{\text{id} \in \mathcal{K}}$ along with the public parameters. The adversary will not issue secret key queries for corrupted authorities.

Definition H.2 (na-cMAFE). A MAFE scheme with non-adaptive authority corruptions (na-cMAFE) scheme for circuit class $\mathcal{C}$ and $\mathcal{GID}$ is defined similarly to Definition 4.1 except that the adversary is prohibited to corrupt any authorities in the post-challenge query phase. That is, all authority corruptions should be made before sending the challenge circuit.

In order to achieve non-adaptive corruptions, we rely on a similar compiler as Section 3. The idea is that instead of a tMAFE scheme, we use a sel-cMAFE scheme to achieve non-adaptive corruptions²². As the pre-challenge simulator BFE ($\text{BFE.S}_0, \text{BFE.S}_1$) behaves identically to an honest challenger, we can reveal BFE.MSK at most by the challenge query phase and proceed honestly. Thus, if all the corruptions are made before challenge query phase, we can readily rely on sel-cMAFE security as well.

For adaptive corruptions, we guess the authorities that will be corrupted a-priori and proceed with the security argument from there. The probability that our guess is correct is at most $2^{-O(\log^2 \lambda)}$.

We now construct a non-adaptively corruptible MAFE scheme (na-cMAFE) using a selectively corruptible MAFE scheme (sel-cMAFE).

²²Note that even in Construction 3.5, we could rely on selectively secure tMAFE to achieve the same result.

Construction H.3 (na-cMAFE). We provide the construction of na-cMAFE for P/Poly circuits and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ using a sel-cMAFE scheme for P/Poly circuits and $\mathcal{GID}$ (Definition H.1) and a BFE scheme for P/Poly circuits (Definition 3.3) as follows.

GSetup($1^\lambda, 1^Q, 1^n, 1^s, 1^\chi$). Output $\text{CRS} := (\lambda, Q, n, s, \chi)$.

AS_{id}Setup($\text{id}, 1^{\ell_{\text{id}}}$). Sample keys $(\text{bfe.mpk}, \text{bfe.msk}) \leftarrow \text{BFE.Setup}(1^\lambda, 1^Q, 1^{\lambda+|\mathcal{GID}|+\ell_{\text{id}}}, 1^{\kappa_{\text{id}}})$ where $\kappa_{\text{id}} \leq \text{poly}(\lambda, Q, |\mathcal{GID}|, \ell_{\text{id}})$. Output $\text{MPK} := (\ell_{\text{id}}, \text{bfe.mpk})$ and $\text{MSK} := (\ell_{\text{id}}, \text{bfe.msk})$.

KGen($\text{id}, \text{MSK}_{\text{id}}, \{\text{MPK}_{\text{id}_x}\}_{\text{id}_x \in [n]}, \mathcal{GID}, x$). Parse MSK_{id} as $(\ell_{\text{id}}, \text{bfe.msk}_{\text{id}})$ and for all $\text{id}_x \in [n]$, MPK_{id_x} as $(\ell_{\text{id}_x}, \text{bfe.mpk}_{\text{id}_x})$. Sample $\text{bfe.sk}_{\text{id}, \mathcal{GID}, x} \leftarrow \text{BFE.KGen}(\text{bfe.msk}_{\text{id}}, (\text{id}, \mathcal{GID}, x))$. Output $\text{SK}_{\text{id}, \mathcal{GID}, x} := \text{bfe.sk}_{\text{id}, \mathcal{GID}, x}$.

Enc($\{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}, C$). Parse MPK_{id} as $(\ell_{\text{id}}, \text{bfe.mpk}_{\text{id}})$ for each $\text{id} \in [n]$.

- Sample $\text{mafe.crs} \leftarrow \text{sel-cMAFE.GSetup}(1^\lambda, 1^Q, 1^n, 1^s, 1^\chi)$ and $\forall \text{id} \in [n]$, $(\text{mafe.mpk}_{\text{id}}, \text{mafe.msk}_{\text{id}}) \leftarrow \text{sel-cMAFE.ASetup}(\text{id}, 1^{\ell_{\text{id}}})$.
- Compute $\text{mafe.ct} \leftarrow \text{sel-cMAFE.Enc}(\{\text{mafe.mpk}_{\text{id}}\}_{\text{id} \in [n]}, C)$.
- $\forall \text{id} \in [n]$, $\text{bfe.ct}_{\text{id}} \leftarrow \text{BFE.Enc}(\text{bfe.mpk}_{\text{id}}, \text{sel-cMAFE.KGen}(\cdot, \text{mafe.msk}_{\text{id}}, \{\text{mafe.mpk}_{\text{id}_x}\}_{\text{id}_x \in [n]}, \cdot, \cdot))$.

Output $\text{CT} := (\text{mafe.ct}, \{\text{bfe.ct}_{\text{id}}\}_{\text{id} \in [n]})$.

Dec($\{\text{SK}_{\text{id}, \mathcal{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}, \text{CT}$). Parse $\text{SK}_{\text{id}, \mathcal{GID}, x_{\text{id}}}$ as $\text{bfe.sk}_{\text{id}, \mathcal{GID}, x_{\text{id}}}$ for each $\text{id} \in [n]$ and $\text{CT} := (\text{mafe.ct}, \{\text{bfe.ct}_{\text{id}}\}_{\text{id} \in [n]})$. For each $\text{id} \in [n]$, compute $\text{mafe.sk}_{\text{id}} = \text{BFE.Dec}(\text{bfe.sk}_{\text{id}, \mathcal{GID}, x}, \text{bfe.ct}_{\text{id}})$. Output $y := \text{sel-cMAFE.Dec}(\{\text{mafe.sk}_{\text{id}}\}_{\text{id} \in [n]}, \text{mafe.ct})$.

Correctness. The correctness of the scheme follows from the correctness of BFE and sel-cMAFE.

Theorem H.4. If sel-cMAFE is a sel-cMAFE scheme (Definition H.1) for P/Poly circuits and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ and BFE is a BFE scheme for P/Poly circuits (Definition 3.3), then Construction H.3 is an na-cMAFE scheme for P/Poly circuits and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ (Definition H.2).

Proof Sketch. The proof of this theorem is similar to the proof of Theorem 3.6. We provide a sketch of the hybrid argument used to prove security of Construction H.3 as follows.

Hybrid 0. This is the real experiment from Definition H.2. We use the stateless sel-cMAFE and BFE algorithms to respond to queries from adversary.

Hybrid 1. In this hybrid, we will simulate the pre-challenge queries using BFE.S_0 and BFE.S_1 . That is, we will use $(\text{BFE.S}_0, \text{BFE.S}_1, \text{BFE.Enc}, \text{BFE.KGen})$ to respond to all secret key queries made to n authorities in the pre-challenge secret key query phase. Note that if id is corrupted, we can proceed with revealing its MSK_{id} as the output distribution remains the same as hybrid 0 by BFE security.

Hybrid 2, j . for $j \in [n+1]$. In this hybrid, we will simulate the honest BFE instantiations for the first $j-1$ authorities using BFE.Sim . And, we will use BFE.Enc and BFE.KGen algorithms for corrupted authorities which we will know by challenge query phase. The indistinguishability of hybrids can be argued from security of BFE.

Hybrid 3. In this hybrid, as we know all the corruptions by challenge query phase, we can simulate the sel-cMAFE instantiation. We can use the master secret keys to encrypt the honest sel-cMAFE.KGen algorithm using the corrupted authorities' BFE instantiations. Moreover, as no authority corruptions are allowed in the post-challenge query phase, indistinguishability can be argued using the security of sel-cMAFE. This is the description of Sim from Definition H.2.

We omit full proof for brevity. $\square$

Remark H.5 (Adaptive Corruptions). Assuming the quasi-polynomial security of sel-cMAFE, we can construct a cMAFE scheme simply by guessing the corruption set a-priori. As we have $\sum_{id} \ell_{id} = \text{poly}(\lambda)$ and out of which we need to guess $\chi = O(\log \lambda)$ bits that get corrupted. We will be correct with probability $1/\binom{\text{poly}(\lambda)}{O(\log \lambda)} = 2^{-O(\log^2 \lambda)}$. Hence, by relying on sub-exponential security of sel-cMAFE, we can get a cMAFE scheme generically. Moreover, we can construct a tight reduction using artificial abort techniques [Wat05, GY24] so that the resulting cMAFE scheme is secure against attackers running in times $2^{O(\log^2 \lambda)}$ with advantage at most $2^{-O(\log^2 \lambda)}$.

I Proofs from Section 5.3

In this section, we provide full proof of Lemma 5.6.

Hyb₀^A(1^λ). This is the real experiment with Chal with $b = 0/1$.

Setup. $\mathcal{A}$ sends $(n, s, \ell_1, \dots, \ell_n, C_0, C_1)$ to Chal. Chal sets $\text{CRS} := (\lambda, n, s)$. Then, Chal samples $(\text{fe.mpk}_{id}, \text{fe.msk}_{id}) \leftarrow \text{FE.Setup}(1^\lambda, 1^{s'_{id}})$ for each $id \in [n]$. Chal sets $\text{MPK}_{id} := (id, \ell_{id}, \text{fe.mpk}_{id})$ and $\text{MSK}_{id} := (id, \text{fe.msk}_{id})$.

Now, Chal computes $\mathbb{I}_1, \dots, \mathbb{I}_n, \alpha_1, \dots, \alpha_n$, samples $K \xleftarrow{\$} \{0, 1\}^\lambda$ and $\text{fe.ct}_{id} \leftarrow \text{FE.Enc}(\text{fe.mpk}_{id}, (C_b, K, \alpha_{id}, \mathbb{I}_{id}, 0, \perp))$, and sets $\text{CT} := (\text{fe.ct}_{id})_{id \in [n]}$.

Chal sends $(\text{CRS}, \{\text{MPK}_{id}\}_{id \in [n]}, \text{CT})$ to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ now sends $Q = \text{poly}(\lambda)$ many secret key queries of the form $(\text{GID}, x_1, \dots, x_n)$.

Chal now samples $\text{ske.ct}_{id}^* \xleftarrow{\$} \{0, 1\}^{\ell_{id}^*}$ for each $id \in [n]$ and creates circuits $F_1, \dots, F_n$ as described in Figure 2. Then sample $\text{fe.sk}_{F_{id}} \leftarrow \text{FE.KGen}(\text{fe.msk}_{id}, F_{id})$. Set $\text{SK}_{id, \text{GID}, x_{id}} := \text{fe.sk}_{F_{id}}$ and send $\{\text{SK}_{id, \text{GID}, x_{id}}\}_{id \in [n]}$ to $\mathcal{A}$.

Guess. Output whatever $\mathcal{A}$ outputs.

Hyb₁^A(1^λ). In this hybrid, we set ske.ct_{id}^* to be an SKE ciphertext of $F_{id}(C_b, K, \alpha_{id}, \mathbb{I}_{id}, 0, \perp)$. The changes are highlighted in red.

Setup. $\mathcal{A}$ sends $(n, s, \ell_1, \dots, \ell_n, C_0, C_1)$. Set $\text{CRS} := (\lambda, n, s)$. Then, sample $(\text{fe.mpk}_{id}, \text{fe.msk}_{id}) \leftarrow \text{FE.Setup}(1^\lambda, 1^{s'_{id}})$ for each $id \in [n]$. Set $\text{MPK}_{id} := (id, \ell_{id}, \text{fe.mpk}_{id})$ and $\text{MSK}_{id} := (id, \text{fe.msk}_{id})$. In addition, sample $\text{ske.sk} \leftarrow \text{SKE.Setup}(1^\lambda)$.

Now, compute $\mathbb{I}_1, \dots, \mathbb{I}_n, \alpha_1, \dots, \alpha_n$, samples $K \xleftarrow{\$} \{0, 1\}^\lambda$ and $\text{fe.ct}_{id} \leftarrow \text{FE.Enc}(\text{fe.mpk}_{id}, (C_b, K, \alpha_{id}, \mathbb{I}_{id}, 0, \perp))$, and set $\text{CT} := (\text{fe.ct}_{id})_{id \in [n]}$.

Send $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}, \text{CT})$ to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ now sends $Q = \text{poly}(\lambda)$ many secret key queries of the form $(\text{GID}, x_1, \dots, x_n)$.

Compute $\tilde{C}, \{w_{i,b}\}_{i \in [\ell_1 + \dots + \ell_n], b \in \{0,1\}} \leftarrow \text{Garb}(1^\lambda, C_b; \text{PRF}(K, \text{GID}))$. Sample $\text{ske.ct}_{\text{id}}^* \leftarrow \text{SKE.Enc}(\text{ske.sk}, (\tilde{C}, \{w_{i,x_{\text{id}}[i-\alpha_{\text{id}}+1]}\}_{i \in \mathbb{I}_{\text{id}}}))$ for each $\text{id} \in [n]$.

Create circuits $F_1, \dots, F_n$ as described in Figure 2. Then sample $\text{fe.sk}_{F_{\text{id}}} \leftarrow \text{FE.KGen}(\text{fe.msk}_{\text{id}}, F_{\text{id}})$. Set $\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}} := \text{fe.sk}_{F_{\text{id}}}$ and send $\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Guess. Output whatever $\mathcal{A}$ outputs.

Hyb_{2,j}^A(1^λ) for $j \in [n+1]$. In this hybrid, we set input to the first $j-1$ instantiations of FE to be $(\bar{0}, 1, \text{ske.sk})$. The changes are highlighted in red.

Setup. $\mathcal{A}$ sends $(n, s, \ell_1, \dots, \ell_n, C_0, C_1)$. Set $\text{CRS} := (\lambda, n, s)$. Then, sample $(\text{fe.mpk}_{\text{id}}, \text{fe.msk}_{\text{id}}) \leftarrow \text{FE.Setup}(1^\lambda, 1^{s_{\text{id}}})$ for each $\text{id} \in [n]$. Set $\text{MPK}_{\text{id}} := (\text{id}, \ell_{\text{id}}, \text{fe.mpk}_{\text{id}})$ and $\text{MSK}_{\text{id}} := (\text{id}, \text{fe.msk}_{\text{id}})$. Sample $\text{ske.sk} \leftarrow \text{SKE.Setup}(1^\lambda)$.

Now, compute $\mathbb{I}_1, \dots, \mathbb{I}_n, \alpha_1, \dots, \alpha_n$, sample $K \xleftarrow{\$} \{0, 1\}^\lambda$. If $\text{id} < j$, $\text{fe.ct}_{\text{id}} \leftarrow \text{FE.Enc}(\text{fe.mpk}_{\text{id}}, (\bar{0}, 1, \text{ske.sk}))$. Otherwise, $\text{fe.ct}_{\text{id}} \leftarrow \text{FE.Enc}(\text{fe.mpk}_{\text{id}}, (C_b, K, \alpha_{\text{id}}, \mathbb{I}_{\text{id}}, 0, \perp))$. Set $\text{CT} := (\text{fe.ct}_{\text{id}})_{\text{id} \in [n]}$. Sends $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}, \text{CT})$ to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ now sends $Q = \text{poly}(\lambda)$ many secret key queries of the form $(\text{GID}, x_1, \dots, x_n)$.

Compute $\tilde{C}, \{w_{i,b}\}_{i \in [\ell_1 + \dots + \ell_n], b \in \{0,1\}} \leftarrow \text{Garb}(1^\lambda, C_b; \text{PRF}(K, \text{GID}))$. Sample $\text{ske.ct}_{\text{id}}^* \leftarrow \text{SKE.Enc}(\text{ske.sk}, (\tilde{C}, \{w_{i,x_{\text{id}}[i-\alpha_{\text{id}}+1]}\}_{i \in \mathbb{I}_{\text{id}}}))$ for each $\text{id} \in [n]$.

Create circuits $F_1, \dots, F_n$ as described in Figure 2. Then sample $\text{fe.sk}_{F_{\text{id}}} \leftarrow \text{FE.KGen}(\text{fe.msk}_{\text{id}}, F_{\text{id}})$. Set $\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}} := \text{fe.sk}_{F_{\text{id}}}$ and send $\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Guess. Output whatever $\mathcal{A}$ outputs.

Hyb₃^A(1^λ). In this hybrid, we sample uniformly random strings for garbling randomness instead of using $\text{PRF}(K, \text{GID})$. The changes are highlighted in red.

Setup. $\mathcal{A}$ sends $(n, s, \ell_1, \dots, \ell_n, C_0, C_1)$. Set $\text{CRS} := (\lambda, n, s)$. Then, sample $(\text{fe.mpk}_{\text{id}}, \text{fe.msk}_{\text{id}}) \leftarrow \text{FE.Setup}(1^\lambda, 1^{s_{\text{id}}})$ for each $\text{id} \in [n]$. Set $\text{MPK}_{\text{id}} := (\text{id}, \ell_{\text{id}}, \text{fe.mpk}_{\text{id}})$ and $\text{MSK}_{\text{id}} := (\text{id}, \text{fe.msk}_{\text{id}})$. Sample $\text{ske.sk} \leftarrow \text{SKE.Setup}(1^\lambda)$.

Now, compute $\mathbb{I}_1, \dots, \mathbb{I}_n, \alpha_1, \dots, \alpha_n$. Sample $\text{fe.ct}_{\text{id}} \leftarrow \text{FE.Enc}(\text{fe.mpk}_{\text{id}}, (\bar{0}, 1, \text{ske.sk}))$. Set $\text{CT} := (\text{fe.ct}_{\text{id}})_{\text{id} \in [n]}$.

Sends $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}, \text{CT})$ to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ now sends $Q = \text{poly}(\lambda)$ many secret key queries of the form $(\text{GID}, x_1, \dots, x_n)$.

Compute $\tilde{C}, \{w_{i,b}\}_{i \in [\ell_1 + \dots + \ell_n], b \in \{0,1\}} \leftarrow \text{Garb}(1^\lambda, C_b; R)$ where $R \xleftarrow{\$} \{0, 1\}^\lambda$. Sample $\text{ske.ct}_{\text{id}}^* \leftarrow \text{SKE.Enc}(\text{ske.sk}, (\tilde{C}, \{w_{i,x_{\text{id}}[i-\alpha_{\text{id}}+1]}\}_{i \in \mathbb{I}_{\text{id}}}))$ for each $\text{id} \in [n]$.

Create circuits $F_1, \dots, F_n$ as described in Figure 2. Then sample $\text{fe.sk}_{F_{\text{id}}} \leftarrow \text{FE.KGen}(\text{fe.msk}_{\text{id}}, F_{\text{id}})$. Set $\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}} := \text{fe.sk}_{F_{\text{id}}}$ and send $\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Guess. Output whatever $\mathcal{A}$ outputs.

Hyb_{4,j}^A(1^λ) for $j \in [Q+1]$. In this hybrid, we simulate the first $j-1$ garbling instantiations using $\text{Garb}(1^\lambda, C_b)$. The changes are highlighted in **red**.

Setup. $\mathcal{A}$ sends $(n, s, \ell_1, \dots, \ell_n, C_0, C_1)$. Set $\text{CRS} := (\lambda, n, s)$. Then, sample $(\text{fe.mpk}_{\text{id}}, \text{fe.msk}_{\text{id}}) \leftarrow \text{FE.Setup}(1^\lambda, 1^{s'_{\text{id}}})$ for each $\text{id} \in [n]$. Set $\text{MPK}_{\text{id}} := (\text{id}, \ell_{\text{id}}, \text{fe.mpk}_{\text{id}})$ and $\text{MSK}_{\text{id}} := (\text{id}, \text{fe.msk}_{\text{id}})$. Sample $\text{ske.sk} \leftarrow \text{SKE.Setup}(1^\lambda)$.

Now, compute $\mathbb{I}_1, \dots, \mathbb{I}_n, \alpha_1, \dots, \alpha_n$. Sample $\text{fe.ct}_{\text{id}} \leftarrow \text{FE.Enc}(\text{fe.mpk}_{\text{id}}, (\bar{0}, 1, \text{ske.sk}))$. Set $\text{CT} := (\text{fe.ct}_{\text{id}})_{\text{id} \in [n]}$.

Sends $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}, \text{CT})$ to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ now sends $Q = \text{poly}(\lambda)$ many secret key queries of the form $(\text{GID}, x_1, \dots, x_n)$.

Let $X = (x_1, \dots, x_n) \in \{0, 1\}^{\ell_1 + \dots + \ell_n}$ and $y = C_b(X)$.

If this is the q -th query where $q < j$, compute $\tilde{C}, \{w_{i,X[i]}\}_{i \in [\ell_1 + \dots + \ell_n]} \leftarrow \text{Garb.Sim}(1^\lambda, 1^{|C_b|}, y)$.

Otherwise, compute $\tilde{C}, \{w_{i,X[i]}\}_{i \in [\ell_1 + \dots + \ell_n]} \leftarrow \text{Garb}(1^\lambda, C_b)$. Sample $\text{ske.ct}_{\text{id}}^* \leftarrow \text{SKE.Enc}(\text{ske.sk}, (\tilde{C}, \{w_{i,X[i]}\}_{i \in \mathbb{I}_{\text{id}}}))$ for each $\text{id} \in [n]$.

Create circuits $F_1, \dots, F_n$ as described in Figure 2. Then sample $\text{fe.sk}_{F_{\text{id}}} \leftarrow \text{FE.KGen}(\text{fe.msk}_{\text{id}}, F_{\text{id}})$.

Set $\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}} := \text{fe.sk}_{F_{\text{id}}}$ and send $\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Guess. Output whatever $\mathcal{A}$ outputs.

Claim I.1. Assuming the pseudorandom ciphertext security of SKE, **Hyb₀^A(1^λ)** and **Hyb₁^A(1^λ)** are computationally indistinguishable.

Proof. Assume that there exists an adversary $\mathcal{A}$ that can distinguish between **Hyb₀^A(1^λ)** and **Hyb₁^A(1^λ)** with non-negligible probability $\epsilon(\lambda)$, i.e.,

$$\left| \Pr \left[1 \leftarrow \mathbf{Hyb}_0^A(1^\lambda) \right] - \Pr \left[1 \leftarrow \mathbf{Hyb}_1^A(1^\lambda) \right] \right| > \epsilon(\lambda)$$

We will construct a reduction adversary $\mathcal{B}$ that can break the security of SKE with non-negligible probability. The description of $\mathcal{B}^\mathcal{O}$ is as follows.

Setup. $\mathcal{A}$ sends $(n, s, \ell_1, \dots, \ell_n, C_0, C_1)$. Set $\text{CRS} := (\lambda, n, s)$. Then, sample $(\text{fe.mpk}_{\text{id}}, \text{fe.msk}_{\text{id}}) \leftarrow \text{FE.Setup}(1^\lambda, 1^{s'_{\text{id}}})$ for each $\text{id} \in [n]$. Set $\text{MPK}_{\text{id}} := (\text{id}, \ell_{\text{id}}, \text{fe.mpk}_{\text{id}})$ and $\text{MSK}_{\text{id}} := (\text{id}, \text{fe.msk}_{\text{id}})$.

Now, compute $\mathbb{I}_1, \dots, \mathbb{I}_n, \alpha_1, \dots, \alpha_n$, samples $K \xleftarrow{\$} \{0, 1\}^\lambda$ and $\text{fe.ct}_{\text{id}} \leftarrow \text{FE.Enc}(\text{fe.mpk}_{\text{id}}, (C_b, K, \alpha_{\text{id}}, \mathbb{I}_{\text{id}}, 0, \perp))$, and set $\text{CT} := (\text{fe.ct}_{\text{id}})_{\text{id} \in [n]}$.

Send $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}, \text{CT})$ to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ now sends $Q = \text{poly}(\lambda)$ many secret key queries of the form $(\text{GID}, x_1, \dots, x_n)$.

Compute $\tilde{C}, \{w_{i,b}\}_{i \in [\ell_1 + \dots + \ell_n], b \in \{0,1\}} \leftarrow \text{Garb}(1^\lambda, C_b; \text{PRF}(K, \text{GID}))$. Query $\mathcal{O}$ to get $\text{ske.ct}_{\text{id}}^* \leftarrow \mathcal{O}((\tilde{C}, \{w_{i,x_{\text{id}}[i-\alpha_{\text{id}}+1]}\}_{i \in \mathbb{I}_{\text{id}}}))$ for each $\text{id} \in [n]$.

Create circuits $F_1, \dots, F_n$ as described in Figure 2. Then sample $\text{fe.sk}_{F_{\text{id}}} \leftarrow \text{FE.KGen}(\text{fe.msk}_{\text{id}}, F_{\text{id}})$.

Set $\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}} := \text{fe.sk}_{F_{\text{id}}}$ and send $\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Guess. Output whatever $\mathcal{A}$ outputs.

As we can see, the running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$, λ . If $\mathcal{O}$ generates uniformly random strings, $\mathcal{B}$ behaves like $\mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)$ and if $\mathcal{O}$ generates encryptions of message, $\mathcal{B}$ behaves like $\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$. Hence, $\mathcal{B}$ is a valid adversary that break the security of SKE with non-negligible probability. Thus, $\mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable. $\square$

Claim I.2. $\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_{2,1}^{\mathcal{A}}(1^\lambda)$ are identically distributed.

Proof. As we are not switching inputs of any FE instantiations in $\mathbf{Hyb}_{2,1}^{\mathcal{A}}(1^\lambda)$, these hybrids are identically distributed. $\square$

Claim I.3. Assuming the IND security of FE, $\mathbf{Hyb}_{2,j}^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_{2,j+1}^{\mathcal{A}}(1^\lambda)$ for any $j \in [n]$ are computationally indistinguishable.

Proof. Assume that there exists an adversary $\mathcal{A}$ that can distinguish between $\mathbf{Hyb}_{2,j}^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_{2,j+1}^{\mathcal{A}}(1^\lambda)$ for some $j \in [n]$ with non-negligible probability $\epsilon(\lambda)$, i.e.,

$$\left| \Pr \left[1 \leftarrow \mathbf{Hyb}_{2,j}^{\mathcal{A}}(1^\lambda) \right] - \Pr \left[1 \leftarrow \mathbf{Hyb}_{2,j+1}^{\mathcal{A}}(1^\lambda) \right] \right| > \epsilon(\lambda)$$

We will construct a reduction adversary $\mathcal{B}$ that can break the IND security of FE with non-negligible probability. The description of $\mathcal{B}^{\mathcal{O}}$ is as follows.

Setup. Receive mpk from $\mathcal{O}$ and set $\text{fe.mpk}_j := \text{mpk}$. $\mathcal{A}$ sends $(n, s, \ell_1, \dots, \ell_n, C_0, C_1)$. Set $\text{CRS} := (\lambda, n, s)$. Then, sample $(\text{fe.mpk}_{\text{id}}, \text{fe.msk}_{\text{id}}) \leftarrow \text{FE.Setup}(1^\lambda, 1^{s_{\text{id}}})$ for each $\text{id} \in [n] \setminus \{j\}$. Set $\text{MPK}_{\text{id}} := (\text{id}, \ell_{\text{id}}, \text{fe.mpk}_{\text{id}})$ and $\text{MSK}_{\text{id}_x} := (\text{id}_x, \text{fe.msk}_{\text{id}_x})$ for $\text{id}_x \in [n] \setminus \{j\}$. Sample $\text{ske.sk} \leftarrow \text{SKE.Setup}(1^\lambda)$.

Now, compute $\mathbb{I}_1, \dots, \mathbb{I}_n, \alpha_1, \dots, \alpha_n$, sample $K \xleftarrow{\$} \{0, 1\}^\lambda$. If $\text{id} < j$, $\text{fe.ct}_{\text{id}} \leftarrow \text{FE.Enc}(\text{fe.mpk}_{\text{id}}, (\bar{0}, 1, \text{ske.sk}))$. Otherwise, if $\text{id} = j$, query $\mathcal{O}$ to receive $\text{fe.ct}_{\text{id}} \leftarrow \mathcal{O}(m_0 := (C_b, K, \alpha_{\text{id}}, \mathbb{I}_{\text{id}}, 0, \perp), m_1 := (\bar{0}, 1, \text{ske.sk}))$. Otherwise, $\text{fe.ct}_{\text{id}} \leftarrow \text{FE.Enc}(\text{fe.mpk}_{\text{id}}, (C_b, K, \alpha_{\text{id}}, \mathbb{I}_{\text{id}}, 0, \perp))$. Set $\text{CT} := (\text{fe.ct}_{\text{id}})_{\text{id} \in [n]}$.

Sends $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}, \text{CT})$ to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ now sends $Q = \text{poly}(\lambda)$ many secret key queries of the form $(\text{GID}, x_1, \dots, x_n)$.

Compute $\tilde{C}, \{w_{i,b}\}_{i \in [\ell_1 + \dots + \ell_n], b \in \{0,1\}} \leftarrow \text{Garb}(1^\lambda, C_b; \text{PRF}(K, \text{GID}))$. Sample $\text{ske.ct}_{\text{id}}^* \leftarrow \text{SKE.Enc}(\text{ske.sk}, (\tilde{C}, \{w_{i,x_{\text{id}}[i - \alpha_{\text{id}} + 1]}\}_{i \in \mathbb{I}_{\text{id}}}))$ for each $\text{id} \in [n]$.

Create circuits $F_1, \dots, F_n$ as described in Figure 2. Then sample $\text{fe.sk}_{F_{\text{id}}} \leftarrow \text{FE.KGen}(\text{fe.msk}_{\text{id}}, F_{\text{id}})$ for $\text{id} \neq j$. Send F_j to $\mathcal{O}$ to receive fe.sk_{F_j} . Set $\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}} := \text{fe.sk}_{F_{\text{id}}}$ and send $\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Guess. Output whatever $\mathcal{A}$ outputs.

As we can see, the running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$, λ . If $\mathcal{O}$ uses m_0 , $\mathcal{B}$ behaves like $\mathbf{Hyb}_{2,j}^{\mathcal{A}}(1^\lambda)$ and if $\mathcal{O}$ generates encryptions of message m_1 , $\mathcal{B}$ behaves like $\mathbf{Hyb}_{2,j+1}^{\mathcal{A}}(1^\lambda)$. Hence, $\mathcal{B}$ is a valid adversary that break the security of SKE with non-negligible probability. Thus, $\mathbf{Hyb}_{2,j}^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_{2,j+1}^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable. $\square$

Claim I.4. Assuming the pseudorandomness property of PRF, $\mathbf{Hyb}_{2,n+1}^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable.

Proof. Assume that there exists an adversary $\mathcal{A}$ that can distinguish between $\mathbf{Hyb}_{2,n+1}^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$ with non-negligible probability $\epsilon(\lambda)$, i.e.,

$$\left| \Pr \left[1 \leftarrow \mathbf{Hyb}_{2,n+1}^{\mathcal{A}}(1^\lambda) \right] - \Pr \left[1 \leftarrow \mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda) \right] \right| > \epsilon(\lambda)$$

We will construct a reduction adversary $\mathcal{B}$ that can break the security of PRF with non-negligible probability. The description of $\mathcal{B}^{\mathcal{O}}$ is as follows.

Setup. $\mathcal{A}$ sends $(n, s, \ell_1, \dots, \ell_n, C_0, C_1)$. Set $\text{CRS} := (\lambda, n, s)$. Then, sample $(\text{fe.mpk}_{\text{id}}, \text{fe.msk}_{\text{id}}) \leftarrow \text{FE.Setup}(1^\lambda, 1^{s'_{\text{id}}})$ for each $\text{id} \in [n]$. Set $\text{MPK}_{\text{id}} := (\text{id}, \ell_{\text{id}}, \text{fe.mpk}_{\text{id}})$ and $\text{MSK}_{\text{id}} := (\text{id}, \text{fe.msk}_{\text{id}})$. Sample $\text{ske.sk} \leftarrow \text{SKE.Setup}(1^\lambda)$.

Now, compute $\mathbb{I}_1, \dots, \mathbb{I}_n, \alpha_1, \dots, \alpha_n$. Sample $\text{fe.ct}_{\text{id}} \leftarrow \text{FE.Enc}(\text{fe.mpk}_{\text{id}}, (\bar{0}, 1, \text{ske.sk}))$. Set $\text{CT} := (\text{fe.ct}_{\text{id}})_{\text{id} \in [n]}$.

Sends $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}, \text{CT})$ to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ now sends $Q = \text{poly}(\lambda)$ many secret key queries of the form $(\text{GID}, x_1, \dots, x_n)$.

Compute $\tilde{C}, \{w_{i,b}\}_{i \in [\ell_1 + \dots + \ell_n], b \in \{0,1\}} \leftarrow \text{Garb}(1^\lambda, C_b; \mathcal{O}(\text{GID}))$. Sample $\text{ske.ct}_{\text{id}}^* \leftarrow \text{SKE.Enc}(\text{ske.sk}, (\tilde{C}, \{w_{i,x_{\text{id}}[i-\alpha_{\text{id}}+1]}\}_{i \in \mathbb{I}_{\text{id}}}))$ for each $\text{id} \in [n]$.

Create circuits $F_1, \dots, F_n$ as described in Figure 2. Then sample $\text{fe.sk}_{F_{\text{id}}} \leftarrow \text{FE.KGen}(\text{fe.msk}_{\text{id}}, F_{\text{id}})$. Set $\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}} := \text{fe.sk}_{F_{\text{id}}}$ and send $\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Guess. Output whatever $\mathcal{A}$ outputs.

As we can see, the running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$, λ . If $\mathcal{O}$ is $\text{PRF}(K, \cdot)$ for some $K \xleftarrow{\$} \{0,1\}^\lambda$, $\mathcal{B}$ behaves like $\mathbf{Hyb}_{2,n+1}^{\mathcal{A}}(1^\lambda)$ and if $\mathcal{O}$ generates uniformly random strings $\mathcal{B}$ behaves like $\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$. Hence, $\mathcal{B}$ is a valid adversary that break the security of PRF with non-negligible probability. Thus, $\mathbf{Hyb}_{2,n+1}^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable. $\square$

Claim I.5. $\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_{4,1}^{\mathcal{A}}(1^\lambda)$ are identically distributed.

Proof. As we are not simulating any garblings of C_b in $\mathbf{Hyb}_{4,1}^{\mathcal{A}}(1^\lambda)$, these hybrids are identically distributed. $\square$

Claim I.6. Assuming the security of $(\text{Garb}, \text{Eval})$, $\mathbf{Hyb}_{4,j}^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_{4,j+1}^{\mathcal{A}}(1^\lambda)$ for any $j \in [Q]$ are computationally indistinguishable.

Proof. Assume that there exists an adversary $\mathcal{A}$ that can distinguish between $\mathbf{Hyb}_{4,j}^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_{4,j+1}^{\mathcal{A}}(1^\lambda)$ for some $j \in [Q]$ with non-negligible probability $\epsilon(\lambda)$, i.e.,

$$\left| \Pr \left[1 \leftarrow \mathbf{Hyb}_{4,j}^{\mathcal{A}}(1^\lambda) \right] - \Pr \left[1 \leftarrow \mathbf{Hyb}_{4,j+1}^{\mathcal{A}}(1^\lambda) \right] \right| > \epsilon(\lambda)$$

We will construct a reduction adversary $\mathcal{B}$ that can break the security of $(\text{Garb}, \text{Eval})$ with non-negligible probability. The description of $\mathcal{B}^{\mathcal{O}}$ is as follows.

Setup. $\mathcal{A}$ sends $(n, s, \ell_1, \dots, \ell_n, C_0, C_1)$. Set $\text{CRS} := (\lambda, n, s)$. Then, sample $(\text{fe.mpk}_{\text{id}}, \text{fe.msk}_{\text{id}}) \leftarrow \text{FE.Setup}(1^\lambda, 1^{s_{\text{id}}})$ for each $\text{id} \in [n]$. Set $\text{MPK}_{\text{id}} := (\text{id}, \ell_{\text{id}}, \text{fe.mpk}_{\text{id}})$ and $\text{MSK}_{\text{id}} := (\text{id}, \text{fe.msk}_{\text{id}})$. Sample $\text{ske.sk} \leftarrow \text{SKE.Setup}(1^\lambda)$.

Now, compute $\mathbb{I}_1, \dots, \mathbb{I}_n, \alpha_1, \dots, \alpha_n$. Sample $\text{fe.ct}_{\text{id}} \leftarrow \text{FE.Enc}(\text{fe.mpk}_{\text{id}}, (\bar{0}, 1, \text{ske.sk}))$. Set $\text{CT} := (\text{fe.ct}_{\text{id}})_{\text{id} \in [n]}$.

Sends $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}, \text{CT})$ to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ now sends $Q = \text{poly}(\lambda)$ many secret key queries of the form $(\text{GID}, x_1, \dots, x_n)$. Let $X = (x_1, \dots, x_n) \in \{0, 1\}^{\ell_1 + \dots + \ell_n}$ and $y = C_b(X)$.

If this is the q -th query where $q < j$, compute $\tilde{C}, \{w_{i,X[i]}\}_{i \in [\ell_1 + \dots + \ell_n]} \leftarrow \text{Garb.Sim}(1^\lambda, 1^{|C_b|}, y)$. Otherwise, if this is the j -th query, send (C_b, X) to $\mathcal{O}$ to receive $\tilde{C}, \{w_{i,X[i]}\}_{i \in [\ell_1 + \dots + \ell_n]}$. Otherwise, compute $\tilde{C}, \{w_{i,X[i]}\}_{i \in [\ell_1 + \dots + \ell_n]} \leftarrow \text{Garb}(1^\lambda, C_b)$. Sample $\text{ske.ct}_{\text{id}}^* \leftarrow \text{SKE.Enc}(\text{ske.sk}, (\tilde{C}, \{w_{i,X[i]}\}_{i \in \mathbb{I}_{\text{id}}}))$ for each $\text{id} \in [n]$.

Create circuits $F_1, \dots, F_n$ as described in Figure 2. Then sample $\text{fe.sk}_{F_{\text{id}}} \leftarrow \text{FE.KGen}(\text{fe.msk}_{\text{id}}, F_{\text{id}})$. Set $\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}} := \text{fe.sk}_{F_{\text{id}}}$ and send $\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Guess. Output whatever $\mathcal{A}$ outputs.

As we can see, the running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$, λ . If $\mathcal{O}$ honestly garbles C_b $\mathcal{B}$ behaves like $\text{Hyb}_{4,j}^{\mathcal{A}}(1^\lambda)$ and if $\mathcal{O}$ simulates garbling using $C_b(X)$, $\mathcal{B}$ behaves like $\text{Hyb}_{4,j+1}^{\mathcal{A}}(1^\lambda)$. Hence, $\mathcal{B}$ is a valid adversary that break the security of $(\text{Garb}, \text{Eval})$ with non-negligible probability. Thus, $\text{Hyb}_{4,j}^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_{4,j+1}^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable. $\square$

J One-Ciphertext Private-Key MAFE

In this section, we show how to construct a one-ciphertext private-key MAFE scheme for P/Poly circuits (Definition J.1) from public-key encryption. At a first glance, it looks like one could follow a similar blueprint as [ABSV15] where we start from private-key 1MAFE ([GY24]), make it function-hiding similarly to [BS18] and then switch the roles of KGen and Enc. However, the major issue is that in 1MAFE, the key generation is split rather than encryption and we can only give out one key for each authority to the attacker for security to hold. The same is true even in the indistinguishability domain.

The proposed definition can be realized via the pipeline specified in [ABSV15] if one starts from a *multi-client* functional encryption [GGG⁺14, CDSG⁺18] (MCFE) rather than a multi-authority functional encryption. Essentially, in the former, the encryption algorithm is performed with respect to a GID^{23} and split across multiple “clients”. The resultant ciphertexts can be combined only if GID is same across all encryptions rather than key generation as in MAFE. We provide construction of IND-secure private-key MCFE for P/Poly circuits in the *trusted setup* model with batched key generation assuming the existence of one-way functions. Then, we make it function hiding using [BS18] at which point, we can semantically switch the roles of Enc and KGen to realize Definition J.1.

²³We remark that in the literature for MCFE, ‘labels’/‘tags’ are used as opposed to GID . However, to maintain notational consistency, we use the language of GIDs .

J.1 Definition

We define our private-key one-ciphertext MAFE scheme for P/Poly circuits below.

Syntax. A private-key one-ciphertext MAFE scheme for circuit class $\mathcal{C} = \{\mathcal{C}_\lambda : \mathcal{X}_{\lambda,1} \times \dots \times \mathcal{X}_{\lambda,n} \rightarrow \mathcal{O}_\lambda\}_{\lambda \in \mathbb{N}}$ and global identifier domain $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ consists of the following polynomial-time algorithms.

GSetup $(1^\lambda, 1^n, 1^s, 1^{\ell_1}, \dots, 1^{\ell_n}) \rightarrow (\text{CRS}, \{\text{MSK}_{\text{id}}\}_{\text{id} \in [n]})$. The probabilistic global setup algorithm takes as input the security parameter λ , number of authorities n , the maximum circuit size s , outputs CRS, and master secret keys for all authorities, $\{\text{MSK}_{\text{id}}\}_{\text{id} \in [n]}$. The following algorithms take CRS implicitly.

KGen $(\text{id}, \text{MSK}_{\text{id}}, \text{GID}, x) \rightarrow \text{SK}_{\text{id}, \text{GID}, x}$. The possibly randomized key generation algorithm takes as input the id-th master secret key MSK_{id} , the global identifier for the user $\text{GID} \in \mathcal{GID}_\lambda$, an input $x \in \mathcal{X}_{\text{id}} = \{0, 1\}^{\ell_{\text{id}}}$, and outputs the secret key $\text{SK}_{\text{id}, \text{GID}, x}$.

Enc $(\{\text{MSK}_{\text{id}}\}_{\text{id} \in [n]}, C) \rightarrow \text{CT}$. The probabilistic encryption algorithm takes as input the master secret keys for all authorities, $\{\text{MSK}_{\text{id}}\}_{\text{id} \in [n]}$, a circuit $C \in \mathcal{C}_\lambda$, and outputs ciphertext CT.

Dec $(\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}, \text{CT}) \rightarrow y$. The deterministic decryption algorithm takes as input the secret keys for all authorities, $\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}$, ciphertext CT, and outputs the value y .

Definition J.1 (opMAFE). An opMAFE scheme (GSetup, KGen, Enc, Dec) is said to be a one-ciphertext private-key MAFE scheme for circuit class $\mathcal{C}$ and $\mathcal{GID}$ if it satisfies the following properties.

Correctness. For any $\lambda \in \mathbb{N}$, $n = n(\lambda)$, $s = s(\lambda)$, $C \in \mathcal{C}_\lambda$, $\text{GID} \in \mathcal{GID}_\lambda$, $\forall \text{id} \in [n]$, $\ell_{\text{id}} = \ell_{\text{id}}(\lambda)$, $x_{\text{id}} \in \{0, 1\}^{\ell_{\text{id}}}$,

$$\Pr \left[\begin{array}{l} C(x_1, \dots, x_n) = \\ \text{Dec}(\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id}}, \text{CT}) \end{array} : \begin{array}{l} (\text{CRS}, \{\text{MSK}_{\text{id}}\}_{\text{id}}) \leftarrow \text{GSetup}(1^\lambda, 1^n, 1^s, 1^{\ell_1}, \dots, 1^{\ell_n}), \\ \forall \text{id} \in [n], \text{SK}_{\text{id}, \text{GID}, x_{\text{id}}} \leftarrow \text{KGen}(\text{id}, \text{MSK}_{\text{id}}, \text{GID}, x_{\text{id}}), \\ \text{CT} \leftarrow \text{Enc}(\{\text{MSK}_{\text{id}}\}_{\text{id}}, C) \end{array} \right] = 1$$

IND Security with Batched Key Generation. For any PPT admissible adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} b \leftarrow \mathcal{A}^{\mathcal{O}(\cdot, \cdot)}(\text{CT}) \\ \text{ } \end{array} : \begin{array}{l} (n, s, \ell_1, \dots, \ell_n) \leftarrow \mathcal{A}(1^\lambda), \\ (\text{CRS}, \{\text{MSK}_{\text{id}}\}_{\text{id} \in [n]}) \leftarrow \text{GSetup}(1^\lambda, 1^n, 1^s, 1^{\ell_1}, \dots, 1^{\ell_n}), \\ (C_0, C_1) \leftarrow \mathcal{A}^{\mathcal{O}(\cdot, \cdot)}(\text{CRS}), \\ b \stackrel{\$}{\leftarrow} \{0, 1\}, \text{CT} \leftarrow \text{Enc}(\{\text{MSK}_{\text{id}}\}_{\text{id} \in [n]}, C_b) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda)$$

where $\mathcal{O}$ is a key-generation oracle that takes as input $(\text{GID}, (x_1, \dots, x_n)) \in \mathcal{GID} \times \{0, 1\}^{\ell_1} \times \dots \times \{0, 1\}^{\ell_n}$ and outputs $(\text{SK}_{1, \text{GID}, x_1}, \dots, \text{SK}_{n, \text{GID}, x_n})$ where $\text{SK}_{\text{id}, \text{GID}, x} \leftarrow \text{KGen}(\text{id}, \text{MSK}_{\text{id}}, \text{GID}, x_{\text{id}})$ for each $\text{id} \in [n]$. A stateful PPT machine $\mathcal{A}$ is said to be an admissible adversary if for each $\text{GID} \in \mathcal{GID}$ queried by $\mathcal{A}$ with the corresponding query $(\text{GID}, x_1, \dots, x_n)$, $C_0(x_1, \dots, x_n) = C_1(x_1, \dots, x_n)$. Moreover, $C_0, C_1 \in \mathcal{C}_\lambda$, $|C_0| = |C_1|$ and are of size at most s .

J.2 Preliminaries

We will use a one-key private-key functional encryption scheme for P/Poly circuits [GVW12] in our construction. We formally define it below. Essentially this is a private-key BFE scheme (Definition 3.3) for P/Poly circuits where the bound $Q = 1$. In addition, we can encrypt an unbounded polynomial number of circuits, but simulation security is only guaranteed for one challenge query.

Definition J.2. A private-key one-key FE scheme for P/Poly circuits is a BFE scheme for P/Poly circuits where the collusion bound $Q = 1$. Hence, we ignore Q from the syntax. The security of the scheme can also be strengthened to allow unbounded encryptions for different circuits but only one challenge circuit against which simulation security holds. Specifically, the security definition is defined below.

Security. For any PPT stateful admissible adversary $\mathcal{A}$, there exists a stateful PPT simulator such that, for all $\lambda \in \mathbb{N}$,

$$\left\{ \begin{array}{l} \mathcal{A}^{\mathcal{O}(\text{MSK}, \cdot, \cdot)}(\text{CT}^*) : \\ \begin{array}{l} s \leftarrow \mathcal{A}(1^\lambda), \\ \text{MSK} \leftarrow \text{Setup}(1^\lambda, 1^s), \\ C^* \leftarrow \mathcal{A}^{\mathcal{O}(\text{MSK}, \cdot, \cdot)}(1^\lambda), \\ \text{CT}^* \leftarrow \text{Enc}(\text{MSK}, C^*) \end{array} \end{array} \right\} \approx_c \left\{ \begin{array}{l} \mathcal{A}^{\text{Sim}^{C^*}(\cdot)(\text{st}, \cdot, \cdot)}(\text{CT}^*) : \\ \begin{array}{l} s \leftarrow \mathcal{A}(1^\lambda), \\ \text{st} \leftarrow \text{Sim}(1^\lambda, 1^s), \\ C^* \leftarrow \mathcal{A}^{\text{Sim}(\text{st}, \cdot, \cdot)}(1^\lambda), \\ (\text{CT}^*, \text{st}) \leftarrow \text{Sim}(1^{|C^*|}, \mathbf{V}) \end{array} \end{array} \right\}$$

where $\mathbf{V} = \{(x, C^*(x)) : \text{if key generation for } x \text{ was queried before challenge query}\}$. $\mathcal{O}$ and Sim respond to the following queries from $\mathcal{A}$.

- (Enc, C). $\mathcal{A}$ queries for an encryption of circuit C with size at most s . $\mathcal{O}$ responds with $\text{CT} \leftarrow \text{Enc}(\text{MSK}, C)$. On the other hand, Sim responds with $\text{CT} \leftarrow \text{Sim}(\text{st}, C)$.
- (KGen, x). $\mathcal{A}$ queries for secret key for input x . $\mathcal{O}$ responds with $\text{SK}_x \leftarrow \text{KGen}(\text{MSK}, x)$ and Sim responds with $\text{SK}_x \leftarrow \text{Sim}(\text{st}, x)$. In addition, if secret key query was done after challenge query for C^* , Sim uses $\mathbf{V} = \{(x, C^*(x))\}$.

A PPT machine $\mathcal{A}$ is said to be admissible if it makes at most one secret key query for x and at most one challenge query for circuit C^* of size at most s .

Theorem J.3 ([GVW12]). A private-key one-key FE scheme for P/Poly circuits exists assuming the hardness of one-way functions.

J.3 One-Key One-Challenge SIM-secure MCFE for P/Poly

In this section, we will construct a private-key MCFE scheme for P/Poly circuits where the scheme remains secure if at most one key for circuit $C \in \mathcal{C}_\lambda$ and at most one challenge ciphertext per client for inputs $x_1, \dots, x_n$ are corrupted by the adversary. Moreover, there is a simulator that only utilizes $(C, C(x_1, \dots, x_n))$ to simulate the challenge ciphertext and secret key. Specifically, we will construct an MCFE scheme that satisfies the following definition.

Syntax. A one-key private-key MCFE scheme for circuit class $\mathcal{C} = \{\mathcal{C}_\lambda : \mathcal{X}_{\lambda,1} \times \dots \times \mathcal{X}_{\lambda,n} \rightarrow \mathcal{O}_\lambda\}_{\lambda \in \mathbb{N}}$ and global identifier domain $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ consists of the following polynomial-time algorithms.

$\text{GSetup}(1^\lambda, 1^n, 1^{\ell_1}, \dots, 1^{\ell_n}, 1^s) \rightarrow (\text{CRS}, \text{MSK}, \{\text{EK}_{\text{id}}\}_{\text{id} \in [n]})$. The probabilistic global setup algorithm takes as input the security parameter λ , number of authorities n , input sizes for all authorities $\ell_1, \dots, \ell_n$, the maximum circuit size s , outputs CRS, master secret key MSK, and encryption keys for all authorities, $\{\text{EK}_{\text{id}}\}_{\text{id} \in [n]}$. The following algorithms take CRS implicitly.

$\text{Enc}(\text{id}, \text{EK}_{\text{id}}, \text{GID}, x) \rightarrow \text{CT}_{\text{id}, \text{GID}}$. The probabilistic encryption algorithm takes as input the id-th encryption key EK_{id} , the global identifier for the user $\text{GID} \in \mathcal{GID}_\lambda$, an input $x \in \{0, 1\}^{\ell_{\text{id}}}$, and outputs ciphertext $\text{CT}_{\text{id}, \text{GID}}$.

$\text{KGen}(\text{MSK}, C) \rightarrow \text{SK}_C$. The possibly randomized key generation algorithm takes as input the master secret key, MSK, a circuit $C \in \mathcal{C}_\lambda$, and outputs secret key SK_C .

$\text{Dec}(\{\text{CT}_{\text{id}, \text{GID}}\}_{\text{id} \in [n]}, \text{SK}_C) \rightarrow y$. The deterministic decryption algorithm takes as input the ciphertexts for all authorities, $\{\text{CT}_{\text{id}, \text{GID}}\}_{\text{id} \in [n]}$, secret key SK_C , and outputs the value y .

Definition J.4 (oMCFE). An oMCFE scheme $(\text{GSetup}, \text{Enc}, \text{KGen}, \text{Dec})$ is said to be a one-key, one-challenge MCFE scheme for circuit class $\mathcal{C}$ and $\mathcal{GID}$ if it satisfies the following properties.

Correctness. For any $\lambda \in \mathbb{N}$, $n = n(\lambda)$, $s = s(\lambda)$, $C \in \mathcal{C}_\lambda$, $\text{GID} \in \mathcal{GID}_\lambda$, $\forall \text{id} \in [n]$, $\ell_{\text{id}} = \ell_{\text{id}}(\lambda)$, $x_{\text{id}} \in \{0, 1\}^{\ell_{\text{id}}}$,

$$\Pr \left[\begin{array}{l} C(x_1, \dots, x_n) = \\ \text{Dec}(\{\text{CT}_{\text{id}, \text{GID}}\}_{\text{id} \in [n]}, \text{SK}_C) \end{array} : \begin{array}{l} (\text{CRS}, \text{MSK}, \{\text{EK}_{\text{id}}\}_{\text{id}}) \leftarrow \text{GSetup}(1^\lambda, 1^n, 1^{\ell_1}, \dots, 1^{\ell_n}, 1^s), \\ \forall \text{id} \in [n], \text{CT}_{\text{id}, \text{GID}} \leftarrow \text{Enc}(\text{id}, \text{EK}_{\text{id}}, \text{GID}, x_{\text{id}}), \\ \text{SK}_C \leftarrow \text{KGen}(\text{MSK}, C) \end{array} \right] = 1$$

SIM Security with Batched Encryption. For any PPT admissible adversary $\mathcal{A}$, there exists a stateful PPT simulator Sim such that,

$$\left\{ \begin{array}{l} \mathcal{A}^{\mathcal{O}(\cdot, \cdot)}(\{\text{CT}_{\text{id}, \text{GID}^*}^*\}_{\text{id} \in [n]}) : \\ \begin{array}{l} n, \ell_1, \dots, \ell_n, s \leftarrow \mathcal{A}(1^\lambda), \\ (\text{CRS}, \text{MSK}, \{\text{EK}_{\text{id}}\}_{\text{id} \in [n]}) \leftarrow \text{GSetup}(1^\lambda, 1^n, 1^{\ell_1}, \dots, 1^{\ell_n}, 1^s), \\ (\text{GID}^*, x_1^*, \dots, x_n^*) \leftarrow \mathcal{A}^{\mathcal{O}(\cdot, \cdot)}(\text{CRS}), \\ \forall \text{id} \in [n], \text{CT}_{\text{id}, \text{GID}^*}^* \leftarrow \text{Enc}(\text{id}, \text{EK}_{\text{id}}, \text{GID}^*, x_{\text{id}}^*) \end{array} \end{array} \right\} \approx_c \left\{ \begin{array}{l} \mathcal{A}^{\text{Sim}^{\mathcal{U}(\cdot, x_1^*, \dots, x_n^*)}(\text{st}, \cdot, \cdot)}(\{\text{CT}_{\text{id}, \text{GID}^*}^*\}_{\text{id} \in [n]}) : \\ \begin{array}{l} n, \ell_1, \dots, \ell_n, s \leftarrow \mathcal{A}(1^\lambda), \\ (\text{CRS}, \text{st}) \leftarrow \text{Sim}(1^\lambda, 1^n, 1^{\ell_1}, \dots, 1^{\ell_n}, 1^s), \\ (\text{GID}^*, x_1^*, \dots, x_n^*) \leftarrow \mathcal{A}^{\text{Sim}(\text{st}, \cdot, \cdot)}(\text{CRS}), \\ \{\text{CT}_{\text{id}, \text{GID}^*}^*\}_{\text{id} \in [n]} \leftarrow \text{Sim}(\text{V}) \end{array} \end{array} \right\}$$

where $\text{V} = \{(C, C(x_1^*, \dots, x_n^*)) : \text{if key generation for } C \text{ was queried before challenge query}\}$. $\mathcal{O}$ and Sim respond to the following queries from $\mathcal{A}$.

- $(\text{Enc}, (\text{GID}, x_1, \dots, x_n))$. $\mathcal{A}$ queries for an encryption of inputs $x_1, \dots, x_n$ and GID . $\mathcal{O}$ responds with $\{\text{CT}_{\text{id}, \text{GID}}\}_{\text{id} \in [n]}$ where $\text{CT}_{\text{id}, \text{GID}} \leftarrow \text{Enc}(\text{id}, \text{EK}_{\text{id}}, \text{GID}, x_{\text{id}})$. On the other hand, Sim responds with $\{\text{CT}_{\text{id}, \text{GID}}\}_{\text{id} \in [n]} \leftarrow \text{Sim}(\text{st}, \text{GID}, x_1, \dots, x_n)$.

- (KGen, x) . $\mathcal{A}$ queries for secret key for circuit C of size at most s . $\mathcal{O}$ responds with $\text{SK}_C \leftarrow \overline{\text{KGen}}(\text{MSK}, C)$ and Sim responds with $\text{SK}_C \leftarrow \text{Sim}(\text{st}, C)$. In addition, if secret key query was done after challenge query for $(\text{GID}^*, x_1^*, \dots, x_n^*)$, Sim uses $\mathbf{V} = \{(C, C(x_1^*, \dots, x_n^*))\}$.

A PPT machine $\mathcal{A}$ is said to be admissible if it makes at most one secret key query for C of size at most s , at most one challenge query for $(\text{GID}^*, x_1^*, \dots, x_n^*)$, and makes unbounded Enc queries with a unique GID each.

Construction J.5. We provide the construction of a MCFE scheme with batched key generation for P/Poly circuits and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ in the trusted setup model using a one-key private-key FE scheme for P/Poly circuits (Definition J.2), a pseudorandom function $\text{PRF} : \{0, 1\}^\lambda \times \mathcal{GID}_\lambda \rightarrow \{0, 1\}^\lambda$ (Definition A.3), and garbled circuit scheme $(\text{Garb}, \text{Eval})$ for P/Poly circuits (Definition A.1) as follows.

$\text{GSetup}(1^\lambda, 1^n, 1^{\ell_1}, \dots, 1^{\ell_n}, 1^s)$. Sample PRF key $K \xleftarrow{\$} \{0, 1\}^\lambda$. For each $\text{id} \in [n]$, sample $1\text{-fe.msk}_{\text{id}} \leftarrow 1\text{-FE.Setup}(1^\lambda, 1^{s'})$ where $s' = \text{poly}(\lambda, n, \ell_1, \dots, \ell_n, s)$ is the maximum size of the circuit in Figure 7. Set $\text{CRS} := (\lambda, n, \ell_1, \dots, \ell_n, s)$, $\text{EK}_{\text{id}} := (K, 1\text{-fe.msk}_{\text{id}})$ for each $\text{id} \in [n]$, $\text{MSK} = \{1\text{-fe.msk}_{\text{id}}\}_{\text{id} \in [n]}$ and output $(\text{CRS}, \text{MSK}, \{\text{EK}_{\text{id}}\}_{\text{id} \in [n]})$.

$\text{Enc}(\text{id}, \text{EK}_{\text{id}}, \text{GID}, x)$. Parse EK_{id} as $(K, 1\text{-fe.msk}_{\text{id}})$. Given x such that $|x| \leq \ell_{\text{id}}$, compute $\alpha_{\text{id}} = \left(\sum_{i=1}^{\text{id}-1} \ell_i\right) + 1$ and $\mathbb{I}_{\text{id}} = \left\{\left(\sum_{i=1}^{\text{id}-1} \ell_i\right) + 1, \dots, \left(\sum_{i=1}^{\text{id}} \ell_i\right)\right\}$. Compute $R = \text{PRF}(K, \text{GID})$, and construct the circuit F as specified in Figure 7.

Encrypt F using 1-FE as $1\text{-fe.ct}_{\text{id}} \leftarrow 1\text{-FE.Enc}(1\text{-fe.msk}_{\text{id}}, F)$. Output $\text{CT}_{\text{id}, \text{GID}} := (\text{GID}, 1\text{-fe.ct}_{\text{id}})$.

$\text{KGen}(\text{MSK}, C)$. Parse MSK as $\{1\text{-fe.msk}_{\text{id}}\}_{\text{id} \in [n]}$.

For each $\text{id} \in [n]$, sample $1\text{-fe.sk}_{C, \text{id}} \leftarrow 1\text{-FE.KGen}(1\text{-fe.msk}_{\text{id}}, C)$. Output $\text{SK}_C := \{1\text{-fe.sk}_{C, \text{id}}\}_{\text{id} \in [n]}$.

$\text{Dec}(\text{SK}_C, \{\text{CT}_{\text{id}, \text{GID}_{\text{id}}}\}_{\text{id} \in [n]})$. Parse SK_C as $\{1\text{-fe.sk}_{C, \text{id}}\}_{\text{id} \in [n]}$ and $\text{CT}_{\text{id}, \text{GID}_{\text{id}}}$ as $(\text{GID}_{\text{id}}, 1\text{-fe.ct}_{\text{id}})$ for each $\text{id} \in [n]$. If all GID_{id} are not the same, abort and output $\perp$. Otherwise, let $\text{GID} := \text{GID}_1$.

For each $\text{id} \in [n]$, compute $y_{\text{id}} := 1\text{-FE.Dec}(1\text{-fe.sk}_{C, \text{id}}, 1\text{-fe.ct}_{\text{id}})$. Parse each y_{id} as $(\tilde{C}_{\text{id}}, \bar{w}_{\text{id}})$. If all $\tilde{C}_{\text{id}}$ are not the same, abort and output $\perp$. Otherwise, let $\tilde{C} := \tilde{C}_{\text{id}}$.

Then, compute and output $y := \text{Eval}(\tilde{C}, \{\bar{w}_{\text{id}}\}_{\text{id} \in [n]})$.

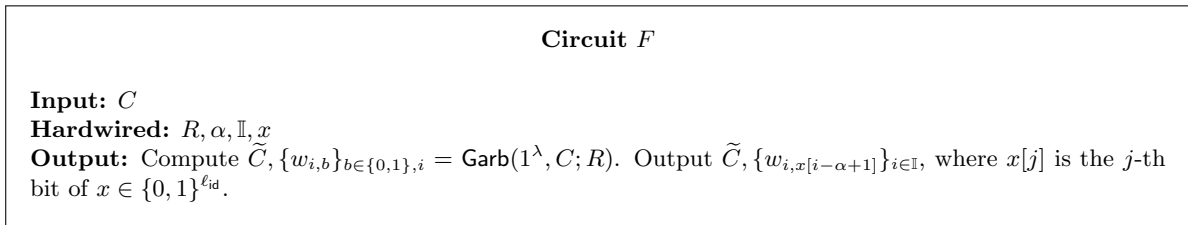


Figure 7: Description of circuit F

Correctness. The correctness of the scheme follows from the correctness of 1-FE, garbling scheme (Garb, Eval), and the deterministic nature of PRF. By correctness of 1-FE, $y_{id} = (\tilde{C}, \bar{w}_{id})$ where $\tilde{C}, \{\bar{w}_{id}\}_{id \in [n]} \leftarrow \text{Garb}(1^\lambda, C; R)$ and $R = \text{PRF}(K, \text{GID})$. If all GID are the same, then so are $\tilde{C}$ due to deterministic nature of PRF. Thus, by correctness of garbling scheme, $y = C(x_1, \dots, x_n)$.

Lemma J.6. If 1-FE is an adaptively secure one-key private-key FE scheme for P/Poly circuits (Definition J.2), $\text{PRF} : \{0, 1\}^\lambda \times \text{GID}_\lambda \rightarrow \{0, 1\}^\lambda$ is a secure pseudorandom function (Definition A.3), and (Garb, Eval) is a secure garbling scheme for P/Poly circuits (Definition A.1), then Construction J.5 is an adaptively secure one-key MCFE scheme for P/Poly circuits and $\mathcal{GID} = \{\text{GID}_\lambda\}_{\lambda \in \mathbb{N}}$.

Proof. We prove this lemma using the following hybrids and claims.

Hyb₀^A(1^λ). This is almost the real experiment with honest challenger. We sample $\alpha_{id}, \mathbb{I}_{id}$ early during setup. Nevertheless, the distributions remain identical.

Setup Phase. $\mathcal{A}$ sends $(n, s, \ell_1, \dots, \ell_n)$ to Chal. Chal samples $1\text{-fe.msk}_{id} \leftarrow 1\text{-FE.Setup}(1^\lambda, 1^{s'})$. Chal also samples $K \leftarrow \{0, 1\}^\lambda$.

Chal also computes $\alpha_{id}, \mathbb{I}_{id}$ as described in Enc of Construction J.5.

Chal sets $\text{CRS} := (\lambda, n, \ell_1, \dots, \ell_n, s)$ and $\text{MSK} = \{1\text{-fe.msk}_{id}\}_{id \in [n]}$, $\text{EK}_{id} := (K, 1\text{-fe.msk}_{id})$ for each $id \in [n]$ and sends CRS to $\mathcal{A}$.

Adaptive Queries. $\mathcal{A}$ can now makes one of each of the following queries in an adaptive manner.

Encryption Query. $\mathcal{A}$ sends $(\text{GID}, x_1, \dots, x_n)$. Chal computes $R = \text{PRF}(K, \text{GID})$, constructs circuits F_{id} , and samples $1\text{-fe.ct}_{id} \leftarrow 1\text{-FE.Enc}(1\text{-fe.msk}_{id}, F_{id})$ for each $id \in [n]$. Chal sets $\text{CT}_{id, \text{GID}} := (\text{GID}, 1\text{-fe.ct}_{id})$ and sends $\{\text{CT}_{id, \text{GID}}\}_{id \in [n]}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends $(\text{GID}^*, x_1^*, \dots, x_n^*)$. Chal computes $R^* = \text{PRF}(K, \text{GID}^*)$, constructs circuits F_{id}^* , and samples $1\text{-fe.ct}_{id}^* \leftarrow 1\text{-FE.Enc}(1\text{-fe.msk}_{id}, F_{id}^*)$ for each $id \in [n]$. Chal sets $\text{CT}_{id, \text{GID}^*}^* := (\text{GID}^*, 1\text{-fe.ct}_{id}^*)$ and sends $\{\text{CT}_{id, \text{GID}^*}^*\}_{id \in [n]}$ to $\mathcal{A}$.

Secret Key Query. $\mathcal{A}$ sends circuit C . Chal computes $1\text{-fe.sk}_{C, id} \leftarrow 1\text{-FE.KGen}(1\text{-fe.msk}_{id}, C)$ for each $id \in [n]$ and sends $\text{SK}_C := \{1\text{-fe.sk}_{C, id}\}_{id \in [n]}$ to $\mathcal{A}$.

Guess Phase. $\mathcal{A}$ sends guess b' . Output b' .

Hyb_{1,j}^A(1^λ) for $j \in [n+1]$. In this hybrid, we simulate the first $j-1$ instantiations of 1-FE. The changes are highlighted in red.

Setup Phase. $\mathcal{A}$ sends $(n, s, \ell_1, \dots, \ell_n)$. For each $id \in [n]$, if $id < j$, sample $\text{st}_{id} \leftarrow 1\text{-FE.Sim}(1^\lambda, 1^{s'})$. Otherwise, if $id \geq j$, sample $1\text{-fe.msk}_{id} \leftarrow 1\text{-FE.Setup}(1^\lambda, 1^{s'})$. Also sample $K \leftarrow \{0, 1\}^\lambda$.

Compute $\alpha_{id}, \mathbb{I}_{id}$ as described in Enc of Construction J.5.

Set $\text{CRS} := (\lambda, n, \ell_1, \dots, \ell_n, s)$ and send CRS to $\mathcal{A}$.

Adaptive Queries. $\mathcal{A}$ can now makes one of each of the following queries in an adaptive manner.

Encryption Query. $\mathcal{A}$ sends $(\text{GID}, x_1, \dots, x_n)$. Compute $R = \text{PRF}(K, \text{GID})$, construct circuits F_{id} . For each $id \in [n]$,

- If $\text{id} < j$, sample $1\text{-fe.ct}_{\text{id}} \leftarrow 1\text{-FE.Sim}(\text{st}_{\text{id}}, \text{Enc}, F_{\text{id}})$.
- Otherwise, sample $1\text{-fe.ct}_{\text{id}} \leftarrow 1\text{-FE.Enc}(1\text{-fe.mpk}_{\text{id}}, F_{\text{id}})$.

Set $\text{CT}_{\text{id}, \text{GID}} := (\text{GID}, 1\text{-fe.ct}_{\text{id}})$ and sends $\{\text{CT}_{\text{id}, \text{GID}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends $(\text{GID}^*, x_1^*, \dots, x_n^*)$. Compute $R^* = \text{PRF}(K, \text{GID}^*)$, construct circuits F_{id}^* . For each $\text{id} \in [n]$,

- If $\text{id} < j$, sample $1\text{-fe.ct}_{\text{id}}^* \leftarrow 1\text{-FE.Sim}(\text{st}_{\text{id}}, 1^{|F_{\text{id}}^*|}, V)$ where $V = F_{\text{id}}^*(C)$ if $\mathcal{A}$ already queried for secret key. Otherwise, $V = \perp$.
- Otherwise, sample $1\text{-fe.ct}_{\text{id}}^* \leftarrow 1\text{-FE.Enc}(1\text{-fe.msk}_{\text{id}}, F_{\text{id}}^*)$.

Set $\text{CT}_{\text{id}, \text{GID}^*}^* := (\text{GID}^*, 1\text{-fe.ct}_{\text{id}}^*)$ and sends $\{\text{CT}_{\text{id}, \text{GID}^*}^*\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Secret Key Query. $\mathcal{A}$ sends circuit C . For each $\text{id} \in [n]$,

- If $\text{id} < j$, sample $1\text{-fe.sk}_{C, \text{id}} \leftarrow 1\text{-FE.Sim}(\text{st}_{\text{id}}, \text{KGen}, C, V)$ where $V = F_{\text{id}}(C)$ if $\mathcal{A}$ already queried for encryption. Otherwise, $V = \perp$.
- Otherwise, sample $1\text{-fe.sk}_{C, \text{id}} \leftarrow 1\text{-FE.KGen}(1\text{-fe.msk}_{\text{id}}, C)$.

Send $\text{SK}_C := \{1\text{-fe.sk}_{C, \text{id}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Guess Phase. $\mathcal{A}$ sends guess b' . Output b' .

Hyb₂^A(1^λ). In this hybrid, we set R to be a uniformly random string instead of PRF output on GID . The changes are highlighted in red.

Setup Phase. $\mathcal{A}$ sends $(n, s, \ell_1, \dots, \ell_n)$. For each $\text{id} \in [n]$, sample $\text{st}_{\text{id}} \leftarrow 1\text{-FE.Sim}(1^\lambda, 1^{s'})$.

Compute $\alpha_{\text{id}}, \mathbb{I}_{\text{id}}$ as described in Enc of Construction J.5.

Set $\text{CRS} := (\lambda, n, \ell_1, \dots, \ell_n, s)$ and send CRS to $\mathcal{A}$.

Adaptive Queries. $\mathcal{A}$ can now makes one of each of the following queries in an adaptive manner.

Encryption Query. $\mathcal{A}$ sends $(\text{GID}, x_1, \dots, x_n)$. Construct the circuit F_{id} with $R \leftarrow \{0, 1\}^\lambda$. Sample $1\text{-fe.ct}_{\text{id}} \leftarrow 1\text{-FE.Sim}(\text{st}_{\text{id}}, \text{Enc}, F_{\text{id}})$. Set $\text{CT}_{\text{id}, \text{GID}} := (\text{GID}, 1\text{-fe.ct}_{\text{id}})$ and sends $\{\text{CT}_{\text{id}, \text{GID}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends $(\text{GID}^*, x_1^*, \dots, x_n^*)$. If $\mathcal{A}$ already queried for secret key with circuit C , sample $(\tilde{C}, \{w_{i,b}\}_{i,b}) \leftarrow \text{Garb}(1^\lambda, C; R)$ where $R \leftarrow \{0, 1\}^\lambda$. Then, set $V_{\text{id}} = (\tilde{C}, \bar{w}_{\text{id}})$ for each $\text{id} \in [n]$. Otherwise, set $V_{\text{id}} = \perp$. Sample $1\text{-fe.ct}_{\text{id}}^* \leftarrow 1\text{-FE.Sim}(\text{st}_{\text{id}}, 1^{|F_{\text{id}}^*|}, V_{\text{id}})$. Set $\text{CT}_{\text{id}, \text{GID}^*}^* := (\text{GID}^*, 1\text{-fe.ct}_{\text{id}}^*)$ and sends $\{\text{CT}_{\text{id}, \text{GID}^*}^*\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Secret Key Query. $\mathcal{A}$ sends circuit C . If $\mathcal{A}$ already queried for encryption with input $\text{GID}, x_1, \dots, x_n$, sample $(\tilde{C}, \{w_{i,b}\}_{i,b}) \leftarrow \text{Garb}(1^\lambda, C; R)$ where $R \leftarrow \{0, 1\}^\lambda$. Then, set $V_{\text{id}} = (\tilde{C}, \bar{w}_{\text{id}})$ for each $\text{id} \in [n]$. Otherwise, set $V_{\text{id}} = \perp$. For each $\text{id} \in [n]$, sample $1\text{-fe.sk}_{C, \text{id}} \leftarrow 1\text{-FE.Sim}(\text{st}_{\text{id}}, \text{KGen}, C, V_{\text{id}})$. Send $\text{SK}_C := \{1\text{-fe.sk}_{C, \text{id}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Guess Phase. $\mathcal{A}$ sends guess b' . Output b' .

Hyb₃^A(1^λ). In this hybrid, we will simulate the garbling scheme to use only $C(x_1, \dots, x_n)$. This is the description of simulator Sim. The changes are highlighted in red.

Setup Phase. $\mathcal{A}$ sends $(n, s, \ell_1, \dots, \ell_n)$. For each $\text{id} \in [n]$, sample $\text{st}_{\text{id}} \leftarrow 1\text{-FE.Sim}(1^\lambda, 1^{s'})$.

Compute $\alpha_{\text{id}}, \mathbb{I}_{\text{id}}$ as described in Enc of Construction J.5.

Set $\text{CRS} := (\lambda, n, \ell_1, \dots, \ell_n, s)$ and send CRS to $\mathcal{A}$.

Adaptive Queries. $\mathcal{A}$ can now makes one of each of the following queries in an adaptive manner.

Encryption Query. $\mathcal{A}$ sends $(\text{GID}, x_1, \dots, x_n)$. **Construct the circuit F_{id} with $R \leftarrow \{0, 1\}^\lambda$.** Sample $1\text{-fe.ct}_{\text{id}} \leftarrow 1\text{-FE.Sim}(\text{st}_{\text{id}}, \text{Enc}, F_{\text{id}})$. Set $\text{CT}_{\text{id}, \text{GID}} := (\text{GID}, 1\text{-fe.ct}_{\text{id}})$ and sends $\{\text{CT}_{\text{id}, \text{GID}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends $(\text{GID}^*, x_1^*, \dots, x_n^*)$. **If $\mathcal{A}$ already queried for secret key with circuit C , sample $(\tilde{C}, \{\bar{w}_{\text{id}}\}_{\text{id}}) \leftarrow \text{Garb.Sim}(1^\lambda, 1^{|C|}, C(x_1^*, \dots, x_n^*))$. Then, set $V_{\text{id}} = (\tilde{C}, \bar{w}_{\text{id}})$ for each $\text{id} \in [n]$. Otherwise, set $V_{\text{id}} = \perp$.** Sample $1\text{-fe.ct}_{\text{id}}^* \leftarrow 1\text{-FE.Sim}(\text{st}_{\text{id}}, 1^{|F_{\text{id}}^*|}, V_{\text{id}})$. Set $\text{CT}_{\text{id}, \text{GID}^*}^* := (\text{GID}^*, 1\text{-fe.ct}_{\text{id}}^*)$ and sends $\{\text{CT}_{\text{id}, \text{GID}^*}^*\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Secret Key Query. $\mathcal{A}$ sends circuit C . **If $\mathcal{A}$ already queried for encryption with input $\text{GID}, x_1, \dots, x_n$, sample $(\tilde{C}, \{\bar{w}_{\text{id}}\}_{\text{id}}) \leftarrow \text{Garb.Sim}(1^\lambda, 1^{|C|}, C(x_1^*, \dots, x_n^*))$. Then, set $V_{\text{id}} = (\tilde{C}, \bar{w}_{\text{id}})$ for each $\text{id} \in [n]$. Otherwise, set $V_{\text{id}} = \perp$.** For each $\text{id} \in [n]$, sample $1\text{-fe.sk}_{C, \text{id}} \leftarrow 1\text{-FE.Sim}(\text{st}_{\text{id}}, \text{KGen}, C, V_{\text{id}})$. Send $\text{SK}_C := \{1\text{-fe.sk}_{C, \text{id}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Guess Phase. $\mathcal{A}$ sends guess b' . Output b' .

Claim J.7. $\text{Hyb}_0^A(1^\lambda)$ and $\text{Hyb}_{1,1}^A(1^\lambda)$ are identically distributed.

Proof. As we are not simulating any 1-FE instantiations in $\text{Hyb}_{1,1}^A(1^\lambda)$, these hybrids are identically distributed. $\square$

Claim J.8. Assuming the security of 1-FE, $\text{Hyb}_{1,j}^A(1^\lambda)$ and $\text{Hyb}_{1,j+1}^A(1^\lambda)$ for any $j \in [n]$ are computationally indistinguishable.

Proof. Assume that there exists an adversary $\mathcal{A}$ that can distinguish between $\text{Hyb}_{1,j}^A(1^\lambda)$ and $\text{Hyb}_{1,j+1}^A(1^\lambda)$ for some $j \in [n]$ with non-negligible probability $\epsilon(\lambda)$, i.e,

$$\left| \Pr \left[1 \leftarrow \text{Hyb}_{1,j}^A(1^\lambda) \right] - \Pr \left[1 \leftarrow \text{Hyb}_{1,j+1}^A(1^\lambda) \right] \right| > \epsilon(\lambda)$$

We will construct a reduction adversary $\mathcal{B}$ that can break the security of 1-FE with non-negligible probability. The description of $\mathcal{B}^\mathcal{O}$ is as follows.

Setup Phase. $\mathcal{A}$ sends $(n, s, \ell_1, \dots, \ell_n)$. For each $\text{id} \in [n]$, if $\text{id} < j$, sample $\text{st}_{\text{id}} \leftarrow 1\text{-FE.Sim}(1^\lambda, 1^{s'})$. Otherwise, if $\text{id} > j$, sample $1\text{-fe.msk}_{\text{id}} \leftarrow 1\text{-FE.Setup}(1^\lambda, 1^{s'})$. Also sample $K \leftarrow \{0, 1\}^\lambda$.

Compute $\alpha_{\text{id}}, \mathbb{I}_{\text{id}}$ as described in Enc of Construction J.5.

Set $\text{CRS} := (\lambda, n, \ell_1, \dots, \ell_n, s)$ and send CRS to $\mathcal{A}$.

Adaptive Queries. $\mathcal{A}$ can now makes one of each of the following queries in an adaptive manner.

Encryption Query. $\mathcal{A}$ sends $(\text{GID}, x_1, \dots, x_n)$. Compute $R = \text{PRF}(K, \text{GID})$, construct circuits F_{id} . For each $\text{id} \in [n]$,

- If $\text{id} < j$, sample $1\text{-fe.ct}_{\text{id}} \leftarrow 1\text{-FE.Sim}(\text{st}_{\text{id}}, \text{Enc}, |F_{\text{id}}|)$.
- Otherwise, if $\text{id} = j$, send F_{id} to $\mathcal{O}$ to receive $1\text{-fe.ct}_{\text{id}}$.
- Otherwise, sample $1\text{-fe.ct}_{\text{id}} \leftarrow 1\text{-FE.Enc}(1\text{-fe.msk}_{\text{id}}, F_{\text{id}})$.

Set $\text{CT}_{\text{id}, \text{GID}} := (\text{GID}, 1\text{-fe.ct}_{\text{id}})$ and sends $\{\text{CT}_{\text{id}, \text{GID}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends $(\text{GID}^*, x_1^*, \dots, x_n^*)$. Compute $R = \text{PRF}(K, \text{GID}^*)$, construct circuits F_{id}^* . For each $\text{id} \in [n]$,

- If $\text{id} < j$, sample $1\text{-fe.ct}_{\text{id}}^* \leftarrow 1\text{-FE.Sim}(\text{st}_{\text{id}}, 1^{|F_{\text{id}}^*|}, V_{\text{id}})$ where $V = F_{\text{id}}(C)$ if $\mathcal{A}$ already queried for secret key for C . Otherwise, $V = \perp$.
- Otherwise, if $\text{id} = j$, send F_{id} to $\mathcal{O}$ to receive $1\text{-fe.ct}_{\text{id}}^*$.
- Otherwise, sample $1\text{-fe.ct}_{\text{id}}^* \leftarrow 1\text{-FE.Enc}(1\text{-fe.msk}_{\text{id}}, F_{\text{id}}^*)$.

Set $\text{CT}_{\text{id}, \text{GID}^*}^* := (\text{GID}^*, 1\text{-fe.ct}_{\text{id}}^*)$ and sends $\{\text{CT}_{\text{id}, \text{GID}^*}^*\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Secret Key Query. $\mathcal{A}$ sends circuit C . For each $\text{id} \in [n]$,

- If $\text{id} < j$, sample $1\text{-fe.sk}_{C, \text{id}} \leftarrow 1\text{-FE.Sim}(\text{st}_{\text{id}}, \text{KGen}, C, V)$ where $V = F_{\text{id}}(C)$ if $\mathcal{A}$ already queried for encryption. Otherwise, $V = \perp$.
- Otherwise, if $\text{id} = j$, send C to $\mathcal{O}$ to receive $1\text{-fe.sk}_{C, \text{id}}$.
- Otherwise, sample $1\text{-fe.sk}_{C, \text{id}} \leftarrow 1\text{-FE.KGen}(1\text{-fe.msk}_{\text{id}}, C)$.

Send $\text{SK}_C := \{1\text{-fe.sk}_{C, \text{id}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Guess Phase. $\mathcal{A}$ sends guess b' . Output b' .

As we can see, the running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$, λ . If $\mathcal{O}$ is an honest challenger, $\mathcal{B}$ behaves like $\mathbf{Hyb}_{1,j}^{\mathcal{A}}(1^\lambda)$ and if $\mathcal{O}$ simulate 1-FE, then $\mathcal{B}$ behaves like $\mathbf{Hyb}_{1,j+1}^{\mathcal{A}}(1^\lambda)$. Hence, $\mathcal{B}$ is a valid adversary that breaks the security of 1-FE with non-negligible probability. Thus, $\mathbf{Hyb}_{1,j}^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_{1,j+1}^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable. $\square$

Claim J.9. Assuming the security of PRF, $\mathbf{Hyb}_{1,n+1}^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable.

Proof. Assume that there exists an adversary $\mathcal{A}$ that can distinguish between $\mathbf{Hyb}_{1,n+1}^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ with non-negligible probability $\epsilon(\lambda)$, i.e,

$$\left| \Pr \left[1 \leftarrow \mathbf{Hyb}_{1,n+1}^{\mathcal{A}}(1^\lambda) \right] - \Pr \left[1 \leftarrow \mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda) \right] \right| > \epsilon(\lambda)$$

We will construct a reduction adversary $\mathcal{B}$ that can break the security of PRF with non-negligible probability. The description of $\mathcal{B}^{\mathcal{O}}$ is as follows.

Setup Phase. $\mathcal{A}$ sends $(n, s, \ell_1, \dots, \ell_n)$. For each $\text{id} \in [n]$, sample $\text{st}_{\text{id}} \leftarrow 1\text{-FE.Sim}(1^\lambda, 1^{s'})$.

Compute $\alpha_{\text{id}}, \mathbb{I}_{\text{id}}$ as described in Enc of Construction J.5.

Set $\text{CRS} := (\lambda, n, \ell_1, \dots, \ell_n, s)$ and send CRS to $\mathcal{A}$.

Adaptive Queries. $\mathcal{A}$ can now makes one of each of the following queries in an adaptive manner.

Encryption Query. $\mathcal{A}$ sends $(\text{GID}, x_1, \dots, x_n)$. Construct the circuit F_{id} with $R \leftarrow \mathcal{O}(\text{GID})$. Sample $1\text{-fe.ct}_{\text{id}} \leftarrow 1\text{-FE.Sim}(\text{st}_{\text{id}}, \text{Enc}, F_{\text{id}})$. Set $\text{CT}_{\text{id}, \text{GID}} := (\text{GID}, 1\text{-fe.ct}_{\text{id}})$ and sends $\{\text{CT}_{\text{id}, \text{GID}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends $(\text{GID}^*, x_1^*, \dots, x_n^*)$. If $\mathcal{A}$ already queried for secret key with circuit C , sample $(\tilde{C}, \{w_{i,b}\}_{i,b}) \leftarrow \text{Garb}(1^\lambda, C; R)$ where $R \leftarrow \mathcal{O}(\text{GID})$. Then, set $V_{\text{id}} = (\tilde{C}, \bar{w}_{\text{id}})$ for each $\text{id} \in [n]$. Otherwise, set $V_{\text{id}} = \perp$. Sample $1\text{-fe.ct}_{\text{id}}^* \leftarrow 1\text{-FE.Sim}(\text{st}_{\text{id}}, 1^{|F_{\text{id}}^*|}, V_{\text{id}})$. Set $\text{CT}_{\text{id}, \text{GID}^*}^* := (\text{GID}^*, 1\text{-fe.ct}_{\text{id}}^*)$ and sends $\{\text{CT}_{\text{id}, \text{GID}^*}^*\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Secret Key Query. $\mathcal{A}$ sends circuit C . If $\mathcal{A}$ already queried for encryption with input $\text{GID}, x_1, \dots, x_n$, sample $(\tilde{C}, \{w_{i,b}\}_{i,b}) \leftarrow \text{Garb}(1^\lambda, C; R)$ where $R \leftarrow \mathcal{O}(\text{GID}^*)$. Then, set $V_{\text{id}} = (\tilde{C}, \bar{w}_{\text{id}})$ for each $\text{id} \in [n]$. Otherwise, set $V_{\text{id}} = \perp$. For each $\text{id} \in [n]$, sample $1\text{-fe.sk}_{C, \text{id}} \leftarrow 1\text{-FE.Sim}(\text{st}_{\text{id}}, \text{KGen}, C, V_{\text{id}})$. Send $\text{SK}_C := \{1\text{-fe.sk}_{C, \text{id}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Guess Phase. $\mathcal{A}$ sends guess b' . Output b' .

As we can see, the running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}, \lambda$. If $\mathcal{O}$ sends pseudorandom strings with some $K \leftarrow \{0, 1\}^\lambda$, $\mathcal{B}$ behaves like $\text{Hyb}_{1, n+1}^{\mathcal{A}}(1^\lambda)$ and if $\mathcal{O}$ sends uniformly random strings, then $\mathcal{B}$ behaves like $\text{Hyb}_2^{\mathcal{A}}(1^\lambda)$. Hence, $\mathcal{B}$ is a valid adversary that breaks the security of PRF with non-negligible probability. Thus, $\text{Hyb}_{1, n+1}^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_2^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable. $\square$

Claim J.10. Assuming the security of garbling scheme $(\text{Garb}, \text{Eval})$, $\text{Hyb}_2^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_3^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable.

Proof. Assume that there exists an adversary $\mathcal{A}$ that can distinguish between $\text{Hyb}_2^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_3^{\mathcal{A}}(1^\lambda)$ with non-negligible probability $\epsilon(\lambda)$, i.e.,

$$\left| \Pr \left[1 \leftarrow \text{Hyb}_2^{\mathcal{A}}(1^\lambda) \right] - \Pr \left[1 \leftarrow \text{Hyb}_3^{\mathcal{A}}(1^\lambda) \right] \right| > \epsilon(\lambda)$$

We will construct a reduction adversary $\mathcal{B}$ that can break the security of $(\text{Garb}, \text{Eval})$ with non-negligible probability. The description of $\mathcal{B}^{\mathcal{O}}$ is as follows.

Setup Phase. $\mathcal{A}$ sends $(n, s, \ell_1, \dots, \ell_n)$. For each $\text{id} \in [n]$, sample $\text{st}_{\text{id}} \leftarrow 1\text{-FE.Sim}(1^\lambda, 1^{s'})$.

Compute $\alpha_{\text{id}}, \mathbb{I}_{\text{id}}$ as described in Enc of Construction J.5.

Set $\text{CRS} := (\lambda, n, \ell_1, \dots, \ell_n, s)$ and send CRS to $\mathcal{A}$.

Adaptive Queries. $\mathcal{A}$ can now makes one of each of the following queries in an adaptive manner.

Encryption Query. $\mathcal{A}$ sends $(\text{GID}, x_1, \dots, x_n)$. Construct the circuit F_{id} with $R \leftarrow \{0, 1\}^\lambda$. Sample $1\text{-fe.ct}_{\text{id}} \leftarrow 1\text{-FE.Sim}(\text{st}_{\text{id}}, \text{Enc}, F_{\text{id}})$. Set $\text{CT}_{\text{id}, \text{GID}} := (\text{GID}, 1\text{-fe.ct}_{\text{id}})$ and sends $\{\text{CT}_{\text{id}, \text{GID}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends $(\text{GID}^*, x_1^*, \dots, x_n^*)$. If $\mathcal{A}$ already queried for secret key with circuit C , sample $(\tilde{C}, \{\bar{w}_{\text{id}}\}_{\text{id}}) \leftarrow \mathcal{O}(C, (x_1^*, \dots, x_n^*))$. Then, set $V_{\text{id}} = (\tilde{C}, \bar{w}_{\text{id}})$ for each $\text{id} \in [n]$. Otherwise, set $V_{\text{id}} = \perp$. Sample $1\text{-fe.ct}_{\text{id}}^* \leftarrow 1\text{-FE.Sim}(\text{st}_{\text{id}}, 1^{|F_{\text{id}}^*|}, V_{\text{id}})$. Set $\text{CT}_{\text{id}, \text{GID}^*}^* := (\text{GID}^*, 1\text{-fe.ct}_{\text{id}}^*)$ and sends $\{\text{CT}_{\text{id}, \text{GID}^*}^*\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Secret Key Query. $\mathcal{A}$ sends circuit C . If $\mathcal{A}$ already queried for encryption with input $\text{GID}, x_1, \dots, x_n$, sample $(\tilde{C}, \{\bar{w}_{\text{id}}\}_{\text{id}}) \leftarrow \mathcal{O}(C, (x_1^*, \dots, x_n^*))$. Then, set $V_{\text{id}} = (\tilde{C}, \bar{w}_{\text{id}})$ for each $\text{id} \in [n]$. Otherwise, set $V_{\text{id}} = \perp$. For each $\text{id} \in [n]$, sample $1\text{-fe.sk}_{C, \text{id}} \leftarrow 1\text{-FE.Sim}(\text{st}_{\text{id}}, \text{KGen}, C, V_{\text{id}})$. Send $\text{SK}_C := \{1\text{-fe.sk}_{C, \text{id}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Guess Phase. $\mathcal{A}$ sends guess b' . Output b' .

As we can see, the running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$, λ . If $\mathcal{O}$ is an honest challenger, $\mathcal{B}$ behaves like $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ and if $\mathcal{O}$ simulates garbling, then $\mathcal{B}$ behaves like $\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$. Hence, $\mathcal{B}$ is a valid adversary that breaks the security of (Garb, Eval) with non-negligible probability. Thus, $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable. $\square$

$\square$

J.4 One-Key Many-Challenge MCFE for P/Poly

In this section, we will move towards IND security rather than SIM security from previous section. This allows us to query multiple challenge inputs rather than a single challenge input via a couple of standard hybrid arguments. We will show that the MCFE construction from previous section already satisfies these IND secure versions defined below.

Definition J.11. An MCFE scheme (GSetup, Enc, KGen, Dec) is said to be a one-key, one-challenge MCFE scheme for circuit class $\mathcal{C}$ and $\mathcal{GTD}$ with IND security if it satisfies the correctness property from Definition J.4 and also the following IND-security property.

IND Security with Batched Encryption. For any PPT admissible adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\left| \Pr \left[\begin{array}{l} b \leftarrow \mathcal{A}^{\mathcal{O}(\cdot, \cdot)}(\text{CT}^*) : \begin{array}{l} b \leftarrow \{0, 1\}, n, \ell_1, \dots, \ell_n, s \leftarrow \mathcal{A}(1^\lambda), \\ (\text{GID}^*, (x_{0,1}^*, \dots, x_{0,n}^*), \\ (x_{1,1}^*, \dots, x_{1,n}^*)) \leftarrow \mathcal{A}^{\mathcal{O}(\cdot, \cdot)}(\text{CRS}), \\ \forall \text{id} \in [n], \text{CT}_{\text{id}, \text{GID}^*}^* \leftarrow \text{Enc}(\text{id}, \text{EK}_{\text{id}}, \text{GID}^*, x_{b, \text{id}}^*), \\ \text{Let } \text{CT}^* := \{\text{CT}_{\text{id}, \text{GID}^*}^*\}_{\text{id} \in [n]} \end{array} \end{array} \right] - \frac{1}{2} \right| \leq \text{negl}(\lambda)$$

where $(\text{CRS}, \text{MSK}, \{\text{EK}_{\text{id}}\}_{\text{id} \in [n]}) \leftarrow \text{GSetup}(1^\lambda, 1^n, 1^{\ell_1}, \dots, 1^{\ell_n}, 1^s)$. $\mathcal{O}$ responds to the following queries from $\mathcal{A}$.

- $(\text{Enc}, (\text{GID}, x_1, \dots, x_n))$. $\mathcal{A}$ queries for an encryption of inputs $x_1, \dots, x_n$ and GID. $\mathcal{O}$ responds with $\{\text{CT}_{\text{id}, \text{GID}}\}_{\text{id} \in [n]}$ where $\text{CT}_{\text{id}, \text{GID}} \leftarrow \text{Enc}(\text{id}, \text{EK}_{\text{id}}, \text{GID}, x_{\text{id}})$.
- (KGen, C) . $\mathcal{A}$ queries for secret key for circuit C of size at most s . $\mathcal{O}$ responds with $\text{SK}_C \leftarrow \text{KGen}(\text{MSK}, C)$.

A PPT machine $\mathcal{A}$ is said to be admissible if it makes at most one KGen query for C of size at most s , at most one challenge query for $(\text{GID}^*, (x_{0,1}^*, \dots, x_{0,n}^*), (x_{1,1}^*, \dots, x_{1,n}^*))$ such that $C(x_{0,1}^*, \dots, x_{0,n}^*) = C(x_{1,1}^*, \dots, x_{1,n}^*)$, and makes unbounded Enc queries with a unique GID each.

Lemma J.12. Any MCFE scheme for P/Poly circuits that satisfies Definition J.4 satisfies Definition J.11.

Proof. This follows from SIM-secure MCFE with batched encryption. Concretely, in order to prove IND-Security between $b = 0$ and $b = 1$, we can use an intermediate hybrid in which we move using oMCFE.Sim . This change is computationally indistinguishable by any PPT $\mathcal{A}$. Moreover, as $C(x_{0,1}^*, \dots, x_{0,n}^*) = C(x_{1,1}^*, \dots, x_{1,n}^*)$, any adversary has zero advantage and this lemma follows. $\square$

Now, we will lift an IND-secure one-key one-challenge MCFE scheme to a one-key many-challenge MCFE by a standard hybrid argument.

Definition J.13. An MCFE scheme $(\text{GSetup}, \text{Enc}, \text{KGen}, \text{Dec})$ is said to be a one-key, *many*-challenge MCFE scheme for circuit class $\mathcal{C}$ and $\mathcal{GID}$ with IND security if it satisfies the correctness property from Definition J.4 and also the following IND-security property.

Many-Challenge IND Security with Batched Encryption. For any PPT admissible adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\left| \Pr \left[b \leftarrow \mathcal{A}^{\mathcal{O}_b(\cdot, \cdot)}(\text{CRS}) : \begin{array}{l} b \leftarrow \{0, 1\}, n, \ell_1, \dots, \ell_n, s \leftarrow \mathcal{A}(1^\lambda), \\ (\text{CRS}, \text{MSK}, \{\text{EK}_{\text{id}}\}_{\text{id} \in [n]}) \leftarrow \\ \text{GSetup}(1^\lambda, 1^n, 1^{\ell_1}, \dots, 1^{\ell_n}, 1^s) \end{array} \right] - \frac{1}{2} \right| \leq \text{negl}(\lambda)$$

where $\mathcal{O}_b$ responds to the following queries from $\mathcal{A}$.

- $(\text{Enc}, (\text{GID}, (x_{0,1}, \dots, x_{0,n}), (x_{1,1}, \dots, x_{1,n})))$. $\mathcal{A}$ queries for an encryption of inputs for GID. $\mathcal{O}$ responds with $\{\text{CT}_{\text{id}, \text{GID}}\}_{\text{id} \in [n]}$ where $\text{CT}_{\text{id}, \text{GID}} \leftarrow \text{Enc}(\text{id}, \text{EK}_{\text{id}}, \text{GID}, x_{b, \text{id}})$.
- (KGen, C) . $\mathcal{A}$ queries for secret key for circuit C of size at most s . $\mathcal{O}$ responds with $\text{SK}_C \leftarrow \text{KGen}(\text{MSK}, C)$.

A PPT machine $\mathcal{A}$ is said to be admissible if it makes at most one KGen query for C of size at most s , and unbounded encryption queries with a unique GID each $(\text{GID}, (x_{0,1}, \dots, x_{0,n}), (x_{1,1}, \dots, x_{1,n}))$ such that $C(x_{0,1}, \dots, x_{0,n}) = C(x_{1,1}, \dots, x_{1,n})$ for each of them.

Lemma J.14. Any MCFE scheme for P/Poly circuits that satisfies Definition J.11 satisfies Definition J.13.

Proof. This follows by a standard hybrid argument. $\square$

J.5 One-CT Private-Key MAFE

Now, we are finally ready to construct a one-ciphertext private-key MAFE for P/Poly circuits that satisfies Definition J.1. The idea is to use function-hiding techniques similar to [BS18]. We provide full construction below.

Construction J.15 (opMAFE). We provide the construction of a one-ciphertext private-key MAFE scheme with batched key generation for P/Poly circuits and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ (Definition J.1) using a one-key private-key MCFE scheme for P/Poly circuits (Definition J.13) and $\mathcal{GID}$ and a secret-key encryption scheme SKE (Definition A.6) as follows.

$\text{GSetup}(1^\lambda, 1^n, 1^s, 1^{\ell_1}, \dots, 1^{\ell_n})$. Let $s' = \text{poly}(\lambda, n, s, \ell_1, \dots, \ell_n)$ be the maximum size of the circuit described in Figure 8 and $\ell'_{\text{id}} = \text{poly}(\lambda, \ell_{\text{id}})$ is the input size of id -th authority. Sample MCFE keys $(\text{mcfe.crs}, \{\text{mcfe.ek}_{\text{id}}\}_{\text{id} \in [n]}, \text{mcfe.msk}) \leftarrow \text{oMCFE.GSetup}(1^\lambda, 1^n, 1^{s'}, 1^{\ell'_1}, \dots, 1^{\ell'_n})$. Sample two SKE key $\text{ske.sk}, \text{ske.sk}' \leftarrow \text{SKE.Setup}(1^\lambda)$. Set $\text{CRS} := \text{mcfe.crs}$, $\text{MSK}_{\text{id}} := (\text{mcfe.ek}_{\text{id}}, \text{mcfe.msk}, \text{ske.sk}, \text{ske.sk}')$ for each $\text{id} \in [n]$.

Output $(\text{CRS}, \{\text{MSK}_{\text{id}}\}_{\text{id} \in [n]})$.

$\text{KGen}(\text{id}, \text{MSK}_{\text{id}}, \text{GID}, x)$. Parse MSK_{id} as $(\text{mcfe.ek}_{\text{id}}, \text{mcfe.msk}, \text{ske.sk}, \text{ske.sk}')$. Sample encryption for MCFE, $\text{mcfe.ct}_{\text{id}, \text{GID}} \leftarrow \text{oMCFE.Enc}(\text{id}, \text{mcfe.ek}_{\text{id}}, \text{GID}, (x, \text{ske.sk}, \perp, \perp))$. Output $\text{SK}_{\text{id}, \text{GID}, x} := (\text{id}, \text{GID}, \text{mcfe.ct}_{\text{id}, \text{GID}})$.

$\text{Enc}(\{\text{MSK}_{\text{id}}\}_{\text{id} \in [n]}, C)$. Parse MSK_{id} as $(\text{mcfe.ek}_{\text{id}}, \text{mcfe.msk}, \text{ske.sk}, \text{ske.sk}')$ for each $\text{id} \in [n]$. Sample ciphertexts $\text{ske.ct} \leftarrow \text{SKE.Enc}(\text{ske.sk}, C)$ and $\text{ske.ct}' \leftarrow \text{SKE.Enc}(\text{ske.sk}', C)$. Construct the circuit F as shown in Figure 8. Sample MCFE secret key $\text{mcfe.sk}_F \leftarrow \text{oMCFE.KGen}(\text{mcfe.msk}, F)$. Output $\text{CT} := \text{mcfe.sk}_F$.

$\text{Dec}(\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}, \text{CT})$. Parse $\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}$ as $(\text{id}, \text{GID}, \text{mcfe.ct}_{\text{id}, \text{GID}})$ for each $\text{id} \in [n]$ and CT as mcfe.sk_F . Output $y = \text{oMCFE.Dec}(\{\text{mcfe.ct}_{\text{id}, \text{GID}}\}_{\text{id} \in [n]}, \text{mcfe.sk}_F)$.

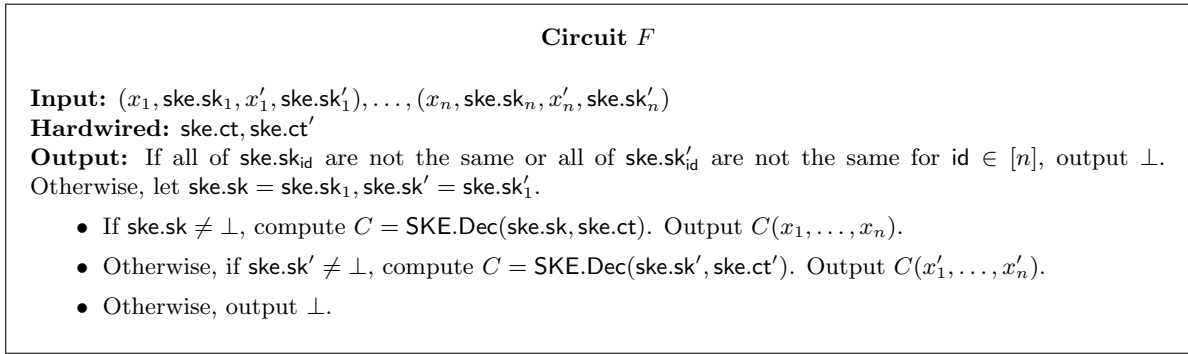


Figure 8: Description of circuit F

Correctness. Correctness of the scheme follows from correctness of SKE and oMCFE. By correctness of oMCFE, $y = F((x_1, \text{ske.sk}, \perp, \perp), \dots, (x_n, \text{ske.sk}, \perp, \perp))$. By the functionality of F and correctness of SKE, we have that $y = C(x_1, \dots, x_n)$.

Lemma J.16. If oMCFE is an MCFE scheme for P/Poly circuits and $\mathcal{GID}$ (Definition J.13), SKE is an IND-CPA secure SKE (Definition A.6), then Construction J.15 is a one-ciphertext private-key MAFE scheme for P/Poly circuits and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$.

Proof. Proof proceeds similar to [BS18]. Hence, we only provide an overview of the hybrids.

Hybrid 0. This is the experiment from Definition J.1 with $b = 0$.

Hybrid 1. In this hybrid, as we only encrypt $(x_{\text{id}}, \text{ske.sk}, \perp, \perp)$ from previous experiment, $\text{ske.sk}'$ is completely hidden from $\mathcal{A}$'s view. Hence, we can set $\text{ske.ct}'$ to be encryption of C_1 . The indistinguishability readily holds by IND-CPA security of SKE.

Hybrid 2. In this hybrid, we leverage the fact that $C_0(x_{1, \text{GID}}, \dots, x_{n, \text{GID}}) = C_1(x_{1, \text{GID}}, \dots, x_{n, \text{GID}})$ for each GID query and encrypt $(\perp, \perp, x_{\text{id}, \text{GID}}, \text{ske.sk}')$. By functionality of F (Figure 8), outputs are equal. Hence, indistinguishability can be argued using many-challenge IND security with batched encryption property of oMCFE.

Hybrid 3. In the hybrid, as we only encrypt $(\perp, \perp, x_{\text{id}}, \text{ske.sk}')$ from previous experiment, ske.sk is completely hidden from $\mathcal{A}$'s view. Hence, we can set ske.ct to be encryption of C_1 . The indistinguishability readily holds by IND-CPA security of SKE.

Hybrid 4. This is the experiment with $b = 1$. Indistinguishability can be argued using many-challenge IND security with batched encryption property of oMCFE.

□

K Proofs from Section 5.4

In this section, we provide full proof of Lemma 5.8.

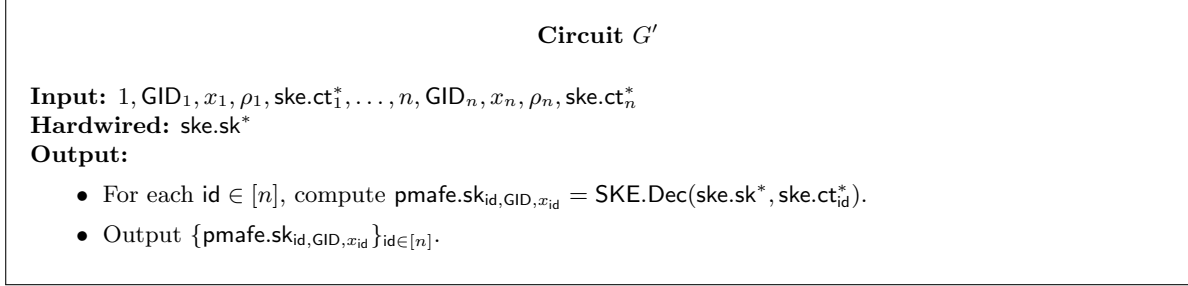


Figure 9: Description of circuit G'

Hyb₀^A(1^λ). This is almost the real experiment with Chal with $b = 0/1$. We sample K and opMAFE keys early. Nonetheless, distribution remains identical.

Setup Phase. $\mathcal{A}$ sends $(n, s, \ell_1, \dots, \ell_n)$ to Chal. Define $L = \max_{\text{id}} \{\ell_{\text{id}}\}$. Chal samples $\text{selmafe.crs} \leftarrow \text{sbMAFE.GSetup}(1^\lambda, 1^n, 1^{s'}, 1^L)$ where $s' = \text{poly}(\lambda, n, L, s)$ is the maximum size of circuit from Figure 3 and Figure 9. Set $\text{CRS} := \text{selmafe.crs}$. Then, Chal samples $(\text{selmafe.mpk}_{\text{id}}, \text{selmafe.msk}_{\text{id}}) \leftarrow \text{sbMAFE.ASetup}(\text{id}, 1^{\ell_{\text{id}}})$ for each $\text{id} \in [n]$. Chal sets $\text{MPK}_{\text{id}} := \text{selmafe.mpk}_{\text{id}}$ and $\text{MSK}_{\text{id}} := \text{selmafe.msk}_{\text{id}}$.

Now, Chal samples $K \xleftarrow{\$} \{0, 1\}^\lambda$ and $\text{pmafe.crs}, \{\text{pmafe.msk}_{\text{id}}\}_{\text{id} \in [n]} \leftarrow \text{opMAFE.GSetup}(1^\lambda, 1^n, 1^s, 1^L)$. Construct circuit G from Figure 3.

Chal sends $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]})$ to $\mathcal{A}$.

Pre-challenge Queries. $\mathcal{A}$ sends queries of the form $(\text{GID}, x_1, \dots, x_n)$.

Chal now samples $\rho_{\text{id}} \xleftarrow{\$} \{0, 1\}^\lambda$, $\text{ske.ct}_{\text{id}}^* \xleftarrow{\$} \{0, 1\}^{\ell_{\text{id}}^*}$ and samples $\text{selmafe.sk}_{\text{id}, \text{GID}, x_{\text{id}}} \leftarrow \text{sbMAFE.KGen}(\text{id}, \text{selmafe.msk}_{\text{id}}, \{\text{selmafe.mpk}_{\text{id}_x}\}_{\text{id}_x \in [n]}, \text{GID}, (\text{id}, \text{GID}, x_{\text{id}}, \rho_{\text{id}}, \text{ske.ct}_{\text{id}}^*))$ for each $\text{id} \in [n]$.

Set $\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}} := \text{selmafe.sk}_{\text{id}, \text{GID}, x_{\text{id}}}$ and send $\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends C_0, C_1 . Sample $\text{pmafe.ct} \leftarrow \text{opMAFE.Enc}(\{\text{pmafe.msk}_{\text{id}}\}_{\text{id} \in [n]}, C_b)$ and $\text{selmafe.ct} \leftarrow \text{sbMAFE.Enc}(\{\text{selmafe.mpk}_{\text{id}}\}_{\text{id} \in [n]}, G)$. Send $\text{CT} := (\text{selmafe.ct}, \text{pmafe.ct})$ to $\mathcal{A}$.

Post-challenge Queries. This is same as pre-challenge queries.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb₁^A(1^λ). In this hybrid, we set $\text{ske.ct}_{\text{id}}^*$ to be an SKE ciphertext of the string $\text{pmafe.sk}_{\text{id,GID},x_{\text{id}}} = \text{opMAFE.KGen}(\text{id}, \text{pmafe.msk}_{\text{id}}, \text{GID}, x_{\text{id}}; \text{PRF}(K, \rho_{\text{id,GID}}))$. The changes are highlighted in red.

Setup Phase. $\mathcal{A}$ sends $(n, s, \ell_1, \dots, \ell_n)$. Define $L = \max_{\text{id}} \{\ell_{\text{id}}\}$. Sample $\text{selmafe.crs} \leftarrow \text{sbMAFE.GSetup}(1^\lambda, 1^n, 1^{s'}, 1^L)$ where $s' = \text{poly}(\lambda, n, L, s)$ is the maximum size of circuit from Figure 3 and Figure 9. Set $\text{CRS} := \text{selmafe.crs}$. Then, sample $(\text{selmafe.mpk}_{\text{id}}, \text{selmafe.msk}_{\text{id}}) \leftarrow \text{sbMAFE.ASetup}(\text{id}, 1^{\ell'_{\text{id}}})$ for each $\text{id} \in [n]$. Set $\text{MPK}_{\text{id}} := \text{selmafe.mpk}_{\text{id}}$ and $\text{MSK}_{\text{id}} := \text{selmafe.msk}_{\text{id}}$.

Now, sample $K \xleftarrow{\$} \{0, 1\}^\lambda$ and $\text{pmafe.crs}, \{\text{pmafe.msk}_{\text{id}}\}_{\text{id} \in [n]} \leftarrow \text{opMAFE.GSetup}(1^\lambda, 1^n, 1^s, 1^L)$. Construct circuit G from Figure 3. In addition, sample $\text{ske.sk}^* \leftarrow \text{SKE.Setup}(1^\lambda)$ and construct G' from Figure 9.

Send $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]})$ to $\mathcal{A}$.

Pre-challenge Queries. $\mathcal{A}$ sends queries of the form $(\text{GID}, x_1, \dots, x_n)$.

Now sample $\rho_{\text{id}} \xleftarrow{\$} \{0, 1\}^\lambda$, $\text{ske.ct}_{\text{id}}^* \leftarrow \text{SKE.Enc}(\text{ske.sk}^*, \text{opMAFE.KGen}(\text{id}, \text{pmafe.msk}_{\text{id}}, \text{GID}, x_{\text{id}}; \text{PRF}(K, \rho_{\text{id}})))$. Sample $\text{selmafe.sk}_{\text{id,GID},x_{\text{id}}} \leftarrow \text{sbMAFE.KGen}(\text{id}, \text{selmafe.msk}_{\text{id}}, \{\text{selmafe.mpk}_{\text{id}_x}\}_{\text{id}_x \in [n]}, \text{GID}, (\text{id}, \text{GID}, x_{\text{id}}, \rho_{\text{id}}, \text{ske.ct}_{\text{id}}^*))$ for each $\text{id} \in [n]$.

Set $\text{SK}_{\text{id,GID},x_{\text{id}}} := \text{selmafe.sk}_{\text{id,GID},x_{\text{id}}}$ and send $\{\text{SK}_{\text{id,GID},x_{\text{id}}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends C_0, C_1 . Sample $\text{pmafe.ct} \leftarrow \text{opMAFE.Enc}(\{\text{pmafe.msk}_{\text{id}}\}_{\text{id} \in [n]}, C_b)$ and $\text{selmafe.ct} \leftarrow \text{sbMAFE.Enc}(\{\text{selmafe.mpk}_{\text{id}}\}_{\text{id} \in [n]}, G)$. Send $\text{CT} := (\text{selmafe.ct}, \text{pmafe.ct})$ to $\mathcal{A}$.

Post-challenge Queries. This is same as pre-challenge queries.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb₂^A(1^λ). In this hybrid, we encrypt G' using sbMAFE. The changes are highlighted in red.

Setup Phase. $\mathcal{A}$ sends $(n, s, \ell_1, \dots, \ell_n)$. Define $L = \max_{\text{id}} \{\ell_{\text{id}}\}$. Sample $\text{selmafe.crs} \leftarrow \text{sbMAFE.GSetup}(1^\lambda, 1^n, 1^{s'}, 1^L)$ where $s' = \text{poly}(\lambda, n, L, s)$ is the maximum size of circuit from Figure 3 and Figure 9. Set $\text{CRS} := \text{selmafe.crs}$. Then, sample $(\text{selmafe.mpk}_{\text{id}}, \text{selmafe.msk}_{\text{id}}) \leftarrow \text{sbMAFE.ASetup}(\text{id}, 1^{\ell'_{\text{id}}})$ for each $\text{id} \in [n]$. Set $\text{MPK}_{\text{id}} := \text{selmafe.mpk}_{\text{id}}$ and $\text{MSK}_{\text{id}} := \text{selmafe.msk}_{\text{id}}$.

Now, sample $K \xleftarrow{\$} \{0, 1\}^\lambda$ and $\text{pmafe.crs}, \{\text{pmafe.msk}_{\text{id}}\}_{\text{id} \in [n]} \leftarrow \text{opMAFE.GSetup}(1^\lambda, 1^n, 1^s, 1^L)$. Construct circuit G from Figure 3. In addition, sample $\text{ske.sk}^* \leftarrow \text{SKE.Setup}(1^\lambda)$ and construct G' from Figure 9.

Send $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]})$ to $\mathcal{A}$.

Pre-challenge Queries. $\mathcal{A}$ sends queries of the form $(\text{GID}, x_1, \dots, x_n)$.

Now sample $\rho_{\text{id}} \xleftarrow{\$} \{0, 1\}^\lambda$, $\text{ske.ct}_{\text{id}}^* \leftarrow \text{SKE.Enc}(\text{ske.sk}^*, \text{opMAFE.KGen}(\text{id}, \text{pmafe.msk}_{\text{id}}, \text{GID}, x_{\text{id}}; \text{PRF}(K, \rho_{\text{id}})))$. Sample $\text{selmafe.sk}_{\text{id,GID},x_{\text{id}}} \leftarrow \text{sbMAFE.KGen}(\text{id}, \text{selmafe.msk}_{\text{id}}, \{\text{selmafe.mpk}_{\text{id}_x}\}_{\text{id}_x \in [n]}, \text{GID}, (\text{id}, \text{GID}, x_{\text{id}}, \rho_{\text{id}}, \text{ske.ct}_{\text{id}}^*))$ for each $\text{id} \in [n]$.

Set $\text{SK}_{\text{id,GID},x_{\text{id}}} := \text{selmafe.sk}_{\text{id,GID},x_{\text{id}}}$ and send $\{\text{SK}_{\text{id,GID},x_{\text{id}}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends C_0, C_1 . Sample $\text{pmafe.ct} \leftarrow \text{opMAFE.Enc}(\{\text{pmafe.msk}_{\text{id}}\}_{\text{id} \in [n]}, C_b)$ and $\text{selmafe.ct} \leftarrow \text{sbMAFE.Enc}(\{\text{selmafe.mpk}_{\text{id}}\}_{\text{id} \in [n]}, G')$. Send $\text{CT} := (\text{selmafe.ct}, \text{pmafe.ct})$ to $\mathcal{A}$.

Post-challenge Queries. This is same as pre-challenge queries.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb₃^A(1^λ). In this hybrid, we sample uniformly random strings for garbling randomness instead of using $\text{PRF}(K, \rho_{\text{id}, \text{GID}})$. The changes are highlighted in **red**.

Setup Phase. $\mathcal{A}$ sends $(n, s, \ell_1, \dots, \ell_n)$. Define $L = \max_{\text{id}} \{\ell_{\text{id}}\}$. Sample $\text{selmafe.crs} \leftarrow \text{sbMAFE.GSetup}(1^\lambda, 1^n, 1^{s'}, 1^L)$ where $s' = \text{poly}(\lambda, n, L, s)$ is the maximum size of circuit from Figure 3 and Figure 9. Set $\text{CRS} := \text{selmafe.crs}$. Then, sample $(\text{selmafe.mpk}_{\text{id}}, \text{selmafe.msk}_{\text{id}}) \leftarrow \text{sbMAFE.ASetup}(\text{id}, 1^{\ell'_{\text{id}}})$ for each $\text{id} \in [n]$. Set $\text{MPK}_{\text{id}} := \text{selmafe.mpk}_{\text{id}}$ and $\text{MSK}_{\text{id}} := \text{selmafe.msk}_{\text{id}}$.

Now, sample $K \xleftarrow{\$} \{0, 1\}^\lambda$ and $\text{pmafe.crs}, \{\text{pmafe.msk}_{\text{id}}\}_{\text{id} \in [n]} \leftarrow \text{opMAFE.GSetup}(1^\lambda, 1^n, 1^s, 1^L)$. Construct circuit G from Figure 3. In addition, sample $\text{ske.sk}^* \leftarrow \text{SKE.Setup}(1^\lambda)$ and construct G' from Figure 9.

Send $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]})$ to $\mathcal{A}$.

Pre-challenge Queries. $\mathcal{A}$ sends queries of the form $(\text{GID}, x_1, \dots, x_n)$.

Now sample $R_{\text{id}, \text{GID}} \xleftarrow{\$} \{0, 1\}^\lambda$, $\text{ske.ct}_{\text{id}}^* \leftarrow \text{SKE.Enc}(\text{ske.sk}^*, \text{opMAFE.KGen}(\text{id}, \text{pmafe.msk}_{\text{id}}, \text{GID}, x_{\text{id}}; R_{\text{id}, \text{GID}}))$. Sample $\text{selmafe.sk}_{\text{id}, \text{GID}, x_{\text{id}}} \leftarrow \text{sbMAFE.KGen}(\text{id}, \text{selmafe.msk}_{\text{id}}, \{\text{selmafe.mpk}_{\text{id}_x}\}_{\text{id}_x \in [n]}, \text{GID}, (\text{id}, \text{GID}, x_{\text{id}}, \rho_{\text{id}}, \text{ske.ct}_{\text{id}}^*))$ for each $\text{id} \in [n]$.

Set $\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}} := \text{selmafe.sk}_{\text{id}, \text{GID}, x_{\text{id}}}$ and send $\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends C_0, C_1 . Sample $\text{pmafe.ct} \leftarrow \text{opMAFE.Enc}(\{\text{pmafe.msk}_{\text{id}}\}_{\text{id} \in [n]}, C_b)$ and $\text{selmafe.ct} \leftarrow \text{sbMAFE.Enc}(\{\text{selmafe.mpk}_{\text{id}}\}_{\text{id} \in [n]}, G')$. Send $\text{CT} := (\text{selmafe.ct}, \text{pmafe.ct})$ to $\mathcal{A}$.

Post-challenge Queries. This is same as pre-challenge queries.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb₄^A(1^λ). In this hybrid, we encrypt C_{1-b} using opMAFE . The changes are highlighted in **red**.

Setup Phase. $\mathcal{A}$ sends $(n, s, \ell_1, \dots, \ell_n)$. Define $L = \max_{\text{id}} \{\ell_{\text{id}}\}$. Sample $\text{selmafe.crs} \leftarrow \text{sbMAFE.GSetup}(1^\lambda, 1^n, 1^{s'}, 1^L)$ where $s' = \text{poly}(\lambda, n, L, s)$ is the maximum size of circuit from Figure 3 and Figure 9. Set $\text{CRS} := \text{selmafe.crs}$. Then, sample $(\text{selmafe.mpk}_{\text{id}}, \text{selmafe.msk}_{\text{id}}) \leftarrow \text{sbMAFE.ASetup}(\text{id}, 1^{\ell'_{\text{id}}})$ for each $\text{id} \in [n]$. Set $\text{MPK}_{\text{id}} := \text{selmafe.mpk}_{\text{id}}$ and $\text{MSK}_{\text{id}} := \text{selmafe.msk}_{\text{id}}$.

Now, sample $K \xleftarrow{\$} \{0, 1\}^\lambda$ and $\text{pmafe.crs}, \{\text{pmafe.msk}_{\text{id}}\}_{\text{id} \in [n]} \leftarrow \text{opMAFE.GSetup}(1^\lambda, 1^n, 1^s, 1^L)$. Construct circuit G from Figure 3. In addition, sample $\text{ske.sk}^* \leftarrow \text{SKE.Setup}(1^\lambda)$ and construct G' from Figure 9.

Send $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]})$ to $\mathcal{A}$.

Pre-challenge Queries. $\mathcal{A}$ sends queries of the form $(\text{GID}, x_1, \dots, x_n)$.

Now sample $R_{\text{id}, \text{GID}} \xleftarrow{\$} \{0, 1\}^\lambda$, $\text{ske.ct}_{\text{id}}^* \leftarrow \text{SKE.Enc}(\text{ske.sk}^*, \text{opMAFE.KGen}(\text{id}, \text{pmafe.msk}_{\text{id}}, \text{GID}, x_{\text{id}}; R_{\text{id}, \text{GID}}))$. Sample $\text{selmafe.sk}_{\text{id}, \text{GID}, x_{\text{id}}} \leftarrow \text{sbMAFE.KGen}(\text{id}, \text{selmafe.msk}_{\text{id}}, \{\text{selmafe.mpk}_{\text{id}_x}\}_{\text{id}_x \in [n]}, \text{GID}, (\text{id}, \text{GID}, x_{\text{id}}, \rho_{\text{id}}, \text{ske.ct}_{\text{id}}^*))$ for each $\text{id} \in [n]$.

Set $\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}} := \text{selmafe.sk}_{\text{id}, \text{GID}, x_{\text{id}}}$ and send $\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends C_0, C_1 . Sample $\text{pmafe.ct} \leftarrow \text{opMAFE.Enc}(\{\text{pmafe.msk}_{\text{id}}\}_{\text{id} \in [n]}, C_{1-b})$ and $\text{selmafe.ct} \leftarrow \text{sbMAFE.Enc}(\{\text{selmafe.mpk}_{\text{id}}\}_{\text{id} \in [n]}, G')$. Send $\text{CT} := (\text{selmafe.ct}, \text{pmafe.ct})$ to $\mathcal{A}$.

Post-challenge Queries. This is same as pre-challenge queries.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

$\text{Hyb}_5^{\mathcal{A}}(1^\lambda)$. This is real experiment with Chal with $1 - b$.

Claim K.1. Assuming the pseudorandom ciphertext security of SKE, $\text{Hyb}_0^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_1^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable.

Proof. Assume that there exists an adversary $\mathcal{A}$ that can distinguish between $\text{Hyb}_0^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_1^{\mathcal{A}}(1^\lambda)$ with non-negligible probability $\epsilon(\lambda)$, i.e.,

$$\left| \Pr \left[1 \leftarrow \text{Hyb}_0^{\mathcal{A}}(1^\lambda) \right] - \Pr \left[1 \leftarrow \text{Hyb}_1^{\mathcal{A}}(1^\lambda) \right] \right| > \epsilon(\lambda)$$

We will construct a reduction adversary $\mathcal{B}$ that can break the security of SKE with non-negligible probability. The description of $\mathcal{B}^{\mathcal{O}}$ is as follows.

Setup Phase. $\mathcal{A}$ sends $(n, s, \ell_1, \dots, \ell_n)$ to Chal . Define $L = \max_{\text{id}} \{\ell_{\text{id}}\}$. Sample $\text{selmafe.crs} \leftarrow \text{sbMAFE.GSetup}(1^\lambda, 1^n, 1^{s'}, 1^L)$ where $s' = \text{poly}(\lambda, n, L, s)$ is the maximum size of circuit from Figure 3 and Figure 9. Set $\text{CRS} := \text{selmafe.crs}$. Then, sample $(\text{selmafe.mpk}_{\text{id}}, \text{selmafe.msk}_{\text{id}}) \leftarrow \text{sbMAFE.ASetup}(\text{id}, 1^{\ell_{\text{id}}})$ for each $\text{id} \in [n]$. Set $\text{MPK}_{\text{id}} := \text{selmafe.mpk}_{\text{id}}$ and $\text{MSK}_{\text{id}} := \text{selmafe.msk}_{\text{id}}$.

Now, sample $K \xleftarrow{\$} \{0, 1\}^\lambda$ and $\text{pmafe.crs}, \{\text{pmafe.msk}_{\text{id}}\}_{\text{id} \in [n]} \leftarrow \text{opMAFE.GSetup}(1^\lambda, 1^n, 1^s, 1^L)$. Construct circuit G from Figure 3.

Send $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]})$ to $\mathcal{A}$.

Pre-challenge Queries. $\mathcal{A}$ sends queries of the form $(\text{GID}, x_1, \dots, x_n)$.

Now sample $\rho_{\text{id}} \xleftarrow{\$} \{0, 1\}^\lambda$, send $\text{opMAFE.KGen}(\text{id}, \text{pmafe.msk}_{\text{id}}, \text{GID}, x_{\text{id}}; \text{PRF}(K, \rho_{\text{id}}))$ to $\mathcal{O}$ to receive $\text{ske.ct}_{\text{id}}^*$. Sample $\text{selmafe.sk}_{\text{id}, \text{GID}, x_{\text{id}}} \leftarrow \text{sbMAFE.KGen}(\text{id}, \text{selmafe.msk}_{\text{id}}, \{\text{selmafe.mpk}_{\text{id}_x}\}_{\text{id}_x \in [n]}, \text{GID}, (\text{id}, \text{GID}, x_{\text{id}}, \rho_{\text{id}}, \text{ske.ct}_{\text{id}}^*))$ for each $\text{id} \in [n]$.

Set $\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}} := \text{selmafe.sk}_{\text{id}, \text{GID}, x_{\text{id}}}$ and send $\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends C_0, C_1 . Sample $\text{pmafe.ct} \leftarrow \text{opMAFE.Enc}(\{\text{pmafe.msk}_{\text{id}}\}_{\text{id} \in [n]}, C_b)$ and $\text{selmafe.ct} \leftarrow \text{sbMAFE.Enc}(\{\text{selmafe.mpk}_{\text{id}}\}_{\text{id} \in [n]}, G)$. Send $\text{CT} := (\text{selmafe.ct}, \text{pmafe.ct})$ to $\mathcal{A}$.

Post-challenge Queries. This is same as pre-challenge queries.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

As we can see, the running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$, λ . If $\mathcal{O}$ generates uniformly random strings, $\mathcal{B}$ behaves like $\text{Hyb}_0^{\mathcal{A}}(1^\lambda)$ and if $\mathcal{O}$ generates encryptions of message, $\mathcal{B}$ behaves like $\text{Hyb}_1^{\mathcal{A}}(1^\lambda)$. Hence, $\mathcal{B}$ is a valid adversary that break the security of SKE with non-negligible probability. Thus, $\text{Hyb}_0^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_1^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable. $\square$

Claim K.2. Assuming the security of **sbMAFE**, $\mathbf{Hyb}_1^A(1^\lambda)$ and $\mathbf{Hyb}_2^A(1^\lambda)$ are computationally indistinguishable.

Proof. Assume that there exists an adversary $\mathcal{A}$ that can distinguish between $\mathbf{Hyb}_1^A(1^\lambda)$ and $\mathbf{Hyb}_2^A(1^\lambda)$ with non-negligible probability $\epsilon(\lambda)$, i.e.,

$$\left| \Pr \left[1 \leftarrow \mathbf{Hyb}_1^A(1^\lambda) \right] - \Pr \left[1 \leftarrow \mathbf{Hyb}_2^A(1^\lambda) \right] \right| > \epsilon(\lambda)$$

We will construct a reduction adversary $\mathcal{B}$ that can break the security of **sbMAFE** with non-negligible probability. The description of $\mathcal{B}^\mathcal{O}$ is as follows.

Setup Phase. $\mathcal{A}$ sends $(n, s, \ell_1, \dots, \ell_n)$. Define $L = \max_{\text{id}} \{\ell_{\text{id}}\}$. Now, sample $K \xleftarrow{\$} \{0, 1\}^\lambda$ and $\text{pmafe.crs}, \{\text{pmafe.msk}_{\text{id}}\}_{\text{id} \in [n]} \leftarrow \text{opMAFE.GSetup}(1^\lambda, 1^n, 1^s, 1^L)$. Construct circuit G from Figure 3. In addition, sample $\text{ske.sk}^* \leftarrow \text{SKE.Setup}(1^\lambda)$ and construct G' from Figure 9.

Send $(n, s', \ell'_1, \dots, \ell'_n, G, G')$ to $\mathcal{O}$ to receive $\text{selmafe.crs}, \{\text{selmafe.mpk}_{\text{id}}\}_{\text{id} \in [n]}, \text{selmafe.ct}$. Set $\text{CRS} := \text{selmafe.crs}$, $\text{MPK}_{\text{id}} := \text{selmafe.mpk}_{\text{id}}$ for each $\text{id} \in [n]$.

Send $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]})$ to $\mathcal{A}$.

Pre-challenge Queries. $\mathcal{A}$ sends queries of the form $(\text{GID}, x_1, \dots, x_n)$.

Now sample $\rho_{\text{id}} \xleftarrow{\$} \{0, 1\}^\lambda$, $\text{ske.ct}_{\text{id}}^* \leftarrow \text{SKE.Enc}(\text{ske.sk}^*, \text{opMAFE.KGen}(\text{id}, \text{pmafe.msk}_{\text{id}}, \text{GID}, x_{\text{id}}; \text{PRF}(K, \rho_{\text{id}})))$ for each $\text{id} \in [n]$. Send $((\text{id}, \text{GID}, x_{\text{id}}, \rho_{\text{id}}, \text{ske.ct}_{\text{id}}^*))_{\text{id} \in [n]}$ to $\mathcal{O}$ to receive $\{\text{selmafe.sk}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}$.

Set $\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}} := \text{selmafe.sk}_{\text{id}, \text{GID}, x_{\text{id}}}$ and send $\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends C_0, C_1 . Sample $\text{pmafe.ct} \leftarrow \text{opMAFE.Enc}(\{\text{pmafe.msk}_{\text{id}}\}_{\text{id} \in [n]}, C_b)$. Send $\text{CT} := (\text{selmafe.ct}, \text{pmafe.ct})$ to $\mathcal{A}$.

Post-challenge Queries. This is same as pre-challenge queries.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

As we can see, the running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$, λ . If $\mathcal{O}$ encrypts G , $\mathcal{B}$ behaves like $\mathbf{Hyb}_1^A(1^\lambda)$ and if $\mathcal{O}$ encrypts G' , $\mathcal{B}$ behaves like $\mathbf{Hyb}_2^A(1^\lambda)$. Hence, $\mathcal{B}$ is a valid adversary that break the security of **sbMAFE** with non-negligible probability. Thus, $\mathbf{Hyb}_1^A(1^\lambda)$ and $\mathbf{Hyb}_2^A(1^\lambda)$ are computationally indistinguishable. $\square$

Claim K.3. Assuming the pseudorandomness property of PRF, $\mathbf{Hyb}_2^A(1^\lambda)$ and $\mathbf{Hyb}_3^A(1^\lambda)$ are computationally indistinguishable.

Proof. Assume that there exists an adversary $\mathcal{A}$ that can distinguish between $\mathbf{Hyb}_2^A(1^\lambda)$ and $\mathbf{Hyb}_3^A(1^\lambda)$ with non-negligible probability $\epsilon(\lambda)$, i.e.,

$$\left| \Pr \left[1 \leftarrow \mathbf{Hyb}_2^A(1^\lambda) \right] - \Pr \left[1 \leftarrow \mathbf{Hyb}_3^A(1^\lambda) \right] \right| > \epsilon(\lambda)$$

We will construct a reduction adversary $\mathcal{B}$ that can break the security of PRF with non-negligible probability. The description of $\mathcal{B}^\mathcal{O}$ is as follows.

Setup Phase. $\mathcal{A}$ sends $(n, s, \ell_1, \dots, \ell_n)$. Define $L = \max_{id} \{\ell_{id}\}$. Sample $\text{selmafe.crs} \leftarrow \text{sbMAFE.GSetup}(1^\lambda, 1^n, 1^{s'}, 1^L)$ where $s' = \text{poly}(\lambda, n, L, s)$ is the maximum size of circuit from Figure 3 and Figure 9. Set $\text{CRS} := \text{selmafe.crs}$. Then, sample $(\text{selmafe.mpk}_{id}, \text{selmafe.msk}_{id}) \leftarrow \text{sbMAFE.ASetup}(id, 1^{\ell_{id}})$ for each $id \in [n]$. Set $\text{MPK}_{id} := \text{selmafe.mpk}_{id}$ and $\text{MSK}_{id} := \text{selmafe.msk}_{id}$. Now, sample $\text{pmafe.crs}, \{\text{pmafe.msk}_{id}\}_{id \in [n]} \leftarrow \text{opMAFE.GSetup}(1^\lambda, 1^n, 1^s, 1^L)$. In addition, sample $\text{ske.sk}^* \leftarrow \text{SKE.Setup}(1^\lambda)$ and construct G' from Figure 9. Send $(\text{CRS}, \{\text{MPK}_{id}\}_{id \in [n]})$ to $\mathcal{A}$.

Pre-challenge Queries. $\mathcal{A}$ sends queries of the form $(\text{GID}, x_1, \dots, x_n)$.

Now sample $\rho_{id} \xleftarrow{\$} \{0, 1\}^\lambda$ and send it to $\mathcal{O}$ to receive $R_{id, \text{GID}}, \text{ske.ct}_{id}^* \leftarrow \text{SKE.Enc}(\text{ske.sk}^*, \text{opMAFE.KGen}(id, \text{pmafe.msk}_{id}, \text{GID}, x_{id}; R_{id, \text{GID}}))$. Sample $\text{selmafe.sk}_{id, \text{GID}, x_{id}} \leftarrow \text{sbMAFE.KGen}(id, \text{selmafe.msk}_{id}, \{\text{selmafe.mpk}_{id_x}\}_{id_x \in [n]}, \text{GID}, (id, \text{GID}, x_{id}, \rho_{id}, \text{ske.ct}_{id}^*))$ for each $id \in [n]$.

Set $\text{SK}_{id, \text{GID}, x_{id}} := \text{selmafe.sk}_{id, \text{GID}, x_{id}}$ and send $\{\text{SK}_{id, \text{GID}, x_{id}}\}_{id \in [n]}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends C_0, C_1 . Sample $\text{pmafe.ct} \leftarrow \text{opMAFE.Enc}(\{\text{pmafe.msk}_{id}\}_{id \in [n]}, C_b)$ and $\text{selmafe.ct} \leftarrow \text{sbMAFE.Enc}(\{\text{selmafe.mpk}_{id}\}_{id \in [n]}, G')$. Send $\text{CT} := (\text{selmafe.ct}, \text{pmafe.ct})$ to $\mathcal{A}$.

Post-challenge Queries. This is same as pre-challenge queries.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

As we can see, the running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$, λ . If $\mathcal{O}$ is $\text{PRF}(K, \cdot)$ for some $K \xleftarrow{\$} \{0, 1\}^\lambda$, $\mathcal{B}$ behaves like $\text{Hyb}_2^{\mathcal{A}}(1^\lambda)$ and if $\mathcal{O}$ generates uniformly random strings $\mathcal{B}$ behaves like $\text{Hyb}_3^{\mathcal{A}}(1^\lambda)$. Hence, $\mathcal{B}$ is a valid adversary that break the security of PRF with non-negligible probability. Thus, $\text{Hyb}_2^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_3^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable. $\square$

Claim K.4. Assuming the security of opMAFE , $\text{Hyb}_3^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_4^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable.

Proof. Assume that there exists an adversary $\mathcal{A}$ that can distinguish between $\text{Hyb}_3^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_4^{\mathcal{A}}(1^\lambda)$ with non-negligible probability $\epsilon(\lambda)$, i.e.,

$$\left| \Pr \left[1 \leftarrow \text{Hyb}_3^{\mathcal{A}}(1^\lambda) \right] - \Pr \left[1 \leftarrow \text{Hyb}_4^{\mathcal{A}}(1^\lambda) \right] \right| > \epsilon(\lambda)$$

We will construct a reduction adversary $\mathcal{B}$ that can break the security of opMAFE with non-negligible probability. The description of $\mathcal{B}^{\mathcal{O}}$ is as follows.

Setup Phase. $\mathcal{A}$ sends $(n, s, \ell_1, \dots, \ell_n)$. Define $L = \max_{id} \{\ell_{id}\}$. Sample $\text{selmafe.crs} \leftarrow \text{sbMAFE.GSetup}(1^\lambda, 1^n, 1^{s'}, 1^L)$ where $s' = \text{poly}(\lambda, n, L, s)$ is the maximum size of circuit from Figure 3 and Figure 9. Set $\text{CRS} := \text{selmafe.crs}$. Then, sample $(\text{selmafe.mpk}_{id}, \text{selmafe.msk}_{id}) \leftarrow \text{sbMAFE.ASetup}(id, 1^{\ell_{id}})$ for each $id \in [n]$. Set $\text{MPK}_{id} := \text{selmafe.mpk}_{id}$ and $\text{MSK}_{id} := \text{selmafe.msk}_{id}$. Now, send $n, s, \ell_1, \dots, \ell_n$ to $\mathcal{O}$ to receive pmafe.crs . In addition, sample $\text{ske.sk}^* \leftarrow \text{SKE.Setup}(1^\lambda)$ and construct G' from Figure 9. Send $(\text{CRS}, \{\text{MPK}_{id}\}_{id \in [n]})$ to $\mathcal{A}$.

Pre-challenge Queries. $\mathcal{A}$ sends queries of the form $(\text{GID}, x_1, \dots, x_n)$.

Now send $(\text{GID}, x_1, \dots, x_n)$ to $\mathcal{O}$ to receive $\{\text{pmafe.sk}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}$ sample $\text{ske.ct}_{\text{id}}^* \leftarrow \text{SKE.Enc}(\text{ske.sk}^*, \text{pmafe.sk}_{\text{id}, \text{GID}, x_{\text{id}}})$. Sample $\text{selmafe.sk}_{\text{id}, \text{GID}, x_{\text{id}}} \leftarrow \text{sbMAFE.KGen}(\text{id}, \text{selmafe.msk}_{\text{id}}, \{\text{selmafe.mpk}_{\text{id}_x}\}_{\text{id}_x \in [n]}, \text{GID}, (\text{id}, \text{GID}, x_{\text{id}}, \rho_{\text{id}}, \text{ske.ct}_{\text{id}}^*))$ for each $\text{id} \in [n]$.

Set $\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}} := \text{selmafe.sk}_{\text{id}, \text{GID}, x_{\text{id}}}$ and send $\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends C_0, C_1 . Send (C_b, C_{1-b}) to $\mathcal{O}$ to receive pmafe.ct and sample $\text{selmafe.ct} \leftarrow \text{sbMAFE.Enc}(\{\text{selmafe.mpk}_{\text{id}}\}_{\text{id} \in [n]}, G')$. Send $\text{CT} := (\text{selmafe.ct}, \text{pmafe.ct})$ to $\mathcal{A}$.

Post-challenge Queries. This is same as pre-challenge queries.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

As we can see, the running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$, λ . If $\mathcal{O}$ encrypts C_b $\mathcal{B}$ behaves like $\text{Hyb}_3^{\mathcal{A}}(1^\lambda)$ and if $\mathcal{O}$ encrypts C_{1-b} , $\mathcal{B}$ behaves like $\text{Hyb}_4^{\mathcal{A}}(1^\lambda)$. Hence, $\mathcal{B}$ is a valid adversary that break the security of opMAFE with non-negligible probability. Thus, $\text{Hyb}_3^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_4^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable. $\square$

Claim K.5. $\text{Hyb}_4^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_5^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable.

Proof. The proof of this claim is similar to proofs of Claim K.1, Claim K.2, Claim K.3, and Claim K.4. $\square$

L Vector Non-Committing Encryption from IBE

In this section, we construct a VNCE scheme (Definition L.1) from any IBE scheme (Definition A.4).

L.1 Definition: Vector Non-Committing Encryption

In this section, we introduce the notion of vector non-committing encryption which can be used to commit a vector of messages $(\mu_1, \dots, \mu_n)$ where each μ_{id} for $\text{id} \in [n]$ is from the id -th party. The idea is that each party encrypts its own input with respect to a specific GID and all other party's public information. These messages will only be revealed after collecting encryptions from all parties.

We need this scheme to be reusable for various messages but for a specific GID . That is, $(\mu_1, \dots, \mu_n)$ can only be revealed if id -th party encrypts $(\text{GID}, \mu_{\text{id}})$ for $\text{id} \in [n]$. We describe the scheme in the notation of MAFE for ease of exposition. In a way, this primitive can be thought of reusable MPC for identity functions with delayed output generation.

Syntax. A vector non-committing encryption scheme (VNCE) for global identifier domain $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ consists of the following polynomial-time algorithms.

$\text{GSetup}(1^\lambda, 1^n, 1^L) \rightarrow \text{CRS}$. The possibly randomized global setup algorithm takes as input the security parameter λ , number of authorities n , each party's input size L , and outputs CRS . The following algorithms take CRS implicitly.

$\text{ASetup}(\text{id}) \rightarrow (\text{MPK}, \text{MSK})$. The probabilistic authority setup algorithm outputs master public and secret key pair (MPK, MSK) .

$\text{Enc}(\text{id}, \text{MSK}_{\text{id}}, \{\text{MPK}_{\text{id}_x}\}_{\text{id}_x \in [n]}, \text{GID}, \mu) \rightarrow \text{ct}_{\text{id}, \text{GID}}$. The probabilistic encryption algorithm takes as input the id-th master secret key MSK_{id} , the global identifier for the user $\text{GID} \in \mathcal{GID}_\lambda$, an input μ such that $|\mu| = L$, and outputs encryption $\text{ct}_{\text{id}, \text{GID}}$.

$\text{Dec}(\{\text{ct}_{\text{id}, \text{GID}}\}_{\text{id} \in [n]}) \rightarrow y$. The deterministic decryption algorithm takes as input encryptions from all authorities, $\{\text{ct}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}$, ciphertext CT , and outputs the value $y \in \{0, 1\}^{nL} \cup \{\perp\}$.

Definition L.1 (VNCE). An VNCE scheme $(\text{GSetup}, \text{ASetup}, \text{Enc}, \text{Dec})$ is said to be a VNCE scheme for $\mathcal{GID}$ if it satisfies the following properties.

Correctness. For any $\lambda \in \mathbb{N}$, $n = n(\lambda)$, $L = L(\lambda)$, $\text{GID} \in \mathcal{GID}_\lambda$, $\forall \text{id} \in [n]$, $\ell_{\text{id}} = \ell_{\text{id}}(\lambda)$, $\mu_{\text{id}} \in \{0, 1\}^L$,

$$\Pr \left[\begin{array}{l} (\mu_1, \dots, \mu_n) = \\ \text{Dec}(\{\text{ct}_{\text{id}, \text{GID}}\}_{\text{id}}) \end{array} : \begin{array}{l} \text{CRS} \leftarrow \text{GSetup}(1^\lambda, 1^n, 1^L), \forall \text{id} \in [n], \\ (\text{MPK}_{\text{id}}, \text{MSK}_{\text{id}}) \leftarrow \text{ASetup}(\text{id}), \\ \text{ct}_{\text{id}, \text{GID}} \leftarrow \text{Enc}(\text{id}, \text{MSK}_{\text{id}}, \{\text{MPK}_{\text{id}_x}\}_{\text{id}_x \in [n]}, \text{GID}, \mu_{\text{id}}) \end{array} \right] = 1$$

Security. There exists a stateful PPT simulator Sim such that for any PPT adversary $\mathcal{A}$, $\forall \lambda \in \mathbb{N}$,

$$\left\{ \begin{array}{l} \mathcal{A}^{\mathcal{O}(\cdot, \cdot, \cdot)}(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}) : \begin{array}{l} n, L \leftarrow \mathcal{A}(1^\lambda), \\ \text{CRS} \leftarrow \text{GSetup}(1^\lambda, 1^n, 1^L), \\ \forall \text{id} \in [n], (\text{MPK}_{\text{id}}, \text{MSK}_{\text{id}}) \leftarrow \text{ASetup}(\text{id}) \end{array} \end{array} \right\} \\ \approx_c \\ \left\{ \begin{array}{l} \mathcal{A}^{\text{Sim}(\cdot, \cdot, \mathbf{V})}(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}) : \begin{array}{l} n, L \leftarrow \mathcal{A}(1^\lambda), \\ \text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]} \leftarrow \text{Sim}(1^\lambda, 1^n, 1^L) \end{array} \end{array} \right\}$$

where both $\mathcal{O}, \text{Sim}$ respond to queries from $\mathcal{A}$ of the form $(\text{id}, \text{GID}, \mu_{\text{id}, \text{GID}})$. $\mathcal{O}$ outputs $\text{ct}_{\text{id}, \text{GID}} \leftarrow \text{Enc}(\text{id}, \text{MSK}_{\text{id}}, \{\text{MPK}_{\text{id}_x}\}_{\text{id}_x \in [n]}, \text{GID}, \mu)$. Sim only uses (id, GID) and additionally uses the value $\mathbf{V}$. Here, $\mathbf{V} = (\mu_{1, \text{GID}}, \dots, \mu_{n, \text{GID}})$ if this is the last authority that needs to be queried for this GID and $\perp$ otherwise.

L.2 Preliminaries

In this section, we provide the definition of a reusable non-committing encryption scheme. Recall that a non-committing encryption scheme (NCE) [CFGN96] is a public-key encryption scheme with an additional property that we can later reveal a ciphertext to be the encryption of any message by tailoring the secret key for the scheme accordingly.

Syntax. A reusable non-committing encryption scheme NCE for $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ consists of the following polynomial time algorithms.

$\text{Setup}(1^\lambda, 1^\ell) \rightarrow (\text{MPK}, \text{MSK})$. The probabilistic setup algorithm takes as input security parameter λ and length of the messages ℓ , outputs master public-secret keys (MPK, MSK) .

$\text{KGen}(\text{MSK}, \text{GID}) \rightarrow \text{SK}$. The probabilistic key generation algorithm takes as input the master secret key MSK , a $\text{GID} \in \mathcal{GID}_\lambda$, and outputs a secret key SK .

$\text{Enc}(\text{MPK}, \text{GID}, m) \rightarrow \text{CT}$. The probabilistic encryption algorithm takes as input master public key MPK, a $\text{GID} \in \mathcal{GID}_\lambda$, a message $m \in \{0, 1\}^\ell$, and outputs the corresponding ciphertext CT.

$\text{Dec}(\text{SK}, \text{CT}) \rightarrow m'$. The decryption algorithm takes as input the secret key SK and a ciphertext CT, outputs m' .

$\text{Fake}(\text{MPK}, \text{GID}) \rightarrow (\widetilde{\text{CT}}, \text{aux})$. The probabilistic faking algorithm takes as input the master public key MPK, a $\text{GID} \in \mathcal{GID}_\lambda$, and outputs a fake ciphertext $\widetilde{\text{CT}}$ and auxiliary information aux.

$\text{Reveal}(\text{MSK}, \text{GID}, \text{aux}, m) \rightarrow \widetilde{\text{SK}}$. The revealing algorithm takes as input the master secret key MSK, a $\text{GID} \in \mathcal{GID}_\lambda$, auxiliary information aux, message $m \in \{0, 1\}^\ell$, and outputs secret key $\widetilde{\text{SK}}$.

Definition L.2. An NCE scheme $\text{NCE} = (\text{Setup}, \text{KGen}, \text{Enc}, \text{Dec}, \text{Fake}, \text{Reveal})$ is said to be a reusable non-committing encryption scheme for $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ if it satisfies the following properties.

Correctness. For any $\lambda \in \mathbb{N}, \ell = \ell(\lambda), \text{GID} \in \mathcal{GID}_\lambda$, any $m \in \{0, 1\}^\ell$,

$$\Pr \left[m = \text{Dec}(\text{SK}, \text{CT}) : \begin{array}{l} (\text{MPK}, \text{MSK}) \leftarrow \text{Setup}(1^\lambda, 1^\ell), \\ \text{CT} \leftarrow \text{Enc}(\text{MPK}, \text{GID}, m), \\ \text{SK} \leftarrow \text{KGen}(\text{MSK}, \text{GID}) \end{array} \right] = 1$$

Security. For any PPT admissible adversary $\mathcal{A}$, there exists a stateful PPT simulator Sim such that,

$$\left\{ \mathcal{A}^{\mathcal{O}(\cdot, \cdot)}(\text{MPK}) : \begin{array}{l} \ell \leftarrow \mathcal{A}(1^\lambda), \\ (\text{MPK}, \text{MSK}) \leftarrow \text{Setup}(1^\lambda, 1^\ell) \end{array} \right\} \\ \approx_c \\ \left\{ \mathcal{A}^{\text{Sim}(\cdot, \mathcal{V})}(\text{MPK}) : \begin{array}{l} \ell \leftarrow \mathcal{A}(1^\lambda), \mathcal{V} := \emptyset, \\ \text{MPK} \leftarrow \text{Sim}(1^\lambda, 1^\ell), \end{array} \right\}$$

A stateful PPT machine $\mathcal{A}$ is said to be admissible if it makes two kinds of queries — encryption query $(\text{Enc}, (\text{GID}, m))$ to receive CT, secret key query $(\text{KGen}, \text{GID})$ to receive SK, and makes at most one encryption and secret key query per GID. $\mathcal{O}$ uses Enc, KGen to respond to $\mathcal{A}$'s queries. $\mathcal{V} = \{(\text{GID}, m) : \text{for each GID queried by } \mathcal{A} \text{ is updated at each query}\}$.

Sim on the other hand uses Enc, KGen if KGen query for GID comes before Enc query and Fake, Reveal otherwise. While using Enc in the former scenario or Reveal in the latter, Sim queries $\mathcal{V}$ to receive m , message for this GID.

Remark L.3. Definition L.2 can be generically obtained by replacing PKE with IBE (Definition A.4) in the non-committing encryption constructions of [GVW12, HMNY22, HKM⁺24].

L.3 VNCE from NCE

In this section, we show how to construct vector NCE (VNCE) for any n parties using NCE. The main idea is that each party encrypts its input under each other party's NCE instantiation and also reveals keys for GID. If there are n parties interacting and each party consists of n independent NCE instantiations, we can form a $n \times n$ matrix of NCE ciphertexts where each party reveals its own column of secret keys. We can use the final column to be revealed to equivocate. We formally implement this idea below.

Notation. For ease of exposition, we introduce some notation for this construction. We denote matrices by bold uppercase letters $\mathbf{M}$, etc. We denote the i -th row, j -th column value of matrix $\mathbf{M}$ by $M_{i,j}$. It would easier to understand the construction if we consider the components of NCE scheme with sub- and superscripts in this construction as follows: superscript values are column numbers and subscript values are row numbers.

Construction L.4. We provide the construction of a vector NCE scheme (Definition L.1) for $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ and n parties using reusable NCE scheme for $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ as follows.

GSetup $(1^\lambda, 1^n, 1^L)$. Output $\text{CRS} := (\lambda, n, L)$.

ASetup (id) . Sample n instantiations of NCE, $(\text{ncc.mpk}_i, \text{ncc.msk}_i) \leftarrow \text{NCE.Setup}(1^\lambda, 1^L)$ for $i \in [n]$. Set $\text{MPK} := (\text{ncc.mpk}_i)_{i \in [n]}$, $\text{MSK} := (\text{ncc.msk}_i)_{i \in [n]}$ and output (MPK, MSK) .

Enc $(\text{id}, \text{MSK}_{\text{id}}, \{\text{MPK}_{\text{idx}}\}_{\text{idx} \in [n]}, \text{GID}, \mu)$. Parse MSK_{id} as $(\text{ncc.msk}_i^{(\text{id})})_{i \in [n]}$ and MPK_{idx} as $(\text{ncc.mpk}_i^{(\text{idx})})_{i \in [n]}$ for each $\text{idx} \in [n]$.

- Sample $R_1, \dots, R_n \leftarrow \{0, 1\}^L$. Set $\tilde{R} = (\bigoplus_{j \in [n]} R_j) \oplus \mu$.
- Sample for each $j \in [n]$, $\text{ncc.ct}_{\text{id}, \text{GID}}^{(j)} \leftarrow \text{NCE.Enc}(\text{ncc.mpk}_{\text{id}}^{(j)}, \text{GID}, R_j)$.
- Sample for each $i \in [n]$, $\text{ncc.sk}_i^{(\text{id})} \leftarrow \text{NCE.KGen}(\text{ncc.msk}_i^{(\text{id})}, \text{GID})$.

Output $\text{ct}_{\text{id}, \text{GID}} := \left((\text{ncc.sk}_i^{(\text{id})})_{i \in [n]}, (\text{ncc.ct}_{\text{id}}^{(j)})_{j \in [n]}, \tilde{R} \right)$.

Dec $(\{\text{ct}_{\text{id}, \text{GID}}\}_{\text{id} \in [n]})$. Parse each $\text{ct}_{\text{id}, \text{GID}}$ as $\left((\text{ncc.sk}_i^{(\text{id})})_{i \in [n]}, (\text{ncc.ct}_{\text{id}}^{(j)})_{j \in [n]}, \tilde{R}_{\text{id}} \right)$.

- Place each $\text{ncc.ct}_{\text{id}}^{(j)}$ as the (id, j) -th entry in the $n \times n$ matrix $\mathbf{C}$ for each $\text{id} \in [n], j \in [n]$.
- Place $\text{ncc.sk}_i^{(\text{id})}$ in the (i, id) -th entry in the $n \times n$ matrix $\mathbf{K}$ for each $i \in [n], \text{id} \in [n]$.
- Perform component-wise decryption of $\mathbf{C}$ and $\mathbf{K}$, i.e., use the (i, j) -th entry of $\mathbf{K}$ to decrypt the (i, j) -th entry of $\mathbf{C}$ for each $i, j \in [n]$ and place the resultant value in the (i, j) -th position, of $n \times n$ matrix $\mathbf{P}$.
- For each $\text{id} \in [n]$, let $\rho_{\text{id}} = \bigoplus_{j \in [n]} P_{\text{id}, j}$. Compute $\mu'_{\text{id}} = \tilde{R}_{\text{id}} \oplus \rho_{\text{id}}$.

Output $(\mu'_1, \dots, \mu'_n)$.

Correctness. Note that for each $\text{id} \in [n], i \in [n]$, the id -th authority samples keys for the (i, id) -th entries for the $n \times n$ matrices $\mathbf{C}, \mathbf{K}$ we form in Dec. That is, id -th column corresponds to id -th authority. Thus, all (i, j) -th entries of matrices $\mathbf{C}, \mathbf{K}$ are generated corresponding to the keys $(\text{ncc.mpk}_i^{(j)}, \text{ncc.msk}_i^{(j)})$ and w.r.to GID . Hence, $P_{\text{id}, j}$ is the value R_j generated by the id -th authority by correctness of NCE. Thus, $\rho_{\text{id}} = \bigoplus_{j \in [n]} R_j$. This in turn implies that $(\mu'_1, \dots, \mu'_n) = (\mu_1, \dots, \mu_n)$.

Lemma L.5. If NCE is a reusable non-committing encryption scheme for $\mathcal{GID}$ (Definition L.2), then Construction L.4 is a vector non-committing encryption scheme for $\mathcal{GID}$.

Proof. We prove this lemma using the following hybrids and claims.

Hyb₀^A(1^λ). This is the real experiment with an honest challenger Chal.

Setup Phase. $\mathcal{A}$ sends n, L . For each $\text{id} \in [n], i \in [n]$, sample $(\text{nce.mpk}_i^{(\text{id})}, \text{nce.msk}_i^{(\text{id})}) \leftarrow \text{NCE.Setup}(1^\lambda, 1^L)$. Set $\text{CRS} := (\lambda, n, L)$, $\text{MPK}_{\text{id}} = (\text{nce.mpk}_i^{(\text{id})})_{i \in [n]}$, $\text{MSK}_{\text{id}} = (\text{nce.msk}_i^{(\text{id})})_{i \in [n]}$. Send $\{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Query Phase. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x) \in [n] \times \mathcal{GID}_\lambda \times \{0, 1\}^L$.

- Sample $R_1, \dots, R_n \leftarrow \{0, 1\}^L$. Set $\tilde{R} = (\bigoplus_{j \in [n]} R_j) \oplus x$.
- Sample for each $j \in [n]$, $\text{nce.ct}_{\text{id}}^{(j)} \leftarrow \text{NCE.Enc}(\text{nce.mpk}_{\text{id}}^{(j)}, \text{GID}, R_j)$.
- Sample for each $i \in [n]$, $\text{nce.sk}_i^{(\text{id})} \leftarrow \text{NCE.KGen}(\text{nce.msk}_i^{(\text{id})}, \text{GID})$.

Send $\text{ct}_{\text{id}, \text{GID}} := ((\text{nce.sk}_i^{(\text{id})})_{i \in [n]}, (\text{nce.ct}_{\text{id}}^{(j)})_{j \in [n]}, \tilde{R})$ to $\mathcal{A}$.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb_{1, \iota, \kappa}^A(1^λ) for $\iota \in [n], \kappa \in [n+1]$. In this hybrid, we will simulate the first $\kappa - 1$ instantiations of NCE by the first ι authorities. Changes are highlighted in red.

Setup Phase. $\mathcal{A}$ sends n, L . For each $\text{id} \in [n], i \in [n]$, if $(\text{id} > \iota)$ or $(\text{id} \leq \iota, i \geq \kappa)$, sample $(\text{nce.mpk}_i^{(\text{id})}, \text{nce.msk}_i^{(\text{id})}) \leftarrow \text{NCE.Setup}(1^\lambda, 1^L)$. Otherwise, sample $\text{nce.mpk}_i^{(\text{id})} \leftarrow \text{NCE.Sim}_i^{(\text{id})}(1^\lambda, 1^L)$. Set $\text{CRS} := (\lambda, n, L)$, $\text{MPK}_{\text{id}} = (\text{nce.mpk}_i^{(\text{id})})_{i \in [n]}$. Send $\{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Start dictionaries $\mathcal{V}_i^{(j)}$ for each $j \in [n], i \in [n]$.

Query Phase. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x) \in [n] \times \mathcal{GID}_\lambda \times \{0, 1\}^L$.

- Sample $R_1, \dots, R_n \leftarrow \{0, 1\}^L$. Set $\tilde{R} = (\bigoplus_{j \in [n]} R_j) \oplus x$. Add (GID, R_j) to $\mathcal{V}_{\text{id}}^{(j)}$.
- Sample for each $j \in [n]$ if $j > \iota$ or $(j \leq \iota, \text{id} \geq \kappa)$, $\text{nce.ct}_{\text{id}}^{(j)} \leftarrow \text{NCE.Enc}(\text{nce.mpk}_{\text{id}}^{(j)}, \text{GID}, R_j)$. Otherwise, $\text{nce.ct}_{\text{id}}^{(j)} \leftarrow \text{NCE.Sim}_i^{(j)}(\text{Enc}, \text{GID})$. Any query made by Sim is answered using dictionaries.
- Sample for each $i \in [n]$, if $\text{id} > \iota$ or $(\text{id} \leq \iota, i \geq \kappa)$, $\text{nce.sk}_i^{(\text{id})} \leftarrow \text{NCE.KGen}(\text{nce.msk}_i^{(\text{id})}, \text{GID})$. Otherwise, $\text{nce.sk}_i^{(\text{id})} \leftarrow \text{NCE.Sim}_i^{(\text{id})}(\text{KGen}, \text{GID})$. Any query made by Sim is answered using dictionaries.

Send $\text{ct}_{\text{id}, \text{GID}} := ((\text{nce.sk}_i^{(\text{id})})_{i \in [n]}, (\text{nce.ct}_{\text{id}}^{(j)})_{j \in [n]}, \tilde{R})$ to $\mathcal{A}$.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb₂^A(1^λ). In this hybrid, all NCE instantiations are simulated. This is same as **Hyb_{1, n, n+1}^A(1^λ)**. We write it explicitly for clarity.

Setup Phase. $\mathcal{A}$ sends n, L . For each $\text{id} \in [n], i \in [n]$, sample $\text{nce.mpk}_i^{(\text{id})} \leftarrow \text{NCE.Sim}_i^{(\text{id})}(1^\lambda, 1^L)$. Set $\text{CRS} := (\lambda, n, L)$, $\text{MPK}_{\text{id}} = (\text{nce.mpk}_i^{(\text{id})})_{i \in [n]}$. Send $\{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Start dictionaries $\mathcal{V}_i^{(j)}$ for each $j \in [n], i \in [n]$.

Query Phase. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x) \in [n] \times \mathcal{GID}_\lambda \times \{0, 1\}^L$.

- Sample $R_1, \dots, R_n \leftarrow \{0, 1\}^L$. Set $\tilde{R} = (\bigoplus_{j \in [n]} R_j) \oplus x$. Add (GID, R_j) to $\mathcal{V}_{\text{id}}^{(j)}$.
- Sample for each $j \in [n]$, $\text{nce.ct}_{\text{id}}^{(j)} \leftarrow \text{NCE.Sim}_{\text{id}}^{(j)}(\text{Enc}, \text{GID})$. Any query made by Sim is answered using dictionaries.
- Sample for each $i \in [n]$, $\text{nce.sk}_i^{(\text{id})} \leftarrow \text{NCE.Sim}_i^{(\text{id})}(\text{KGen}, \text{GID})$. Any query made by Sim is answered using dictionaries.

Send $\text{ct}_{\text{id}, \text{GID}} := \left((\text{nce.sk}_i^{(\text{id})})_{i \in [n]}, (\text{nce.ct}_{\text{id}}^{(j)})_{j \in [n]}, \tilde{R} \right)$ to $\mathcal{A}$.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb₃^A(1^λ). In this hybrid, for each GID query, we sample a matrix $\mathbf{R}^{\text{GID}} \xleftarrow{\$} (\{0, 1\}^L)^{n \times n}$ and use the values in this matrix for responding to queries corresponding to GID. The changes are highlighted in **red**.

Setup Phase. $\mathcal{A}$ sends n, L . For each $\text{id} \in [n], i \in [n]$, sample $\text{nce.mpk}_i^{(\text{id})} \leftarrow \text{NCE.Sim}_i^{(\text{id})}(1^\lambda, 1^L)$. Set $\text{CRS} := (\lambda, n, L)$, $\text{MPK}_{\text{id}} = (\text{nce.mpk}_i^{(\text{id})})_{i \in [n]}$. Send $\{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Start dictionary $\mathcal{R}$ which is initially empty.

Query Phase. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x) \in [n] \times \mathcal{GID}_\lambda \times \{0, 1\}^L$. **If this is a new GID query, sample $\mathbf{R}^{\text{GID}} \xleftarrow{\$} (\{0, 1\}^L)^{n \times n}$. Otherwise, retrieve $\mathbf{R}^{\text{GID}}$ from $\mathcal{R}$. Parse $\mathbf{R}^{\text{GID}}$'s id -th row as $R_1, \dots, R_n$.**

- Set $\tilde{R} = (\bigoplus_{j \in [n]} R_j) \oplus x$.
- Sample for each $j \in [n]$, $\text{nce.ct}_{\text{id}}^{(j)} \leftarrow \text{NCE.Sim}_{\text{id}}^{(j)}(\text{Enc}, \text{GID})$. Any query made by Sim is answered **using $\mathbf{R}^{\text{GID}}$** .
- Sample for each $i \in [n]$, $\text{nce.sk}_i^{(\text{id})} \leftarrow \text{NCE.Sim}_i^{(\text{id})}(\text{KGen}, \text{GID})$. Any query made by Sim is answered **using $\mathbf{R}^{\text{GID}}$** .

Send $\text{ct}_{\text{id}, \text{GID}} := \left((\text{nce.sk}_i^{(\text{id})})_{i \in [n]}, (\text{nce.ct}_{\text{id}}^{(j)})_{j \in [n]}, \tilde{R} \right)$ to $\mathcal{A}$.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb₄^A(1^λ). In this hybrid, we will set $\tilde{R}$ to be uniformly random if this is not the last authority query and program the values of $x_1, \dots, x_n$ if this is the last query. This is the description of Sim from Definition L.1. The changes are highlighted in **red**.

Setup Phase. $\mathcal{A}$ sends n, L . For each $\text{id} \in [n], i \in [n]$, sample $\text{nce.mpk}_i^{(\text{id})} \leftarrow \text{NCE.Sim}_i^{(\text{id})}(1^\lambda, 1^L)$. Set $\text{CRS} := (\lambda, n, L)$, $\text{MPK}_{\text{id}} = (\text{nce.mpk}_i^{(\text{id})})_{i \in [n]}$. Send $\{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Start dictionary $\mathcal{R}$ which is initially empty. Also start dictionary $\mathcal{V}, \mathcal{R}'$ initially empty.

Query Phase. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x) \in [n] \times \mathcal{GID}_\lambda \times \{0, 1\}^L$. **Store $(\text{id}, \text{GID}, x)$ in $\mathcal{V}$. If this is a new GID query, sample $\mathbf{R}^{\text{GID}} \xleftarrow{\$} (\{0, 1\}^L)^{n \times n}$. Otherwise, retrieve $\mathbf{R}^{\text{GID}}$ from $\mathcal{R}$.**

- If this is not the last authority to be queried for GID , sample $\tilde{R} \leftarrow \{0,1\}^L$. Store $(\text{id}, \text{GID}, \tilde{R})$ in $\mathcal{R}'$. Otherwise, retrieve and $(x_1, \dots, x_n)$ from $\mathcal{V}$ and $(\tilde{R}_{\text{id}x})_{\text{id}x \neq \text{id}}$ from $\mathcal{R}'$ for this GID . Sample $\tilde{R}_{\text{id}} \xleftarrow{\$} \{0,1\}^L$. Replace id -th column of $\mathbf{R}^{\text{GID}}$ with values $(\tilde{R} \oplus (\bigoplus_{j \neq \text{id}} R_{\text{id},j}^{\text{GID}}) \oplus x_{\text{id}})$.
- Sample for each $j \in [n]$, $\text{ncc.ct}_{\text{id}}^{(j)} \leftarrow \text{NCE.Sim}_{\text{id}}^{(j)}(\text{Enc}, \text{GID})$. Any query made by Sim is answered using $\mathbf{R}^{\text{GID}}$.
- Sample for each $i \in [n]$, $\text{ncc.sk}_i^{(\text{id})} \leftarrow \text{NCE.Sim}_i^{(\text{id})}(\text{KGen}, \text{GID})$. Any query made by Sim is answered using $\mathbf{R}^{\text{GID}}$.

Send $\text{ct}_{\text{id}, \text{GID}} := ((\text{ncc.sk}_i^{(\text{id})})_{i \in [n]}, (\text{ncc.ct}_{\text{id}}^{(j)})_{j \in [n]}, \tilde{R})$ to $\mathcal{A}$.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Claim L.6. $\text{Hyb}_0^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_{1,1,1}^{\mathcal{A}}(1^\lambda)$ are identically distributed.

Proof. As we are not simulating any VNCE instantiations in $\text{Hyb}_{1,1,1}^{\mathcal{A}}(1^\lambda)$, these two hybrids are identically distributed. $\square$

Claim L.7. Assuming the security of NCE , $\text{Hyb}_{1,\ell,\kappa}^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_{1,\ell,\kappa+1}^{\mathcal{A}}(1^\lambda)$ for any $\ell \in [n], \kappa \in [n]$ are computationally indistinguishable.

Proof. Assume that there exists an adversary $\mathcal{A}$ that can distinguish between $\text{Hyb}_{1,\ell,\kappa}^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_{1,\ell,\kappa+1}^{\mathcal{A}}(1^\lambda)$ for some $\ell \in [n], \kappa \in [n]$ with non-negligible probability $\epsilon(\lambda)$, i.e.,

$$\left| \Pr \left[1 \leftarrow \text{Hyb}_{1,\ell,\kappa}^{\mathcal{A}}(1^\lambda) \right] - \Pr \left[1 \leftarrow \text{Hyb}_{1,\ell,\kappa+1}^{\mathcal{A}}(1^\lambda) \right] \right| > \epsilon(\lambda)$$

We will construct a reduction adversary $\mathcal{B}$ that can break the security of NCE with non-negligible probability. The description of $\mathcal{B}^{\mathcal{O}}$ is as follows.

Setup Phase. $\mathcal{A}$ sends n, L . Send 1^L to $\mathcal{O}$ and receive ncc.mpk . For each $\text{id} \in [n], i \in [n]$, if $(\text{id} > \ell)$ or $(\text{id} \leq \ell, i > \kappa)$, sample $(\text{ncc.mpk}_i^{(\text{id})}, \text{ncc.msk}_i^{(\text{id})}) \leftarrow \text{NCE.Setup}(1^\lambda, 1^L)$. Otherwise, if $\text{id} = \ell, i = \kappa$, set $\text{ncc.mpk}_i^{(\text{id})} := \text{ncc.mpk}$. Otherwise, sample $\text{ncc.mpk}_i^{(\text{id})} \leftarrow \text{NCE.Sim}_i^{(\text{id})}(1^\lambda, 1^L)$. Set $\text{CRS} := (\lambda, n, L)$, $\text{MPK}_{\text{id}} = (\text{ncc.mpk}_i^{(\text{id})})_{i \in [n]}$. Send $\{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Start dictionaries $\mathcal{V}_i^{(j)}$ for each $j \in [n], i \in [n]$.

Query Phase. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x) \in [n] \times \mathcal{GID}_\lambda \times \{0,1\}^L$.

- Sample $R_1, \dots, R_n \leftarrow \{0,1\}^L$. Set $\tilde{R} = (\bigoplus_{j \in [n]} R_j) \oplus x$. Add (GID, R_j) to $\mathcal{V}_{\text{id}}^{(j)}$.
- Sample for each $j \in [n]$ if $j > \ell$ or $(j \leq \ell, \text{id} > \kappa)$, $\text{ncc.ct}_{\text{id}}^{(j)} \leftarrow \text{NCE.Enc}(\text{ncc.mpk}_{\text{id}}^{(j)}, \text{GID}, R_j)$. Otherwise, if $j = \ell, \text{id} = \kappa$, send (Enc, R_j) to $\mathcal{O}$ to receive $\text{ncc.ct}_{\text{id}}^{(j)}$. Otherwise, $\text{ncc.ct}_{\text{id}}^{(j)} \leftarrow \text{NCE.Sim}_{\text{id}}^{(j)}(\text{Enc}, \text{GID})$. Any query made by Sim is answered using dictionaries.

- Sample for each $i \in [n]$, if $\text{id} > \iota$ or $(\text{id} \leq \iota, i > \kappa)$, $\text{nce.sk}_i^{(\text{id})} \leftarrow \text{NCE.KGen}(\text{nce.msk}_i^{(\text{id})}, \text{GID})$. Otherwise, if $\text{id} = \iota, i = \kappa$, send $(\text{KGen}, \text{GID})$ to $\mathcal{O}$ to receive $\text{nce.sk}_i^{(\text{id})}$. Otherwise, $\text{nce.sk}_i^{(\text{id})} \leftarrow \text{NCE.Sim}_i^{(\text{id})}(\text{KGen}, \text{GID})$. Any query made by Sim is answered using dictionaries.

Send $\text{ct}_{\text{id}, \text{GID}} := \left((\text{nce.sk}_i^{(\text{id})})_{i \in [n]}, (\text{nce.ct}_{\text{id}}^{(j)})_{j \in [n]}, \tilde{R} \right)$ to $\mathcal{A}$.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

As we can see, the running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$, λ . If $\mathcal{O}$ uses Enc, KGen $\mathcal{B}$ behaves like $\mathbf{Hyb}_{1, \iota, \kappa}^{\mathcal{A}}(1^\lambda)$ and if $\mathcal{O}$ use $\text{Fake}, \text{Reveal}$, $\mathcal{B}$ behaves like $\mathbf{Hyb}_{1, \iota, \kappa+1}^{\mathcal{A}}(1^\lambda)$. Hence, $\mathcal{B}$ is a valid adversary that break the security of NCE with non-negligible probability. Thus, $\mathbf{Hyb}_{1, \iota, \kappa}^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_{1, \iota, \kappa+1}^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable. $\square$

Claim L.8. Assuming the security of NCE, $\mathbf{Hyb}_{1, \iota, n+1}^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_{1, \iota+1, 1}^{\mathcal{A}}(1^\lambda)$ for any $\iota \in [n-1]$ are computationally indistinguishable.

Proof. As the only difference between these hybrids is that we simulate the (ι, n) -th instantiation of the ι -th authority, the proof of this claim is similar to proof of Claim L.7. $\square$

Claim L.9. $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$, $\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$, and $\mathbf{Hyb}_4^{\mathcal{A}}(1^\lambda)$ are identically distributed.

Proof. As the only difference between $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$ is that in $\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$, we are sampling all the random strings $\{R_i\}_i$ a-priori for each authority and in $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$, on-the-fly. As both as identically distributed, these two hybrids are identically distributed.

For the case of $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$, consider the matrix $\mathbf{R}^{\text{GID}}$ and add *one* more column to it corresponding to $\tilde{R}_{\text{id}}$ sampled by each id for GID . The i -th row of this resultant matrix are additive secret shares of x_i . It is the same for both $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$. We are just changing where the input x_i is added. Thus, both of these hybrids are identically distributed. $\square$

$\square$

M Proofs from Section 5.5

In this section, we give the full proof of Theorem 5.10.

$\mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)$. This is the real experiment with Chal with $b = 0/1$.

Setup Phase. $\mathcal{A}$ sends $n, s, \ell_1, \dots, \ell_n$. Define $L = \max_{\text{id}} \{\ell_{\text{id}}\}$. Chal samples $\text{mafe.crs} \leftarrow \text{abMAFE.GSetup}(1^\lambda, 1^n, 1^s, 1^L)$ and $\text{nce.crs} \leftarrow \text{VNCE.GSetup}(1^\lambda, 1^n, 1^{L'})$. Chal sets $\text{CRS} := (\text{mafe.crs}, \text{nce.crs})$.

Chal then samples for each $\text{id} \in [n]$, $(\text{mafe.mpk}_{\text{id}}, \text{mafe.msk}_{\text{id}}) \leftarrow \text{abMAFE.ASetup}(\text{id}, 1^{\ell_{\text{id}}})$ and $(\text{nce.mpk}_{\text{id}}, \text{nce.msk}_{\text{id}}) \leftarrow \text{VNCE.ASetup}(\text{id})$. Set $\text{MPK}_{\text{id}} := (\text{mafe.mpk}_{\text{id}}, \text{nce.mpk}_{\text{id}})$ and $\text{MSK}_{\text{id}} := (\text{mafe.msk}_{\text{id}}, \text{nce.msk}_{\text{id}})$.

Chal sends $\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Pre-challenge Queries. $\mathcal{A}$ sends queries of the form $(\text{id}, \text{GID}, x)$. Sample $\text{mafe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{abMAFE.KGen}(\text{id}, \text{mafe.msk}_{\text{id}}, \{\text{mafe.mpk}_{\text{id}, x}\}_{x \in [n]}, \text{GID}, x)$. Then encrypt using VNCE, $\text{nce.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{VNCE.Enc}(\text{id}, \text{nce.msk}_{\text{id}}, \{\text{nce.mpk}_{\text{id}, x}\}_{x \in [n]}, \text{GID}, \text{mafe.sk}_{\text{id}, \text{GID}, x})$.

Send $\text{SK}_{\text{id}, \text{GID}, x} := \text{nce.sk}_{\text{id}, \text{GID}, x}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends circuits $C_0, C_1 \in \mathcal{C}_\lambda$ such that $|C_0| = |C_1|$ and they are of size at most s . Chal samples $\text{mafe.ct} \leftarrow \text{abMAFE.Enc}(\{\text{mafe.mpk}_{\text{id}}\}_{\text{id} \in [n]}, C_b)$ and sends $\text{CT} := \text{mafe.ct}$ to $\mathcal{A}$.

Post-challenge Queries. This is same as pre-challenge queries.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb₁^A(1^λ). In this hybrid, we simulate the VNCE instantiation. Changes are highlighted in red.

Setup Phase. $\mathcal{A}$ sends $n, s, \ell_1, \dots, \ell_n$. Define $L = \max_{\text{id}} \{\ell_{\text{id}}\}$. Sample $\text{mafe.crs} \leftarrow \text{abMAFE.GSetup}(1^\lambda, 1^n, 1^s, 1^L)$. Then, for each $\text{id} \in [n]$, $\text{mafe.mpk}_{\text{id}}, \text{mafe.msk}_{\text{id}} \leftarrow \text{abMAFE.ASetup}(\text{id}, 1^{\ell_{\text{id}}})$. Also, $(\text{nce.crs}, \{\text{nce.mpk}_{\text{id}}\}_{\text{id} \in [n]}) \leftarrow \text{VNCE.Sim}(1^\lambda, 1^n, 1^L)$. Also initiate an empty dictionary of queries $\mathcal{Q}$.

Set $\text{CRS} := (\text{mafe.crs}, \text{nce.crs})$. Set $\text{MPK}_{\text{id}} := (\text{mafe.mpk}_{\text{id}}, \text{nce.mpk}_{\text{id}})$ and $\text{MSK}_{\text{id}} := \text{mafe.msk}_{\text{id}}$.

Send $\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Pre-challenge Queries. $\mathcal{A}$ sends queries of the form $(\text{id}, \text{GID}, x)$.

- If this is the last query for this GID, sample for each $\text{id} \in [n]$, $\text{mafe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{abMAFE.KGen}(\text{id}, \text{mafe.msk}_{\text{id}}, \{\text{mafe.mpk}_{\text{id}, x}\}_{x \in [n]}, \text{GID}, x_{\text{id}, \text{GID}})$ by retrieving $((1, \text{GID}, x_{1, \text{GID}}), \dots, (n, \text{GID}, x_{n, \text{GID}}))$ from $\mathcal{Q}$. Let $\mathbf{V} := (\text{mafe.sk}_{1, \text{GID}, x_{1, \text{GID}}}, \dots, \text{mafe.sk}_{n, \text{GID}, x_{n, \text{GID}}})$. Sample $\text{nce.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{VNCE.Sim}(\text{id}, \text{GID}, x, \mathbf{V})$.
- Otherwise, add $(\text{id}, \text{GID}, x)$ to $\mathcal{Q}$ and simulate $\text{nce.sk}_{\text{id}, \text{GID}, x_{\text{id}}} \leftarrow \text{VNCE.Sim}(\text{id}, \text{GID}, x, \perp)$.

Send $\text{SK}_{\text{id}, \text{GID}, x} := \text{nce.sk}_{\text{id}, \text{GID}, x}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends circuits $C_0, C_1 \in \mathcal{C}_\lambda$ such that $|C_0| = |C_1|$ and they are of size at most s . Sample $\text{mafe.ct} \leftarrow \text{abMAFE.Enc}(\{\text{mafe.mpk}_{\text{id}}\}_{\text{id} \in [n]}, C_b)$ and sends $\text{CT} := \text{mafe.ct}$ to $\mathcal{A}$.

Post-challenge Queries. This is same as pre-challenge queries.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb₂^A(1^λ). In this hybrid, we change the circuit encrypted in abMAFE to C_{1-b} . Changes are highlighted in red.

Setup Phase. $\mathcal{A}$ sends $n, s, \ell_1, \dots, \ell_n$. Define $L = \max_{\text{id}} \{\ell_{\text{id}}\}$. Sample $\text{mafe.crs} \leftarrow \text{abMAFE.GSetup}(1^\lambda, 1^n, 1^s, 1^L)$. Then, for each $\text{id} \in [n]$, $\text{mafe.mpk}_{\text{id}}, \text{mafe.msk}_{\text{id}} \leftarrow \text{abMAFE.ASetup}(\text{id}, 1^{\ell_{\text{id}}})$. Also, $(\text{nce.crs}, \{\text{nce.mpk}_{\text{id}}\}_{\text{id} \in [n]}) \leftarrow \text{VNCE.Sim}(1^\lambda, 1^n, 1^L)$. Also initiate an empty dictionary of queries $\mathcal{Q}$.

Set $\text{CRS} := (\text{mafe.crs}, \text{nce.crs})$. Set $\text{MPK}_{\text{id}} := (\text{mafe.mpk}_{\text{id}}, \text{nce.mpk}_{\text{id}})$ and $\text{MSK}_{\text{id}} := (\text{mafe.msk}_{\text{id}}, \text{nce.msk}_{\text{id}})$.

Send $\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Pre-challenge Queries. $\mathcal{A}$ sends queries of the form $(\text{id}, \text{GID}, x)$.

- If this is the last query for this GID, sample for each $\text{id} \in [n]$, $\text{mafe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{abMAFE.KGen}(\text{id}, \text{mafe.msk}_{\text{id}}, \{\text{mafe.mpk}_{\text{id}, x}\}_{x \in [n]}, \text{GID}, x_{\text{id}, \text{GID}})$ by retrieving $((1, \text{GID}, x_{1, \text{GID}}), \dots, (n, \text{GID}, x_{n, \text{GID}}))$ from $\mathcal{Q}$. Let $V := (\text{mafe.sk}_{1, \text{GID}, x_{1, \text{GID}}}, \dots, \text{mafe.sk}_{n, \text{GID}, x_{n, \text{GID}}})$. Sample $\text{nce.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{VNCE.Sim}(\text{id}, \text{GID}, x, V)$.
- Otherwise, add $(\text{id}, \text{GID}, x)$ to $\mathcal{Q}$ and simulate $\text{nce.sk}_{\text{id}, \text{GID}, x_{\text{id}}} \leftarrow \text{VNCE.Sim}(\text{id}, \text{GID}, x, \perp)$.

Send $\text{SK}_{\text{id}, \text{GID}, x} := \text{nce.sk}_{\text{id}, \text{GID}, x}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends circuits $C_0, C_1 \in \mathcal{C}_\lambda$ such that $|C_0| = |C_1|$ and they are of size at most s . Sample $\text{mafe.ct} \leftarrow \text{abMAFE.Enc}(\{\text{mafe.mpk}_{\text{id}}\}_{\text{id} \in [n]}, C_{1-b})$ and sends $\text{CT} := \text{mafe.ct}$ to $\mathcal{A}$.

Post-challenge Queries. This is same as pre-challenge queries.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb₃^A(1^λ). This is experiment Hybrid 0 with $1 - b$.

Lemma M.1. Assuming the security of VNCE, **Hyb₀^A(1^λ)** and **Hyb₁^A(1^λ)** are computationally indistinguishable.

Proof. Assume that there exists an adversary $\mathcal{A}$ that can distinguish between **Hyb₀^A(1^λ)** and **Hyb₁^A(1^λ)** with non-negligible probability $\epsilon(\lambda)$, i.e.,

$$\left| \Pr \left[1 \leftarrow \mathbf{Hyb}_0^A(1^\lambda) \right] - \Pr \left[1 \leftarrow \mathbf{Hyb}_1^A(1^\lambda) \right] \right| > \epsilon(\lambda)$$

We will construct a reduction adversary $\mathcal{B}$ that can break the security of VNCE with non-negligible probability. The description of $\mathcal{B}^\mathcal{O}$ is as follows.

Setup Phase. $\mathcal{A}$ sends $n, s, \ell_1, \dots, \ell_n$. Define $L = \max_{\text{id}} \{\ell_{\text{id}}\}$. Sample $\text{mafe.crs} \leftarrow \text{abMAFE.GSetup}(1^\lambda, 1^n, 1^s, 1^L)$. Then, for each $\text{id} \in [n]$, $(\text{mafe.mpk}_{\text{id}}, \text{mafe.msk}_{\text{id}}) \leftarrow \text{abMAFE.ASetup}(\text{id}, 1^{\ell_{\text{id}}})$. Receive, $(\text{nce.crs}, \{\text{nce.mpk}_{\text{id}}\}_{\text{id} \in [n]})$ from $\mathcal{O}$. Also initiate an empty dictionary of queries $\mathcal{Q}$.

Set $\text{CRS} := (\text{mafe.crs}, \text{nce.crs})$. Set $\text{MPK}_{\text{id}} := (\text{mafe.mpk}_{\text{id}}, \text{nce.mpk}_{\text{id}})$ and $\text{MSK}_{\text{id}} := \text{mafe.msk}_{\text{id}}$.

Send $\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Pre-challenge Queries. $\mathcal{A}$ sends queries of the form $(\text{id}, \text{GID}, x)$. Sample $\text{mafe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{abMAFE.KGen}(\text{id}, \text{mafe.msk}_{\text{id}}, \{\text{mafe.mpk}_{\text{id}, x}\}_{x \in [n]}, \text{GID}, x)$. Then send $(\text{id}, \text{GID}, \text{mafe.sk}_{\text{id}, \text{GID}, x})$ to $\mathcal{O}$ to receive $\text{nce.sk}_{\text{id}, \text{GID}, x}$.

Send $\text{SK}_{\text{id}, \text{GID}, x} := \text{nce.sk}_{\text{id}, \text{GID}, x}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends circuits $C_0, C_1 \in \mathcal{C}_\lambda$ such that $|C_0| = |C_1|$ and they are of size at most s . Sample $\text{mafe.ct} \leftarrow \text{abMAFE.Enc}(\{\text{mafe.mpk}_{\text{id}}\}_{\text{id} \in [n]}, C_b)$ and sends $\text{CT} := \text{mafe.ct}$ to $\mathcal{A}$.

Post-challenge Queries. This is same as pre-challenge queries.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

As we can see, the running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$, λ . If $\mathcal{O}$ is an honest challenger for VNCE $\mathcal{B}$ behaves like $\mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)$ and if $\mathcal{O}$ simulates VNCE $\mathcal{B}$ behaves like $\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$. Hence, $\mathcal{B}$ is a valid adversary that break the security of VNCE with non-negligible probability. Thus, $\mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable. $\square$

Lemma M.2. Assuming the security of abMAFE, $\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable.

Proof. Assume that there exists an adversary $\mathcal{A}$ that can distinguish between $\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ with non-negligible probability $\epsilon(\lambda)$, i.e.,

$$\left| \Pr \left[1 \leftarrow \mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda) \right] - \Pr \left[1 \leftarrow \mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda) \right] \right| > \epsilon(\lambda)$$

We will construct a reduction adversary $\mathcal{B}$ that can break the security of abMAFE with non-negligible probability. The description of $\mathcal{B}^{\mathcal{O}}$ is as follows.

Setup Phase. $\mathcal{A}$ sends $n, s, \ell_1, \dots, \ell_n$. Define $L = \max_{id} \{\ell_{id}\}$. Receive $\text{mafe.crs}, \{\text{mafe.mpk}_{id}\}_{id \in [n]}$ from $\mathcal{O}$. Also, $(\text{nce.crs}, \{\text{nce.mpk}_{id}\}_{id \in [n]}) \leftarrow \text{VNCE.Sim}(1^\lambda, 1^n, 1^{L'})$. Also, initiate an empty dictionary of queries $\mathcal{Q}$.

Set $\text{CRS} := (\text{mafe.crs}, \text{nce.crs})$. Set $\text{MPK}_{id} := (\text{mafe.mpk}_{id}, \text{nce.mpk}_{id})$.

Send $\text{CRS}, \{\text{MPK}_{id}\}_{id \in [n]}$ to $\mathcal{A}$.

Pre-challenge Queries. $\mathcal{A}$ sends queries of the form (id, GID, x) .

- If this is the last query for this GID, retrieve $((1, \text{GID}, x_{1,\text{GID}}), \dots, (n, \text{GID}, x_{n,\text{GID}}))$ from $\mathcal{Q}$. Send $\text{GID}, x_{1,\text{GID}}, \dots, x_{n,\text{GID}}$ to $\mathcal{O}$ to receive $\text{mafe.sk}_{1,\text{GID},x_{1,\text{GID}}}, \dots, \text{mafe.sk}_{n,\text{GID},x_{n,\text{GID}}}$. Let $V := (\text{mafe.sk}_{1,\text{GID},x_{1,\text{GID}}}, \dots, \text{mafe.sk}_{n,\text{GID},x_{n,\text{GID}}})$. Sample $\text{nce.sk}_{id,\text{GID},x} \leftarrow \text{VNCE.Sim}(id, \text{GID}, x, V)$.
- Otherwise, add (id, GID, x) to $\mathcal{Q}$ and simulate $\text{nce.sk}_{id,\text{GID},x_{id}} \leftarrow \text{VNCE.Sim}(id, \text{GID}, x, \perp)$.

Send $\text{SK}_{id,\text{GID},x} := \text{nce.sk}_{id,\text{GID},x}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends circuits $C_0, C_1 \in \mathcal{C}_\lambda$ such that $|C_0| = |C_1|$ and they are of size at most s . Send C_b, C_{1-b} to $\mathcal{O}$ to receive mafe.ct and sends $\text{CT} := \text{mafe.ct}$ to $\mathcal{A}$.

Post-challenge Queries. This is same as pre-challenge queries.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

As we can see, the running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$, λ . If $\mathcal{O}$ encryption C_b $\mathcal{B}$ behaves like $\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$ and if $\mathcal{O}$ encrypts C_{1-b} $\mathcal{B}$ behaves like $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$. Hence, $\mathcal{B}$ is a valid adversary that break the security of abMAFE with non-negligible probability. Thus, $\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable. $\square$

Lemma M.3. Assuming the security of VNCE, $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable.

Proof. The proof of this lemma is similar to proof of Lemma M.1. $\square$

N Proofs from Section 6.2

In this section, we provide full proof of Theorem 6.3.

Hyb₀^A(1^λ). This is the honest experiment with Chal with $b = 0/1$.

Setup. $\mathcal{A}$ sends $Q, n, s, k, \chi, \ell_1, \dots, \ell_n$ to Chal. For each $i \in \mathbb{I}, j \in \mathbb{J}$, Chal samples $\text{crs}^{(i,j)} \leftarrow \text{MAFE.GSetup}(1^\lambda, 1^Q, 1^{n-k}, 1^s, 1^{L+\lambda+\ell_{\text{out}}})$. In addition, sample for each $\text{id} \in [n]$, $(\text{mpk}_{\text{id}}^{(i,j)}, \text{msk}_{\text{id}}^{(i,j)}) \leftarrow \text{MAFE.ASetup}(\text{id}, 1^{\ell_{\text{id}}+\lambda+\ell_{\text{out}}})$ for i such that $\text{id} \notin \mathcal{K}_i$. Set $\text{CRS} := (\text{crs}^{(i,j)})_{i,j}$, $\text{MPK}_{\text{id}} := (\text{mpk}_{\text{id}}^{(i,j)})_{i,j}$, and $\text{MSK}_{\text{id}} := (\text{msk}_{\text{id}}^{(i,j)})_{i,j}$ and send $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]})$ to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$ for $q \in [Q]$ to Chal such that id is not corrupted. For each $i \in \mathbb{I}'_{\text{id}}, j \in \mathbb{J}$, sample $r_{ij} \xleftarrow{\$} \{0, 1\}^\lambda$, $\text{ske.ct}_{ij}^* \leftarrow \{0, 1\}^{\ell_{\text{out}}}$, and $\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \text{MAFE.KGen}(\text{id}, \text{msk}_{\text{id}}^{(i,j)}, \{\text{mpk}_{\text{id}_x}^{(i,j)}\}_{\text{id}_x}, \text{GID}, (x, r_{ij}, \text{ske.ct}_{ij}^*))$. Set $\text{SK}_{\text{id}, \text{GID}, x} = (\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)})_{i \in \mathbb{I}'_{\text{id}}, j}$ and send it to $\mathcal{A}$.

Corruption Queries. $\mathcal{A}$ sends (Corr, id) to Chal. Send $(\text{msk}_{\text{id}}^{(i,j)})_{i,j}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends circuits C_0, C_1 to Chal. Chal samples $\{\tilde{C}_i\}_{i \in \mathbb{I}'} \leftarrow \text{HSS.Share}(1^\lambda, 1^{|\mathbb{I}'|}, C)$. For each $i \in \mathbb{I}', j \in \mathbb{J}$, sample $\text{ct}^{(i,j)} \leftarrow \text{MAFE.Enc}(\{\text{mpk}_{\text{id}}^{(i,j)}\}_{\text{id} \notin \mathcal{K}_i}, F_{ij})$. Set $\text{CT} = (\text{ct}^{(i,j)})_{i,j}$ and send it to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$ to Chal such that id is not corrupted. For each $i \in \mathbb{I}'_{\text{id}}, j \in \mathbb{J}$, sample $r_{ij} \xleftarrow{\$} \{0, 1\}^\lambda$, $\text{ske.ct}_{ij}^* \leftarrow \{0, 1\}^{\ell_{\text{out}}}$, and $\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \text{MAFE.KGen}(\text{id}, \text{msk}_{\text{id}}^{(i,j)}, \{\text{mpk}_{\text{id}_x}^{(i,j)}\}_{\text{id}_x}, \text{GID}, (x, r_{ij}, \text{ske.ct}_{ij}^*))$. Set $\text{SK}_{\text{id}, \text{GID}, x} = (\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)})_{i \in \mathbb{I}'_{\text{id}}, j}$ and send it to $\mathcal{A}$.

Corruption Queries. $\mathcal{A}$ sends (Corr, id) to Chal. Send $(\text{msk}_{\text{id}}^{(i,j)})_{i,j}$ to $\mathcal{A}$.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb₁^A(1^λ). In this hybrid, we simply guess $i^* \xleftarrow{\$} \mathbb{I}'$. If any $\text{id} \notin \mathcal{K}_{i^*}$ is corrupted, we will abort. The changes are highlighted in red.

Setup. $\mathcal{A}$ sends $Q, n, s, k, \chi, \ell_1, \dots, \ell_n$. For each $i \in \mathbb{I}, j \in \mathbb{J}$, sample $\text{crs}^{(i,j)} \leftarrow \text{MAFE.GSetup}(1^\lambda, 1^Q, 1^{n-k}, 1^s, 1^{L+\lambda+\ell_{\text{out}}})$. In addition, sample for each $\text{id} \in [n]$, $(\text{mpk}_{\text{id}}^{(i,j)}, \text{msk}_{\text{id}}^{(i,j)}) \leftarrow \text{MAFE.ASetup}(\text{id}, 1^{\ell_{\text{id}}+\lambda+\ell_{\text{out}}})$ for i such that $\text{id} \notin \mathcal{K}_i$. Set $\text{CRS} := (\text{crs}^{(i,j)})_{i,j}$, $\text{MPK}_{\text{id}} := (\text{mpk}_{\text{id}}^{(i,j)})_{i,j}$, and $\text{MSK}_{\text{id}} := (\text{msk}_{\text{id}}^{(i,j)})_{i,j}$ and send $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]})$ to $\mathcal{A}$.

Sample $i^* \xleftarrow{\$} \mathbb{I}'$.

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$ for $q \in [Q]$ such that id is not corrupted. For each $i \in \mathbb{I}'_{\text{id}}, j \in \mathbb{J}$, sample $r_{ij} \xleftarrow{\$} \{0, 1\}^\lambda$, $\text{ske.ct}_{ij}^* \leftarrow \{0, 1\}^{\ell_{\text{out}}}$, and $\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \text{MAFE.KGen}(\text{id}, \text{msk}_{\text{id}}^{(i,j)}, \{\text{mpk}_{\text{id}_x}^{(i,j)}\}_{\text{id}_x}, \text{GID}, (x, r_{ij}, \text{ske.ct}_{ij}^*))$. Set $\text{SK}_{\text{id}, \text{GID}, x} = (\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)})_{i \in \mathbb{I}'_{\text{id}}, j}$ and send it to $\mathcal{A}$.

Corruption Queries. $\mathcal{A}$ sends (Corr, id) . **If $\text{id} \notin \mathcal{K}_{i^*}$, abort.** Send $(\text{msk}_{\text{id}}^{(i,j)})_{i,j}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends circuit C_0, C_1 . Sample $\{\tilde{C}_i\}_{i \in \mathbb{I}'} \leftarrow \text{HSS.Share}(1^\lambda, 1^{|\mathbb{I}'|}, C_b)$. For each $i \in \mathbb{I}', j \in \mathbb{J}$, sample $\text{ct}^{(i,j)} \leftarrow \text{MAFE.Enc}(\{\text{mpk}_{\text{id}}^{(i)}\}_{\text{id} \notin \mathcal{K}_i}, F_{ij})$. Set $\text{CT} = (\text{ct}^{(i,j)})_{i,j}$ and send it to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$ such that id is not corrupted. For each $i \in \mathbb{I}'_{\text{id}}, j \in \mathbb{J}$, sample $r_{ij} \xleftarrow{\$} \{0, 1\}^\lambda$, $\text{ske.ct}_{ij}^* \leftarrow \{0, 1\}^{\ell_{\text{out}}}$, and $\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \text{MAFE.KGen}(\text{id}, \text{msk}_{\text{id}}^{(i,j)}, \{\text{mpk}_{\text{id}_x}^{(i,j)}\}_{\text{id}_x}, \text{GID}, (x, r_{ij}, \text{ske.ct}_{ij}^*))$. Set $\text{SK}_{\text{id}, \text{GID}, x} = (\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)})_{i \in \mathbb{I}'_{\text{id}}, j}$ and send it to $\mathcal{A}$.

Corruption Queries. $\mathcal{A}$ sends (Corr, id) . If $\text{id} \notin \mathcal{K}_{i^*}$, **abort**. Send $(\text{msk}_{\text{id}}^{(i,j)})_{i,j}$ to $\mathcal{A}$.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb₂^A(1^λ). In this hybrid, we will sample a unique ρ_{i^*j} per GID for each $j \in \mathbb{J}$. We will set each r_{i^*j} for each authority as additive secret shares of ρ_{i^*j} . The changes are highlighted in **red**.

Setup. $\mathcal{A}$ sends $Q, n, s, k, \chi, \ell_1, \dots, \ell_n$. For each $i \in \mathbb{I}, j \in \mathbb{J}$, sample $\text{crs}^{(i,j)} \leftarrow \text{MAFE.GSetup}(1^\lambda, 1^Q, 1^{n-k}, 1^s, 1^{L+\lambda+\ell_{\text{out}}})$. In addition, sample for each $\text{id} \in [n]$, $(\text{mpk}_{\text{id}}^{(i,j)}, \text{msk}_{\text{id}}^{(i,j)}) \leftarrow \text{MAFE.ASetup}(\text{id}, 1^{\ell_{\text{id}}+\lambda+\ell_{\text{out}}})$ for i such that $\text{id} \notin \mathcal{K}_i$. Set $\text{CRS} := (\text{crs}^{(i,j)})_{i,j}$, $\text{MPK}_{\text{id}} := (\text{mpk}_{\text{id}}^{(i,j)})_{i,j}$, and $\text{MSK}_{\text{id}} := (\text{msk}_{\text{id}}^{(i,j)})_{i,j}$ and send $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]})$ to $\mathcal{A}$.

Sample $i^* \xleftarrow{\$} \mathbb{I}'$. **Initiate an empty dictionary $\mathcal{R}$. Sample Q many ρ_j for each $j \in \mathbb{J}$. If any of these are repeated, abort and output $\perp$. Otherwise, compute $r_j^{(1)}, \dots, r_j^{(n)}$ such that $\bigoplus_{\text{id}} r_j^{(\text{id})} = \rho_j$ and add $(\text{GID}, (r_j^{(\text{id})})_{j, \text{id}})$ to $\mathcal{R}$.**

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$ for $q \in [Q]$ such that id is not corrupted. For each $i \in \mathbb{I}'_{\text{id}}, j \in \mathbb{J}$,

- If $i = i^*$, **retrieve $(r_j^{(\text{id})})_{j, \text{id}}$ from $\mathcal{R}$. Set $r_{ij} := r_j^{(\text{id})}$.** Sample $\text{ske.ct}_{ij}^* \leftarrow \{0, 1\}^{\ell_{\text{out}}}$, and $\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \text{MAFE.KGen}(\text{id}, \text{msk}_{\text{id}}^{(i,j)}, \{\text{mpk}_{\text{id}_x}^{(i,j)}\}_{\text{id}_x}, \text{GID}, (x, r_{ij}, \text{ske.ct}_{ij}^*))$.
- Otherwise, proceed similarly to **Hyb₁^A(1^λ)**.

Set $\text{SK}_{\text{id}, \text{GID}, x} = (\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)})_{i \in \mathbb{I}'_{\text{id}}, j}$ and send it to $\mathcal{A}$.

Corruption Queries. $\mathcal{A}$ sends (Corr, id) . If $\text{id} \notin \mathcal{K}_{i^*}$, **abort**. Send $(\text{msk}_{\text{id}}^{(i,j)})_{i,j}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends circuit C_0, C_1 . Sample $\{\tilde{C}_i\}_{i \in \mathbb{I}'} \leftarrow \text{HSS.Share}(1^\lambda, 1^{|\mathbb{I}'|}, C_b)$. For each $i \in \mathbb{I}', j \in \mathbb{J}$, sample $\text{ct}^{(i,j)} \leftarrow \text{MAFE.Enc}(\{\text{mpk}_{\text{id}}^{(i)}\}_{\text{id} \notin \mathcal{K}_i}, F_{ij})$. Set $\text{CT} = (\text{ct}^{(i,j)})_{i,j}$ and send it to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$ such that id is not corrupted. For each $i \in \mathbb{I}'_{\text{id}}, j \in \mathbb{J}$, sample $r_{ij} \xleftarrow{\$} \{0, 1\}^\lambda$, $\text{ske.ct}_{ij}^* \leftarrow \{0, 1\}^{\ell_{\text{out}}}$, and $\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \text{MAFE.KGen}(\text{id}, \text{msk}_{\text{id}}^{(i,j)}, \{\text{mpk}_{\text{id}_x}^{(i,j)}\}_{\text{id}_x}, \text{GID}, (x, r_{ij}, \text{ske.ct}_{ij}^*))$. Set $\text{SK}_{\text{id}, \text{GID}, x} = (\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)})_{i \in \mathbb{I}'_{\text{id}}, j}$ and send it to $\mathcal{A}$.

Corruption Queries. $\mathcal{A}$ sends (Corr, id) . If $\text{id} \notin \mathcal{K}_{i^*}$, **abort**. Send $(\text{msk}_{\text{id}}^{(i,j)})_{i,j}$ to $\mathcal{A}$.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb₃^A(1^λ). In this hybrid, we will set $\text{ske.ct}_{i^*j}^*$ to be encryptions of output of F_{i^*j} in the post-challenge phase for i^* . The changes are highlighted in red.

Setup. $\mathcal{A}$ sends $Q, n, s, k, \chi, \ell_1, \dots, \ell_n$. For each $i \in \mathbb{I}, j \in \mathbb{J}$, sample $\text{crs}^{(i,j)} \leftarrow \text{MAFE.GSetup}(1^\lambda, 1^Q, 1^{n-k}, 1^s, 1^{L+\lambda+\ell_{\text{out}}})$. In addition, sample for each $\text{id} \in [n]$, $(\text{mpk}_{\text{id}}^{(i,j)}, \text{msk}_{\text{id}}^{(i,j)}) \leftarrow \text{MAFE.ASetup}(\text{id}, 1^{\ell_{\text{id}}+\lambda+\ell_{\text{out}}})$ for i such that $\text{id} \notin \mathcal{K}_i$. Set $\text{CRS} := (\text{crs}^{(i,j)})_{i,j}$, $\text{MPK}_{\text{id}} := (\text{mpk}_{\text{id}}^{(i,j)})_{i,j}$, and $\text{MSK}_{\text{id}} := (\text{msk}_{\text{id}}^{(i,j)})_{i,j}$ and send $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]})$ to $\mathcal{A}$.

Sample $i^* \xleftarrow{\$} \mathbb{I}'$. Initiate an empty dictionary $\mathcal{R}$. Sample Q many ρ_j for each $j \in \mathbb{J}$. If any of these are repeated, abort and output $\perp$. Otherwise, compute $r_j^{(1)}, \dots, r_j^{(n)}$ such that $\bigoplus_{\text{id}} r_j^{(\text{id})} = \rho_j$ and add $(\text{GID}, (r_j^{(\text{id})})_{j,\text{id}})$ to $\mathcal{R}$. **Sample $\text{ske.sk}^* \leftarrow \text{SKE.Setup}(1^\lambda)$.**

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$ for $q \in [Q]$ such that id is not corrupted. For each $i \in \mathbb{I}'_{\text{id}}, j \in \mathbb{J}$,

- If $i = i^*$, retrieve $(r_j^{(\text{id})})_{\text{id},j}$ from $\mathcal{R}$. Set $r_{ij} := r_j^{(\text{id})}$. **Sample $\text{ske.ct}_{ij}^* \leftarrow \text{SKE.Enc}(\text{ske.sk}^*, r_{ij}^*)$ for random r_{ij}^* whose size is same as output size of F_{i^*j} , and $\text{sk}_{\text{id},\text{GID},x}^{(i,j)} \leftarrow \text{MAFE.KGen}(\text{id}, \text{msk}_{\text{id}}^{(i,j)}, \{\text{mpk}_{\text{id}_x}^{(i,j)}\}_{\text{id}_x}, \text{GID}, (x, r_{ij}, \text{ske.ct}_{ij}^*))$.**
- Otherwise, proceed similarly to **Hyb₂^A(1^λ)**.

Set $\text{SK}_{\text{id},\text{GID},x} = (\text{sk}_{\text{id},\text{GID},x}^{(i,j)})_{i \in \mathbb{I}'_{\text{id}}, j}$ and send it to $\mathcal{A}$.

Corruption Queries. $\mathcal{A}$ sends (Corr, id) . If $\text{id} \notin \mathcal{K}_{i^*}$, abort. Send $(\text{msk}_{\text{id}}^{(i,j)})_{i,j}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends circuit C_0, C_1 . Sample $\{\tilde{C}_i\}_{i \in \mathbb{I}'} \leftarrow \text{HSS.Share}(1^\lambda, 1^{|\mathbb{I}'|}, C_b)$. For each $i \in \mathbb{I}', j \in \mathbb{J}$, sample $\text{ct}^{(i,j)} \leftarrow \text{MAFE.Enc}(\{\text{mpk}_{\text{id}}^{(i,j)}\}_{\text{id} \notin \mathcal{K}_i}, F_{ij})$. Set $\text{CT} = (\text{ct}^{(i,j)})_{i,j}$ and send it to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$ for $q \in [Q]$ such that id is not corrupted. For each $i \in \mathbb{I}'_{\text{id}}, j \in \mathbb{J}$,

- If $i = i^*$, retrieve $(r_j^{(\text{id})})_{\text{id},j}$ from $\mathcal{R}$. Set $r_{ij} := r_j^{(\text{id})}$. **If this is the last authority that will be queried for this GID, retrieve $X = \{x_{\text{id}}\}_{\text{id} \notin \mathcal{K}_{i^*}}$ and $\{r_{i^*j}^{(\text{id}_x)}\}_{\text{id}_x \neq \text{id}}$ which are encrypted in each of $\text{ske.ct}_{i^*j}^*$ for other authorities. Sample $\text{ske.ct}_{ij}^* = \text{SKE.Enc}(\text{ske.sk}^*, \bigoplus_{\text{id}_x \neq \text{id}} r_{ij}^{(\text{id}_x)} \oplus \text{HSS.Eval}(i^*, \tilde{C}_{i^*}, \{X, j\}))$. Otherwise, sample $\text{ske.ct}_{ij}^* \leftarrow \text{SKE.Enc}(\text{ske.sk}^*, r_{ij}^*)$ for random r_{ij}^* whose size is same as output size of F_{i^*j} , and $\text{sk}_{\text{id},\text{GID},x}^{(i,j)} \leftarrow \text{MAFE.KGen}(\text{id}, \text{msk}_{\text{id}}^{(i,j)}, \{\text{mpk}_{\text{id}_x}^{(i,j)}\}_{\text{id}_x}, \text{GID}, (x, r_{ij}, \text{ske.ct}_{ij}^*))$.**
- Otherwise, proceed similarly to **Hyb₂^A(1^λ)**.

Corruption Queries. $\mathcal{A}$ sends (Corr, id) . If $\text{id} \notin \mathcal{K}_{i^*}$, abort. Send $(\text{msk}_{\text{id}}^{(i,j)})_{i,j}$ to $\mathcal{A}$.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Circuit F'_{i^*j}
Input: $x_1, r_1, \text{ske.ct}_1^*, \dots, x_{n-k}, r_{n-k}, \text{ske.ct}_{n-k}^*$ Hardwired: $i, j, \text{ske.sk}^*, \{(\rho_j^{\text{GID}}, y_j)\}_{\text{GID}}$ for each GID queried in pre-challenge secret key queries. Output: If $(\bigoplus_i r_i)$ is any ρ_j^{GID} , output y_j . Otherwise, let $r_i^* = \text{SKE.Dec}(\text{ske.sk}^*, \text{ske.ct}_i^*)$ and output $\bigoplus_i r_i^*$.

Figure 10: Description of circuit F'_{i^*j}

Hyb $_{4,\kappa}^A(1^\lambda)$. For $\kappa \in [2^\chi + 1]$. In this hybrid, we will encrypt F'_{i^*j} for $j \in [\kappa - 1]$ instantiations of MAFE. The changes are highlighted in red.

Setup, Secret Key Queries, Corruption Queries. Same as **Hyb $_3^A(1^\lambda)$** .

Challenge Query. $\mathcal{A}$ sends circuit C_0, C_1 . Sample $\{\tilde{C}_i\}_{i \in \mathbb{I}'} \leftarrow \text{HSS.Share}(1^\lambda, 1^{|\mathbb{I}'|}, C_b)$. For each $i \in \mathbb{I}', j \in \mathbb{J}$, if $i = i^*$, compute $y_j = \text{HSS.Eval}(i^*, \tilde{C}_{i^*}, \{X, j\})$ for each (GID, X) of the pre-challenge queries where for this GID, all $n - k$ authorities are queried and sample $\text{ct}^{(i,j)} \leftarrow \text{MAFE.Enc}(\{\text{mpk}_{\text{id}}^{(i)}\}_{\text{id} \notin \mathcal{K}_i}, F'_{i^*j})$. Otherwise, sample $\text{ct}^{(i,j)} \leftarrow \text{MAFE.Enc}(\{\text{mpk}_{\text{id}}^{(i)}\}_{\text{id} \notin \mathcal{K}_i}, F_{ij})$. Set $\text{CT} = (\text{ct}^{(i,j)})_{i,j}$ and send it to $\mathcal{A}$.

Secret Key Queries, Corruption Queries, Guess Phase. Same as **Hyb $_3^A(1^\lambda)$** .

Hyb $_5^A(1^\lambda)$. In this hybrid, we simulate HSS. The changes are highlighted in red.

Setup, Secret Key Queries, Corruption Queries. Same as **Hyb $_{4,2^\chi+1}^A(1^\lambda)$** .

Challenge Query. $\mathcal{A}$ sends circuit C_0, C_1 . Sample $\{\tilde{C}_i\}_{i \neq i^*, i \in \mathbb{I}'} \leftarrow \text{HSS.Sim}(1^\lambda, 1^{|\mathbb{I}'|}, 1^{|\mathcal{C}_b|}, 1^*)$. For each $i \in \mathbb{I}', j \in \mathbb{J}$, if $i = i^*$, compute $y_j = \text{HSS.Sim}(X, C_b(X))$ for each (GID, X) of the pre-challenge queries where for this GID and sample $\text{ct}^{(i,j)} \leftarrow \text{MAFE.Enc}(\{\text{mpk}_{\text{id}}^{(i)}\}_{\text{id} \notin \mathcal{K}_i}, F'_{i^*j})$. Otherwise, sample $\text{ct}^{(i,j)} \leftarrow \text{MAFE.Enc}(\{\text{mpk}_{\text{id}}^{(i)}\}_{\text{id} \notin \mathcal{K}_i}, F_{ij})$. Set $\text{CT} = (\text{ct}^{(i,j)})_{i,j}$ and send it to $\mathcal{A}$.

Secret Key Queries, Corruption Queries, Guess Phase. Same as **Hyb $_{4,2^\chi+1}^A(1^\lambda)$** except use **HSS.Sim** for the output of HSS evaluation on $\tilde{C}_{i^*}$.

Lemma N.1. $\Pr[1 \leftarrow \text{Hyb}_1^A(1^\lambda)] \geq \Pr[1 \leftarrow \text{Hyb}_0^A(1^\lambda)] / \text{poly}(\lambda)$.

Proof. In **Hyb $_1^A(1^\lambda)$** , we guess an i^* such that each authority in $\mathcal{K}_{i^*}$ is corrupted. As $\mathcal{A}$ corrupts at most k authorities, we can see that our guess is correct with at least $1/\binom{n}{k}$ probability. Hence, in **Hyb $_1^A(1^\lambda)$** we do not abort with at least $1/\binom{n}{k} = 1/\text{poly}(\lambda)$ probability. Thus, the lemma holds. $\square$

Lemma N.2. **Hyb $_1^A(1^\lambda)$** and **Hyb $_2^A(1^\lambda)$** are statistically close.

Proof. In **Hyb $_1^A(1^\lambda)$** , we sample r_{i^*j} for each id independently whereas in **Hyb $_2^A(1^\lambda)$** , we sample a random ρ_{i^*j} and then set r_{i^*j} for each authority as additive secret shares of ρ_{i^*j} . Both of these are identically distributed. In addition, we abort when each ρ_{i^*j} is not unique. As this only happens with negligible probability, the lemma holds. $\square$

Lemma N.3. Assuming the security of SKE, **Hyb $_2^A(1^\lambda)$** and **Hyb $_3^A(1^\lambda)$** are computationally close.

Proof. In both hybrids ske.ct^* is not present in the view of $\mathcal{A}$. Thus, the lemma follows by security of SKE with pseudorandom ciphertexts. Otherwise, we can reduce a distinguishing adversary to the break the security of SKE similarly to Claim K.1. $\square$

Lemma N.4. $\text{Hyb}_3^A(1^\lambda)$ and $\text{Hyb}_{4,1}^A(1^\lambda)$ are identically distributed.

Proof. As we are not encrypting F'_{i^*j} for any j in $\text{Hyb}_{4,1}^A(1^\lambda)$, these hybrids are identically distributed. $\square$

Lemma N.5. Assuming the security of MAFE, $\text{Hyb}_{4,\kappa}^A(1^\lambda)$ and $\text{Hyb}_{4,\kappa+1}^A(1^\lambda)$ for $\kappa \in [2^\lambda]$ are computationally indistinguishable.

Proof. Assume that there exists an adversary $\mathcal{A}$ that can distinguish between $\text{Hyb}_{4,\kappa}^A(1^\lambda)$ and $\text{Hyb}_{4,\kappa+1}^A(1^\lambda)$ with non-negligible probability $\epsilon(\lambda)$, i.e.,

$$\left| \Pr \left[1 \leftarrow \text{Hyb}_{4,\kappa}^A(1^\lambda) \right] - \Pr \left[1 \leftarrow \text{Hyb}_{4,\kappa+1}^A(1^\lambda) \right] \right| > \epsilon(\lambda)$$

We will construct a reduction adversary $\mathcal{B}$ that can break the security of MAFE with non-negligible probability. The description of $\mathcal{B}^\mathcal{O}$ is as follows.

Setup, Secret Key Queries, Corruption Queries. Same as $\text{Hyb}_{4,\kappa}^A(1^\lambda)$ except when generating secret key for (i^*, κ) -th instantiation, ask $\mathcal{O}$ for a secret key for the corresponding input.

Challenge Query. $\mathcal{A}$ sends circuit C_0, C_1 . Sample $\{\tilde{C}_i\}_{i \in \mathbb{I}'} \leftarrow \text{HSS.Share}(1^\lambda, 1^{|\mathbb{I}'|}, C_b)$. For each $i \in \mathbb{I}', j \in \mathbb{J}$, if $i = i^*$, compute $y_j = \text{HSS.Eval}(i^*, \tilde{C}_{i^*}, \{X, j\})$ for each (GID, X) of the pre-challenge queries where for this GID, all $n - k$ authorities are queried and sample

- If $j < \kappa$, $\text{ct}^{(i,j)} \leftarrow \text{MAFE.Enc}(\{\text{mpk}_{\text{id}}^{(i)}\}_{\text{id} \notin \mathcal{K}_i}, F'_{i^*j})$.
- Otherwise, if $j > \kappa$, sample $\text{ct}^{(i,j)} \leftarrow \text{MAFE.Enc}(\{\text{mpk}_{\text{id}}^{(i)}\}_{\text{id} \notin \mathcal{K}_i}, F_{ij})$.
- Otherwise, $\text{ct}^{(i,j)} \leftarrow \mathcal{O}(F_{ij}, F'_{i^*j})$.

Set $\text{CT} = (\text{ct}^{(i,j)})_{i,j}$ and send it to $\mathcal{A}$.

Secret Key Queries, Corruption Queries, Guess Phase. Same as $\text{Hyb}_{4,\kappa}^A(1^\lambda)$ except when generating secret key for (i^*, κ) -th instantiation, ask $\mathcal{O}$ for a secret key for the corresponding input.

As we can see, running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$ and λ . If $\mathcal{O}$ encrypts F'_{i^*j} , $\mathcal{B}$ behaves like $\text{Hyb}_{4,\kappa+1}^A(1^\lambda)$ and if $\mathcal{O}$ encrypts F_{i^*j} , $\mathcal{B}$ behaves like $\text{Hyb}_{4,\kappa}^A(1^\lambda)$. In addition, as long as $\mathcal{B}$ does not abort, it is a successful reduction adversary that can break the security of MAFE with non-negligible probability. $\square$

Lemma N.6. Assuming the security of HSS, $\text{Hyb}_{4,2^\lambda+1}^A(1^\lambda)$ and $\text{Hyb}_5^A(1^\lambda)$ are computationally indistinguishable.

Proof. As the only difference between these hybrids is that we are simulating HSS using output values, these two hybrids can be argued to be computationally indistinguishable similarly to Lemma G.3. $\square$

O Lower Bounds on MAFE with Authority Corruptions

In this section, we provide lower bounds on the efficiency of MAFE schemes that support authority corruptions. Specifically, we show that if an MAFE scheme supports at most χ -bit corruptions, the size of its ciphertexts and master keys *must* grow with 2^χ . We show that any non-trivial efficiency in this regard, i.e, $2^{\chi(1-\xi)}$ for $\xi > 0$, will imply an indistinguishability obfuscation (iO) scheme²⁴. This result implies that Construction 4.3 is optimal in certain regards.

The main idea as explained in overview is that MAFE schemes that satisfy a weaker notion of security such as static security (Definition 0.5) with a non-trivial efficiency guarantee as described above can be bootstrapped to an iO scheme. Here, by static security, we mean that adversary makes secret key and corruption queries before receiving the public parameters of the system.

We show that an MAFE scheme that obeys this definition implies an XiO scheme for $P^{\log}/Poly$ [LPST16] if it supports one GID query and a weakly succinct FE scheme for $P/Poly$ [BV15] if it supports at most Q GID queries. Our implications are formally proved in Section 0.3 and Section 0.4. Note that an XiO scheme for $P^{\log}/Poly$ in conjunction with LWE implies an iO scheme whereas a weakly succinct FE scheme for $P/Poly$ implies it generically. We provide the definitions of these preliminaries in Section 0.1 and formally define statically secure MAFE schemes in Section 0.2.

O.1 Preliminaries

We provide definitions of some preliminaries we use in this section.

Definition 0.1 (iO). An $i\mathcal{O}$ scheme with polynomial-time algorithms $(\text{Obf}, \text{Eval})$ is said to be an indistinguishability obfuscation (iO) for the circuit class $\mathcal{C} = \{\mathcal{C}_\lambda\}_{\lambda \in \mathbb{N}}$ if it satisfies the following properties.

Correctness. For any $\lambda \in \mathbb{N}, x, C \in \mathcal{C}_\lambda$,

$$\Pr[C(x) = \text{Eval}(\tilde{C}, x) : \tilde{C} \leftarrow \text{Obf}(1^\lambda, C)] = 1$$

Indistinguishability. For any stateful PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that $\forall \lambda \in \mathbb{N}, C_0, C_1 \in \mathcal{C}_\lambda, |C_0| = |C_1|, \forall x, C_0(x) = C_1(x)$,

$$\Pr \left[b \leftarrow \mathcal{A}(\tilde{C}) \quad : \quad \begin{array}{l} b \xleftarrow{\$} \{0, 1\}, (C_0, C_1) \leftarrow \mathcal{A}(1^\lambda), \\ \tilde{C} \leftarrow \text{Obf}(1^\lambda, C_b) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda)$$

Definition 0.2 (XiO). An exponentially-efficient indistinguishability obfuscation (XiO) scheme XiO as defined by [LPST16] is an $i\mathcal{O}$ scheme for the circuit class $\mathcal{C} = \{\mathcal{C}_\lambda : \{0, 1\}^{n(\lambda)} \rightarrow \{0, 1\}^{m(\lambda)}\}_{\lambda \in \mathbb{N}}$ that satisfies Definition 0.1 and the following property.

Non-trivial Efficiency. There exists a constant $\xi > 0$ such that for any $\lambda \in \mathbb{N}$, circuit $C \in \mathcal{C}_\lambda$, the probabilistic algorithm Obf runs in time $\text{poly}(\lambda, |C|, 2^n)$ and output size $|\tilde{C}| = \text{poly}(\lambda, |C|) \cdot 2^{n(1-\xi)}$.

²⁴We consider indistinguishability-based security for our schemes in this section and simulation-based security is impossible for such schemes (cf. Remark 0.13).

Definition O.3 (Circuit class $P^{\log}/\text{Poly}$, [LPST16]). $P^{\log}/\text{Poly}$ is a subclass of P/Poly circuits that takes $O(\log \lambda)$ many bits as input. The output of any circuit $C \in P^{\log}/\text{Poly}$ is polynomially bounded. Note that $\mathcal{XiO}$ is efficient for $P^{\log}/\text{Poly}$.

Definition O.4 (sFE, [BV15]). An sFE scheme $(\text{Setup}, \text{Enc}, \text{Dec})$ is said to be a weakly size-succinct functional encryption (sFE) scheme for circuit class $\mathcal{C} = \{\mathcal{C}_\lambda\}_{\lambda \in \mathbb{N}}$ if it satisfies the following properties.

Correctness. For any $\lambda \in \mathbb{N}$, $Q = Q(\lambda)$, and any $C_q \in \mathcal{C}_\lambda$ for $q \in [Q]$, any x ,

$$\Pr \left[y = C_q(x) \quad : \quad \begin{array}{l} (\text{MPK}, \{\text{SK}_q\}_q) \leftarrow \text{Setup}(1^\lambda, \{C_q\}_q), \\ \text{CT} \leftarrow \text{Enc}(\text{MPK}, x), y = \text{Dec}(\text{SK}_q, \text{CT}) \end{array} \right] = 1$$

Weak size-succinctness. The size of the encryption circuit, $|\text{Enc}(\text{MPK}, \cdot)| \leq Q^{1-\xi} \cdot \text{poly}(\lambda, |C|)$ where $\xi > 0$ and $|C|$ is the maximum circuit size for $\mathcal{C}_\lambda$.

Security. For any stateful PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that $\forall \lambda \in \mathbb{N}$,

$$\Pr \left[b = b' \quad : \quad \begin{array}{l} \forall q \in [Q], \{C_q\}_q \leftarrow \mathcal{A}(1^\lambda), b \xleftarrow{\$} \{0, 1\}, \\ (\text{MPK}, \{\text{SK}_q\}_q) \leftarrow \text{Setup}(1^\lambda, \{C_q\}_q), \\ (x_0, x_1) \leftarrow \mathcal{A}(\text{MPK}, \{\text{SK}_q\}_q), \\ \text{CT} \leftarrow \text{Enc}(\text{MPK}, x_b), b' \leftarrow \mathcal{A}(\text{CT}) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda)$$

O.2 Definitions

We provide the definitions of primitives with non-trivial efficiency for MAFE with authority corruptions.

Syntax. A static Q -GID MAFE (stMAFE) scheme for the circuit class $\mathcal{C} = \{\mathcal{C}_\lambda\}_{\lambda \in \mathbb{N}}$ and $\mathcal{GID} = \{\text{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ that can handle corruption of χ bits with non-trivial efficiency consists of algorithms $(\text{GSetup}, \text{ASetup}, \text{KGen}, \text{Enc}, \text{Dec})$ defined similar to Definition 4.1.

Definition O.5 (stMAFE). An stMAFE scheme $(\text{GSetup}, \text{ASetup}, \text{KGen}, \text{Enc}, \text{Dec})$ is said to be an stMAFE scheme if it satisfies the following properties.

Correctness. For any $\lambda \in \mathbb{N}$, $Q = Q(\lambda)$, $n = n(\lambda)$, $s = s(\lambda)$, $k < n$, $\chi = \chi(\lambda)$, $C \in \mathcal{C}_\lambda$, $\text{GID} \in \mathcal{GID}_\lambda$, $\forall \text{id} \in [n]$, $\ell_{\text{id}} = \ell_{\text{id}}(\lambda)$,

$$\Pr \left[\begin{array}{l} C(x_1, \dots, x_n) = \\ \text{Dec}(\{\text{SK}_{\text{id}, \text{GID}, x}\}_{\text{id}}, \text{CT}) \end{array} \quad : \quad \begin{array}{l} \text{CRS} \leftarrow \text{GSetup}(1^\lambda, 1^Q, 1^n, 1^s, 1^k, 1^\chi), \\ \forall \text{id}, (\text{MPK}_{\text{id}}, \text{MSK}_{\text{id}}) \leftarrow \text{ASetup}(\text{id}, 1^{\ell_{\text{id}}}), \\ \text{CT} \leftarrow \text{Enc}(\{\text{MPK}_{\text{id}}\}_{\text{id}}, C) \end{array} \right] = 1$$

where $\text{SK}_{\text{id}, \text{GID}, x} \leftarrow \text{KGen}(\text{id}, \text{MSK}_{\text{id}}, \{\text{MPK}_{\text{id}_x}\}_{\text{id}_x \in [n]}, \text{GID}, x)$.

Non-trivial efficiency. The size of the encryption circuit, $|\text{Enc}(\{\text{MPK}_{\text{id}}\}_{\text{id}}, \cdot)|$ is at most $(Q \cdot 2^\chi)^{1-\xi} \cdot \text{poly}(\lambda, \ell_1, \dots, \ell_n, |C|)$.

Static IND security. For any stateful PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that $\forall \lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \left(Q, n, s, k, \chi, \mathcal{K}, \{\ell_{\text{id}}\}_{\text{id} \in [n]}, \right) \leftarrow \mathcal{A}(1^\lambda), \\ \forall \text{id}, q, (\text{MPK}_{\text{id}}, \text{MSK}_{\text{id}}) \leftarrow \text{ASetup}(\text{id}, 1^{\ell_{\text{id}}}), \\ \text{SK}_{\text{id},q} \leftarrow \text{KGen}(\text{id}, \text{MSK}_{\text{id}}, \{\text{MPK}_{\text{id}_x}\}_{\text{id}_x}, \text{GID}_q, x_{q,\text{id}}), \\ (C_0, C_1) \leftarrow \mathcal{A} \left(\begin{array}{l} \text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}, \\ \{\text{MSK}_{\text{id}}\}_{\text{id} \in \mathcal{K}}, \{\text{SK}_{\text{id},q}\}_{q,\text{id}} \end{array} \right), \\ b \xleftarrow{\$} \{0, 1\}, \text{CT} \leftarrow \text{Enc}(\{\text{MPK}_{\text{id}}\}_{\text{id}}, C_b) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda)$$

where $\text{CRS} \leftarrow \text{GSetup}(1^\lambda, 1^Q, 1^n, 1^s, 1^k, 1^\chi)$ and KGen is defined similarly. (C_0, C_1) are such that for any $\text{GID} \in \{\text{GID}_q\}, x \in \mathbf{X}_{\text{GID},1} \times \dots \times \mathbf{X}_{\text{GID},n}$, $C_0(x) = C_1(x)$ where $\mathbf{X}_{\text{GID},\text{id}} = \{0, 1\}^{\ell_{\text{id}}}$ if $\text{id} \in \mathcal{K}$ and $\mathbf{X}_{\text{GID},\text{id}} = \{x_{\text{GID},\text{id}}\}$ otherwise. In addition, $\sum_{\text{id} \in \mathcal{K}} \ell_{\text{id}} \leq \chi$.

Definition O.6 (fst1MAFE). A fst1MAFE scheme $(\text{GSetup}, \text{ASetup}, \text{KGen}, \text{Enc}, \text{Dec})$ is said to be a fully static fixed-corruptible 1-GID MAFE scheme for circuit class $\mathcal{C} = \{\mathcal{C}_\lambda\}_{\lambda \in \mathbb{N}}$ and a single GID if in the above definition, $Q = 1$ (and we ignore it in syntax) and $\mathcal{A}$ sends challenge circuits (C_0, C_1) along with secret key queries. In addition, we make the following change to efficiency.

Non-trivial efficiency. The sizes of CRS , MSK_{id} , MPK_{id} for any $\text{id} \in [n]$, $\text{SK}_{\text{id},x}, x \in \{0, 1\}^{\ell_{\text{id}}}$, and CT for the circuit C are at most $\text{poly}(\lambda, \ell_1, \dots, \ell_n, |C|) \cdot 2^{\chi(1-\xi)}$ where $\xi > 0$. In addition, the running times of $\text{GSetup}, \text{ASetup}, \text{KGen}, \text{Enc}$ are at most $\text{poly}(\lambda, \ell_1, \dots, \ell_n, |C|, 2^\chi)$.

O.3 XiO from fst1MAFE

Construction O.7 (XiO). We provide the construction of an XiO scheme (Definition O.2) for $\text{P}^{\log}/\text{Poly}$ circuits (Definition O.3) $\mathcal{C} = \{\mathcal{C}_\lambda : \{0, 1\}^{n(\lambda)} \rightarrow \mathcal{O}_\lambda\}_{\lambda \in \mathbb{N}}$ from an fst1MAFE scheme for P/Poly circuits (Definition O.6) that can handle n corruptions as follows.

Obf($1^\lambda, C$). Let $C : \{0, 1\}^{n(\lambda)} \rightarrow \mathcal{O}_\lambda$ for $n(\lambda) = O(\log \lambda)$. Sample an fst1MAFE instantiation with $N = 2$ authorities, $k = 1$ corrupts where $\ell_1 = n, \ell_2 = \text{poly}(\lambda), \chi = \ell_1$,

$\text{crs} \leftarrow \text{fst1MAFE.GSetup}(1^\lambda, 1^N, 1^{s'}, 1^n, 1^k, 1^\chi)$. $\forall \text{id} \in \{1, 2\}, (\text{mpk}_{\text{id}}, \text{msk}_{\text{id}}) \leftarrow \text{fst1MAFE.ASetup}(\text{id}, 1^{\ell_{\text{id}}})$.

Sample a secret key for a random $\alpha \in \{0, 1\}^{\ell_2}$, $\text{sk}_{2,\alpha} \leftarrow \text{fst1MAFE.KGen}(2, \text{msk}_2, \alpha)$. Now, generate a ciphertext for the circuit C' of size $s' = \Theta(|C|)$, $\text{ct} \leftarrow \text{fst1MAFE.Enc}(\text{mpk}_1, \text{mpk}_2, C')$. C' simply ignores the second input.

Output $\tilde{C} := (\text{crs}, \text{mpk}_1, \text{mpk}_2, \text{msk}_1, \text{sk}_{2,\alpha}, \text{ct})$.

Eval($\tilde{C}, x$). Parse $\tilde{C}$ as $(\text{crs}, \text{mpk}_1, \text{mpk}_2, \text{msk}_1, \text{sk}_{2,\alpha}, \text{ct})$. Run KGen for x , $\text{sk}_{1,x} \leftarrow \text{fst1MAFE.KGen}(1, \text{msk}_1, \{\text{mpk}_{\text{id}}\}_{\text{id}}, x)$.

Output $y := \text{fst1MAFE.Dec}(\text{sk}_{1,x}, \text{sk}_{2,\alpha}, \text{ct})$.

Correctness. This follows from the correctness of fst1MAFE.

Non-trivial Efficiency. As all algorithms of fst1MAFE used in Obf run in $\text{poly}(\lambda, n, \ell_2, |C|, 2^n)$, running time of Obf is $\text{poly}(\lambda, n, \ell_2, |C|, 2^n) \leq \text{poly}(\lambda, |C|, |C|, 2^n) = \text{poly}(\lambda, |C|, 2^n)$. The size of $\tilde{C}$ can be similarly shown to be $\text{poly}(\lambda, |C|) \cdot 2^{n(1-\xi')}$ where $\xi' > 0$.

Theorem O.8. If fst1MAFE is a fully static 1-GID MAFE scheme for P/Poly circuits that can handle $O(\log \lambda)$ corruptions (Definition O.6), then Construction O.7 is an XiO scheme for $\text{P}^{\log}/\text{Poly}$ circuits (Definition O.2).

Proof. We prove the security of Construction O.7 by showing a reduction from an adversary that breaks XiO security to an adversary that breaks fst1MAFE security. Assume that there exists an adversary $\mathcal{A}$ that runs in time t against XiO that can distinguish between $\tilde{C}_0$ and $\tilde{C}_1$ with advantage ϵ . Here, C_0 and C_1 are such that $|C_0| = |C_1|$ and $\forall x, C_0(x) = C_1(x)$. We prove the security of XiO by constructing a reduction $\mathcal{B}$ between $\mathcal{A}$ and a challenger Chal for fst1MAFE for P/Poly circuits. The description of $\mathcal{B}$ is as follows.

$\mathcal{B}^{\text{Chal}}(1^\lambda)$. $\mathcal{A}$ sends C_0, C_1 to $\mathcal{B}$. Let $C_0, C_1 : \{0, 1\}^n \rightarrow \mathcal{O}_\lambda$ for $n = O(\log \lambda)$. Set $\ell_2 := \text{poly}(\lambda), N = 2, k = 1, \chi = n$, and sample $\alpha \xleftarrow{\$} \{0, 1\}^{\ell_2}$. Send $N, s', k, \chi, \mathcal{K} = \{1\}, \alpha, (C'_0, C'_1)$ to Chal . Parse the information sent by Chal and set $\tilde{C} := (\text{crs}, \text{mpk}_1, \text{mpk}_2, \text{msk}_1, \text{sk}_{2,\alpha}, \text{ct})$. Run $\mathcal{A}$ on $\tilde{C}$. Output whatever $\mathcal{A}$ outputs. Note that the running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$ and λ . In addition, $\mathcal{B}$ acts as a valid adversary for Chal and a valid challenger for $\mathcal{A}$. Advantage of $\mathcal{B}$ is exactly ϵ . $\square$

Thus, if we have a sub-exponentially secure fst1MAFE for P/Poly circuits, we can build a sub-exponentially secure XiO scheme for $\text{P}^{\log}/\text{Poly}$ circuits. The following theorem from [LPST16] shows that we can build sub-exponentially secure $i\mathcal{O}$ for P/Poly circuits from XiO for $\text{P}^{\log}/\text{Poly}$.

Theorem O.9 ([LPST16]). Assuming the existence of sub-exponentially secure XiO for $\text{P}^{\log}/\text{Poly}$ and sub-exponentially secure LWE , there exists a sub-exponentially secure $i\mathcal{O}$ scheme for P/Poly circuits (Definition O.1).

O.4 Weakly Size-Succinct FE from stMAFE

Construction O.10 (sFE). We provide the construction of a weakly size-succinct FE scheme for P/Poly circuits (Definition O.4) from an stMAFE scheme (Definition O.5) for P/Poly circuits and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ as follows.

$\text{Setup}(1^\lambda, \{C_q\}_{q \in [Q]})$. W.l.o.g., assume that all circuits are of size s and output size $m = m(\lambda)$. We will use $\ell_1 = \lceil \log m \rceil, \ell_2 = s, N = 2, k = 1, \chi = \ell_1, \mathcal{K} := \{1\}$ and s' to be the size of the circuit from Enc .

Generate an stMAFE instantiation, $\text{crs} \leftarrow \text{stMAFE.GSetup}(1^\lambda, 1^Q, 1^N, 1^{s'}, 1^k, 1^\chi)$. For each $\text{id} = \{1, 2\}$, $(\text{mpk}_{\text{id}}, \text{msk}_{\text{id}}) \leftarrow \text{stMAFE.ASetup}(\text{id}, 1^{\ell_{\text{id}}})$.

Sample Q GIDs randomly from $\mathcal{GID}_\lambda$. That is $\forall q \in [Q], \text{GID}_q \leftarrow \mathcal{GID}_\lambda$. Sample secret keys for the $\text{id} = 2$, for $\text{GID}_q, C_q, \text{sk}_{2,q,C_q} \leftarrow \text{KGen}(2, \text{msk}_2, \text{GID}_q, C_q)$.

Output $\text{MPK} := (\text{crs}, \text{mpk}_1, \text{mpk}_2)$ and $\text{SK}_q := (\text{MPK}, \text{msk}_1, \text{GID}_q, C_q, \text{sk}_{2,q,C_q})$.

Enc(MPK, x). Parse MPK as $(\text{crs}, \{\text{mpk}_{\text{id}}\}_{\text{id}})$. Construct the circuit $F = F(x, \cdot, \cdot)$. Here, $F(x, i, C)$ outputs the i -th bit of $C(x)$ for $i \in [m]$. Note that the size of circuit F is $O(s^2)$. Generate ciphertext for F using stMAFE , $\text{ct} \leftarrow \text{stMAFE.Enc}(\text{mpk}_1, \text{mpk}_2, F)$ and output $\text{CT} := \text{ct}$.

Dec(SK, CT). Parse CT as ct and SK as $(\text{crs}, \text{mpk}_1, \text{mpk}_2, \text{msk}_1, \text{GID}_q, C_q, \text{sk}_{2,q,C})$. Run stMAFE key generation m times for $\text{id} = 1$, for $\text{sk}_{1,q,i} \leftarrow \text{stMAFE.KGen}(1, \text{msk}_1, \{\text{mpk}_{\text{id}}\}_{\text{id}}, \text{GID}_q, i)$. Output $y := (y_1, \dots, y_m)$ where $\forall i \in [m], y_i = \text{stMAFE.Dec}(\text{sk}_{1,q,i}, \text{sk}_{2,q,C}, \text{ct})$.

Correctness. This follows from the correctness of stMAFE .

Weak size-succinctness. The size of $\text{Enc}(\text{MPK}, \cdot)$ is at most

$$|F| + |\text{stMAFE.Enc}(\{\text{mpk}_{\text{id}}\}_{\text{id}}, \cdot)| \leq s^2 + (Q2^{\lceil \log m \rceil})^{1-\xi} \cdot \text{poly}(\lambda, s, \lceil \log m \rceil, s) \\ \leq Q^{1-\xi} \text{poly}(\lambda, s)$$

Theorem O.11. If stMAFE is a secure static fixed-corruptible Q -GID MAFE scheme for P/Poly circuits and $\mathcal{GID}$ (Definition O.5), then Construction O.10 is a secure weakly size-succinct FE scheme for P/Poly circuits.

Proof. We prove the security of Construction O.10 by showing a reduction between any adversary that breaks the security of sFE and a challenger for stMAFE . Assume that there exists a t -time adversary $\mathcal{A}$ that runs in time t against sFE as per Definition O.4 with advantage ϵ . We prove the security of sFE by constructing a reduction $\mathcal{B}$ between $\mathcal{A}$ and a challenger Chal for stMAFE for P/Poly circuits. The description of $\mathcal{B}$ is as follows.

$\mathcal{B}^{\text{Chal}}(1^\lambda)$. $\mathcal{A}$ sends size- s circuits with output size m , $\{C_q\}_{q \in [Q]}$. Set $\ell_1 := \lceil \log m \rceil, \ell_2 = s, N = 2, k = 1, \chi = \ell_1, \mathcal{K} = \{1\}$, and $s' = O(s^2)$. Sample $\text{GID}_q \xleftarrow{\$} \mathcal{GID}_\lambda$ and sets $x_{2,q} := C_q$ for $q \in [Q]$. Send $Q, N, s', k, \chi, \ell_1, \ell_2, \mathcal{K}, \{\text{GID}_q, \{x_{2,q}\}\}_q$ to Chal . Receive $(\text{crs}, \text{mpk}_1, \text{mpk}_2, \text{msk}_1, \{\text{sk}_{2,q,C_q}\}_q)$ from Chal . Set $\text{MPK} := (\text{crs}, \text{mpk}_1, \text{mpk}_2)$, $\text{SK}_q := (\text{MPK}, \text{msk}_1, \text{GID}_q, C_q, \text{sk}_{2,q,C_q})$, and send $(\text{MPK}, \{\text{SK}_q\}_q)$ to $\mathcal{A}$. $\mathcal{A}$ sends (x_0, x_1) . Construct $F_0 = F_0(x_0, \cdot, \cdot)$, $F_1 = F_1(x_1, \cdot, \cdot)$ where $F_b(x_b, i, C)$ outputs the i -th bit of $C(x_b)$ for $b \in \{0, 1\}$. Send (F_0, F_1) to Chal . Receive CT from Chal and run $\mathcal{A}$ on CT. Output whatever $\mathcal{A}$ outputs. Note that the running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$ and λ . In addition, $\mathcal{B}$ acts as a valid adversary for Chal and a valid challenger for $\mathcal{A}$. Advantage of $\mathcal{B}$ is exactly ϵ . $\square$

Thus, if we have a sub-exponentially secure fst1MAFE for P/Poly circuits, we can build a sub-exponentially secure sFE for P/Poly circuits. The following theorem from [BV15] shows that we can build sub-exponentially secure iO for P/Poly circuits from sFE for P/Poly circuits.

Theorem O.12 ([BV15]). Assuming the existence of sub-exponentially secure sFE for P/Poly circuits, there exists a sub-exponentially secure iO scheme for P/Poly circuits.

Remark O.13 (Impossibilities for SIM-security). By an incompressibility argument similar to [BSW11, AGVW13], SIM-secure fst1MAFE and stMAFE are impossible in the standard model.

P Distributed Functional Tag System

In this section, we will construct a distributed functional tag system scheme (dFTS) that we use in our adaptive MA-ABE construction. This is a generalization of the functional tag system [WW24] in which the dummy functional tags can be split across multiple authorities. The formal definition is provided in Appendix P.1. We construct this scheme from one-way functions and specifically using somewhere-equivocal PRFs [HJO⁺16] and blind garbled circuits [BLSV18, WW24]. Both of these primitives can be constructed from one-way functions and are defined in Appendix P.2.

P.1 Definition

In this section, we provide the definition of a distributed functional tag system for policy class $\{\mathcal{P}_s\}_{s \in \mathbb{N}}$ which extends the notion of functional tag system to n distinct authorities.

Syntax. A distributed functional tag system dFTS for policy class $\{\mathcal{P}_s\}_{s \in \mathbb{N}}$ consists of the following polynomial time algorithms.

GSetup $(1^\lambda, 1^n, 1^s, 1^L) \rightarrow \text{CRS}$. The possibly randomized global setup algorithm takes as input security parameter λ , number of authorities n , maximum policy size s , maximum input size of each authority L , and outputs common reference string CRS. The following algorithms take CRS as an implicit input.

DFuncTag $(P) \rightarrow \text{tg}_P$. The probabilistic dummy function tag generation algorithm takes as input policy $P \in \mathcal{P}_s$ and outputs dummy function tag tg_P .

DInpTag $(\text{id}, \text{GID}, x) \rightarrow \text{tg}_{\text{id}, \text{GID}, x}$. The probabilistic dummy input tag generation algorithm takes as input authority identifier id , a $\text{GID} \in \mathcal{GID}_\lambda$, input $x \in \{0, 1\}^{\ell_{\text{id}}}$, and outputs dummy input tag $\text{tg}_{\text{id}, \text{GID}, x}$.

KGen $(\text{CRS}) \rightarrow \text{MSK}$. The probabilistic master secret key generation algorithm takes as input CRS, and outputs master secret key MSK.

SFuncTag $(\text{MSK}, P) \rightarrow \text{tg}_P$. The probabilistic smart function tag generation algorithm takes as input master secret key MSK, policy $P \in \mathcal{P}_s$, and outputs smart function tag tg_P .

SInpTag $(\text{MSK}, (1, \text{GID}, x_1), \dots, (n, \text{GID}, x_n)) \rightarrow (\text{tg}_{1, \text{GID}, x_1}, \dots, \text{tg}_{n, \text{GID}, x_n})$. The probabilistic smart input tag generation algorithm takes as input master secret key MSK, inputs to all authorities for a $\text{GID} \in \mathcal{GID}_\lambda$, $(1, \text{GID}, x_1), \dots, (n, \text{GID}, x_n)$, and outputs smart input tags $(\text{tg}_{1, \text{GID}, x_1}, \dots, \text{tg}_{n, \text{GID}, x_n})$.

Trig $(\text{tg}_P, \{\text{tg}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}) \rightarrow 0/1$. The deterministic triggering algorithm, takes as input function tag tg_P , input tags for all authorities $\{\text{tg}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}$, and outputs either 0 (reject) or 1 (accept).

Definition P.1 (dFTS). A dFTS scheme (GSetup, DFuncTag, DInpTag, KGen, SFuncTag, SInpTag) is said to be a distributed functional tag system for policy class $\{\mathcal{P}_s\}_{s \in \mathbb{N}}$ and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ if it satisfies the following properties.

Dummy Correctness. For any $\lambda \in \mathbb{N}$, $n = n(\lambda)$, $s = s(\lambda)$, $\forall \text{id} \in [n]$, $\ell_{\text{id}} = \ell_{\text{id}}(\lambda)$, $P \in \mathcal{P}_s$, $\text{GID} \in \mathcal{GID}_\lambda$, $x_{\text{id}} \in \{0, 1\}^{\ell_{\text{id}}}$

$$\Pr \left[1 = \text{Trig}(\text{tg}_P, \{\text{tg}_{\text{id}, \text{GID}, x_{\text{id}}} \}_{\text{id} \in [n]}) : \begin{array}{l} \text{CRS} \leftarrow \text{GSetup}(1^\lambda, 1^n, 1^s, 1^L), \\ \text{tg}_P \leftarrow \text{DFuncTag}(P), \forall \text{id} \in [n], \\ \text{tg}_{\text{id}, \text{GID}, x_{\text{id}}} \leftarrow \text{DInpTag}(\text{id}, \text{GID}, x_{\text{id}}) \end{array} \right] \leq \text{negl}(\lambda)$$

Smart Correctness. For any $\lambda \in \mathbb{N}$, $n = n(\lambda)$, $s = s(\lambda)$, $\forall \text{id} \in [n]$, $\ell_{\text{id}} = \ell_{\text{id}}(\lambda)$, $P \in \mathcal{P}_s$, $\text{GID} \in \mathcal{GID}_\lambda$, $x_{\text{id}} \in \{0, 1\}^{\ell_{\text{id}}}$, if $P(x_1, \dots, x_n) = 1$,

$$\Pr \left[1 = \text{Trig}(\text{tg}_P, \{\text{tg}_{\text{id}, \text{GID}, x_{\text{id}}} \}_{\text{id} \in [n]}) : \begin{array}{l} \text{CRS} \leftarrow \text{GSetup}(1^\lambda, 1^n, 1^s, 1^L), \\ \text{MSK} \leftarrow \text{KGen}(\text{CRS}), \\ \text{tg}_P \leftarrow \text{SFuncTag}(\text{MSK}, P), \forall \text{id} \in [n], \\ \{\text{tg}_{\text{id}, \text{GID}, x_{\text{id}}} \}_{\text{id}} \leftarrow \text{SInpTag}(\text{MSK}, \\ (1, \text{GID}, x_1), \dots, (n, \text{GID}, x_n)) \end{array} \right] = 1$$

Mode Indistinguishability. For any stateful PPT admissible adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that $\forall \lambda \in \mathbb{N}$,

$$\left| \Pr \left[b \leftarrow \mathcal{A}^{\mathcal{O}_b(\cdot)}(\text{tg}_{P^*}) : \begin{array}{l} b \xleftarrow{\$} \{0, 1\}, (n, s, \ell_1, \dots, \ell_n) \leftarrow \mathcal{A}(1^\lambda), \\ L = \max_{\text{id}} \{\ell_{\text{id}}\}, \text{CRS} \leftarrow \text{GSetup}(1^\lambda, 1^n, 1^s, 1^L), \\ \text{if } b = 1, \text{MSK} \leftarrow \text{KGen}(\text{CRS}), \\ P^* \leftarrow \mathcal{A}^{\mathcal{O}_b(\cdot)}(\text{CRS}), \\ \text{if } b = 0, \text{tg}_{P^*} \leftarrow \text{DFuncTag}(P^*), \\ \text{if } b = 1, \text{tg}_{P^*} \leftarrow \text{SFuncTag}(\text{MSK}, P^*) \end{array} \right] - \frac{1}{2} \right| \leq \text{negl}(\lambda)$$

where $\mathcal{O}_0$ and $\mathcal{O}_1$ respond to queries of the form $(\text{GID}, x_1, \dots, x_n) \in \mathcal{GID}_\lambda \times \{0, 1\}^{\ell_1} \times \dots \times \{0, 1\}^{\ell_n}$. $\mathcal{O}_0$ computes $\forall \text{id} \in [n]$, $\text{tg}_{\text{id}, \text{GID}, x_{\text{id}}} \leftarrow \text{DInpTag}(\text{id}, \text{GID}, x_{\text{id}})$ whereas $\mathcal{O}_1$ uses MSK to generate $\{\text{tg}_{\text{id}, \text{GID}, x_{\text{id}}} \}_{\text{id} \in [n]} \leftarrow \text{SInpTag}(\text{MSK}, (\text{id}, \text{GID}, x_{\text{id}})_{\text{id} \in [n]})$.

A stateful PPT machine is said to be admissible if it makes at most one function tag query with P^* whose size is at most s and for each input tag query, $P^*(x_1, \dots, x_n) = 0$.

P.2 Preliminaries

In this section, we define the preliminaries we use for our construction. Specifically, we define blind garbled circuits and somewhere-equivocal PRFs.

Somewhere Equivocal PRFs. A somewhere-equivocal PRF (SEPRF) for input size λ consists of the following polynomial-time algorithms.

Setup $(1^\lambda, 1^\ell) \rightarrow \text{key}$. The probabilistic setup algorithm takes as input security parameter λ , output size ℓ , and outputs key key .

Sim $_1(1^\lambda, 1^\ell, x^*) \rightarrow \text{eqKey}$. The probabilistic equivocal key generation algorithm takes as input security parameter λ , output size ℓ , input $x^* \in \{0, 1\}^\lambda$ and outputs equivocation key eqKey .

Sim $_2(\text{eqKey}, y^*) \rightarrow \text{key}$. The possibly randomized simulated key generation algorithm takes as input equivocation key eqKey , output $y^* \in \{0, 1\}^\ell$, and outputs key key .

$\text{Eval}(\text{key}, x) \rightarrow y$. The deterministic evaluation algorithm takes as input key key , input $x \in \{0, 1\}^\lambda$, and outputs value $y \in \{0, 1\}^\ell$.

Definition P.2. An SEPRF scheme $(\text{Setup}, \text{Sim}_1, \text{Sim}_2, \text{Eval})$ is said to be a somewhere equivocal PRF scheme (SEPRF) if it satisfies the following properties.

Correctness. For any $\lambda \in \mathbb{N}, \ell = \ell(\lambda), x^* \in \{0, 1\}^\lambda, y^* \in \{0, 1\}^\ell$, and for any $x \neq x^*, \text{Eval}(\text{key}, x) = \text{Eval}(\text{eqKey}, x)$. In addition, $\text{Eval}(\text{key}, x^*) = y^*$. Here, $\text{eqKey} \leftarrow \text{Sim}_1(1^\lambda, 1^\ell, x^*)$ and $\text{key} \leftarrow \text{Sim}_2(\text{eqKey}, y^*)$.

Security. For any stateful PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that $\forall \lambda \in \mathbb{N}$,

$$\left| \Pr \left[\begin{array}{l} b \leftarrow \mathcal{A}(\text{key}_b) \quad : \quad \begin{array}{l} b \xleftarrow{\$} \{0, 1\}, (\ell, x^*) \leftarrow \mathcal{A}(1^\lambda), \\ \text{if } b = 0, \text{key}_0 \leftarrow \text{Setup}(1^\lambda, 1^\ell), \\ \text{if } b = 1, \text{eqKey} \leftarrow \text{Sim}_1(1^\lambda, 1^\ell, x^*), \\ y^* \xleftarrow{\$} \{0, 1\}^\ell, \text{key}_1 \leftarrow \text{Sim}_2(\text{eqKey}, y^*) \end{array} \end{array} \right] - \frac{1}{2} \right| \leq \text{negl}(\lambda)$$

Theorem P.3 ([HJO⁺16]). Assuming the existence of one-way functions, there exists an SEPRF scheme for any positive polynomial $\ell(\lambda)$.

Blind Garbled Circuits. A blind garbled circuit scheme is a strengthening of a garbled circuit scheme (Definition A.1) with the additional property that the simulated garbled circuit is a uniformly random string as long as the circuit's output is also uniformly random. We also include a reusable property as provide in [WW24]. Specifically, a blind garbled circuit scheme for a class of circuit $\mathcal{C} = \{\mathcal{C}_s\}_{s \in \mathbb{N}}$ with λ -bit output and at most size s consists of the following polynomial-time algorithms.

$\text{Setup}(1^\lambda, 1^s, 1^L) \rightarrow \text{SK}$. The probabilistic setup algorithm takes as input security parameter λ , maximum circuit size s , and input size L , outputs secret key SK .

$\text{InpGarb}(\text{SK}, x) \rightarrow \tilde{x}$. The deterministic input garbling algorithm takes as input secret key SK , circuit $x \in \{0, 1\}^L$ output garbled input $\tilde{x}$.

$\text{CircGarb}(\text{SK}, C) \rightarrow \tilde{C}$. The probabilistic circuit garbling algorithm takes as input secret key SK , circuit $C \in \mathcal{C}_s$, and outputs garbled circuit $\tilde{C}$.

$\text{Eval}(\tilde{C}, \tilde{x}) \rightarrow y$. The deterministic evaluation algorithm takes as input garbled circuit $\tilde{C}$, garbled input $\tilde{x}$, and outputs value y .

$\text{Sim}(\text{SK}, x, C(x)) \rightarrow \tilde{C}$. The probabilistic circuit garbling simulation algorithm takes as input secret key SK , input x , output value $C(x)$, and outputs garbled circuit $\tilde{C}$.

Definition P.4. A blind garbled circuit scheme $(\text{Setup}, \text{InpGarb}, \text{CircGarb}, \text{Eval})$ is said to be a blind garbled circuit scheme for circuit class $\mathcal{C} = \{\mathcal{C}_s\}_{s \in \mathbb{N}}$, if it satisfies the following properties.

Correctness. For any $\lambda \in \mathbb{N}, s = s(\lambda), L = L(\lambda), C \in \mathcal{C}_s, x \in \{0, 1\}^L, C(x) = \text{Eval}(\tilde{C}, \tilde{x})$ where $\tilde{C} \leftarrow \text{CircGarb}(\text{SK}, C), \tilde{x} \leftarrow \text{InpGarb}(\text{SK}, x)$, and $\text{SK} \leftarrow \text{Setup}(1^\lambda, 1^s, 1^L)$.

Security. For any stateful admissible PPT adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for any $\lambda \in \mathbb{N}$,

$$\left| \Pr \left[\begin{array}{l} b \leftarrow \mathcal{A}^{\text{CircGarb}(\text{SK}, \cdot)}(\tilde{C}) : \begin{array}{l} b \leftarrow \{0, 1\}, (s, L) \leftarrow \mathcal{A}(1^\lambda), \\ \text{SK} \leftarrow \text{Setup}(1^\lambda, 1^s, 1^L), \\ x \leftarrow \mathcal{A}^{\text{CircGarb}(\text{SK}, \cdot)}(1^\lambda), \\ \tilde{x} \leftarrow \text{InpGarb}(\text{SK}, x), \\ C \leftarrow \mathcal{A}^{\text{CircGarb}(\text{SK}, \cdot)}(\tilde{x}), \\ \text{if } b = 0, \tilde{C} \leftarrow \text{CircGarb}(\text{SK}, C), \\ \text{if } b = 1, \tilde{C} \leftarrow \text{Sim}(\text{SK}, x, C(x)) \end{array} \end{array} \right] - \frac{1}{2} \right| \leq \text{negl}(\lambda)$$

A stateful PPT machine $\mathcal{A}$ is said to be admissible if it makes at most one challenge input garbling and circuit garbling queries.

Blindness. For any $\lambda \in \mathbb{N}, s = s(\lambda), L = L(\lambda), C \in \mathcal{C}_s, x \in \{0, 1\}^L$,

$$\left\{ \tilde{C} : y \xleftarrow{\$} \{0, 1\}^\ell, \tilde{C} \leftarrow \text{Sim}(\text{SK}, x, y) \right\} \equiv \left\{ \tilde{C} : \tilde{C} \xleftarrow{\$} \{0, 1\}^\ell \right\}$$

where $\text{SK} \leftarrow \text{Setup}(1^\lambda, 1^s, 1^L)$ and ℓ is the length of the garbled circuit of C .

Theorem P.5 ([BLSV18, WW24]). Assuming the existence of one-way functions, there exists a blind garbled circuit scheme for any positive polynomial s, L .

P.3 Construction

In this section, we construct a distributed functional tag system for any circuit class $\mathcal{C} = \{\mathcal{C}_s\}_{s \in \mathbb{N}}$. The main idea is that as dummy input tags of [WW24] are uniform random strings, we can secret share them among multiple authorities. We leverage this idea to construct distributed FTS below.

Construction P.6. We provide the construction of a dFTS scheme for circuit class $\mathcal{C} = \{\mathcal{C}_s\}_{s \in \mathbb{N}}$ and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ using a blind garbled circuit scheme (Garb) for P/Poly circuits (Definition P.4) and SEPRF scheme (Definition P.2) as follows.

GSetup($1^\lambda, 1^n, 1^s, 1^L$). Let $s' = \text{poly}(\lambda, n, s, L)$ be the size of circuit from Figure 11. Let L' be the size of garbled circuit using blind garbled circuit scheme for the circuit from Figure 11 and $\sigma = \text{poly}(s)$ be the description size of circuits of size s . Output $\text{CRS} := (\lambda, n, s, L, s', L', \sigma)$.

DFuncTag(P). Sample $\text{garb.sk} \leftarrow \text{Garb.Setup}(1^\lambda, 1^{s'}, 1^\sigma)$, for each $\text{id} \in [n]$, $\text{key}_{\text{id}} \leftarrow \text{SEPRF.Setup}(1^\lambda, 1^L)$. Compute $\tilde{P} = \text{Garb.InpGarb}(\text{garb.sk}, P)$. Output $\text{tg}_P := (\text{key}_1, \dots, \text{key}_n, \tilde{P})$.

DInpTag(id, GID, x). Sample $t_0 \xleftarrow{\$} \{0, 1\}^\lambda, t_1 \xleftarrow{\$} \{0, 1\}^{L'}, t_2 \xleftarrow{\$} \{0, 1\}^\lambda$ where L is defined similarly to DFuncTag. Output $\text{tg}_{\text{id}, \text{GID}, x} := (t_0, t_1, t_2)$.

KGen(CRS). Sample $\text{garb.sk} \leftarrow \text{Garb.Setup}(1^\lambda, 1^{s'}, 1^\sigma)$, for each $\text{id} \in [n]$, $\text{key}_{\text{id}} \leftarrow \text{SEPRF.Setup}(1^\lambda, 1^L)$. Output $\text{MSK} := (\text{key}_1, \dots, \text{key}_n, \text{garb.sk})$.

SFuncTag(MSK, P). Parse MSK as $(\text{key}_1, \dots, \text{key}_n, \text{garb.sk})$. Compute $\tilde{P} = \text{Garb.InpGarb}(\text{garb.sk}, P)$. Output $\text{tg}_P := (\text{key}_1, \dots, \text{key}_n, \tilde{P})$.

SInpTag(MSK, (1, GID₁, x₁), ..., (n, GID_n, x_n)). If all GID_{id} are not same for all id ∈ [n], abort and output ⊥. Otherwise, let GID := GID₁. Parse MSK as (key₁, ..., key_n, garb.sk).

Sample $t_0^{(1)}, \dots, t_0^{(n)}, t_2^{(1)}, \dots, t_2^{(n)} \xleftarrow{\$} \{0, 1\}^\lambda$. Let $a := \bigoplus_{id \in [n]} t_2^{(id)}$. Sample $r \xleftarrow{\$} \{0, 1\}^\lambda$. Construct the circuit F as defined in Figure 11. Garble F , $\tilde{F} \leftarrow \text{Garb.CircGarb}(\text{garb.sk}, F)$.

Sample $i \xleftarrow{\$} [n]$ and $\{\text{sh}_j\}_{j \in [n], j \neq i} \xleftarrow{\$} \{0, 1\}^{L'}$. Set $\text{sh}_i := (\bigoplus_{j \neq i} \text{sh}_j) \oplus \tilde{F}$.

Compute for each id ∈ [n], $t_1^{(id)} = \text{SEPRF.Eval}(\text{key}_{id}, t_0^{(id)}) \oplus \text{sh}_{id}$.

Set $\text{tg}_{id} := (t_0^{(id)}, t_1^{(id)}, t_2^{(id)})$ and output $(\text{tg}_{id})_{id \in [n]}$.

Trig(tg_P, (tg_{id,GID,x})_{id ∈ [n]}) Parse tg_P as (key₁, ..., key_n, $\tilde{P}$) and tg_{id,GID,x} as $(t_0^{(id)}, t_1^{(id)}, t_2^{(id)})$ for each id ∈ [n]. For each id ∈ [n], compute $\text{sh}_{id} = \text{SEPRF.Eval}(\text{key}_{id}, t_0^{(id)}) \oplus t_1^{(id)}$. Let $\tilde{F} = \bigoplus_{id \in [n]} \text{sh}_{id}$. Also, compute $a = \bigoplus_{id \in [n]} t_2^{(id)}$.

Compute $y = \text{Garb.Eval}(\tilde{F}, \tilde{P})$. Output 1 if and only if $y = a$.

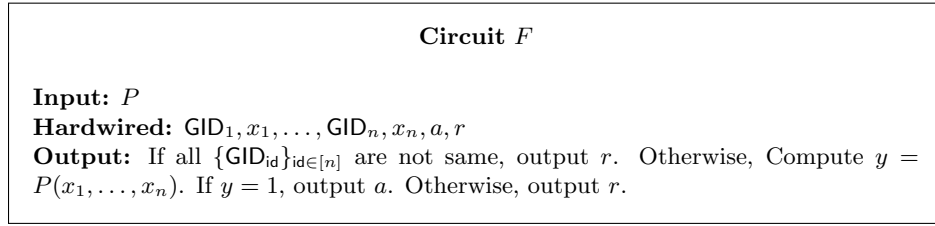


Figure 11: Description of circuit F

Dummy Correctness. Let $\tilde{F} = \bigoplus_{id \in [n]} \text{SEPRF.Eval}(\text{key}_{id}, t_0^{(id)}) \oplus t_1^{(id)}$ be obtained using dummy input tags, which are uniformly random strings. Thus the probability that $\text{Garb.Eval}(\tilde{F}, \tilde{P}) = \bigoplus_{id} t_2^{(id)}$ only happens with negligible probability.

Smart Correctness. Using smart tags, $\tilde{F} = \bigoplus_{id \in [n]} \text{SEPRF.Eval}(\text{key}_{id}, t_0^{(id)}) \oplus t_1^{(id)} = \bigoplus_{id} \text{sh}_{id} = \text{Garb.CircGarb}(\text{garb.sk}, F)$. Hence, by correctness of Garb, we have that if $P(x_1, \dots, x_n) = 1$, then output is 1.

Lemma P.7. If Garb is a blind garbled circuit scheme for P/Poly circuits and SEPRF is a somewhere-equivocal PRF scheme, then Construction P.6 is a distributed functional tag system for P/Poly circuits and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$.

Proof. We prove mode indistinguishability using the following hybrids and claims.

Hyb₀^A(1^λ). This is almost the real experiment with $b = 0$. We check if all all $t_0^{(id)}$ for all id ∈ [n] and each query made by $\mathcal{A}$ are unique. In addition, we also sample SEPRF keys and garb.sk during setup. This hybrid is statistically close to real experiment.

Setup Phase. $\mathcal{A}$ sends $(n, s, \ell_1, \dots, \ell_n)$. Calculate s', L, σ, L' , set $\text{CRS} := (\lambda, n, s, L, s', L', \sigma)$. Send CRS to $\mathcal{A}$.

Sample $\forall \text{id} \in [n], \text{key}_{\text{id}} \leftarrow \text{SEPRF.Setup}(1^\lambda, 1^L)$ and $\text{garb.sk} \leftarrow \text{Garb.Setup}(1^\lambda, 1^{s'}, 1^\sigma)$.

Pre-Challenge Queries. $\mathcal{A}$ sends $(\text{GID}, x_1, \dots, x_n)$. For each $\text{id} \in [n]$, sample $(t_0^{(\text{id})}, t_1^{(\text{id})}, t_2^{(\text{id})}) \xleftarrow{\$} \{0, 1\}^\lambda \times \{0, 1\}^L \times \{0, 1\}^\lambda$. Send $((t_0^{(1)}, t_1^{(1)}, t_2^{(1)}), \dots, (t_0^{(n)}, t_1^{(n)}, t_2^{(n)}))$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends P^* . Compute $\tilde{P} = \text{Garb.InpGarb}(\text{garb.msk}, P^*)$. Send $(\text{key}_1, \dots, \text{key}_n, \tilde{P})$ to $\mathcal{A}$.

Post-Challenge Queries. $\mathcal{A}$ sends $(\text{GID}, x_1, \dots, x_n)$. For each $\text{id} \in [n]$, sample $(t_0^{(\text{id})}, t_1^{(\text{id})}, t_2^{(\text{id})}) \xleftarrow{\$} \{0, 1\}^\lambda \times \{0, 1\}^L \times \{0, 1\}^\lambda$. Send $((t_0^{(1)}, t_1^{(1)}, t_2^{(1)}), \dots, (t_0^{(n)}, t_1^{(n)}, t_2^{(n)}))$ to $\mathcal{A}$.

Check Phase. If any of $t_0^{(\text{id})}$ for all the polynomially many queries made by $\mathcal{A}$ and each $\text{id} \in [n]$ are non-unique, abort.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb_{1,j,1}^A(1^λ) for $j \in [Q_1 + 1]$. where Q_1 is the number of pre-challenge queries made by $\mathcal{A}$. In this hybrid, we switch the first $j - 1$ input tag queries made by $\mathcal{A}$ to use **SInpTag**. And equivocate the values of $t_0^{(1)}, \dots, t_0^{(n)}$ early. The changes are highlighted in **red**.

Setup Phase. $\mathcal{A}$ sends $(n, s, \ell_1, \dots, \ell_n)$. Calculate s', L, σ, L' , set $\text{CRS} := (\lambda, n, s, L, s', L', \sigma)$. Send CRS to $\mathcal{A}$.

Sample $\forall \text{id} \in [n], r_{\text{id}}^* \xleftarrow{\$} \{0, 1\}^{L'}, t_0^{(\text{id})} \xleftarrow{\$} \{0, 1\}^\lambda, \text{eqKey}_{\text{id}} \leftarrow \text{SEPRF.Sim}_1(1^\lambda, 1^L, t_0^{(\text{id})}), \text{key}_{\text{id}} \leftarrow \text{SEPRF.Sim}_2(\text{eqKey}_{\text{id}}, r_{\text{id}}^*)$ and $\text{garb.sk} \leftarrow \text{Garb.Setup}(1^\lambda, 1^{s'}, 1^\sigma)$. Set **MSK** := $(\text{key}_1, \dots, \text{key}_n, \text{garb.sk})$.

Pre-Challenge Queries. $\mathcal{A}$ sends $(\text{GID}, x_1, \dots, x_n)$. If this is the i -th query with $i > j$, for each $\text{id} \in [n]$, sample $(t_0^{(\text{id})}, t_1^{(\text{id})}, t_2^{(\text{id})}) \xleftarrow{\$} \{0, 1\}^\lambda \times \{0, 1\}^L \times \{0, 1\}^\lambda$. Send $((t_0^{(1)}, t_1^{(1)}, t_2^{(1)}), \dots, (t_0^{(n)}, t_1^{(n)}, t_2^{(n)}))$ to $\mathcal{A}$.

Otherwise, if $i < j$, send $((t_0^{(1)}, t_1^{(1)}, t_2^{(1)}), \dots, (t_0^{(n)}, t_1^{(n)}, t_2^{(n)})) \leftarrow \text{SInpTag}(\text{MSK}, (1, \text{GID}, x_1), \dots, (1, \text{GID}, x_n))$ to $\mathcal{A}$.

Otherwise, if $i = j$, use $t_0^{(\text{id})}$ generated during setup phase for each $\text{id} \in [n]$ and $(t_1^{(\text{id})}, t_2^{(\text{id})}) \xleftarrow{\$} \{0, 1\}^L \times \{0, 1\}^\lambda$. Send $((t_0^{(1)}, t_1^{(1)}, t_2^{(1)}), \dots, (t_0^{(n)}, t_1^{(n)}, t_2^{(n)}))$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends P^* . Compute $\tilde{P} = \text{Garb.InpGarb}(\text{garb.msk}, P^*)$. Send $(\text{key}_1, \dots, \text{key}_n, \tilde{P})$ to $\mathcal{A}$.

Post-Challenge Queries. $\mathcal{A}$ sends $(\text{GID}, x_1, \dots, x_n)$. For each $\text{id} \in [n]$, sample $(t_0^{(\text{id})}, t_1^{(\text{id})}, t_2^{(\text{id})}) \xleftarrow{\$} \{0, 1\}^\lambda \times \{0, 1\}^L \times \{0, 1\}^\lambda$. Send $((t_0^{(1)}, t_1^{(1)}, t_2^{(1)}), \dots, (t_0^{(n)}, t_1^{(n)}, t_2^{(n)}))$ to $\mathcal{A}$.

Check Phase. If any of $t_0^{(\text{id})}$ for all the polynomially many queries made by $\mathcal{A}$ and each $\text{id} \in [n]$ are non-unique, abort.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb_{1,j,2}^A(1^λ) for $j \in [Q_1 + 1]$. where Q_1 is the number of pre-challenge queries made by $\mathcal{A}$. In this hybrid, we switch the first $j - 1$ input tag queries made by $\mathcal{A}$ to use **SInpTag**. We will now wait equivocate the values of $t_0^{(1)}, \dots, t_0^{(n)}$ and use equivocal key in generating **SInpTags**. The changes are highlighted in **red**.

Setup Phase. $\mathcal{A}$ sends $(n, s, \ell_1, \dots, \ell_n)$. Calculate s', L, σ, L' , set $\text{CRS} := (\lambda, n, s, L, s', L', \sigma)$. Send CRS to $\mathcal{A}$.

Sample $\forall \text{id} \in [n], t_0^{(\text{id})} \xleftarrow{\$} \{0, 1\}^\lambda, \text{eqKey}_{\text{id}} \leftarrow \text{SEPRF.Sim}_1(1^\lambda, 1^L, t_0^{(\text{id})})$ and $\text{garb.sk} \leftarrow \text{Garb.Setup}(1^\lambda, 1^{s'}, 1^\sigma)$. **Set** $\text{MSK} := (\text{eqKey}_1, \dots, \text{eqKey}_n, \text{garb.sk})$.

Pre-Challenge Queries. $\mathcal{A}$ sends $(\text{GID}, x_1, \dots, x_n)$. If this is the i -th query with $i > j$, for each $\text{id} \in [n]$, sample $(t_0^{(\text{id})}, t_1^{(\text{id})}, t_2^{(\text{id})}) \xleftarrow{\$} \{0, 1\}^\lambda \times \{0, 1\}^L \times \{0, 1\}^\lambda$. Send $((t_0^{(1)}, t_1^{(1)}, t_2^{(1)}), \dots, (t_0^{(n)}, t_1^{(n)}, t_2^{(n)}))$ to $\mathcal{A}$.

Otherwise, if $i < j$, send $((t_0^{(1)}, t_1^{(1)}, t_2^{(1)}), \dots, (t_0^{(n)}, t_1^{(n)}, t_2^{(n)})) \leftarrow \text{SInpTag}(\text{MSK}, (1, \text{GID}, x_1), \dots, (1, \text{GID}, x_n))$ to $\mathcal{A}$.

Otherwise, if $i = j$, use $t_0^{(\text{id})}$ generated during setup phase for each $\text{id} \in [n]$ and $(t_1^{(\text{id})}, t_2^{(\text{id})}) \xleftarrow{\$} \{0, 1\}^{L'} \times \{0, 1\}^\lambda$. Send $((t_0^{(1)}, t_1^{(1)}, t_2^{(1)}), \dots, (t_0^{(n)}, t_1^{(n)}, t_2^{(n)}))$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends P^* . Compute $\tilde{P} = \text{Garb.InpGarb}(\text{garb.msk}, P^*)$. **Let** $t_1^{(1)}, \dots, t_1^{(n)}$ **denote the values sampled in pre-challenge query phase for the j -th query.** **Sample** $\tilde{F} \xleftarrow{\$} \{0, 1\}^{L'}, \text{id}' \xleftarrow{\$} [n]$ **Sample** $\{\text{sh}_j\}_{j \in [n], j \neq \text{id}'} \xleftarrow{\$} \{0, 1\}^{L'}$, **set** $\text{sh}_{\text{id}'} = \bigoplus_{j \neq \text{id}'} \text{sh}_j \oplus \tilde{F}$. **Sample** $\text{key}_1 \leftarrow \text{SEPRF.Sim}_2(\text{eqKey}_1, t_1^{(1)} \oplus \text{sh}_1), \dots, \text{key}_n \leftarrow \text{SEPRF.Sim}_2(\text{eqKey}_n, t_1^{(n)} \oplus \text{sh}_n)$. Compute $\tilde{P} = \text{Garb.InpGarb}(\text{garb.sk}, P^*)$. Send $(\text{key}_1, \dots, \text{key}_n, \tilde{P})$ to $\mathcal{A}$.

Post-Challenge Queries. $\mathcal{A}$ sends $(\text{GID}, x_1, \dots, x_n)$. For each $\text{id} \in [n]$, sample $(t_0^{(\text{id})}, t_1^{(\text{id})}, t_2^{(\text{id})}) \xleftarrow{\$} \{0, 1\}^\lambda \times \{0, 1\}^L \times \{0, 1\}^\lambda$. Send $((t_0^{(1)}, t_1^{(1)}, t_2^{(1)}), \dots, (t_0^{(n)}, t_1^{(n)}, t_2^{(n)}))$ to $\mathcal{A}$.

Check Phase. If any of $t_0^{(\text{id})}$ for all the polynomially many queries made by $\mathcal{A}$ and each $\text{id} \in [n]$ are non-unique, abort.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb_{1,j,3}^A(1^λ) for $j \in [Q_1 + 1]$. where Q_1 is the number of pre-challenge queries made by $\mathcal{A}$. In this hybrid, instead of choosing $\tilde{F} \xleftarrow{\$} \{0, 1\}^{L'}$, we chose it as a simulated garbled circuit. The changes are highlighted in **red**.

Setup Phase, Pre-challenge Queries. Same as **Hyb_{1,j,2}^A(1^λ)**.

Challenge Query. $\mathcal{A}$ sends P^* . Compute $\tilde{P} = \text{Garb.InpGarb}(\text{garb.msk}, P^*)$. Let $t_1^{(1)}, \dots, t_1^{(n)}$ denote the values sampled in pre-challenge query phase for the j -th query. **Sample** $r \xleftarrow{\$} \{0, 1\}^\lambda, \tilde{F} \leftarrow \text{Garb.Sim}(\text{garb.sk}, P^*, r)$, $\text{id}' \xleftarrow{\$} [n]$ **Sample** $\{\text{sh}_j\}_{j \in [n], j \neq \text{id}'} \xleftarrow{\$} \{0, 1\}^{L'}$, **set** $\text{sh}_{\text{id}'} = \bigoplus_{j \neq \text{id}'} \text{sh}_j \oplus \tilde{F}$. **Sample** $\text{key}_1 \leftarrow \text{SEPRF.Sim}_2(\text{eqKey}_1, t_1^{(1)} \oplus \text{sh}_1), \dots, \text{key}_n \leftarrow \text{SEPRF.Sim}_2(\text{eqKey}_n, t_1^{(n)} \oplus \text{sh}_n)$. Compute $\tilde{P} = \text{Garb.InpGarb}(\text{garb.sk}, P^*)$. Send $(\text{key}_1, \dots, \text{key}_n, \tilde{P})$ to $\mathcal{A}$.

Post-Challenge Queries, Check Phase, Guess Phase. Same as **Hyb_{1,j,2}^A(1^λ)**.

Hyb_{1,j,4}^A(1^λ) for $j \in [Q_1 + 1]$. where Q_1 is the number of pre-challenge queries made by $\mathcal{A}$. In this hybrid, simulating $\tilde{F}$ during challenge query phase, we garble it using garb.sk . The changes are highlighted in red.

Setup Phase, Pre-challenge Queries. Same as **Hyb_{1,j,3}^A(1^λ)**.

Challenge Query. $\mathcal{A}$ sends P^* . Compute $\tilde{P} = \text{Garb.InpGarb}(\text{garb.msk}, P^*)$. Let $t_1^{(1)}, \dots, t_1^{(n)}, t_2^{(1)}, \dots, t_2^{(n)}$ denote the values sampled in pre-challenge query phase for the j -th query. Let $(\text{GID}, x_1, \dots, x_n)$ denote the j -th query by $\mathcal{A}$. Sample $r \xleftarrow{\$} \{0, 1\}^\lambda$, compute $a = \bigoplus_{\text{id}} t_2^{(\text{id})}$, $\tilde{F} \leftarrow \text{Garb.CircGarb}(\text{garb.sk}, F)$ where F is from Figure 11, $\text{id}' \xleftarrow{\$} [n]$ Sample $\{\text{sh}_j\}_{j \in [n], j \neq \text{id}'} \xleftarrow{\$} \{0, 1\}^{L'}$, set $\text{sh}_{\text{id}'} = \bigoplus_{j \neq \text{id}'} \text{sh}_j \oplus \tilde{F}$. Sample $\text{key}_1 \leftarrow \text{SEPRF.Sim}_2(\text{eqKey}_1, t_1^{(1)} \oplus \text{sh}_1), \dots, \text{key}_n \leftarrow \text{SEPRF.Sim}_2(\text{eqKey}_n, t_1^{(n)} \oplus \text{sh}_n)$. Compute $\tilde{P} = \text{Garb.InpGarb}(\text{garb.sk}, P^*)$. Send $(\text{key}_1, \dots, \text{key}_n, \tilde{P})$ to $\mathcal{A}$.

Post-Challenge Queries, Check Phase, Guess Phase. Same as **Hyb_{1,j,3}^A(1^λ)**.

Hyb_{1,j,5}^A(1^λ) for $j \in [Q_1 + 1]$. where Q_1 is the number of pre-challenge queries made by $\mathcal{A}$. In this hybrid, simulating $\tilde{F}$ during challenge query phase, we garble it using garb.sk . The changes are highlighted in red.

Setup Phase. $\mathcal{A}$ sends $(n, s, \ell_1, \dots, \ell_n)$. Calculate s', L, σ, L' , set $\text{CRS} := (\lambda, n, s, L, s', L', \sigma)$. Send CRS to $\mathcal{A}$.

Sample $\forall \text{id} \in [n], r_{\text{id}}^* \xleftarrow{\$} \{0, 1\}^{L'}, t_0^{(\text{id})} \xleftarrow{\$} \{0, 1\}^\lambda, \text{eqKey}_{\text{id}} \leftarrow \text{SEPRF.Sim}_1(1^\lambda, 1^L, t_0^{(\text{id})}), \text{key}_{\text{id}} \leftarrow \text{SEPRF.Sim}_2(\text{eqKey}_{\text{id}}, r_{\text{id}}^*)$, and $\text{garb.sk} \leftarrow \text{Garb.Setup}(1^\lambda, 1^{s'}, 1^\sigma)$. Set $\text{MSK} := (\text{key}_1, \dots, \text{key}_n, \text{garb.sk})$.

Pre-Challenge Queries. $\mathcal{A}$ sends $(\text{GID}, x_1, \dots, x_n)$. If this is the i -th query with $i > j$, for each $\text{id} \in [n]$, sample $(t_0^{(\text{id})}, t_1^{(\text{id})}, t_2^{(\text{id})}) \xleftarrow{\$} \{0, 1\}^\lambda \times \{0, 1\}^L \times \{0, 1\}^\lambda$. Send $((t_0^{(1)}, t_1^{(1)}, t_2^{(1)}), \dots, (t_0^{(n)}, t_1^{(n)}, t_2^{(n)}))$ to $\mathcal{A}$.

Otherwise, if $i < j$, send $((t_0^{(1)}, t_1^{(1)}, t_2^{(1)}), \dots, (t_0^{(n)}, t_1^{(n)}, t_2^{(n)})) \leftarrow \text{SInpTag}(\text{MSK}, (1, \text{GID}, x_1), \dots, (1, \text{GID}, x_n))$ to $\mathcal{A}$.

Otherwise, if $i = j$, use $t_0^{(\text{id})}$ generated during setup phase for each $\text{id} \in [n]$ and $t_2^{(\text{id})} \xleftarrow{\$} \{0, 1\}^\lambda$. Set $a = \bigoplus_{\text{id}} t_2^{(\text{id})}$, sample $r \xleftarrow{\$} \{0, 1\}^\lambda$. Sample $\tilde{F} \leftarrow \text{Garb.CircGarb}(\text{garb.sk}, F)$ where F is from Figure 11, $\text{id}' \xleftarrow{\$} [n]$ Sample $\{\text{sh}_j\}_{j \in [n], j \neq \text{id}'} \xleftarrow{\$} \{0, 1\}^{L'}$, set $\text{sh}_{\text{id}'} = \bigoplus_{j \neq \text{id}'} \text{sh}_j \oplus \tilde{F}$. Set $t_1^{(\text{id})} = \text{SEPRF.Eval}(\text{key}_{\text{id}}, t_0^{(\text{id})}) \oplus \text{sh}_{\text{id}}$ for each $\text{id} \in [n]$. Send $((t_0^{(1)}, t_1^{(1)}, t_2^{(1)}), \dots, (t_0^{(n)}, t_1^{(n)}, t_2^{(n)}))$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends P^* . Compute $\tilde{P} = \text{Garb.InpGarb}(\text{garb.sk}, P^*)$. Send $(\text{key}_1, \dots, \text{key}_n, \tilde{P})$ to $\mathcal{A}$.

Post-Challenge Queries, Check Phase, Guess Phase. Same as **Hyb_{1,j,4}^A(1^λ)**.

Hyb_{2,k,1}^A(1^λ) for $k \in [Q_2 + 1]$. where Q_2 is the number of post-challenge queries made by $\mathcal{A}$. In this hybrid, we sample the input tags for the first $k - 1$ post-challenge queries using **SInpTag** and k -th one differently. The changes are highlighted in **red**.

Setup Phase, Pre-Challenge Queries, Challenge Query. Same as **Hyb_{1,Q₁+1,5}^A(1^λ)**.

Post-Challenge Queries. $\mathcal{A}$ sends $(\text{GID}, x_1, \dots, x_n)$. If this is the i -th query with $i > k$, for each $\text{id} \in [n]$, sample $(t_0^{(\text{id})}, t_1^{(\text{id})}, t_2^{(\text{id})}) \xleftarrow{\$} \{0, 1\}^\lambda \times \{0, 1\}^L \times \{0, 1\}^\lambda$. Send $((t_0^{(1)}, t_1^{(1)}, t_2^{(1)}), \dots, (t_0^{(n)}, t_1^{(n)}, t_2^{(n)}))$ to $\mathcal{A}$.

Otherwise, if $i < k$, send $((t_0^{(1)}, t_1^{(1)}, t_2^{(1)}), \dots, (t_0^{(n)}, t_1^{(n)}, t_2^{(n)})) \leftarrow \text{SInpTag}(\text{MSK}, (1, \text{GID}, x_1), \dots, (1, \text{GID}, x_n))$ to $\mathcal{A}$.

Otherwise, if $i = k$, use $t_0^{(\text{id})}$ generated during setup phase for each $\text{id} \in [n]$ and $t_2^{(\text{id})} \xleftarrow{\$} \{0, 1\}^\lambda$. Set $a = \bigoplus_{\text{id}} t_2^{(\text{id})}$, sample $r \xleftarrow{\$} \{0, 1\}^\lambda$. Sample $\tilde{F} \leftarrow \text{Garb.Sim}(\text{garb.sk}, P^*, r)$, $\text{id}' \xleftarrow{\$} [n]$ Sample $\{\text{sh}_j\}_{j \in [n], j \neq \text{id}'} \xleftarrow{\$} \{0, 1\}^{L'}$, set $\text{sh}_{\text{id}'} = \bigoplus_{j \neq \text{id}'} \text{sh}_j \oplus \tilde{F}$. Set $t_1^{(\text{id})} = \text{SEPRF.Eval}(\text{key}_{\text{id}}, t_0^{(\text{id})}) \oplus \text{sh}_{\text{id}}$ for each $\text{id} \in [n]$. Send $((t_0^{(1)}, t_1^{(1)}, t_2^{(1)}), \dots, (t_0^{(n)}, t_1^{(n)}, t_2^{(n)}))$ to $\mathcal{A}$.

Check Phase, Guess Phase. This is same as **Hyb_{1,Q₁+1,5}^A(1^λ)**.

Hyb_{2,k,2}^A(1^λ) for $k \in [Q_2 + 1]$. where Q_2 is the number of post-challenge queries made by $\mathcal{A}$. In this hybrid, we honestly garble F for the k -th query. The changes are highlighted in **red**.

Setup Phase, Pre-Challenge Queries, Challenge Query. Same as **Hyb_{1,Q₁+1,5}^A(1^λ)**.

Post-Challenge Queries. $\mathcal{A}$ sends $(\text{GID}, x_1, \dots, x_n)$. If this is the i -th query with $i > k$, for each $\text{id} \in [n]$, sample $(t_0^{(\text{id})}, t_1^{(\text{id})}, t_2^{(\text{id})}) \xleftarrow{\$} \{0, 1\}^\lambda \times \{0, 1\}^L \times \{0, 1\}^\lambda$. Send $((t_0^{(1)}, t_1^{(1)}, t_2^{(1)}), \dots, (t_0^{(n)}, t_1^{(n)}, t_2^{(n)}))$ to $\mathcal{A}$.

Otherwise, if $i < k$, send $((t_0^{(1)}, t_1^{(1)}, t_2^{(1)}), \dots, (t_0^{(n)}, t_1^{(n)}, t_2^{(n)})) \leftarrow \text{SInpTag}(\text{MSK}, (1, \text{GID}, x_1), \dots, (1, \text{GID}, x_n))$ to $\mathcal{A}$.

Otherwise, if $i = k$, use $t_0^{(\text{id})}$ generated during setup phase for each $\text{id} \in [n]$ and $t_2^{(\text{id})} \xleftarrow{\$} \{0, 1\}^\lambda$. Set $a = \bigoplus_{\text{id}} t_2^{(\text{id})}$, sample $r \xleftarrow{\$} \{0, 1\}^\lambda$. Sample $\tilde{F} \leftarrow \text{Garb.CircGarb}(\text{garb.sk}, F)$, where F is from Figure 11 $\text{id}' \xleftarrow{\$} [n]$ Sample $\{\text{sh}_j\}_{j \in [n], j \neq \text{id}'} \xleftarrow{\$} \{0, 1\}^{L'}$, set $\text{sh}_{\text{id}'} = \bigoplus_{j \neq \text{id}'} \text{sh}_j \oplus \tilde{F}$. Set $t_1^{(\text{id})} = \text{SEPRF.Eval}(\text{key}_{\text{id}}, t_0^{(\text{id})}) \oplus \text{sh}_{\text{id}}$ for each $\text{id} \in [n]$. Send $((t_0^{(1)}, t_1^{(1)}, t_2^{(1)}), \dots, (t_0^{(n)}, t_1^{(n)}, t_2^{(n)}))$ to $\mathcal{A}$.

Check Phase, Guess Phase. This is same as **Hyb_{1,Q₁+1,5}^A(1^λ)**.

Hyb₃^A(1^λ). This the experiment with $b = 1$.

We now prove indistinguishability of these hybrid. For ease of understanding, we note that in our claims, the hybrid indistinguishability flows as follows:

$$\begin{aligned} \mathbf{Hyb}_0^A(1^\lambda) &\approx_c \mathbf{Hyb}_{1,1,1}^A(1^\lambda) \equiv \mathbf{Hyb}_{1,1,2}^A(1^\lambda) \equiv \mathbf{Hyb}_{1,1,3}^A(1^\lambda) \approx_c \mathbf{Hyb}_{1,1,4}^A(1^\lambda) \equiv \mathbf{Hyb}_{1,1,5}^A(1^\lambda) \\ &\approx_c \mathbf{Hyb}_{1,2,1}^A(1^\lambda) \equiv \dots \\ &\approx_c \mathbf{Hyb}_{2,1,1}^A(1^\lambda) \approx_c \mathbf{Hyb}_{2,1,2}^A(1^\lambda) \equiv \mathbf{Hyb}_{2,2,1}^A(1^\lambda) \approx_c \dots \approx_c \mathbf{Hyb}_3^A(1^\lambda) \end{aligned}$$

The proof closely follows the proof of functional tag system from [WW24].

Claim P.8. Assuming the security of SEPRF, $\mathbf{Hyb}_0^A(1^\lambda)$ and $\mathbf{Hyb}_{1,1,1}^A(1^\lambda)$ are computationally indistinguishable.

Proof. This follows from SEPRF security. We can consider n distinct sub-hybrids where in each sub-hybrid i of the we only switch the key sampling procedure for the first i keys. The i -th and $(i+1)$ -th sub-hybrid can be reduced to security of SEPRF readily by sending $t_0^{(i)}$ to challenger sampled during setup phase. $\square$

Claim P.9. $\mathbf{Hyb}_{1,j,1}^A(1^\lambda)$ and $\mathbf{Hyb}_{1,j,2}^A(1^\lambda)$ are identically distributed for any $j \in [Q_1 + 1]$.

Proof. This is because by correctness of SEPRF, `eqKey` and `key` output the same value everywhere except $t_0^{(\text{id})}$ for each `id`. Hence, we can generate `SInpTag` using `eqKeys`. Moreover, each `shid` is a uniformly random string and so is $t_1^{(\text{id})}$. Thus, both hybrids are identically distributed. $\square$

Claim P.10. $\mathbf{Hyb}_{1,j,2}^A(1^\lambda)$ and $\mathbf{Hyb}_{1,j,3}^A(1^\lambda)$ are identically distributed for any $j \in [Q_1 + 1]$.

Proof. This is because by blindness property of Garb. As r is a uniformly random string, it holds that simulated garbled circuit is a uniform random string. $\square$

Claim P.11. Assuming the security of blind garbled circuit scheme, $\mathbf{Hyb}_{1,j,3}^A(1^\lambda)$ and $\mathbf{Hyb}_{1,j,4}^A(1^\lambda)$ are computationally indistinguishable for any $j \in [Q_1 + 1]$.

Proof. The reduction algorithm can query blind garbled circuit challenger to get garblings of F for each of the pre-challenge queries before the j -th one. From j to Q_1 , we have random strings anyway. Then once adversary queries for P^* , the reduction sends P^* to challenger to receive $\tilde{P}$. Then, it sends F as calculated in challenge query phase of $\mathbf{Hyb}_{1,j,4}^A(1^\lambda)$ to challenger to receive challenge circuit garbling $\tilde{F}$. If $\tilde{F}$ was simulated, reduction acts like $\mathbf{Hyb}_{1,j,3}^A(1^\lambda)$ and if it was generated honestly, reduction acts like $\mathbf{Hyb}_{1,j,4}^A(1^\lambda)$. Thus, these two hybrids are computationally indistinguishable. $\square$

Claim P.12. $\mathbf{Hyb}_{1,j,4}^A(1^\lambda)$ and $\mathbf{Hyb}_{1,j,5}^A(1^\lambda)$ are identically distributed for any $j \in [Q_1 + 1]$.

Proof. This is by correctness of SEPRF and as all $t_0^{(\text{id})}$ are unique for each `id` $\in [n]$ and all queries. So, using `eqKeyid` and `keyid` are same. Moreover, in both hybrids, $t_1^{(\text{id})}$ is subject to $\text{SEPRF.Eval}(\text{key}_{\text{id}}, t_0^{(\text{id})}) \oplus \text{sh}_{\text{id}}$. $t_1^{(\text{id})}$ was explicitly random in $\mathbf{Hyb}_{1,j,4}^A(1^\lambda)$ whereas in $\mathbf{Hyb}_{1,j,5}^A(1^\lambda)$ it is $r_{\text{id}}^* \oplus \text{sh}_{\text{id}}$ which is identically distributed to random as r_{id}^* is random. $\square$

Claim P.13. Assuming the security of SEPRF, $\mathbf{Hyb}_{1,j,5}^A(1^\lambda)$ and $\mathbf{Hyb}_{1,j+1,1}^A(1^\lambda)$ are computationally indistinguishable for any $j \in [Q_1]$.

Proof. This is similar to proof of Claim P.8. We can consider a sub-hybrid where we switch from equivocating SEPRF keys to generating all of them using $\text{SEPRF.Setup}(1^\lambda, 1^L)$. These remain indistinguishable due to SEPRF security. Then we can equivocate them all using the $(j+1)$ -th query's $t_0^{(\text{id})}$. These remain indistinguishable by SEPRF security. $\square$

Claim P.14. $\text{Hyb}_{1,Q_1+1,5}^A(1^\lambda)$ and $\text{Hyb}_{2,1,1}^A(1^\lambda)$ are identically distributed.

Proof. This follows from blindness property of Garb as $\tilde{F}$ is a uniformly random string, and thus sh_{id} are also uniformly random. Hence, $t_1^{(\text{id})}$ are uniformly random in both hybrids. $\square$

Claim P.15. Assuming the security of blind garbled circuit scheme, $\text{Hyb}_{2,k,1}^A(1^\lambda)$ and $\text{Hyb}_{2,k+1,1}^A(1^\lambda)$ are computationally indistinguishable for any $j \in [Q_2 + 1]$.

Proof. The reduction algorithm can query blind garbled circuit challenger to get garblings of F for each of the pre-challenge queries. Then once adversary queries for P^* , the reduction sends P^* to challenger to receive $\tilde{P}$. Then for all post-challenge queries before k -th one, it can query the challenger for garblings of F . Then, it sends F as calculated in $\text{Hyb}_{2,k,2}^A(1^\lambda)$ to challenger to receive challenge circuit garbling $\tilde{F}$. If $\tilde{F}$ was simulated, reduction acts like $\text{Hyb}_{2,k,1}^A(1^\lambda)$ and if it was generated honestly, reduction acts like $\text{Hyb}_{2,k,2}^A(1^\lambda)$. For the rest of post-challenge queries, we have uniformly random strings anyway. Thus, these two hybrids are computationally indistinguishable. $\square$

Claim P.16. $\text{Hyb}_{2,Q_2+1,2}^A(1^\lambda)$ and $\text{Hyb}_3^A(1^\lambda)$ are statistically indistinguishable.

Proof. The only change is we stop the check phase which holds with probability $q^2/2^{n^\lambda}$. Thus, these two hybrids are statistically close. $\square$

$\square$

Q Proofs from Section 7.2

In this section, we provide full proof of Theorem 7.5.

$\text{Hyb}_0^A(1^\lambda)$. This is the real experiment with honest challenger from Definition 7.3 with $b = 0/1$.

Setup. $\mathcal{A}$ sends (n, s, L) to Chal. Chal sample $\text{fts.crs} \leftarrow \text{dFTS.GSetup}(1^\lambda, 1^n, 1^s, 1^L)$ and $\text{com.crs} \leftarrow \text{COM.Setup}(1^\lambda)$. Set $\text{CRS} := (n, s, L, \text{com.crs}, \text{fts.crs})$.

Chal samples $\forall \text{id} \in [n], \text{com}_{\text{id}}^{(0)} \leftarrow \text{COM.Com}(1^\lambda, 0; r_{\text{id}}^{(0)})$, $\text{com}_{\text{id}}^{(1)} \leftarrow \text{COM.Com}(1^\lambda, 0^\lambda; r_{\text{id}}^{(1)})$, and $\text{nizk.crs}_{\text{id}} \leftarrow \text{NIZK.Setup}(1^\lambda)$ (for $\mathcal{L}_{\text{NIZK}}$, Figure 5).

Chal sets $\text{MPK}_{\text{id}} := (\text{com}_{\text{id}}^{(0)}, \text{com}_{\text{id}}^{(1)}, \text{nizk.crs}_{\text{id}})$ and sends $\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Pre-Challenge Queries. $\mathcal{A}$ sends $(\text{GID}, x_1, \dots, x_n)$ to Chal. For each $\text{id} \in [n]$, sample $\text{tg}_{\text{id}} \leftarrow \text{dFTS.DInpTag}(\text{id}, \text{GID}, x_{\text{id}})$ and $\pi_{\text{id}} \leftarrow \text{NIZK.Prove}(\text{nizk.crs}_{\text{id}}, (\text{CRS}, \text{id}, \text{GID}, x_{\text{id}}, \text{tg}_{\text{id}}, \text{com}_{\text{id}}^{(0)}, \text{com}_{\text{id}}^{(1)}), (r_{\text{id}}^{(0)}, \bar{0}))$. Set $\text{SK}_{\text{id}, \text{GID}, x} := (\text{id}, \text{GID}, x, \text{tg}_{\text{id}}, \pi_{\text{id}})$. Send $\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends $(P^*, (m_0, m_1))$ to Chal. Chal sample $\text{tg}_{P^*} \leftarrow \text{dFTS.DFuncTag}(P^*)$ and encrypts $\text{we.ct} \leftarrow \text{WE.Enc}(1^\lambda, (\text{CRS}, P^*, \text{tg}_{P^*}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}), m_b)$. Send $(\text{we.ct}, P, \text{tg}_{P^*})$ to $\mathcal{A}$.

Post-Challenge Queries. These are handled similarly to pre-challenge queries.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb₁^A(1^λ). In this hybrid, we switch the mode from using dummy input tags to smart input tags. Changes are highlighted in **red**.

Setup. $\mathcal{A}$ sends (n, s, L) . Sample $\text{fts.crs} \leftarrow \text{dFTS.GSetup}(1^\lambda, 1^n, 1^s, 1^L)$ and $\text{com.crs} \leftarrow \text{COM.Setup}(1^\lambda)$. Set $\text{CRS} := (n, s, L, \text{com.crs}, \text{fts.crs})$. **In addition, sample $\text{fts.msk} \leftarrow \text{dFTS.KGen}(\text{fts.crs})$.**

Sample $\forall \text{id} \in [n], \text{com}_{\text{id}}^{(0)} \leftarrow \text{COM.Com}(1^\lambda, 0; r_{\text{id}}^{(0)})$, $\text{com}_{\text{id}}^{(1)} \leftarrow \text{COM.Com}(1^\lambda, 0^\lambda; r_{\text{id}}^{(1)})$, and $\text{nizk.crs}_{\text{id}} \leftarrow \text{NIZK.Setup}(1^\lambda)$ (for $\mathcal{L}_{\text{NIZK}}$, Figure 5).

Set $\text{MPK}_{\text{id}} := (\text{com}_{\text{id}}^{(0)}, \text{com}_{\text{id}}^{(1)}, \text{nizk.crs}_{\text{id}})$ and sends $\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Pre-Challenge Queries. $\mathcal{A}$ sends $(\text{GID}, x_1, \dots, x_n)$. **Sample $(\text{tg}_{\text{id}})_{\text{id} \in [n]} \leftarrow \text{dFTS.SInpTag}(\text{fts.msk}, (1, \text{GID}, x_1), \dots, (1, \text{GID}, x_n))$.** For each $\text{id} \in [n]$, $\pi_{\text{id}} \leftarrow \text{NIZK.Prove}(\text{nizk.crs}_{\text{id}}, (\text{CRS}, \text{id}, \text{GID}, x_{\text{id}}, \text{tg}_{\text{id}}, \text{com}_{\text{id}}^{(0)}, \text{com}_{\text{id}}^{(1)}), (r_{\text{id}}^{(0)}, \bar{0}))$. Set $\text{SK}_{\text{id}, \text{GID}, x} := (\text{id}, \text{GID}, x, \text{tg}_{\text{id}}, \pi_{\text{id}})$. Send $\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends $(P^*, (m_0, m_1))$ to Chal. Chal sample **$\text{tg}_{P^*} \leftarrow \text{dFTS.SFuncTag}(\text{fts.msk}, P^*)$** and encrypts $\text{we.ct} \leftarrow \text{WE.Enc}(1^\lambda, (\text{CRS}, P^*, \text{tg}_{P^*}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}), m_b)$. Send $(\text{we.ct}, P, \text{tg}_{P^*})$ to $\mathcal{A}$.

Post-Challenge Queries. These are handled similarly to pre-challenge queries.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb₂^A(1^λ). In this hybrid, we set $\text{com}_{\text{id}}^{(1)}$ for each id to be commitments of r^* where r^* is the randomness used to generate fts.msk . Changes are highlighted in **red**.

Setup. $\mathcal{A}$ sends (n, s, L) . Sample $\text{fts.crs} \leftarrow \text{dFTS.GSetup}(1^\lambda, 1^n, 1^s, 1^L)$ and $\text{com.crs} \leftarrow \text{COM.Setup}(1^\lambda)$. Set $\text{CRS} := (n, s, L, \text{com.crs}, \text{fts.crs})$. In addition, sample $\text{fts.msk} \leftarrow \text{dFTS.KGen}(\text{fts.crs}; r^*)$ for $r^* \xleftarrow{\$} \{0, 1\}^\lambda$.

Sample $\forall \text{id} \in [n], \text{com}_{\text{id}}^{(0)} \leftarrow \text{COM.Com}(1^\lambda, 0; r_{\text{id}}^{(0)})$, **$\text{com}_{\text{id}}^{(1)} \leftarrow \text{COM.Com}(1^\lambda, r^*; r_{\text{id}}^{(1)})$** , and $\text{nizk.crs}_{\text{id}} \leftarrow \text{NIZK.Setup}(1^\lambda)$ (for $\mathcal{L}_{\text{NIZK}}$, Figure 5).

Set $\text{MPK}_{\text{id}} := (\text{com}_{\text{id}}^{(0)}, \text{com}_{\text{id}}^{(1)}, \text{nizk.crs}_{\text{id}})$ and sends $\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Pre-Challenge Queries. $\mathcal{A}$ sends $(\text{GID}, x_1, \dots, x_n)$. Sample $(\text{tg}_{\text{id}})_{\text{id} \in [n]} \leftarrow \text{dFTS.SInpTag}(\text{fts.msk}, (1, \text{GID}, x_1), \dots, (1, \text{GID}, x_n))$. For each $\text{id} \in [n]$, $\pi_{\text{id}} \leftarrow \text{NIZK.Prove}(\text{nizk.crs}_{\text{id}}, (\text{CRS}, \text{id}, \text{GID}, x_{\text{id}}, \text{tg}_{\text{id}}, \text{com}_{\text{id}}^{(0)}, \text{com}_{\text{id}}^{(1)}), (r_{\text{id}}^{(0)}, \bar{0}))$. Set $\text{SK}_{\text{id}, \text{GID}, x} := (\text{id}, \text{GID}, x, \text{tg}_{\text{id}}, \pi_{\text{id}})$. Send $\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends $(P^*, (m_0, m_1))$ to Chal. Chal sample $\text{tg}_{P^*} \leftarrow \text{dFTS.SFuncTag}(\text{fts.msk}, P^*)$ and encrypts $\text{we.ct} \leftarrow \text{WE.Enc}(1^\lambda, (\text{CRS}, P^*, \text{tg}_{P^*}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}), m_b)$. Send $(\text{we.ct}, P, \text{tg}_{P^*})$ to $\mathcal{A}$.

Post-Challenge Queries. These are handled similarly to pre-challenge queries.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb $_{3,j}^{\mathcal{A}}(1^\lambda)$ for $j \in [n+1]$. In this hybrid, we simulate first $j-1$ NIZK instantiations. Changes are highlighted in **red**.

Setup. $\mathcal{A}$ sends (n, s, L) . Sample $\text{fts.crs} \leftarrow \text{dFTS.GSetup}(1^\lambda, 1^n, 1^s, 1^L)$ and $\text{com.crs} \leftarrow \text{COM.Setup}(1^\lambda)$.

Set $\text{CRS} := (n, s, L, \text{com.crs}, \text{fts.crs})$. In addition, sample $\text{fts.msk} \leftarrow \text{dFTS.KGen}(\text{fts.crs}; r^*)$ for $r^* \xleftarrow{\$} \{0, 1\}^\lambda$.

Sample $\forall \text{id} \in [n], \text{com}_{\text{id}}^{(0)} \leftarrow \text{COM.Com}(1^\lambda, 0; r_{\text{id}}^{(0)})$, $\text{com}_{\text{id}}^{(1)} \leftarrow \text{COM.Com}(1^\lambda, r^*; r_{\text{id}}^{(1)})$. **If $\text{id} < j$, $\text{nizk.crs}_{\text{id}} \leftarrow \text{NIZK.Sim}_{\text{id}}(1^\lambda)$. Otherwise, $\text{nizk.crs}_{\text{id}} \leftarrow \text{NIZK.Setup}(1^\lambda)$ (for $\mathcal{L}_{\text{NIZK}}$, Figure 5).**

Set $\text{MPK}_{\text{id}} := (\text{com}_{\text{id}}^{(0)}, \text{com}_{\text{id}}^{(1)}, \text{nizk.crs}_{\text{id}})$ and sends $\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Pre-Challenge Queries. $\mathcal{A}$ sends $(\text{GID}, x_1, \dots, x_n)$. Sample $(\text{tg}_{\text{id}})_{\text{id} \in [n]} \leftarrow \text{dFTS.SInpTag}(\text{fts.msk}, (1, \text{GID}, x_1), \dots, (1, \text{GID}, x_n))$.

For each $\text{id} \in [n]$, if $\text{id} < j$, $\pi_{\text{id}} \leftarrow \text{NIZK.Sim}_{\text{id}}((\text{CRS}, \text{id}, \text{GID}, x_{\text{id}}, \text{tg}_{\text{id}}, \text{com}_{\text{id}}^{(0)}, \text{com}_{\text{id}}^{(1)}))$. Otherwise, $\pi_{\text{id}} \leftarrow \text{NIZK.Prove}(\text{nizk.crs}_{\text{id}}, (\text{CRS}, \text{id}, \text{GID}, x_{\text{id}}, \text{tg}_{\text{id}}, \text{com}_{\text{id}}^{(0)}, \text{com}_{\text{id}}^{(1)}), (r_{\text{id}}^{(0)}, \bar{0}))$.

Set $\text{SK}_{\text{id}, \text{GID}, x} := (\text{id}, \text{GID}, x, \text{tg}_{\text{id}}, \pi_{\text{id}})$. Send $\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends $(P^*, (m_0, m_1))$ to Chal. Chal sample $\text{tg}_{P^*} \leftarrow \text{dFTS.SFuncTag}(\text{fts.msk}, P^*)$ and encrypts $\text{we.ct} \leftarrow \text{WE.Enc}(1^\lambda, (\text{CRS}, P^*, \text{tg}_{P^*}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}), m_b)$. Send $(\text{we.ct}, P, \text{tg}_{P^*})$ to $\mathcal{A}$.

Post-Challenge Queries. These are handled similarly to pre-challenge queries.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb $_{4,j}^{\mathcal{A}}(1^\lambda)$ for $j \in [n+1]$. In this hybrid, we use the alternate witness to generate proofs for first $j-1$ NIZK instantiations during key generation. Changes are highlighted in **red**.

Setup. $\mathcal{A}$ sends (n, s, L) . Sample $\text{fts.crs} \leftarrow \text{dFTS.GSetup}(1^\lambda, 1^n, 1^s, 1^L)$ and $\text{com.crs} \leftarrow \text{COM.Setup}(1^\lambda)$.

Set $\text{CRS} := (n, s, L, \text{com.crs}, \text{fts.crs})$. In addition, sample $\text{fts.msk} \leftarrow \text{dFTS.KGen}(\text{fts.crs}; r^*)$ for $r^* \xleftarrow{\$} \{0, 1\}^\lambda$.

Sample $\forall \text{id} \in [n], \text{com}_{\text{id}}^{(0)} \leftarrow \text{COM.Com}(1^\lambda, 0; r_{\text{id}}^{(0)})$, $\text{com}_{\text{id}}^{(1)} \leftarrow \text{COM.Com}(1^\lambda, r^*; r_{\text{id}}^{(1)})$. **If $\text{id} < j$, $\text{nizk.crs}_{\text{id}} \leftarrow \text{NIZK.Setup}(1^\lambda)$. Otherwise, $\text{nizk.crs}_{\text{id}} \leftarrow \text{NIZK.Sim}_{\text{id}}(1^\lambda)$ (for $\mathcal{L}_{\text{NIZK}}$, Figure 5).**

Set $\text{MPK}_{\text{id}} := (\text{com}_{\text{id}}^{(0)}, \text{com}_{\text{id}}^{(1)}, \text{nizk.crs}_{\text{id}})$ and sends $\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Pre-Challenge Queries. $\mathcal{A}$ sends $(\text{GID}, x_1, \dots, x_n)$. Sample $(\text{tg}_{\text{id}})_{\text{id} \in [n]} \leftarrow \text{dFTS.SInpTag}(\text{fts.msk}, (1, \text{GID}, x_1), \dots, (1, \text{GID}, x_n); r_{\text{tg}})$ for some $r_{\text{tg}} \xleftarrow{\$} \{0, 1\}^\lambda$.

For each $\text{id} \in [n]$, if $\text{id} < j$, $\pi_{\text{id}} \leftarrow \text{NIZK.Prove}(\text{nizk.crs}_{\text{id}}, (\text{CRS}, \text{id}, \text{GID}, x_{\text{id}}, \text{tg}_{\text{id}}, \text{com}_{\text{id}}^{(0)}, \text{com}_{\text{id}}^{(1)}), (\bar{0}, r_{\text{id}}^{(1)}, r^*, r_{\text{tg}}, \{(\text{idx}, \text{GID}, x_{\text{idx}}, \text{tg}_{\text{idx}})\}_{\text{idx} \neq \text{id}}))$. Otherwise, $\pi_{\text{id}} \leftarrow \text{NIZK.Sim}_{\text{id}}((\text{CRS}, \text{id}, \text{GID}, x_{\text{id}}, \text{tg}_{\text{id}}, \text{com}_{\text{id}}^{(0)}, \text{com}_{\text{id}}^{(1)}))$.

Set $\text{SK}_{\text{id}, \text{GID}, x} := (\text{id}, \text{GID}, x, \text{tg}_{\text{id}}, \pi_{\text{id}})$. Send $\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends $(P^*, (m_0, m_1))$ to Chal. Chal sample $\text{tg}_{P^*} \leftarrow \text{dFTS.SFuncTag}(\text{fts.msk}, P^*)$ and encrypts $\text{we.ct} \leftarrow \text{WE.Enc}(1^\lambda, (\text{CRS}, P^*, \text{tg}_{P^*}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}), m_b)$. Send $(\text{we.ct}, P, \text{tg}_{P^*})$ to $\mathcal{A}$.

Post-Challenge Queries. These are handled similarly to pre-challenge queries.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb₅^A(1^λ). In this hybrid, we set $\text{com}_{\text{id}}^{(0)}$ to be 1 for each id. Changes are highlighted in red.

Setup. $\mathcal{A}$ sends (n, s, L) . Sample $\text{fts.crs} \leftarrow \text{dFTS.GSetup}(1^\lambda, 1^n, 1^s, 1^L)$ and $\text{com.crs} \leftarrow \text{COM.Setup}(1^\lambda)$. Set $\text{CRS} := (n, s, L, \text{com.crs}, \text{fts.crs})$. In addition, sample $\text{fts.msk} \leftarrow \text{dFTS.KGen}(\text{fts.crs}; r^*)$ for $r^* \xleftarrow{\$} \{0, 1\}^\lambda$.

Sample $\forall \text{id} \in [n], \text{com}_{\text{id}}^{(0)} \leftarrow \text{COM.Com}(1^\lambda, 1; r_{\text{id}}^{(0)})$, $\text{com}_{\text{id}}^{(1)} \leftarrow \text{COM.Com}(1^\lambda, r^*; r_{\text{id}}^{(1)})$, and $\text{nizk.crs}_{\text{id}} \leftarrow \text{NIZK.Setup}(1^\lambda)$. (for $\mathcal{L}_{\text{NIZK}}$, Figure 5).

Set $\text{MPK}_{\text{id}} := (\text{com}_{\text{id}}^{(0)}, \text{com}_{\text{id}}^{(1)}, \text{nizk.crs}_{\text{id}})$ and sends $\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Pre-Challenge Queries. $\mathcal{A}$ sends $(\text{GID}, x_1, \dots, x_n)$. Sample $(\text{tg}_{\text{id}})_{\text{id} \in [n]} \leftarrow \text{dFTS.SInpTag}(\text{fts.msk}, (1, \text{GID}, x_1), \dots, (1, \text{GID}, x_n); r_{\text{tg}})$ for some $r_{\text{tg}} \xleftarrow{\$} \{0, 1\}^\lambda$.

For each $\text{id} \in [n]$, $\pi_{\text{id}} \leftarrow \text{NIZK.Prove}(\text{nizk.crs}_{\text{id}}, (\text{CRS}, \text{id}, \text{GID}, x_{\text{id}}, \text{tg}_{\text{id}}, \text{com}_{\text{id}}^{(0)}, \text{com}_{\text{id}}^{(1)}), (\bar{0}, r_{\text{id}}^{(1)}, r^*, r_{\text{tg}}, \{(\text{id}_x, \text{GID}, x_{\text{id}_x}, \text{tg}_{\text{id}_x})\}_{\text{id}_x \neq \text{id}}))$.

Set $\text{SK}_{\text{id}, \text{GID}, x} := (\text{id}, \text{GID}, x, \text{tg}_{\text{id}}, \pi_{\text{id}})$. Send $\{\text{SK}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends $(P^*, (m_0, m_1))$ to Chal. Chal sample $\text{tg}_{P^*} \leftarrow \text{dFTS.SFuncTag}(\text{fts.msk}, P^*)$ and encrypts $\text{we.ct} \leftarrow \text{WE.Enc}(1^\lambda, (\text{CRS}, P^*, \text{tg}_{P^*}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}), m_b)$. Send $(\text{we.ct}, P, \text{tg}_{P^*})$ to $\mathcal{A}$.

Post-Challenge Queries. These are handled similarly to pre-challenge queries.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb₆^A(1^λ). In this hybrid, we change the message encrypted by WE to be an all-zero string. Changes are highlighted in red.

Setup, Pre-Challenge Queries. Same as **Hyb₅^A(1^λ)**.

Challenge Query. $\mathcal{A}$ sends $(P^*, (m_0, m_1))$ to Chal. Chal sample $\text{tg}_{P^*} \leftarrow \text{dFTS.SFuncTag}(\text{fts.msk}, P^*)$ and encrypts $\text{we.ct} \leftarrow \text{WE.Enc}(1^\lambda, (\text{CRS}, P^*, \text{tg}_{P^*}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}), 0^{|m_b|})$. Send $(\text{we.ct}, P, \text{tg}_{P^*})$ to $\mathcal{A}$.

Post-Challenge Queries, Guess Phase. Same as **Hyb₅^A(1^λ)**.

Lemma Q.1. Assuming the mode indistinguishability property of dFTS, **Hyb₀^A(1^λ)** and **Hyb₁^A(1^λ)** are computationally indistinguishable.

Proof. Assume that there exists an adversary $\mathcal{A}$ that can distinguish between $\mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$ with non-negligible probability $\epsilon(\lambda)$, i.e.,

$$\left| \Pr \left[1 \leftarrow \mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda) \right] - \Pr \left[1 \leftarrow \mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda) \right] \right| > \epsilon(\lambda)$$

We will construct a reduction adversary $\mathcal{B}$ that can break the mode indistinguishability property of dFTS with non-negligible probability. The description of $\mathcal{B}^{\mathcal{O}}$ is as follows.

Setup. Receive fts.crs from $\mathcal{O}$. $\mathcal{A}$ sends (n, s, L) . Sample $\text{com.crs} \leftarrow \text{COM.Setup}(1^\lambda)$. Set $\text{CRS} := (n, s, L, \text{com.crs}, \text{fts.crs})$.

Sample $\forall \text{id} \in [n], \text{com}_{\text{id}}^{(0)} \leftarrow \text{COM.Com}(1^\lambda, 0; r_{\text{id}}^{(0)})$, $\text{com}_{\text{id}}^{(1)} \leftarrow \text{COM.Com}(1^\lambda, 0^\lambda; r_{\text{id}}^{(1)})$, and $\text{nizk.crs}_{\text{id}} \leftarrow \text{NIZK.Setup}(1^\lambda)$ (for $\mathcal{L}_{\text{NIZK}}$, Figure 5).

Set $\text{MPK}_{\text{id}} := (\text{com}_{\text{id}}^{(0)}, \text{com}_{\text{id}}^{(1)}, \text{nizk.crs}_{\text{id}})$ and sends $\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Pre-Challenge Queries. $\mathcal{A}$ sends $(\text{GID}, x_1, \dots, x_n)$. Send $(\text{GID}, x_1, \dots, x_n)$ to $\mathcal{O}$ to receive $\{\text{tg}_{\text{id}}\}_{\text{id} \in [n]}$.

For each $\text{id} \in [n]$, $\pi_{\text{id}} \leftarrow \text{NIZK.Prove}(\text{nizk.crs}_{\text{id}}, (\text{CRS}, \text{id}, \text{GID}, x_{\text{id}}, \text{tg}_{\text{id}}, \text{com}_{\text{id}}^{(0)}, \text{com}_{\text{id}}^{(1)}), (r_{\text{id}}^{(0)}, \bar{0}))$.

Set $\text{SK}_{\text{id}, \text{GID}, x} := (\text{id}, \text{GID}, x, \text{tg}_{\text{id}}, \pi_{\text{id}})$. Send $\{\text{SK}_{\text{id}, \text{GID}, x}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends $(P^*, (m_0, m_1))$. Sample $\text{tg}_{P^*} \leftarrow \mathcal{O}(P^*)$ and encrypts $\text{we.ct} \leftarrow \text{WE.Enc}(1^\lambda, (\text{CRS}, P^*, \text{tg}_{P^*}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}), m_b)$. Send $(\text{we.ct}, P, \text{tg}_{P^*})$ to $\mathcal{A}$.

Post-Challenge Queries. These are handled similarly to pre-challenge queries.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

As we can see, the running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$ and λ . If $\mathcal{O}$ uses dummy input tags and dummy function tags, $\mathcal{B}$ behaves like $\mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)$ and if $\mathcal{O}$ use smart input and function tags, $\mathcal{B}$ behaves like $\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$. Hence, $\mathcal{B}$ is a valid adversary against the mode indistinguishability property of dFTS that can break its security with probability $\epsilon(\lambda)$. Thus $\mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable. $\square$

Lemma Q.2. Assuming the computational hiding property of COM, $\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable.

Proof. Assume that there exists an adversary $\mathcal{A}$ that can distinguish between $\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ with non-negligible probability $\epsilon(\lambda)$, i.e.,

$$\left| \Pr \left[1 \leftarrow \mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda) \right] - \Pr \left[1 \leftarrow \mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda) \right] \right| > \epsilon(\lambda)$$

We will construct a reduction adversary $\mathcal{B}$ that can break the computational hiding property of COM with non-negligible probability. The description of $\mathcal{B}^{\mathcal{O}}$ is as follows.

Setup. Receive com.crs from $\mathcal{O}$. $\mathcal{A}$ sends (n, s, L) . Sample $\text{fts.crs} \leftarrow \text{dFTS.GSetup}(1^\lambda, 1^n, 1^s, 1^L)$.

Set $\text{CRS} := (n, s, L, \text{com.crs}, \text{fts.crs})$. In addition, sample $\text{fts.msk} \leftarrow \text{dFTS.KGen}(\text{fts.crs}; r^*)$ for $r^* \xleftarrow{\$} \{0, 1\}^\lambda$.

Sample $\forall \text{id} \in [n], \text{com}_{\text{id}}^{(0)} \leftarrow \text{COM.Com}(1^\lambda, 0; r_{\text{id}}^{(0)})$, $\text{com}_{\text{id}}^{(1)} \leftarrow \mathcal{O}(0^\lambda, r^*)$, and $\text{nizk.crs}_{\text{id}} \leftarrow \text{NIZK.Setup}(1^\lambda)$ (for $\mathcal{L}_{\text{NIZK}}$, Figure 5).

Set $\text{MPK}_{\text{id}} := (\text{com}_{\text{id}}^{(0)}, \text{com}_{\text{id}}^{(1)}, \text{nizk.crs}_{\text{id}})$ and sends $\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Pre-Challenge Queries, Challenge Query, Post-Challenge Queries, Guess Phase. Similar to $\mathbf{Hyb}_2^A(1^\lambda)$.

As we can see, the running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$ and λ . If $\mathcal{O}$ returns a commitment of r^* , $\mathcal{B}$ behaves like $\mathbf{Hyb}_2^A(1^\lambda)$ and if $\mathcal{O}$ returns a commitment of 0^λ , $\mathcal{B}$ behaves like $\mathbf{Hyb}_1^A(1^\lambda)$. Hence, $\mathcal{B}$ is a valid adversary against the computational hiding property of COM that can break its security with probability $\epsilon(\lambda)$. Thus $\mathbf{Hyb}_1^A(1^\lambda)$ and $\mathbf{Hyb}_2^A(1^\lambda)$ are computationally indistinguishable. $\square$

Lemma Q.3. $\mathbf{Hyb}_2^A(1^\lambda)$ and $\mathbf{Hyb}_{3,1}^A(1^\lambda)$ are identical.

Proof. As we are not changing any NIZK proofs in $\mathbf{Hyb}_{3,1}^A(1^\lambda)$, these two hybrids are identically distributed. $\square$

Lemma Q.4. Assuming the computational zero-knowledge property of NIZK, $\mathbf{Hyb}_{3,j}^A(1^\lambda)$, $\mathbf{Hyb}_{3,j+1}^A(1^\lambda)$ for $j \in [n]$ are computationally indistinguishable.

Proof. Assume that there exists an adversary $\mathcal{A}$ that can distinguish between $\mathbf{Hyb}_{3,j}^A(1^\lambda)$ and $\mathbf{Hyb}_{3,j+1}^A(1^\lambda)$ with non-negligible probability $\epsilon(\lambda)$, i.e.,

$$\left| \Pr \left[1 \leftarrow \mathbf{Hyb}_{3,j}^A(1^\lambda) \right] - \Pr \left[1 \leftarrow \mathbf{Hyb}_{3,j+1}^A(1^\lambda) \right] \right| > \epsilon(\lambda)$$

We will construct a reduction adversary $\mathcal{B}$ that can break the computational zero-knowledge property of NIZK with non-negligible probability. The description of $\mathcal{B}^\mathcal{O}$ is as follows.

Setup. Receive nizk.crs from $\mathcal{O}$. $\mathcal{A}$ sends (n, s, L) . Sample $\text{fts.crs} \leftarrow \text{dFTS.GSetup}(1^\lambda, 1^n, 1^s, 1^L)$ and $\text{com.crs} \leftarrow \text{COM.Setup}(1^\lambda)$. Set $\text{CRS} := (n, s, L, \text{com.crs}, \text{fts.crs})$. In addition, sample $\text{fts.msk} \leftarrow \text{dFTS.KGen}(\text{fts.crs}; r^*)$ for $r^* \xleftarrow{\$} \{0, 1\}^\lambda$.

Sample $\forall \text{id} \in [n]$, $\text{com}_{\text{id}}^{(0)} \leftarrow \text{COM.Com}(1^\lambda, 0; r_{\text{id}}^{(0)})$, $\text{com}_{\text{id}}^{(1)} \leftarrow \text{COM.Com}(1^\lambda, r^*; r_{\text{id}}^{(1)})$. If $\text{id} < j$, $\text{nizk.crs}_{\text{id}} \leftarrow \text{NIZK.Sim}_{\text{id}}(1^\lambda)$. Otherwise, if $\text{id} = j$, set $\text{nizk.crs}_{\text{id}} := \text{nizk.crs}$. Otherwise, $\text{nizk.crs}_{\text{id}} \leftarrow \text{NIZK.Setup}(1^\lambda)$ (for $\mathcal{L}_{\text{NIZK}}$, Figure 5).

Set $\text{MPK}_{\text{id}} := (\text{com}_{\text{id}}^{(0)}, \text{com}_{\text{id}}^{(1)}, \text{nizk.crs}_{\text{id}})$ and sends $\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Pre-Challenge Queries. $\mathcal{A}$ sends $(\text{GID}, x_1, \dots, x_n)$. Sample $(\text{tg}_{\text{id}})_{\text{id} \in [n]} \leftarrow \text{dFTS.SInpTag}(\text{fts.msk}, (1, \text{GID}, x_1), \dots, (1, \text{GID}, x_n))$.

For each $\text{id} \in [n]$, if $\text{id} < j$, $\pi_{\text{id}} \leftarrow \text{NIZK.Sim}_{\text{id}}((\text{CRS}, \text{id}, \text{GID}, x_{\text{id}}, \text{tg}_{\text{id}}, \text{com}_{\text{id}}^{(0)}, \text{com}_{\text{id}}^{(1)}))$. Otherwise, if $\text{id} = j$, send $((\text{CRS}, \text{id}, \text{GID}, x_{\text{id}}, \text{tg}_{\text{id}}, \text{com}_{\text{id}}^{(0)}, \text{com}_{\text{id}}^{(1)}), (r_{\text{id}}^{(0)}, \bar{0}))$ to $\mathcal{O}$ to receive $\text{nizk}.\pi_{\text{id}}$. Otherwise, $\pi_{\text{id}} \leftarrow \text{NIZK.Prove}(\text{nizk.crs}_{\text{id}}, (\text{CRS}, \text{id}, \text{GID}, x_{\text{id}}, \text{tg}_{\text{id}}, \text{com}_{\text{id}}^{(0)}, \text{com}_{\text{id}}^{(1)}), (r_{\text{id}}^{(0)}, \bar{0}))$.

Set $\text{SK}_{\text{id}, \text{GID}, x} := (\text{id}, \text{GID}, x, \text{tg}_{\text{id}}, \pi_{\text{id}})$. Send $\{\text{SK}_{\text{id}, \text{GID}, x}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends $(P^*, (m_0, m_1))$ to Chal. Chal sample $\text{tg}_{P^*} \leftarrow \text{dFTS.SFuncTag}(\text{fts.msk}, P^*)$ and encrypts $\text{we.ct} \leftarrow \text{WE.Enc}(1^\lambda, (\text{CRS}, P^*, \text{tg}_{P^*}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}), m_b)$. Send $(\text{we.ct}, P, \text{tg}_{P^*})$ to $\mathcal{A}$.

Post-Challenge Queries. These are handled similarly to pre-challenge queries.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

As we can see, the running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$ and λ . If $\mathcal{O}$ is a simulator, $\mathcal{B}$ behaves like $\mathbf{Hyb}_{3,j+1}^{\mathcal{A}}(1^\lambda)$ and if $\mathcal{O}$ honestly generates proofs using $(r_{\text{id}}^{(0)}, \bar{0})$, $\mathcal{B}$ behaves like $\mathbf{Hyb}_{3,j}^{\mathcal{A}}(1^\lambda)$. Hence, $\mathcal{B}$ is a valid adversary against computational zero-knowledge of NIZK that can break its security with probability $\epsilon(\lambda)$. Thus $\mathbf{Hyb}_{3,j}^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_{3,j+1}^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable. $\square$

Lemma Q.5. $\mathbf{Hyb}_{3,n+1}^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_{4,1}^{\mathcal{A}}(1^\lambda)$ are identical.

Proof. As we are not honestly instantiating any NIZK for any id in $\mathbf{Hyb}_{4,1}^{\mathcal{A}}(1^\lambda)$, these two hybrids are identically distributed. $\square$

Lemma Q.6. Assuming the computational zero-knowledge property of NIZK, $\mathbf{Hyb}_{4,j}^{\mathcal{A}}(1^\lambda)$, $\mathbf{Hyb}_{4,j+1}^{\mathcal{A}}(1^\lambda)$ for $j \in [n]$ are computationally indistinguishable.

Proof. The proof of this lemma is similar to proof of Lemma Q.4. $\square$

Lemma Q.7. Assuming the computational hiding property of COM, $\mathbf{Hyb}_{4,n+1}^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_5^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable.

Proof. As we are not revealing $r_{\text{id}}^{(0)}$ for any id $\in [n]$ in these hybrids, computational hiding holds and the proof of this lemma is similar to proof of Lemma Q.2. $\square$

Lemma Q.8. Assuming the security of WE, $\mathbf{Hyb}_5^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_6^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable.

Proof. This follows because in both these hybrid, the instance used for WE during encryption is no longer in $\mathcal{L}_{\text{WE}}$ (Figure 6). To prove this assume for contradiction that there is a witness for the statement. This would mean that $P^*(x_1, \dots, x_n) = 1$. That means we have each π_{id} for id $\in [n]$ be valid proofs. Since each $\text{com}_{\text{id}}^{(0)}$ is a commitment of 1 in both these hybrid, and due to the statistical binding property of COM, $\mathcal{A}$ cannot produce another r' such that any $\text{com}_{\text{id}}^{(0)}$ is a commitment of 1. Thus any verifying π_{id} would imply that $\text{com}_{\text{id}}^{(1)}$ is a commitment of r^* and that $(\text{tg}_i)_{i \in [n]}$ are smart input tags. Hence, by smart correctness of dFTS, $\text{dFTS.Trig}(\text{tg}_{P^*}, (\text{tg}_i)_{i \in [n]}) = 1$. But if π_i verifies, it means that $\text{dFTS.Trig}(\text{tg}_{P^*}, (\text{tg}_i)_{i \in [n]}) = 0$ which leads us to a contradiction. As NIZK is statistically sound, π_{id} verifies for instances not in $\mathcal{L}_{\text{NIZK}}$ (Figure 5) only with negligible probability. Thus, with all but negligible probability, WE ciphertext's instance is invalid. Hence, by security of WE, these two hybrids are indistinguishable. $\square$

R Full MA-ABE from VNCE

Construction R.1. We provide the construction of an MA-ABE scheme for P/Poly circuits and $\mathcal{GID} = \{\mathcal{GID}_\lambda\}_{\lambda \in \mathbb{N}}$ using an MA-ABE scheme with batched key generation (abMA-ABE) for P/Poly circuits (Definition 7.3) and $\mathcal{GID}$, and a VNCE scheme for $\mathcal{GID}$ (Definition L.1) as follows.

$\text{GSetup}(1^\lambda, 1^n, 1^L, 1^s)$. Sample $\text{abe.crs} \leftarrow \text{abMA-ABE.GSetup}(1^\lambda, 1^n, 1^L, 1^s)$ and $\text{nce.crs} \leftarrow \text{VNCE.GSetup}(1^\lambda, 1^n, 1^{L'})$ where $L' = \text{poly}(\lambda, n, s, L)$ is the maximum length of abMA-ABE secret keys for any authority. Output $\text{CRS} := (\text{abe.crs}, \text{nce.crs})$.

AS_{id}(id, 1^{ℓ_{id}}). Given $\text{id} \in [n]$, $\ell_{\text{id}} \leq L$, sample abMA-ABE keys, $(\text{abe.mpk}, \text{abe.msk}) \leftarrow \text{abMA-ABE.ASetup}(\text{id}, 1^{\ell_{\text{id}}})$ and VNCE keys $(\text{nce.mpk}, \text{nce.msk}) \leftarrow \text{VNCE.ASetup}(\text{id})$. Output $\text{MPK} := (\text{abe.mpk}, \text{nce.mpk})$ and $\text{MSK} := (\text{abe.msk}, \text{nce.msk})$.

KGen(id, MSK_{id}, {MPK_{idx}}_{idx ∈ [n]}, GID, x). Parse MSK_{id} as $(\text{abe.msk}_{\text{id}}, \text{nce.msk}_{\text{id}})$ and for each $\text{idx} \in [n]$, MPK_{idx} as $(\text{abe.mpk}_{\text{idx}}, \text{nce.mpk}_{\text{idx}})$.

Sample an abMA-ABE secret key $\text{abe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{abMA-ABE.KGen}(\text{id}, \text{abe.msk}_{\text{id}}, \{\text{abe.mpk}_{\text{idx}}\}_{\text{idx} \in [n]}, \text{GID}, x)$. Now, encrypt this string under VNCE, $\text{nce.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{VNCE.Enc}(\text{id}, \text{nce.msk}_{\text{id}}, \{\text{nce.mpk}_{\text{idx}}\}_{\text{idx} \in [n]}, \text{GID}, \text{abe.sk}_{\text{id}, \text{GID}, x})$.

Output $\text{SK}_{\text{id}, \text{GID}, x} := \text{nce.sk}_{\text{id}, \text{GID}, x}$.

Enc({MPK_{id}}_{id ∈ [n]}, P, m). Parse MPK_{id} as $(\text{abe.mpk}_{\text{id}}, \text{nce.mpk}_{\text{id}})$ for each $\text{id} \in [n]$.

Sample an abMA-ABE ciphertext for C , $\text{abe.ct} \leftarrow \text{abMA-ABE.Enc}(\{\text{abe.mpk}_{\text{id}}\}_{\text{id} \in [n]}, P, m)$.

Output $\text{CT} := \text{abe.ct}$.

Dec({SK_{id, GID, x_{id}}}_{id ∈ [n]}, CT). Parse SK_{id, GID, x_{id}} as $\text{nce.sk}_{\text{id}, \text{GID}, x_{\text{id}}}$ for each $\text{id} \in [n]$ and CT as abe.ct .

Compute $\{\text{abe.sk}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]} = \text{VNCE.Dec}(\{\text{nce.sk}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]})$. If this aborts, then abort and output $\perp$.

Otherwise, compute and output $m' = \text{abMA-ABE.Dec}(\{\text{abe.sk}_{\text{id}, \text{GID}, x_{\text{id}}}\}_{\text{id} \in [n]}, \text{abe.ct})$.

Correctness. Correctness of Construction R.1 follows from correctness of abMA-ABE and VNCE.

Proof of Theorem 7.6. We prove this theorem using the following hybrids and lemmas.

Hyb₀^A(1^λ). This is the real experiment with Chal with $b = 0/1$.

Setup Phase. $\mathcal{A}$ sends $n, s, \ell_1, \dots, \ell_n$. Define $L = \max_{\text{id}} \{\ell_{\text{id}}\}$. Chal samples $\text{abe.crs} \leftarrow \text{abMA-ABE.GSetup}(1^\lambda, 1^n, 1^s, 1^L)$ and $\text{nce.crs} \leftarrow \text{VNCE.GSetup}(1^\lambda, 1^n, 1^{L'})$. Chal sets $\text{CRS} := (\text{abe.crs}, \text{nce.crs})$.

Chal then samples for each $\text{id} \in [n]$, $(\text{abe.mpk}_{\text{id}}, \text{abe.msk}_{\text{id}}) \leftarrow \text{abMA-ABE.ASetup}(\text{id}, 1^{\ell_{\text{id}}})$ and $(\text{nce.mpk}, \text{nce.msk}_{\text{id}}) \leftarrow \text{VNCE.ASetup}(\text{id})$. Set $\text{MPK}_{\text{id}} := (\text{abe.mpk}_{\text{id}}, \text{nce.mpk}_{\text{id}})$ and $\text{MSK}_{\text{id}} := (\text{abe.msk}_{\text{id}}, \text{nce.msk}_{\text{id}})$.

Chal sends $\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Pre-challenge Queries. $\mathcal{A}$ sends queries of the form $(\text{id}, \text{GID}, x)$. Sample $\text{abe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{abMA-ABE.KGen}(\text{id}, \text{abe.msk}_{\text{id}}, \{\text{abe.mpk}_{\text{idx}}\}_{\text{idx} \in [n]}, \text{GID}, x)$. Then encrypt using VNCE, $\text{nce.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{VNCE.Enc}(\text{id}, \text{nce.msk}_{\text{id}}, \{\text{nce.mpk}_{\text{idx}}\}_{\text{idx} \in [n]}, \text{GID}, \text{abe.sk}_{\text{id}, \text{GID}, x})$.

Send $\text{SK}_{\text{id}, \text{GID}, x} := \text{nce.sk}_{\text{id}, \text{GID}, x}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends P, m_0, m_1 . Chal samples $\text{abe.ct} \leftarrow \text{abMA-ABE.Enc}(\{\text{abe.mpk}_{\text{id}}\}_{\text{id} \in [n]}, P, m_b)$ and sends $\text{CT} := \text{abe.ct}$ to $\mathcal{A}$.

Post-challenge Queries. This is same as pre-challenge queries.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb₁^A(1^λ). In this hybrid, we simulate the VNCE instantiation. Changes are highlighted in red.

Setup Phase. $\mathcal{A}$ sends $n, s, \ell_1, \dots, \ell_n$. Define $L = \max_{\text{id}} \{\ell_{\text{id}}\}$. Sample $\text{abe.crs} \leftarrow \text{abMA-ABE.GSetup}(1^\lambda, 1^n, 1^s, 1^L)$. Then, for each $\text{id} \in [n]$, $\text{abe.mpk}_{\text{id}}, \text{abe.msk}_{\text{id}} \leftarrow \text{abMA-ABE.ASetup}(\text{id}, 1^{\ell_{\text{id}}})$. Also, $(\text{nce.crs}, \{\text{nce.mpk}_{\text{id}}\}_{\text{id} \in [n]}) \leftarrow \text{VNCE.Sim}(1^\lambda, 1^n, 1^{L'})$. Also initiate an empty dictionary of queries $\mathcal{Q}$.

Set $\text{CRS} := (\text{abe.crs}, \text{nce.crs})$. Set $\text{MPK}_{\text{id}} := (\text{abe.mpk}_{\text{id}}, \text{nce.mpk}_{\text{id}})$ and $\text{MSK}_{\text{id}} := \text{abe.msk}_{\text{id}}$.

Send $\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Pre-challenge Queries. $\mathcal{A}$ sends queries of the form $(\text{id}, \text{GID}, x)$.

- If this is the last query for this GID, sample for each $\text{id} \in [n]$, $\text{abe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{abMA-ABE.KGen}(\text{id}, \text{abe.msk}_{\text{id}}, \{\text{abe.mpk}_{\text{id}, x}\}_{\text{id}, x \in [n]}, \text{GID}, x_{\text{id}, \text{GID}})$ by retrieving $((1, \text{GID}, x_{1, \text{GID}}), \dots, (n, \text{GID}, x_{n, \text{GID}}))$ from $\mathcal{Q}$. Let $V := (\text{abe.sk}_{1, \text{GID}, x_{1, \text{GID}}}, \dots, \text{abe.sk}_{n, \text{GID}, x_{n, \text{GID}}})$. Sample $\text{nce.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{VNCE.Sim}(\text{id}, \text{GID}, x, V)$.
- Otherwise, add $(\text{id}, \text{GID}, x)$ to $\mathcal{Q}$ and simulate $\text{nce.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{VNCE.Sim}(\text{id}, \text{GID}, x, \perp)$.

Send $\text{SK}_{\text{id}, \text{GID}, x} := \text{nce.sk}_{\text{id}, \text{GID}, x}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends P, m_0, m_1 . Sample $\text{abe.ct} \leftarrow \text{abMA-ABE.Enc}(\{\text{abe.mpk}_{\text{id}}\}_{\text{id} \in [n]}, P, m_b)$ and sends $\text{CT} := \text{abe.ct}$ to $\mathcal{A}$.

Post-challenge Queries. This is same as pre-challenge queries.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb₂^A(1^λ). In this hybrid, we change the message encrypted in abMA-ABE to m_{1-b} . Changes are highlighted in red.

Setup Phase. $\mathcal{A}$ sends $n, s, \ell_1, \dots, \ell_n$. Define $L = \max_{\text{id}} \{\ell_{\text{id}}\}$. Sample $\text{abe.crs} \leftarrow \text{abMA-ABE.GSetup}(1^\lambda, 1^n, 1^s, 1^L)$. Then, for each $\text{id} \in [n]$, $\text{abe.mpk}_{\text{id}}, \text{abe.msk}_{\text{id}} \leftarrow \text{abMA-ABE.ASetup}(\text{id}, 1^{\ell_{\text{id}}})$. Also, $(\text{nce.crs}, \{\text{nce.mpk}_{\text{id}}\}_{\text{id} \in [n]}) \leftarrow \text{VNCE.Sim}(1^\lambda, 1^n, 1^{L'})$. Also initiate an empty dictionary of queries $\mathcal{Q}$.

Set $\text{CRS} := (\text{abe.crs}, \text{nce.crs})$. Set $\text{MPK}_{\text{id}} := (\text{abe.mpk}_{\text{id}}, \text{nce.mpk}_{\text{id}})$ and $\text{MSK}_{\text{id}} := (\text{abe.msk}_{\text{id}}, \text{nce.msk}_{\text{id}})$.

Send $\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Pre-challenge Queries. $\mathcal{A}$ sends queries of the form $(\text{id}, \text{GID}, x)$.

- If this is the last query for this GID, sample for each $\text{id} \in [n]$, $\text{abe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{abMA-ABE.KGen}(\text{id}, \text{abe.msk}_{\text{id}}, \{\text{abe.mpk}_{\text{id}, x}\}_{\text{id}, x \in [n]}, \text{GID}, x_{\text{id}, \text{GID}})$ by retrieving $((1, \text{GID}, x_{1, \text{GID}}), \dots, (n, \text{GID}, x_{n, \text{GID}}))$ from $\mathcal{Q}$. Let $V := (\text{abe.sk}_{1, \text{GID}, x_{1, \text{GID}}}, \dots, \text{abe.sk}_{n, \text{GID}, x_{n, \text{GID}}})$. Sample $\text{nce.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{VNCE.Sim}(\text{id}, \text{GID}, x, V)$.
- Otherwise, add $(\text{id}, \text{GID}, x)$ to $\mathcal{Q}$ and simulate $\text{nce.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{VNCE.Sim}(\text{id}, \text{GID}, x, \perp)$.

Send $\text{SK}_{\text{id}, \text{GID}, x} := \text{nce.sk}_{\text{id}, \text{GID}, x}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends P, m_0, m_1 . Sample $\text{abe.ct} \leftarrow \text{abMA-ABE.Enc}(\{\text{abe.mpk}_{\text{id}}\}_{\text{id} \in [n]}, P, m_{1-b})$ and sends $\text{CT} := \text{abe.ct}$ to $\mathcal{A}$.

Post-challenge Queries. This is same as pre-challenge queries.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

$\text{Hyb}_3^{\mathcal{A}}(1^\lambda)$. This is experiment Hybrid 0 with $1 - b$.

Lemma R.2. Assuming the security of VNCE, $\text{Hyb}_0^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_1^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable.

Proof. Assume that there exists an adversary $\mathcal{A}$ that can distinguish between $\text{Hyb}_0^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_1^{\mathcal{A}}(1^\lambda)$ with non-negligible probability $\epsilon(\lambda)$, i.e.,

$$\left| \Pr \left[1 \leftarrow \text{Hyb}_0^{\mathcal{A}}(1^\lambda) \right] - \Pr \left[1 \leftarrow \text{Hyb}_1^{\mathcal{A}}(1^\lambda) \right] \right| > \epsilon(\lambda)$$

We will construct a reduction adversary $\mathcal{B}$ that can break the security of VNCE with non-negligible probability. The description of $\mathcal{B}^{\mathcal{O}}$ is as follows.

Setup Phase. $\mathcal{A}$ sends $n, s, \ell_1, \dots, \ell_n$. Define $L = \max_{\text{id}} \{\ell_{\text{id}}\}$. Sample $\text{abe.crs} \leftarrow \text{abMA-ABE.GSetup}(1^\lambda, 1^n, 1^s, 1^L)$. Then, for each $\text{id} \in [n]$, $(\text{abe.mpk}_{\text{id}}, \text{abe.msk}_{\text{id}}) \leftarrow \text{abMA-ABE.ASetup}(\text{id}, 1^{\ell_{\text{id}}})$. Receive, $(\text{nce.crs}, \{\text{nce.mpk}_{\text{id}}\}_{\text{id} \in [n]})$ from $\mathcal{O}$. Also initiate an empty dictionary of queries $\mathcal{Q}$.

Set $\text{CRS} := (\text{abe.crs}, \text{nce.crs})$. Set $\text{MPK}_{\text{id}} := (\text{abe.mpk}_{\text{id}}, \text{nce.mpk}_{\text{id}})$ and $\text{MSK}_{\text{id}} := \text{abe.msk}_{\text{id}}$.

Send $\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Pre-challenge Queries. $\mathcal{A}$ sends queries of the form $(\text{id}, \text{GID}, x)$. Sample $\text{abe.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{abMA-ABE.KGen}(\text{id}, \text{abe.msk}_{\text{id}}, \{\text{abe.mpk}_{\text{id}, x}\}_{\text{id}, x \in [n]}, \text{GID}, x)$. Then send $(\text{id}, \text{GID}, \text{abe.sk}_{\text{id}, \text{GID}, x})$ to $\mathcal{O}$ to receive $\text{nce.sk}_{\text{id}, \text{GID}, x}$.

Send $\text{SK}_{\text{id}, \text{GID}, x} := \text{nce.sk}_{\text{id}, \text{GID}, x}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends P, m_0, m_1 . Sample $\text{abe.ct} \leftarrow \text{abMA-ABE.Enc}(\{\text{abe.mpk}_{\text{id}}\}_{\text{id} \in [n]}, P, m_b)$ and sends $\text{CT} := \text{abe.ct}$ to $\mathcal{A}$.

Post-challenge Queries. This is same as pre-challenge queries.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

As we can see, the running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$, λ . If $\mathcal{O}$ is an honest challenger for VNCE $\mathcal{B}$ behaves like $\text{Hyb}_0^{\mathcal{A}}(1^\lambda)$ and if $\mathcal{O}$ simulates VNCE $\mathcal{B}$ behaves like $\text{Hyb}_1^{\mathcal{A}}(1^\lambda)$. Hence, $\mathcal{B}$ is a valid adversary that break the security of VNCE with non-negligible probability. Thus, $\text{Hyb}_0^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_1^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable. $\square$

Lemma R.3. Assuming the security of abMA-ABE, $\text{Hyb}_1^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_2^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable.

Proof. Assume that there exists an adversary $\mathcal{A}$ that can distinguish between $\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ with non-negligible probability $\epsilon(\lambda)$, i.e.,

$$\left| \Pr \left[1 \leftarrow \mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda) \right] - \Pr \left[1 \leftarrow \mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda) \right] \right| > \epsilon(\lambda)$$

We will construct a reduction adversary $\mathcal{B}$ that can break the security of abMA-ABE with non-negligible probability. The description of $\mathcal{B}^\mathcal{O}$ is as follows.

Setup Phase. $\mathcal{A}$ sends $n, s, \ell_1, \dots, \ell_n$. Define $L = \max_{\text{id}} \{\ell_{\text{id}}\}$. Receive $\text{abe.crs}, \{\text{abe.mpk}_{\text{id}}\}_{\text{id} \in [n]}$ from $\mathcal{O}$. Also, $(\text{ncc.crs}, \{\text{ncc.mpk}_{\text{id}}\}_{\text{id} \in [n]}) \leftarrow \text{VNCE.Sim}(1^\lambda, 1^n, 1^{L'})$. Also, initiate an empty dictionary of queries $\mathcal{Q}$.

Set $\text{CRS} := (\text{abe.crs}, \text{ncc.crs})$. Set $\text{MPK}_{\text{id}} := (\text{abe.mpk}_{\text{id}}, \text{ncc.mpk}_{\text{id}})$.

Send $\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]}$ to $\mathcal{A}$.

Pre-challenge Queries. $\mathcal{A}$ sends queries of the form $(\text{id}, \text{GID}, x)$.

- If this is the last query for this GID, retrieve $((1, \text{GID}, x_{1, \text{GID}}), \dots, (n, \text{GID}, x_{n, \text{GID}}))$ from $\mathcal{Q}$. Send $\text{GID}, x_{1, \text{GID}}, \dots, x_{n, \text{GID}}$ to $\mathcal{O}$ to receive $\text{abe.sk}_{1, \text{GID}, x_{1, \text{GID}}}, \dots, \text{abe.sk}_{n, \text{GID}, x_{n, \text{GID}}}$. Let $V := (\text{abe.sk}_{1, \text{GID}, x_{1, \text{GID}}}, \dots, \text{abe.sk}_{n, \text{GID}, x_{n, \text{GID}}})$. Sample $\text{ncc.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{VNCE.Sim}(\text{id}, \text{GID}, x, V)$.
- Otherwise, add $(\text{id}, \text{GID}, x)$ to $\mathcal{Q}$ and simulate $\text{ncc.sk}_{\text{id}, \text{GID}, x} \leftarrow \text{VNCE.Sim}(\text{id}, \text{GID}, x, \perp)$.

Send $\text{SK}_{\text{id}, \text{GID}, x} := \text{ncc.sk}_{\text{id}, \text{GID}, x}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends P, m_0, m_1 . Send P, m_b, m_{1-b} to $\mathcal{O}$ to receive abe.ct and sends $\text{CT} := \text{abe.ct}$ to $\mathcal{A}$.

Post-challenge Queries. This is same as pre-challenge queries.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

As we can see, the running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$, λ . If $\mathcal{O}$ encryption m_b $\mathcal{B}$ behaves like $\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$ and if $\mathcal{O}$ encrypts m_{1-b} $\mathcal{B}$ behaves like $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$. Hence, $\mathcal{B}$ is a valid adversary that break the security of abMA-ABE with non-negligible probability. Thus, $\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable. $\square$

Lemma R.4. Assuming the security of VNCE, $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$ are computationally indistinguishable.

Proof. The proof of this lemma is similar to proof of Lemma R.2. $\square$

$\square$

S Proofs from Section 7.4

In this section, we provide full proof of Theorem 7.8.

Hyb₀^A(1^λ). This is the real experiment with Chal.

Setup. $\mathcal{A}$ sends $n, s, k, \chi, \ell_1, \dots, \ell_n$ to Chal. Let $L = \max_{\text{id}} \{\ell_{\text{id}}\}$. For each $i \in \mathbb{I}, j \in \mathbb{J}$, Chal samples $\text{crs}^{(i,j)} \leftarrow \text{MA-ABE.GSetup}(1^\lambda, 1^{n-k}, 1^s, 1^L)$. In addition, sample for each $\text{id} \in [n]$, $(\text{mpk}_{\text{id}}^{(i,j)}, \text{msk}_{\text{id}}^{(i,j)}) \leftarrow \text{MA-ABE.ASetup}(\text{id}, 1^{\ell_{\text{id}}})$ for i such that $\text{id} \notin \mathcal{K}_i$. Set $\text{CRS} := (\text{crs}^{(i,j)})_{i,j}$, $\text{MPK}_{\text{id}} := (\text{mpk}_{\text{id}}^{(i,j)})_{i,j}$, and $\text{MSK}_{\text{id}} := (\text{msk}_{\text{id}}^{(i,j)})_{i,j}$ and send $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]})$ to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$ to Chal such that id is not corrupted. For each $i \in \mathbb{I}'_{\text{id}}, j \in \mathbb{J}$, sample $\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \text{MA-ABE.KGen}(\text{id}, \text{msk}_{\text{id}}^{(i,j)}, \{\text{mpk}_{\text{id}_x}^{(i,j)}\}_{\text{id}_x}, \text{GID}, x)$. Set $\text{SK}_{\text{id}, \text{GID}, x} = (\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)})_{i \in \mathbb{I}'_{\text{id}}, j}$ and send it to $\mathcal{A}$.

Corruption Queries. $\mathcal{A}$ sends (Corr, id) to Chal. Send $(\text{msk}_{\text{id}}^{(i,j)})_{i,j}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends P, m_0, m_1 to Chal. Chal samples $\{\text{sh}_i\}_{i \in \mathbb{I}'}$ such that $\bigoplus_{i \in \mathbb{I}'} \text{sh}_i = m_b$. For each $i \in \mathbb{I}', j \in \mathbb{J}$, sample $\text{ct}^{(i,j)} \leftarrow \text{MA-ABE.Enc}(\{\text{mpk}_{\text{id}}^{(i,j)}\}_{\text{id} \notin \mathcal{K}_i}, F_{ij}, \text{sh}_i)$. Set $\text{CT} = (\text{ct}^{(i,j)})_{i,j}$ and send it to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$ to Chal such that id is not corrupted. For each $i \in \mathbb{I}'_{\text{id}}, j \in \mathbb{J}$, sample $\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \text{MA-ABE.KGen}(\text{id}, \text{msk}_{\text{id}}^{(i,j)}, \{\text{mpk}_{\text{id}_x}^{(i,j)}\}_{\text{id}_x}, \text{GID}, x)$. Set $\text{SK}_{\text{id}, \text{GID}, x} = (\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)})_{i,j}$ and send it to $\mathcal{A}$.

Corruption Queries. $\mathcal{A}$ sends (Corr, id) to Chal. Send $(\text{msk}_{\text{id}}^{(i,j)})_{i,j}$ to $\mathcal{A}$.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

Hyb_{1,κ}^A(1^λ). For $\kappa \in [2^\chi + 1]$. In this hybrid, we randomly select $i^* \leftarrow \mathbb{I}$ and set sh_{i^*} to be a uniformly random string in the $(i^*, 1), \dots, (i^*, \kappa - 1)$ instantiations of MA-ABE. The changes are highlighted in red.

Setup, Secret Key Queries, Corruption Queries. Same as **Hyb₀^A(1^λ)**.

Challenge Query. $\mathcal{A}$ sends P, m_0, m_1 . Sample $\{\tilde{C}_i\}_{i \in \mathbb{I}'}$ such that $\bigoplus_{i \in \mathbb{I}'} \text{sh}_i = m_b$ and $\text{sh}'_{i^*} \xleftarrow{\$} \{0, 1\}^{|m_b|}$. For each $i \in \mathbb{I}', j \in \mathbb{J}$, sample

- If $i \neq i^*$ or $i = i^*$ and $j \geq \kappa$, $\text{ct}^{(i,j)} \leftarrow \text{MA-ABE.Enc}(\{\text{mpk}_{\text{id}}^{(i,j)}\}_{\text{id} \notin \mathcal{K}_i}, F_{ij}, \text{sh}_i)$.
- Otherwise, sample $\text{ct}^{(i,j)} \leftarrow \text{MA-ABE.Enc}(\{\text{mpk}_{\text{id}}^{(i,j)}\}_{\text{id} \notin \mathcal{K}_i}, F_{ij}, \text{sh}'_{i^*})$.

Set $\text{CT} = (\text{ct}^{(i,j)})_{i,j}$ and send it to $\mathcal{A}$.

Secret Key Queries, Corruption Queries, Guess Phase. Same as **Hyb₀^A(1^λ)**.

Hyb₂^A(1^λ). In this hybrid, we encrypt a random string. The changes are highlighted in red.

Setup, Secret Key Queries, Corruption Queries. Same as **Hyb_{1,2^χ+1}^A(1^λ)**.

Challenge Query. $\mathcal{A}$ sends P, m_0, m_1 . Sample $\{\text{sh}_i\}_{i \in \mathbb{I}'}$ such that $\bigoplus_{i \in \mathbb{I}'} \text{sh}_i = m'$ and $m' \xleftarrow{\$} \{0, 1\}^{|m_0|}$. For each $i \in \mathbb{I}', j \in \mathbb{J}$, sample $\text{ct}^{(i,j)} \leftarrow \text{MA-ABE.Enc}(\{\text{mpk}_{\text{id}}^{(i)}\}_{\text{id} \notin \mathcal{K}_i}, F_{ij}, \text{sh}_i)$. Set $\text{CT} = (\text{ct}^{(i,j)})_{i,j}$ and send it to $\mathcal{A}$.

Secret Key Queries, Corruption Queries, Guess Phase. Same as $\text{Hyb}_{1,2\kappa+1}^{\mathcal{A}}(1^\lambda)$.

Lemma S.1. $\text{Hyb}_0^{\mathcal{A}}(1^\lambda), \text{Hyb}_{1,1}^{\mathcal{A}}(1^\lambda)$ are identically distributed.

Proof. As we are not changing the message in any MA-ABE instantiations in $\text{Hyb}_{1,1}^{\mathcal{A}}(1^\lambda)$, and simply sampling i^* , these hybrids are identically distributed. $\square$

Lemma S.2. Assuming the security of MA-ABE, $\text{Hyb}_{1,\kappa}^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_{1,\kappa+1}^{\mathcal{A}}(1^\lambda)$ for $\kappa \in [\mathbb{J}]$ are computationally indistinguishable.

Proof. Note that the probability that $\text{Hyb}_{1,\kappa}^{\mathcal{A}}(1^\lambda)$ does not abort is $1/|\mathbb{J}| = 1/\text{poly}(\lambda)$. Let us denote this event by $\overline{\text{Abort}}$. Assume that there exists an adversary $\mathcal{A}$ that can distinguish between $\text{Hyb}_{1,\kappa}^{\mathcal{A}}(1^\lambda)$ and $\text{Hyb}_{1,\kappa+1}^{\mathcal{A}}(1^\lambda)$ with non-negligible probability $\epsilon(\lambda)$, i.e.,

$$\left| \Pr \left[1 \leftarrow \text{Hyb}_{1,\kappa}^{\mathcal{A}}(1^\lambda) \right] - \Pr \left[1 \leftarrow \text{Hyb}_{1,\kappa+1}^{\mathcal{A}}(1^\lambda) \right] \right| > \epsilon(\lambda)$$

We will construct a reduction adversary $\mathcal{B}$ that can break the security of MA-ABE with non-negligible probability. The description of $\mathcal{B}^{\mathcal{O}}$ is as follows.

Setup. $\mathcal{A}$ sends $n, s, k, \chi, \ell_1, \dots, \ell_n$. Randomly choose $i^* \leftarrow \mathbb{I}$. For $i \in \mathbb{I}, j \in \mathbb{J}$, if $i \neq i^*$ and $j \neq \kappa$, $\text{crs}^{(i,j)} \leftarrow \text{MA-ABE.GSetup}(1^\lambda, 1^{n-k}, 1^s, 1^L)$. Sample, for $\text{id} \in [n], \text{id} \notin \mathcal{K}_i$, $\text{mpk}_{\text{id}}^{(i,j)} \leftarrow \text{MA-ABE.ASetup}(\text{id}, 1^{\ell_{\text{id}}})$. Otherwise, $\text{crs}^{(i,j)}, \{\text{mpk}_{\text{id}}^{(i,j)}\}_{\text{id}} \leftarrow \mathcal{O}(n-k, s, \ell_1, \dots, \ell_n)$. Set $\text{CRS} := (\text{crs}^{(i,j)})_{i,j}$, $\text{MPK}_{\text{id}} := (\text{mpk}_{\text{id}}^{(i,j)})_{i,j}$, and $\text{MSK}_{\text{id}} := ((\text{msk}_{\text{id}}^{(i,j)})_{i \neq i^*, j}, (\text{msk}_{\text{id}}^{(i,j)})_{i=i^*, j \geq \kappa})$ and send $(\text{CRS}, \{\text{MPK}_{\text{id}}\}_{\text{id} \in [n]})$ to $\mathcal{A}$.

Secret Key Queries. $\mathcal{A}$ sends $(\text{id}, \text{GID}, x)$ such that id is not corrupted. For each $i \in \mathbb{I}'_i, j \in \mathbb{J}$, if $i \neq i^*$ and $j \neq \kappa$, $\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \text{MA-ABE.KGen}(\text{id}, \text{msk}_{\text{id}}^{(i,j)}, \{\text{mpk}_{\text{id}_x}^{(i,j)}\}_{\text{id}_x}, \text{GID}, x)$. Otherwise, $\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)} \leftarrow \mathcal{O}(\text{id}, \text{GID}, x)$. Set $\text{SK}_{\text{id}, \text{GID}, x} = (\text{sk}_{\text{id}, \text{GID}, x}^{(i,j)})_{i,j}$ and send it to $\mathcal{A}$.

Corruption Queries. $\mathcal{A}$ sends (Corr, id) . If $\text{id} \notin \mathcal{K}_{i^*}$, abort. Otherwise, send $(\text{msk}_{\text{id}}^{(i,j)})_{i,j}$ to $\mathcal{A}$.

Challenge Query. $\mathcal{A}$ sends P, m_0, m_1 . Sample $\{\text{sh}_i\}_{i \in \mathbb{I}'}$ such that $\bigoplus_{i \in \mathbb{I}'} \text{sh}_i = m_b$ and $\text{sh}'_{i^*} \xleftarrow{\$} \{0, 1\}^{|m_b|}$. For each $i \in \mathbb{I}', j \in \mathbb{J}$, if $i \neq i^*$ and $j \neq \kappa$, $\text{ct}^{(i,j)} \leftarrow \text{MA-ABE.Enc}(\{\text{mpk}_{\text{id}}^{(i)}\}_{\text{id} \notin \mathcal{K}_i}, F_{ij})$. Otherwise, $\text{ct}^{(i,j)} \leftarrow \mathcal{O}(F_{ij}, \text{sh}_{i^*}, \text{sh}'_{i^*})$. Set $\text{CT} = (\text{ct}^{(i,j)})_{i,j}$ and send it to $\mathcal{A}$.

Secret Key Queries, Corruption Queries. Same as pre-challenge queries.

Guess Phase. Output whatever $\mathcal{A}$ outputs.

As we can see, running time of $\mathcal{B}$ is polynomial in the running time of $\mathcal{A}$ and λ . In addition, the probability that $\mathcal{B}$ does not abort is exactly $\Pr[\overline{\text{Abort}}]$. Hence,

$$\begin{aligned}
\text{Adv}_{\mathcal{B}} &= \left| \Pr[\text{Abort}] \frac{1}{2} + \Pr[\overline{\text{Abort}}] \Pr[\mathcal{A} \text{ wins}] - \frac{1}{2} \right| \\
&= \left| \left(1 - \frac{1}{|\mathbb{I}|}\right) \frac{1}{2} + \frac{1}{|\mathbb{I}|} \Pr[\mathcal{A} \text{ wins}] - \frac{1}{2} \right| \\
&= \frac{\text{Adv}_{\mathcal{A}}}{|\mathbb{I}|}
\end{aligned}$$

If $\mathcal{O}$ encrypts sh'_{i^*} , $\mathcal{B}$ behaves like $\mathbf{Hyb}_{1,\kappa+1}^{\mathcal{A}}(1^\lambda)$ and if $\mathcal{O}$ encrypts sh_{i^*} , $\mathcal{B}$ behaves like $\mathbf{Hyb}_{1,\kappa}^{\mathcal{A}}(1^\lambda)$. In addition, as long as $|\mathbb{I}| = \text{poly}(\lambda)$, we have that $\mathcal{B}$ is a successful reduction adversary that can break the security of MA-ABE with non-negligible probability. $\square$

Lemma S.3. $\mathbf{Hyb}_{1,|\mathbb{I}|+1}^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ are identically distributed.

Proof. As we are encrypting shares $\text{sh}_1, \dots, \text{sh}_{i^*-1}, \text{sh}'_{i^*}, \text{sh}_{i^*+1}, \dots, \text{sh}_{|\mathbb{I}|}$ in $\mathbf{Hyb}_{1,|\mathbb{I}|+1}^{\mathcal{A}}(1^\lambda)$, these correspond to a random message. This is because one of the additive shares of m_b is missing which means all information about m_b is completely lost. Thus in both hybrids, we are encryption random shares which correspond to a random message. $\square$